

Numerical Analysis

A Learning Guide

Gustav Delius

Eric Sullivan

2026-09-22

Table of contents

Introduction	1
What Is Numerical Analysis?	1
Example from Algebra	1
Example from Calculus	2
Example from Differential Equations	3
Reasons to study Numerical Analysis	3
How this module works	5
Assessment	7
Textbooks	8
Your Jobs	8
Python	9
Google Colab	9
The Use of AI	10
About You	11
References	12
1. Numbers	13
1.1. Binary Numbers	14
1.2. Floating Point Numbers	18
1.3. Rounding Errors	22
1.4. Loss of Significant Digits	23
1.5. Exam-Style Question	25
2. Functions	27
2.1. Polynomial Approximations	27
2.1.1. Approximating the Exponential Function	29
2.1.2. Taylor Series	32
2.2. Truncation Error	36
2.3. Exam-Style Question	39
2.4. Neural Networks	40
2.4.1. A Single Neuron	40
2.4.2. Combining Neurons into a Layer	42
2.4.3. A Two-Layer Neural Network	43
2.4.4. Deep Neural Networks	46
2.4.5. Why Depth Matters: The Sawtooth Example	47

Table of contents

3. Roots 1	53
3.1. Introduction to Numerical Root Finding	53
3.2. The Bisection Method	55
3.2.1. Intuition	55
3.2.2. Implementation	58
3.2.3. Analysis	60
3.3. Fixed Point Iteration	67
3.4. Exam-Style Question	72
4. Roots 2	75
4.1. Newton's Method	75
4.1.1. Intuition	75
4.1.2. Implementation	78
4.1.3. Failures	79
4.1.4. Rate of Convergence	82
4.2. The Secant Method	83
4.2.1. Intuition and Implementation	83
4.2.2. Analysis	85
4.3. Order of Convergence	86
4.3.1. Definition	86
4.3.2. Fixed Point Iteration	87
4.3.3. Newton's Method	88
4.3.4. Secant Method	89
4.4. Exam-Style Question	91
5. Calculus	93
5.1. Intro to Numerical Calculus	93
5.2. Differentiation with Finite Differences	95
5.2.1. The First Derivative	95
5.2.2. Truncation Error	99
5.2.3. Efficient Coding	102
5.2.4. A Better First Derivative	107
5.2.5. The Second Derivative	109
5.3. Integration	113
5.3.1. Riemann Sums	115
5.3.2. Trapezoidal Rule	118
5.3.3. Simpson's Rule	121
5.4. Algorithm Summaries	126
5.5. Truncation Errors	127
5.5.1. Truncation Error in Finite-Difference Formulae	127
5.5.2. Truncation Error in Numerical Integration	128
5.6. Exam-Style Question	132

6. Optimisation	135
6.1. Single Variable Optimisation	135
6.1.1. Brute Force Search	137
6.1.2. Golden Section Search	138
6.1.3. Gradient Descent	142
6.1.4. Order of Convergence of Gradient Descent	145
6.2. Multivariable Optimisation	146
6.2.1. Gradient Descent Algorithm	148
6.3. Machine Learning	151
Least Squares Regression	153
6.4. Algorithm Summaries	156
6.5. Exam-Style Question	157
7. ODE 1	159
7.1. Euler's Method	160
7.2. Higher-order equations and systems of equations	167
7.3. The Midpoint Method	171
7.4. Numerical Instabilities	176
7.5. Algorithm Summaries	177
7.6. Truncation Errors	178
7.6.1. Truncation Error in Euler's Method	178
7.6.2. Truncation Error in the Midpoint Method	179
7.7. Exam-Style Question	180
8. ODE 2	183
8.1. Searching for a Better Method	183
8.2. Runge-Kutta Method	186
8.3. The Backward Euler Method	188
8.4. Numerical Instabilities	192
8.5. Stiff Equations	195
8.6. Algorithm Summaries	198
8.7. Exam-Style Question	199
9. PDE 1	201
9.1. Intro to PDEs	201
9.2. The Heat Equation	202
9.2.1. In One Spatial Dimension	202
9.2.2. Different Boundary Conditions	210
9.3. Other Parabolic PDEs	210
9.3.1. Reaction-Diffusion Equations	211
9.3.2. Advective-Diffusion Equations	211
9.4. Stability of Forward Difference Method	212
9.5. Exam-Style Question	214

10. PDE 2	217
10.1. Two-dimensional Heat Equation	217
10.2. Implicit Methods	222
10.2.1. Backward Difference Method	223
10.2.2. Stability of the Backward Difference Method	226
10.2.3. The Crank-Nicolson Method	227
10.2.4. Stability of the Crank-Nicolson Method	229
10.3. Exam-Style Question	230
 Appendices	 233
A. Python	233
A.1. Why Python?	233
A.2. Python Programming Basics	234
A.2.1. Variables	234
A.2.2. Indexing and Lists	236
A.2.3. List Operations	238
A.2.4. Tuples	240
A.2.5. Control Flow: Loops and If Statements	240
A.2.6. Functions	247
A.2.7. Lambda Functions	250
A.2.8. Packages	251
A.3. Numerical Python with NumPy	253
A.3.1. Numpy Arrays, Array Operations, and Matrix Operations	255
A.3.2. <code>arange</code> , <code>linspace</code> , <code>zeros</code> , <code>ones</code> , and <code>meshgrid</code>	260
A.4. Plotting with Matplotlib	262
A.4.1. Basics with <code>plt.plot()</code>	262
A.4.2. Subplots	265
A.4.3. Logarithmic Scaling with <code>semilogy</code> , <code>semilogx</code> , and <code>loglog</code>	266
A.5. Dataframes with Pandas	268
 B. Exam solutions	 269
B.1. Numbers	269
B.2. Functions	271
B.3. Roots 1	271
B.4. Roots 2	273
B.5. Calculus	274
B.6. Optimisation	275
B.7. ODE 1	276
B.8. ODE 2	277
B.9. PDE 1	279
B.10. PDE 2	280

C. Linear Algebra	283
C.1. Intro to Numerical Linear Algebra	283
C.2. Notation	284
C.3. Vectors and Matrices in Python	285
C.4. Matrix and Vector Operations	289
C.4.1. The Dot Product	289
C.4.2. Matrix Multiplication	291
C.5. The LU Factorization	296
C.5.1. A Recap of Row Reduction	296
C.5.2. The LU Decomposition	299
C.5.3. Solving Triangular Systems	305
C.5.4. Solving Systems with LU Decomposition	310
C.6. The QR Factorization	311
C.7. Over-determined Systems and Curve Fitting	321
C.8. The Eigenvalue-Eigenvector Problem	324
C.9. Algorithm Summaries	330
D. Automatic Differentiation	333
D.1. Forward Mode AD	335
D.2. Reverse Mode AD	338
D.3. Algorithm Summaries	342
E. PDE 3	343
E.1. The Wave Equation	343
E.2. The 2D Wave Equation	346
E.3. The Transport Equation	346
E.4. The Laplace and Poisson Equations	352
E.5. Algorithm Summaries	360

Introduction

What I cannot create, I do not understand.
–Richard P. Feynman

Mathematics is not just an abstract pursuit; it is an essential tool that powers a vast array of applications. From weather forecasting to black hole simulations, from urban planning to medical research, from ecology to epidemiology, the application of mathematics has become indispensable. Central to this applied force is Numerical Analysis.

What Is Numerical Analysis?

Numerical Analysis is the discipline that bridges continuous mathematical theories with their concrete implementation on digital computers. These computers, by design, work with discrete quantities, and translating continuous problems into this discrete realm is not always straightforward.

In this module, we will explore some key techniques, algorithms, and principles of Numerical Analysis that enable us to translate mathematical problems into computational solutions. We will delve into the challenges that arise in this translation, the strategies to overcome them, and the interaction of theory and practice.

Many mathematical problems cannot be solved analytically in closed form. In Numerical Analysis, we aim to find *approximation algorithms* for mathematical problems, i.e., schemes that allow us to compute the solution approximately. These algorithms use only elementary operations that computers know how to do ($+$, $-$, $\times$, $/$), but often a long sequence of them, so that in practice they need to be run on computers.

Example from Algebra

Solve the equation

$$\log(x) = \sin(x)$$

for x in the interval $x \in (0, \pi)$. Stop and try using all of the algebra that you ever learned to find x . You will quickly realize that there are no by-hand techniques that can solve this problem! A numerical approximation, however, is not so hard to come by. The following graph shows that there is a solution to this equation somewhere between $x = 2$ and $x = 2.5$.

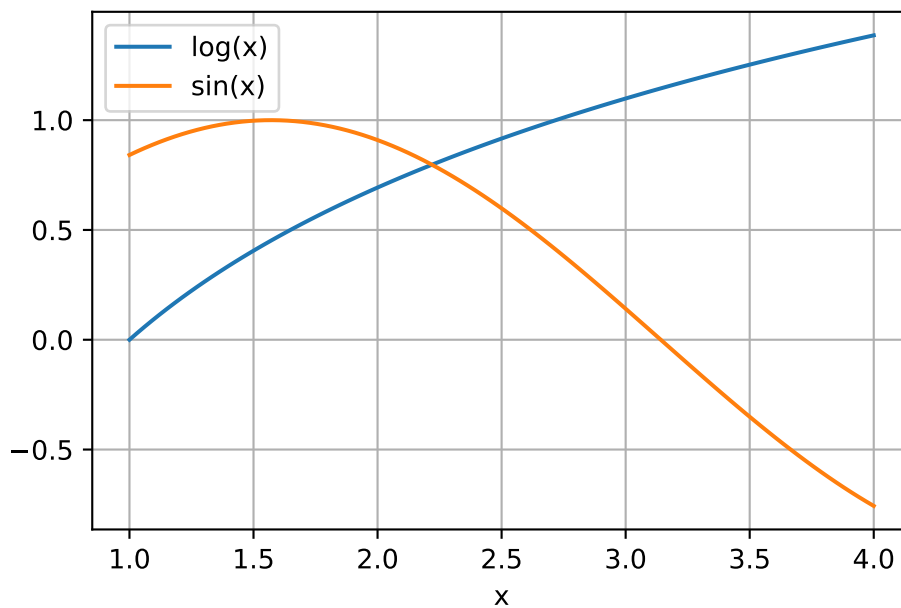


Figure 1.: The graphs of the real functions $\log(x)$ and $\sin(x)$ intersect at exactly one point, giving the solution to the equation $\log(x) = \sin(x)$.

Example from Calculus

What if we want to evaluate

$$\int_0^{\pi} \sin(x^2) dx?$$

Again, trying to use any of the possible techniques for using the Fundamental Theorem of Calculus, and hence finding an antiderivative, on the function $\sin(x^2)$ is completely hopeless. Substitution, integration by parts, and all of the other techniques that you know will all fail. Again, a numerical approximation is not so difficult and is very fast and gives the value

0.7726517138019184

By the way, this integral (called the Fresnel Sine Integral) actually shows up naturally in the field of optics and electromagnetism, so it is not just some arbitrary integral that was cooked up just for fun.

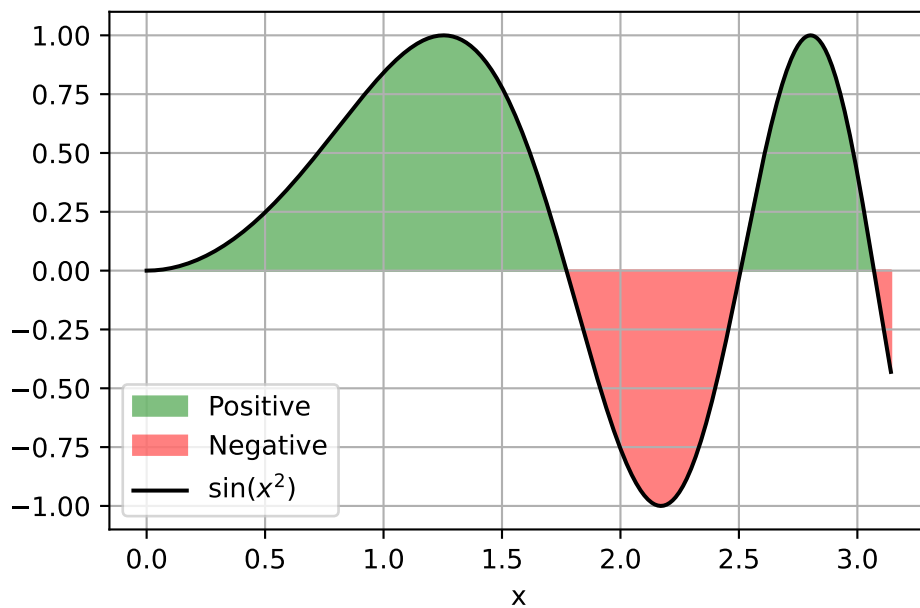


Figure 2.: Visual representation of the integral of $\sin(x^2)$ from 0 to π .

Example from Differential Equations

Say we needed to solve the differential equation

$$\frac{dy}{dt} = \sin(y^2) + t.$$

The nonlinear nature of the problem precludes us from using most of the typical techniques (e.g. separation of variables, undetermined coefficients, Laplace Transforms, etc). However, computational methods that result in a plot of an approximate solution can be made very quickly. Here is a plot of the solution up to time $t = 2.5$ with initial condition $y(0) = 0.1$:

This was an artificial example, but differential equations are central to modelling the real world in order to predict the future. They are the closest thing we have to a crystal ball. Here is a plot of a numerical solution of the SIR model of the evolution of an epidemic over time:

Reasons to study Numerical Analysis

So why should you want to venture into Numerical Analysis rather than just use the computer as a black box?

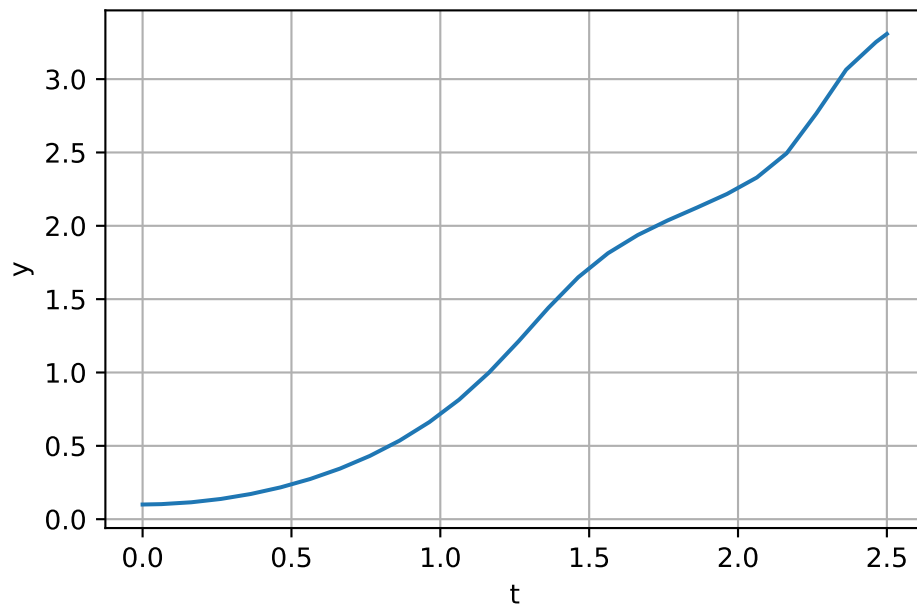


Figure 3.: Plot of numerical solution of $dy/dt = \sin(y^2) + t$ with $y(0) = 0.1$.

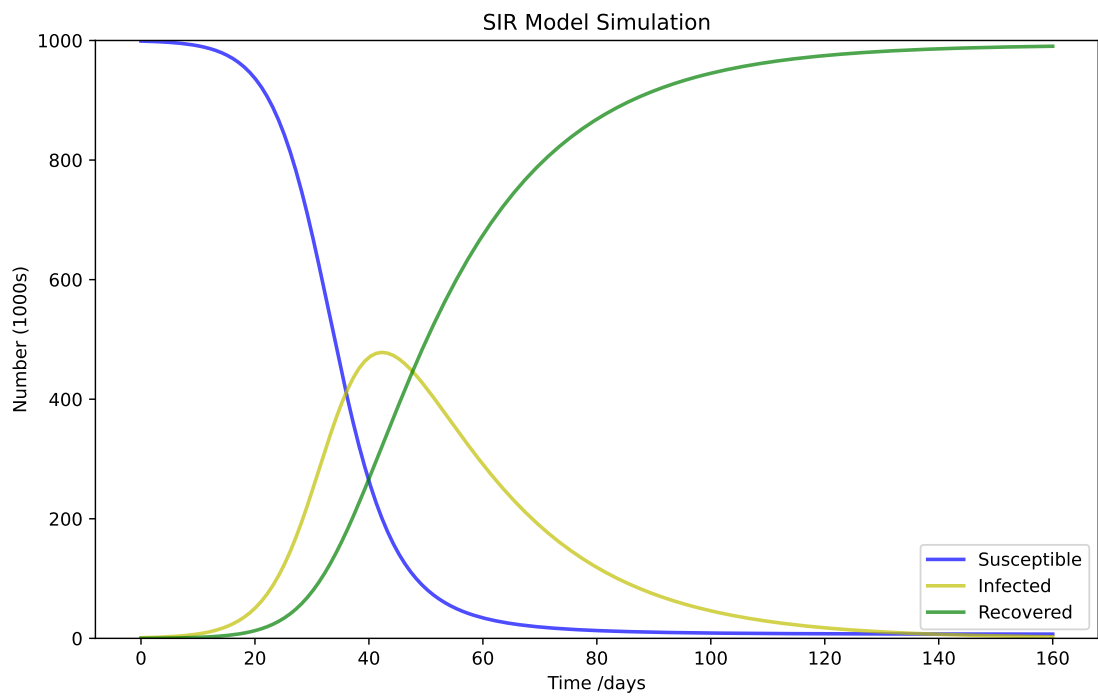


Figure 4.: Plot of a numerical solution of the SIR model

1. **Precision and Stability:** Computers, despite their power, can introduce significant errors if mathematical problems are implemented without care. Numerical Analysis offers techniques to ensure we obtain results that are both accurate and stable.
2. **Efficiency:** Real-world applications often demand not just correctness, but efficiency. By grasping the methods of Numerical Analysis, we can design algorithms that are both accurate and resource-efficient.
3. **Broad Applications:** Whether your interest lies in physics, engineering, biology, finance, or many other scientific fields, Numerical Analysis provides the computational tools to tackle complex problems in these areas.
4. **Basis for Modern Technologies:** Core principles of Numerical Analysis are foundational in emerging fields such as artificial intelligence, quantum computing, and data science.

The prerequisites for this material include a firm understanding of calculus and linear algebra and a good understanding of the basics of differential equations.

By the end of this module, you will not merely understand the methods of Numerical Analysis; you will be equipped to apply them efficiently and effectively in diverse scenarios: you will be able to tackle problems in physics, engineering, biology, finance, and many other fields; you will be able to design algorithms that are both accurate and resource-efficient; you will be able to ensure that your computational solutions are both accurate and stable; you will be able to leverage the power of computers to solve complex problems.

How this module works

There are 4 one-hour **whole-class sessions** every week. Three of these are listed on your timetable as “Lecture” and one as “Computer Practical”. However, in all these sessions you, the student, are the one that is doing the work; discovering methods, writing code, working problems, leading discussions, and pushing the pace. I, the lecturer, will act as a guide who only steps in to redirect conversations or to provide necessary insight. You will use the whole-class sessions to share and discuss your work with the other members of your group. There will also be some whole-class discussions moderated by me.

You will find that this text is not a set of lecture notes. Instead it mostly just contains collections of exercises with minimal interweaving exposition. It is expected that you **do every one of the exercises** in the main body of each chapter and use the sequencing of the exercises to guide your learning and understanding.

Therefore the whole-class sessions form only a very small part of your work on this module. For each hour of whole-class work you should timetable yourself about two and a half hours of **work outside class** for working through the exercises on your own. I strongly recommend that you put those two and a half hours (ten hours spread throughout the week) into your timetable.

Introduction

In order to enable you to get immediate feedback on your work also in between class sessions, I have made **feedback quizzes** where you can test your understanding of the material and your results from some of the exercises. Exercises that have an associated question in the feedback quiz are marked with a .

There are no traditional problem sheets in this module. In this module you will be working on exercises continuously throughout the week rather than working through a problem sheet only every other week.

At the end of each chapter there is a section entitled “**Problems**” that contains additional exercises aimed at consolidating your new understanding and skills. These are optional. Many of the chapters also have a section entitled “**Projects**”. These projects are more open-ended investigations, designed to encourage creative mathematics, to push your coding skills and to require you to write and communicate mathematics. These projects are entirely optional and perhaps you will like to return to one of these even after the module has finished. If you do work on one of the projects, be sure to share your work with me at gustav.deliuss@york.ac.uk because I will be very interested, also after the end of the module.

If you notice any mistakes or unclear things in the learning guide, please point them out to me in the class sessions or at gustav.deliuss@york.ac.uk. Many thanks go to Ben Mason and Toby Cheshire for the corrections they had sent in for a previous version of this guide.

You will need two **notebooks** for working through the exercises in this guide: one in **paper** form and one **electronic**. Some of the exercises are pen-and-paper exercises (and often these are marked with a pen icon) while others are coding exercises (often marked with a computer icon) and some require both writing or sketching and coding. You can keep your two notebooks linked through the numbering of the exercises. There are also some exercises that are marked with a Discussion icon . These are exercises that you should be discussing with other members of your group.

You will keep your coding notebooks in **Google Colab**, which we will discuss below. Most students find it easiest to have one dedicated Colab notebook per section, but some students will want to have one per chapter. You are highly encouraged to write explanatory text into your Google Colab notebooks as you go so that future-you can tell what it is that you were doing, which problem(s) you were solving, and what your thought processes were.

In the end, your collection of notebooks will contain solutions to every exercise in the guide and can serve as a reference manual for future numerical analysis problems. At the end of each of your notebooks you may also want to add a summary of what you have learned, which will both consolidate your learning and make it easier for you to remind yourself of your new skills later.

One request: do not share your notebooks publicly on the internet because that would create temptation for future students to not put in the work themselves, thereby robbing them of the learning experience.

If you have a **computer**, bring it along to the class sessions. However this is not a requirement. I will bring along some spare machines to make sure that every group has at least one computer to use during every session. The only requirements for a computer to be useful for this module is that it can connect to the campus WiFi, can run a web browser, and has a physical keyboard (typing code on virtual keyboards is too slow). The “Computer Practical” takes place in a PC classroom, so there will of course be plenty of machines available then.

Assessment

Unfortunately, your learning in the module also needs to be assessed. The final mark will be made up of 40% coursework and 60% final exam.

The **40% coursework** mark will come from **10 short quizzes** that will take place during the “Computer practical” in weeks 2 to 11. Answering each quiz should take less than 5 minutes but you will be given 16 minutes each to complete the quizzes in order to give you a large safety margin and remove stress. Late answers will not be accepted. The quizzes will be based on exercises that you will already have worked through and for which you will have had time to discuss them in class, so they will be really easy if you have engaged with the exercises as intended. Each quiz will be worth 5 points. There will be a practice quiz in the computer practical in week 1.

While working on the assessment quizzes you can already check your answers. If one of your answers is incorrect you can correct it and submit it again. However this will attract a penalty of 30% of the total mark for that answer. For multiple choice questions the penalty for a wrong answer may be even higher to discourage guessing. So it pays to be careful.

During the assessment quizzes you will be required to work exclusively on one of the PCs in the **computer practical room** rather than your own machine. This means in particular that you need to be physically present for the assessment. While working on the quiz you are only allowed to use a web browser, and the only pages you are allowed to have open are this guide, the quiz page on Moodle and any of your notebooks on Google Colab, with the AI features switched off. You are not allowed to use any AI assistants or other web pages. Besides your digital notebooks on Google Colab you may also use any hand-written notes as long as you have written them yourself.

To allow for the fact that there may be weeks in which you are ill or otherwise prevented from performing to your best in the assessment quizzes, your final coursework mark will be calculated as the average over your 8 best marks. If exceptional circumstances

Introduction

affect more than two of the 10 quizzes then you would need to submit an exceptional circumstances claim.

The **60% final exam** will be a 2 hour exam of the usual closed-book form in an exam room during the exam period. To prepare yourself for the final exam, there will be an exam style question at the end of each chapter, and I will make a practice exam available at the end of the module.

Textbooks

In this module we will only scratch the surface of the vast subject that is Numerical Analysis. The aim is for you at the end of this module to be familiar with some key ideas and to have the confidence to engage with new methods when they become relevant to you.

There are many textbooks on Numerical Analysis. Standard textbooks are (Burden and Faires 2010) and (Kincaid and Cheney 2009). They contain much of the material from this module. A less structured and more opinionated account can be found in (Acton 1990). Another well known reference that researchers often turn to for solutions to specific tasks is (Press et al. 2007). You will find many others in the library. They may go also under alternative names like “Numerical Methods” or “Scientific Computing”.

You may also want to look at textbooks for specific topics covered in this module, like for example (Butcher 2016) for methods for ordinary differential equations.

Your Jobs

You have the following jobs as a student in this module:

1. **Fight!** You will have to fight hard to work through this material. The fight is exactly what we are after since it is ultimately what leads to innovative thinking.
2. **Screw Up!** More accurately, do not be afraid to screw up. You should write code, work problems, and develop methods, then be completely unafraid to scrap what you have done and redo it from scratch.
3. **Collaborate!** You should collaborate with your peers, both within your group and across the whole class. Discuss exercises, ask questions, help others.
4. **Enjoy!** Part of the fun of inquiry-based learning is that you get to experience what it is like to think like a true mathematician / scientist. It takes hard work but ultimately this should be fun!

Python

To properly understand numerical analysis, you will need to write code in order to experiment with the methods we discuss. You will be using Python for this purpose. or most of you, coding in Python will not be new. For example, you have probably seen it in your first year “Mathematical Programming & Skills” module. But perhaps you have never seen Python before. You will have some notion of what a programming language “is” and “does”, but you may never have written any code. That is all right. You will pick it up as you go along.

Appendix A covers some of the basics of Python programming that we will use in this module. If you are new to Python, don’t feel that you need to work through this appendix in one go. Instead, spread the work over the first two weeks of the course and interlace it with your work on the first two chapters.

Google Colab

Every computer is its own unique flower with its own unique requirements. Hence, we will not spend time here giving you all of the ways that you can install Python and all of the associated packages necessary for this module. Instead I ask that you use the Google Colab notebook tool for writing and running your Python code: <https://colab.research.google.com>.

Google Colab allows you to keep all of your Python code on your Google Drive. The Colab environment is a free and collaborative version of the popular Jupyter notebook project. Jupyter notebooks allow you to write and test code as well as to mix writing (including LaTeX formatting) in along with your code and your output. I recommend that if you are new to Google Colab, you start by watching the brief introductory video.

Now the time has come for the first two exercises of this module.

Exercise 0.1. Spend a bit of time poking around in Colab. Figure out how to

- Create new Colab notebooks.
- Add and delete code cells.
- Type a simple calculation like $1+1$ into a code cell and evaluate it.
- Add and delete text cells.
- Add an equation to a text cell using LaTeX notation.
- Save a notebook to your Google Drive.
- Open a notebook from Google Drive.

Introduction

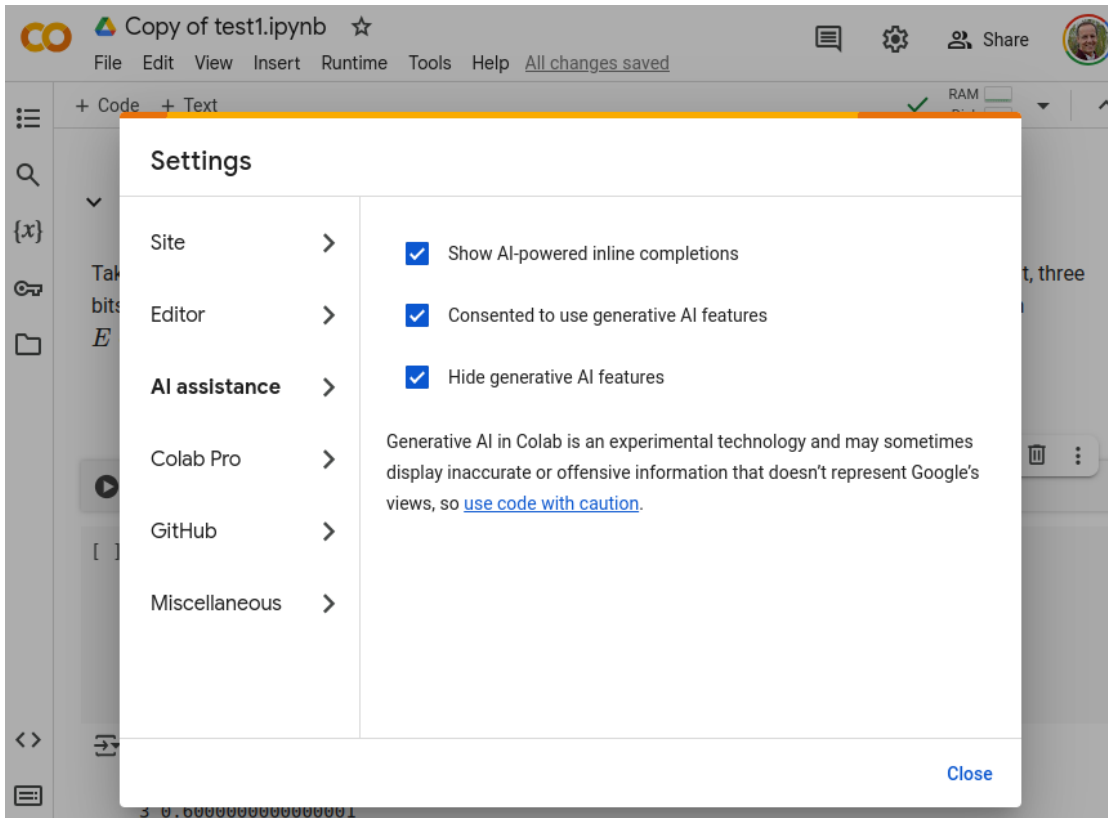
- Share a notebook with other members of your group and see if you can collaboratively edit it.
-

Exercise 0.2. Click on this link to a Colab notebook. It should open it in Colab. Then save a **copy** of it to your Google Drive. You need a copy because you will not have permission to edit the original. Follow the instructions in the notebook.

The Use of AI

You will have gathered from the previous exercise that in this module you are not only allowed to use AI, **you are encouraged to use AI**. However you have probably already discovered that you get more out of an AI if you are already familiar with the basic concepts of a subject. You will need to be able to understand and check any answer an AI gives you. If there is something in an AI answer that is not totally clear or not obviously correct, always ask the AI to explain the details of its answer and ask follow-on questions until everything is crystal-clear.

During the 10 assessment quizzes you will not be allowed to use any AI. In particular you will be required to switch off the AI features in Google Colab. It is thus a good idea when working on practice exercises to also switch off the AI features to make sure you know what you are doing even when there is no AI assistance. To switch off the AI features you should tick the “Hide generative AI” checkbox on the “AI assistance” tab of the “Settings” page in Google Colab.



About You

Exercise 0.3. I am interested to learn more about you to help me make this module work well for you. It would be helpful if you would answer the questionnaire at <https://forms.gle/YHHExaAnVzcLtBcU7>. Of course you are free to share as much or as little as you like.

© Gustav Delius. This learning guide is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. You may copy, distribute, display, remix, rework, and perform this copyrighted work, as long as you give credit to Gustav Delius gustav.delius@york.ac.uk and to the late Eric Sullivan, formerly Mathematics Faculty at Carroll College, on whose learning guide this one is based.

Introduction

References

1. Numbers

We think in generalities, but we live in details.
–Alfred North Whitehead

Have you ever wondered how computers, which operate in a realm of zeros and ones, manage to perform mathematical calculations with real numbers? The secret lies in approximation.

In this chapter and the next we will investigate the foundations that allow a computer to do mathematical calculations at all. How can it store real numbers? How can it calculate the values of mathematical functions? We will understand that the computer can do these things only approximately and will thus always make errors. Numerical Analysis is all about keeping these errors as small as possible while still being able to do efficient calculations.

We will meet the two kinds of errors that a computer makes: **rounding errors** and **truncation errors**. Rounding errors arise from the way the computer needs to approximate real numbers by binary floating-point numbers, which are the numbers it knows how to add, subtract, multiply and divide. We'll discuss this in this chapter. Truncation errors arise from the way the computer needs to reduce all calculations to a finite number of these four basic arithmetic operations. We will see this for the first time in Chapter 2 when we discuss how computers approximate functions by power series and then have to truncate these at some finite order.

Let's start with a striking example of how bad computers actually are at doing even simple calculations:

Exercise 1.1. By hand (no computers!) compute the first few terms of the sequence

$$x_{n+1} = \begin{cases} 2x_n, & x_n \in [0, \frac{1}{2}) \\ 2x_n - 1, & x_n \in [\frac{1}{2}, 1] \end{cases} \quad (1.1)$$

with the initial condition $x_0 = 1/10$. Calculate enough terms so that you can see a pattern.

Exercise 1.2. Now use a spreadsheet to do the computations. Do you get the same answers?

1. Numbers

Exercise 1.3. Finally, solve this problem with Python. Some starter code is given to you below.

```
x = 1.0/10
for n in range(50):
    if x < 0.5:
        # put the correct assignment here
    else:
        # put the correct assignment here
    print(x)
```

(Even if you don't know Python, you should be able to do this exercise after having read up to Section A.2.1 in the chapter on Essential Python.)

In the previous exercise it seems like the computer has failed you! What do you think happened on the computer and why did it give you a different answer? What, do you suppose, is the cautionary tale hiding behind the scenes with this problem?

Exercise 1.4. Now calculate the sequence that you get when starting with $x_0 = 1/8$? Do you again get a different answer from the computer than when doing the calculations by hand?

1.1. Binary Numbers

A computer circuit knows two states: on and off. As such, anything saved in computer memory is stored using base-2 numbers. This is called a binary number system. To fully understand a binary number system it is worthwhile to pause and reflect on our base-10 number system for a few moments.

What do the digits in the number “735” really mean? The position of each digit tells us something particular about the magnitude of the overall number. The number 735 can be represented as a sum of powers of 10 as

$$735 = 700 + 30 + 5 = 7 \times 10^2 + 3 \times 10^1 + 5 \times 10^0 \quad (1.2)$$

and we can read this number as 7 hundreds, 3 tens, and 5 ones.

Now let us switch to the number system used by computers: the binary number system. In a binary number system the base is 2 so the only allowable digits are 0 and 1 (just like in base-10 the allowable digits were 0 through 9). In binary (base-2), the number “101,101” can be interpreted as

$$101,101_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \quad (1.3)$$

(where the subscript “2” indicates the base). If we put this back into base 10, so that we can read it more comfortably, we get

$$101,101_2 = 32 + 0 + 8 + 4 + 0 + 1 = 45_{10}.$$

(The commas in the numbers are only to allow for greater readability – we can easily see groups of three digits and mentally keep track of what we are reading.)

Exercise 1.5. Express the following binary numbers in base-10.

1. 111_2
2. $10,101_2$
3. $1,111,111,111_2$

For the last one you can save yourself some work by noticing that $1,111,111,111_2 = 10,000,000,000_2 - 1$. This is a generally useful trick that will help you again in some later exercises.

Exercise 1.6. Explain the joke: *There are 10 types of people. Those who understand binary and those who do not.*

1. Numbers

Exercise 1.7. Discussion: With your group, discuss how you would convert a base-10 number into its binary representation, without using a calculator or computer. Once you have a proposed method put it into action on the number 237_{10} to show that the base-2 expression is $11,101,101_2$.

Exercise 1.8. By hand, using the methods you developed above, convert the following numbers from base 10 to base 2 or vice versa.

- Write 12_{10} in binary
- Write 11_{10} in binary
- Write 23_{10} in binary
- Write 11_2 in base 10
- What is 100101_2 in base 10?

The icons indicate that you should enter your answers into the feedback quiz. This gives you feedback on whether or not your answer is correct, but just as importantly, it lets your lecturer know how you are progressing with the material. The feedback quizzes are not used for assessment purposes, solely for feedback.

Exercise 1.9. Write down an explanation of the technique that your group has come up with to do the conversion from base 10 to base 2.

Next we will work with fractions and decimals.

Example 1.1. Let us take the base 10 number 5.341_{10} and expand it out to get

$$5.341_{10} = 5 + \frac{3}{10} + \frac{4}{100} + \frac{1}{1000} = 5 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} + 1 \times 10^{-3}.$$

The position to the right of the decimal point gives the negative power of 10 for that digit.

We can do a similar thing with binary decimals.

Exercise 1.10. The base-2 number $1,101.01_2$ can be expanded in powers of 2. Fill in the question marks below and observe the pattern in the powers.

$$1,101.01_2 = ? \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + ? \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}.$$

Example 1.2. Convert 11.01011_2 to base 10.

Solution:

$$\begin{aligned} 11.01011_2 &= 2 + 1 + \frac{0}{2} + \frac{1}{4} + \frac{0}{8} + \frac{1}{16} + \frac{1}{32} \\ &= 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} \\ &= 3.34375_{10}. \end{aligned}$$

Exercise 1.11. Convert the base 10 decimal 0.15625_{10} to binary using the following steps.

1. Multiply 0.15625 by 2. The whole number part of the result is the first binary digit to the right of the decimal point.
2. Take the result of the previous multiplication and ignore the digit to the left of the decimal point. Multiply the remaining decimal by 2. The whole number part is the second binary decimal digit.
3. Repeat the previous step until you have nothing left.

Explain to each other in the group why each step gives the binary digit that it does.

Exercise 1.12. Convert the base 10 fraction 0.1 into binary. Use this to explain why the computer made errors when it calculated with this number in Exercise 1.3.

1. Numbers

1.2. Floating Point Numbers

In this section we will discuss how a computer actually stores a number. More specifically, since computers only have finite memory, we would really like to know the full range of numbers that are possible to store in a computer. Clearly, given the uncountable nature of the real numbers, there will be gaps between the numbers that can be stored. We would like to know what gaps in our number system to expect when using a computer to store and do computations on numbers. For this it is important to know that computers store numbers in a way that is similar to how we write numbers in scientific notation.

Example 1.3. Let us start the discussion with a very concrete example. Consider the number $x = -123.15625$ (in base 10). As we have seen this number can be converted into binary. Indeed

$$x = -123.15625_{10} = -1111011.00101_2$$

(you should check this).

If a computer needs to store this number, it first puts it in the binary version of scientific notation. In this case this will be

$$x = -1.11101100101_2 \times 2^6.$$

This is the **floating-point representation** of the number.

Definition 1.1. For any non-zero base-2 number x the **floating-point representation** is given by

$$x = (-1)^s \times (1 + m) \times 2^E$$

where $s \in \{0, 1\}$, m is a binary number such that $0 \leq m < 1$, and E is an integer.

The number $1 + m$ is called the **significand**, s is known as the **sign bit**, and E is known as the **exponent**. We will refer to m , the fractional part of the significand that actually contains the information, as the **mantissa**, but this use is not universal.

To allow for both very large and very small numbers, the exponent E can be positive or negative. However, inside the computer it is efficiently stored as an unsigned integer e called the **biased exponent**. The true exponent E is obtained by subtracting a fixed **bias** B from e :

$$E = e - B.$$

The bias is chosen to be roughly half of the maximum possible value of the stored exponent e .

Example 1.4. What are the mantissa, sign bit, and unbiased exponent for the numbers 7_{10} , -7_{10} , and $(0.1)_{10}$?

Solution:

- For the number $7_{10} = 111_2 = 1.11 \times 2^2$ we have $s = 0, m = 0.11$ and $E = 2$.
 - For the number $-7_{10} = -111_2 = -1.11 \times 2^2$ we have $s = 1, m = 0.11$ and $E = 2$.
 - For the number $\frac{1}{10} = 0.000110011001100 \dots = 1.100110011 \dots \times 2^{-4}$ we have $s = 0, m = 0.100110011 \dots$, and $E = -4$.
-

In the last part of the previous example we saw that the number $(0.1)_{10}$ is actually a repeating decimal in base-2. This means that in order to completely represent the number $(0.1)_{10}$ in base-2 we need infinitely many decimal places. Obviously that cannot happen since we are dealing with computers with finite memory. Each number can only be allocated a finite number of bits. Thus the number needs to be rounded to the nearest number that can be represented with that number of bits. That leads to an error called the **rounding error** (sometimes also called *round-off error*). We'll look into these in more detail in Section 1.3 below.

Definition 1.2. Machine precision is the gap between the number 1 and the next larger floating-point number. Often it is represented by the symbol ϵ . To clarify: the number 1 can always be stored in a computer system exactly and if ϵ is machine precision for that computer then $1 + \epsilon$ is the next largest number that can be stored with that machine.

1. Numbers

For all practical purposes the computer cannot tell the difference between two numbers if the relative difference is smaller than machine precision. It is important to remember this when you want to check the equality of two numbers in a computer.

Exercise 1.13. To make all of these ideas concrete let us play with a small computer system where each number is stored in the following format, using 6 bits:

$$s e_1 e_2 b_1 b_2 b_3$$

The first bit is for the sign ($0 = +$ and $1 = -$). The next two bits, e_1 and e_2 , are for the biased exponent, and we will assume in this example that the bias is $B = 1$. The three bits on the right represent the fractional part of the significand. Hence, every number in this number system takes the form

$$(-1)^s \times (1.b_1 b_2 b_3)_2 \times 2^{(e_1 e_2)_2 - 1}$$

- What is the smallest positive number that can be represented in this form?
- What is the largest positive number that can be represented in this form?
- What is the machine precision in this number system?

Over the course of the past several decades there have been many systems developed to properly store numbers. The IEEE standard that we now use is the accumulated effort of many computer scientists, much trial and error, and deep scientific research. We now have two standard precisions for storing numbers on a computer: single and double precision. The double precision standard is what most of our modern computers use.

Definition 1.3. According to the IEEE 754 standard:

- A **single-precision** number consists of 32 bits, with 1 bit for the sign, 8 for the exponent, and 23 for the mantissa. The bias is $B = 127$.
- A **double-precision** number consists of 64 bits with 1 bit for the sign, 11 for the exponent, and 52 for the mantissa. The bias is $B = 1023$.

Exercise 1.14. What are the largest numbers that can be stored in single and double precision?

Exercise 1.15. What is machine precision for the single- and double-precision standards?

Exercise 1.16. What is the gap between 2^n and the next largest number that can be stored in double precision?

Exercise 1.17. Computers contain hardware that can perform the basic operations of addition, subtraction, multiplication, and division on floating-point numbers. To get a feel for what goes on under the hood, figure out how to add the numbers $x = 1.010_2 \times 2^3$ and $y = 1.110_2 \times 2^1$. How do you deal with the different exponents? What do you do so that the result is in the correct floating-point format?

Much more can be said about floating-point numbers, such as how we store infinity, how we store NaN, and how we store 0. The Wikipedia page for floating-point arithmetic might be of interest for the curious reader. It is beyond the scope of this module to go into all of those details here.

The biggest takeaway points from this section and the previous are:

- Real numbers are stored with finite precision in a computer.
- Nice rational numbers like 0.1 are sometimes not machine representable in binary.
- Machine precision is the gap between 1 and the next largest number that can be stored.
- The gap between one number and the next grows in proportion to the number.

1.3. Rounding Errors

When the binary representation of a real number has too many binary digits to be represented faithfully by a floating-point number, we need to round it to the nearest floating-point number that can be represented. That introduces rounding errors. We have seen above that the gap between two consecutive floating-point numbers grows in proportion to the number. This means that the relative error of rounding is bounded by $\epsilon/2$ where ϵ is the machine precision.

The rounding rule that is used is “**round to nearest, ties to even**”, which means that if the number is exactly halfway between two numbers that can be represented then we round the mantissa to an even binary number, i.e., to a mantissa that ends in 0.

Example 1.5. If we want to store the number $1.625 = 1.101_2$ in a floating-point number system where the mantissa has only 2 bits then we round to $1.10_2 = 1.5_{10}$ because 1.101_2 is exactly halfway between 1.100_2 and 1.110_2 and the rounding rule is “round to nearest, ties to even”.

To dive a little deeper into what happened in Exercise 1.3, simplify the detailed analysis by working with only a 4-bit mantissa:

Exercise 1.18. Write down how the number $1/10$ is represented in a floating-point number system where the mantissa has only 4 bits. Then calculate the first 10 terms of the sequence

$$x_{n+1} = \begin{cases} 2x_n, & x_n \in [0, \frac{1}{2}) \\ 2x_n - 1, & x_n \in [\frac{1}{2}, 1] \end{cases} \quad \text{with } x_0 = \frac{1}{10} \quad (1.4)$$

using this number system.

Exercise 1.19. (This problem is modified from (Greenbaum and Chartier 2012))

Sometimes floating-point arithmetic does not work like we would expect (and hope) as compared to exact mathematics. In each of the following problems we have a mathematical problem that the computer gets wrong. Explain why the computer is getting these wrong.

1. Mathematically we know that $\sqrt{5}^2$ should just give us 5 back. In Python type `np.sqrt(5)**2 == 5`. What do you get and why do you get it?
2. Mathematically we know that $49 \cdot (\frac{1}{49})$ should just be 1. In Python type `49*(1/49) == 1`. What do you get and why do you get it?

3. Mathematically we know that $e^{\log(3)}$ should just give us 3 back. In Python type `np.exp(np.log(3)) == 3`. What do you get and why do you get it?
4. Create your own example of where Python gets something incorrect because of floating-point arithmetic.

1.4. Loss of Significant Digits

As we have discussed, when representing real numbers by floating-point numbers in the computer, rounding errors will usually occur. When doing a calculation with double-precision floating-point numbers then the rounding error is only a tiny fraction of the actual number, so one might think that they really don't matter. However, calculations usually involve a number of steps, and we saw in Exercise 1.3 that the rounding errors can accumulate and become quite noticeable after a large number of steps.

But the problem is even worse. If we are not careful, then the rounding errors can get magnified already after very few steps if we perform the steps in an unfortunate way. The following examples and exercises will illustrate this.

Example 1.6. Consider the expression

$$(10^{10} + 0.123456789) - 10^{10}.$$

Mathematically, this is strictly equal to

$$(10^{10} - 10^{10}) + 0.123456789 = 0.123456789.$$

However, let us evaluate this in Python:

```
0.12345695495605469
```

Only the first six digits after the decimal point were preserved, the other digits were replaced by something seemingly random. The reason should be clear. The computer makes a rounding error when it tries to store the `10000000000.123456789`. This is known as the loss of significant digits. It can occur whenever you subtract two nearly equal numbers from each other.

Exercise 1.20. Consider these two mathematically equivalent ways to compute the same thing:

1. Numbers

- 1) $(a + b) - c$
- 2) $a + (b - c)$

- a) Why might these give different results in floating-point arithmetic?
 - b) If a is very small compared to b and c , which form would you expect to be more accurate? Why?
-

Exercise 1.21. Consider the trigonometric identity

$$2 \sin^2(x/2) = 1 - \cos(x).$$

It gives us two different methods to calculate the same quantity. Ask Python to evaluate both sides of the identity when $x = 0.0001$. Hint: as described in Section A.2.8, use `import math` so that you can then use `math.cos()` and `math.sin()`. Also remember that exponentiation in Python is represented by `**`.

What do you observe? If you want to calculate $1 - \cos(x)$ with the highest precision, which expression would you use? Discuss.

Exercise 1.22. You know how to find the solutions to the quadratic equation

$$ax^2 + bx + c = 0.$$

You know the quadratic formula. For the larger of the two solutions the formula is

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}.$$

Let's assume that the parameters are given as

$$a = 1, \quad b = 1000000, \quad c = 1.$$

Use the quadratic formula to find the larger of the two solutions, by coding the formula up in Python. You should get a solution slightly smaller than -10^{-6} . Hint: use `math.sqrt()` to code up the square root.

Then check whether your value for x really does solve the quadratic equation by evaluating $ax^2 + bx + c$ with your value of x . You will notice that it does not work. Discuss the cause of the error.

1.5. Exam-Style Question

Now, on a piece of paper, rearrange the quadratic formula for the larger solution by multiplying both the numerator and denominator by $-b - \sqrt{b^2 - 4ac}$ and then simplify by multiplying out the resulting numerator. This should give you the alternative formula

$$x = \frac{2c}{-b - \sqrt{b^2 - 4ac}}.$$

Can you see why this expression will work better for the given parameter values? Again evaluate x with Python and then check it by substituting into the quadratic expression. What do you find?

Exercise 1.23. Google the term “catastrophic cancellation” to find more examples of this phenomenon of loss of significant digits.

These exercises will give much material for in-class discussion. The aim is to make you sensitive to the issue of loss of significant digits and the fact that expressions that are mathematically equal are not always computationally equal.

1.5. Exam-Style Question

Consider a hypothetical “8-bit Mini-Float” system based on the IEEE 754 standard. The 8 bits are allocated as follows:

- **Sign bit (s):** 1 bit (Bit 7)
- **Exponent (e):** 3 bits (Bits 6-4), using a bias of 3.
- **Mantissa (m):** 4 bits (Bits 3-0), normalized with an implied leading 1.

The value of a number in this system is given by: $x = (-1)^s \times (1.m)_2 \times 2^{e-3}$.

- How is the **machine precision** ϵ defined? Give its value for this floating-point system. How is machine precision related to rounding errors? [3 marks]
- Convert the decimal number **13.5** into this 8-bit floating-point representation. Write your final answer as an 8-bit binary pattern (e.g., 0 101 1010). [4 marks]
- Using this specific floating-point system, perform the addition of the number **13.5** (from part b) and **0.25**.
 - Write 0.25 in this floating-point system.

1. Numbers

- Perform the addition simulating the hardware: align exponents, add significands.
 - Apply the rounding rule (“Round to Nearest, Ties to Even”) to determine the final stored bits.
 - What is the final decimal value stored by the system, and what is the absolute error compared to the exact mathematical sum? [4 marks]
- (d) A student attempts to calculate the function $f(x) = \sqrt{x^2 + 1} - x$ for a very large value $x = 10^8$.
- Explain why the result computed by a standard computer might be inaccurate (specifically naming the type of error).
 - Propose an algebraically equivalent formula for $f(x)$ that avoids this error. [3 marks]

2. Functions

How does a computer *understand* a function like $f(x) = e^x$ or $f(x) = \sin(x)$ or $f(x) = \log(x)$? What happens under the hood, so to speak, when you ask a computer to do a computation with one of these functions? A computer is good at arithmetic operations, but working with transcendental functions like these, or really any other sufficiently complicated functions for that matter, is not something that comes naturally to a computer. What is actually happening under the hood is that the computer only approximates the functions.

2.1. Polynomial Approximations

A class of functions that computers are very good at working with are polynomial functions. This is because to evaluate a polynomial function at any point we only need addition and multiplication operations. In this section we will explore how we can use polynomial functions to approximate other functions.

Exercise 2.1. In this exercise you are going to make a bit of a wish list for all of the things that a computer will do when approximating a function. We are going to complete the following sentence:

If we are going to approximate a smooth function $f(x)$ near the point $x = x_0$ with a simpler function $g(x)$ then ...

(I will get us started with the first two things that seem natural to wish for. The rest of the wish list is for you to complete.)

- the functions $f(x)$ and $g(x)$ should agree at $x = x_0$. In other words, $f(x_0) = g(x_0)$
- the function $g(x)$ should only involve addition, subtraction, multiplication, division, and integer exponents since computers are very good at those sorts of operations.
- if $f(x)$ is increasing / decreasing near $x = x_0$ then $g(x)$...
- if $f(x)$ is concave up / down near $x = x_0$ then $g(x)$...
- if we zoom into plots of the functions $f(x)$ and $g(x)$ near $x = x_0$ then ...

2. Functions

- ... is there anything else that you would add?
-

Exercise 2.2. Discuss: Could a polynomial function with a high enough degree satisfy everything in the wish list from the previous problem? Explain your reasoning.

Exercise 2.3. Let us put some parts of the wish list into action. If $f(x)$ is a differentiable function at $x = x_0$ and if $g(x) = A + B(x - x_0) + C(x - x_0)^2 + D(x - x_0)^3$ then

1. What is the value of A such that $f(x_0) = g(x_0)$? (*Hint: substitute $x = x_0$ into the $g(x)$ function*)
 2. What is the value of B such that at x_0 f and g have the same slope? In other words, what is the value of B such that $f'(x_0) = g'(x_0)$? (*Hint: Start by taking the derivative of $g(x)$*)
 3. What is the value of C such that at x_0 f and g have the same concavity? In other words, what is the value of C such that $f''(x_0) = g''(x_0)$?
 4. What is the value of D such that at x_0 f and g have the same third derivative? In other words, what is the value of D such that $f'''(x_0) = g'''(x_0)$?
-

In the previous 3 exercises you have built up some basic intuition for what we would want out of a mathematical operation that might build an approximation of a complicated function. What we have built is actually a way to get better and better approximations for functions out to pretty much any arbitrary accuracy that we like so long as we are near some anchor point (which we called x_0 in the previous exercises).

In the next several problems you will unpack the polynomial approximations of $f(x) = e^x$ and we will wrap the whole discussion with a little bit of formal mathematical language. Then we will examine other functions like $\sin(x)$ and $\log(x)$. One of the points of this whole discussion is to give you a little glimpse as to what is happening behind the scenes in scientific programming languages when you do computations with these functions. A bigger point is to start getting a feel for how we might go in reverse and approximate an unknown function out of much simpler parts. This last goal is one of the big takeaways from numerical analysis: *we can mathematically model highly complicated functions out of fairly simple pieces.*

2.1.1. Approximating the Exponential Function

Exercise 2.4. What is Euler's number e ? You have been using this number often in Calculus and Differential Equations. Do you know the decimal approximation for this number? Moreover, is there a way that we could approximate something like $\sqrt{e} = e^{0.5}$ or e^{-1} without actually having access to the full decimal expansion?

For all of the questions below let us work with the function $f(x) = e^x$.

1. The function $g_0(x) = 1$ matches $f(x) = e^x$ exactly at the point $x = 0$ since $f(0) = e^0 = 1$. Furthermore if x is very close to 0 then the functions $f(x)$ and $g_0(x)$ are really close to each other. Hence we could say that $g_0(x) = 1$ is an approximation of the function $f(x) = e^x$ for values of x very close to $x = 0$. Admittedly, though, it is probably pretty clear that this is a horrible approximation for any x just a little bit away from $x = 0$.
2. Let us get a better approximation. What if we insist that our approximation $g_1(x)$ matches $f(x) = e^x$ exactly at $x = 0$ and ALSO has exactly the same first derivative as $f(x)$ at $x = 0$.
 1. What is the first derivative of $f(x)$?
 2. What is $f'(0)$?
 3. Use the point-slope form of a line to write the equation of the function $g_1(x)$ that goes through the point $(0, f(0))$ and has slope $f'(0)$. Recall from algebra that the point-slope form of a line is $y = f(x_0) + m(x - x_0)$. In this case we are taking $x_0 = 0$ so we are using the formula $g_1(x) = f(0) + f'(0)(x - 0)$ to get the equation of the line.
3. Write Python code to build a plot like Figure 2.1. This plot shows $f(x) = e^x$, our first approximation $g_0(x) = 1$ and our second approximation $g_1(x) = 1 + x$. You may want to look at Example A.43 in the Python chapter for a refresher on how to build plots containing the graphs of several functions. If you need a hint on how to plot the function $g_0(x)$ since it is a constant function, take a look at the `np.ones_like()` function in Example A.40.

Exercise 2.5. Let us extend the idea from the previous problem to much better approximations of the function $f(x) = e^x$.

1. Let us build a function $g_2(x)$ that matches $f(x)$ exactly at $x = 0$, has exactly the same first derivative as $f(x)$ at $x = 0$, AND has exactly the same second derivative as $f(x)$ at $x = 0$. To do this we will use a quadratic function. For a quadratic approximation of a function we just take a slight extension to the point-slope form of a line and use the equation

$$g_2(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2.$$

2. Functions

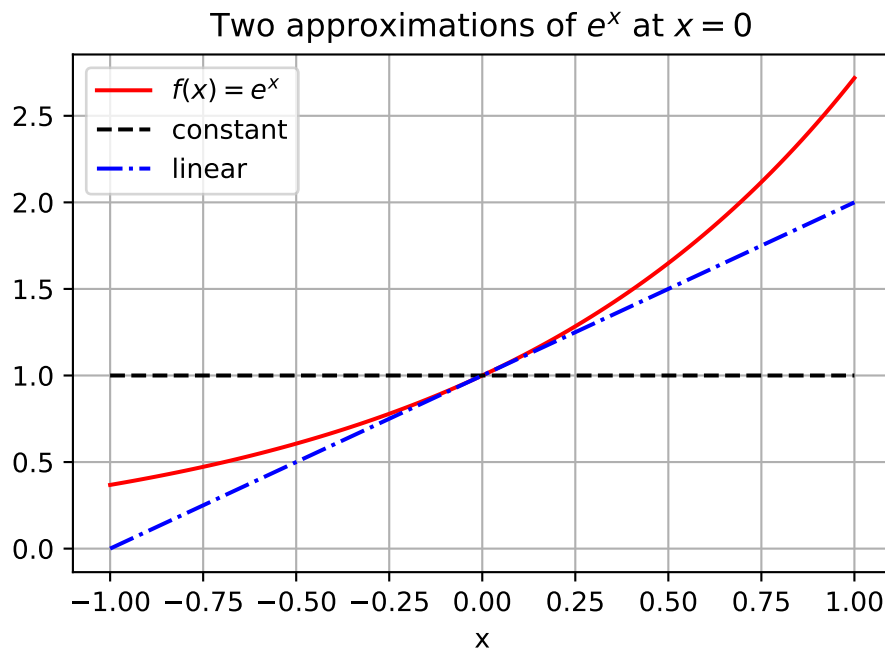


Figure 2.1.: The first two polynomial approximations of the exponential function.

In this case we are using $x_0 = 0$ so the quadratic approximation function looks like

$$g_2(x) = f(0) + f'(0)x + \frac{f''(0)}{2}x^2.$$

1. Find the quadratic approximation for $f(x) = e^x$.
2. Add your new function to the plot you created in the previous problem.
2. Let us keep going!! Next we will do a cubic approximation. A cubic approximation takes the form

$$g_3(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2 + \frac{f'''(x_0)}{3!}(x - x_0)^3$$

1. Find the cubic approximation for $f(x) = e^x$.
 2. How do we know that this function matches the first, second, and third derivatives of $f(x)$ at $x = 0$? What's the deal with the $3!$ on the cubic term?
 3. Add your function to the plot.
-

Exercise 2.6. Write a function that takes the arguments `x` and `n` and returns the `n`th order Taylor series approximation of $f(x) = e^x$. To remind yourself of how functions are defined in Python, you may want to look at Section A.2.6 in the Python chapter. You will also need a loop, see Section A.2.5. Please start from the following skeleton and put your code where it says “TODO”.

```
def exp_approx(x, n):
    """
    Computes the nth order Taylor series approximation of e^x at x=0.

    Parameters:
    x (float): The value at which to evaluate the approximation.
    n (int): The order of the Taylor series expansion.

    Returns:
    float: The nth order Taylor approximation of e^x.
    """
    if n < 0:
        raise ValueError("n must be at least 0")
    # Start with zero-order approximation
    approximation = 1.0
    # Add higher-order terms
    for i in range(1, n + 1):
        term = ... # TODO: calculate the ith term of the Taylor series
        approximation += term
    return approximation
```

Make sure you perfectly understand the code. Use the AI to explain everything to you in detail. You can select parts of the code and ask the AI questions like “Why do we need to use `range(1, n + 1)`?” and “What does `approximation +=` do?”

Exercise 2.7. Use the function `exp_approx` that you have built in Exercise 2.6 to approximate $\frac{1}{e} = e^{-1}$. Check the accuracy of your answer using `np.exp(-1)` in Python.

Exercise 2.8. Brainstorm within your group to see if you can make your function more efficient by writing it without using exponentiation and the factorial function, coding all multiplications and divisions explicitly. Try to minimise the number of arithmetic

2. Functions

operations that you need to perform? Can you make it so that it only needs $n - 1$ multiplications, $n - 1$ divisions and n additions?

Use the `%timeit` magic command to measure how fast your `exp_approx` function is. In a new code cell, run

```
%timeit exp_approx(1.5, 100)
```

This will run the function multiple times and give you an estimate of the execution time.

2.1.2. Taylor Series

What we have been exploring so far in this section is the **Taylor Series** of a function.

Definition 2.1 (Taylor Series). If $f(x)$ is analytic at the point x_0 , then

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2 + \dots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + \dots$$

on some interval around x_0 . The infinite polynomial expansion is called the **Taylor Series** of the function $f(x)$. Taylor Series are named for the mathematician Brook Taylor.

The Taylor Series of a function is often written with summation notation as

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k.$$

Do not let the notation scare you. In a Taylor Series you are just saying: *give me a function that*

- *matches $f(x)$ at $x = x_0$ exactly,*
- *matches $f'(x)$ at $x = x_0$ exactly,*
- *matches $f''(x)$ at $x = x_0$ exactly,*
- *matches $f'''(x)$ at $x = x_0$ exactly,*
- *etc.*

(Take a moment and make sure that the summation notation makes sense to you.)

Moreover, Taylor Series are built out of the easiest types of functions: polynomials. Computers are rather good at doing computations with addition, subtraction, multiplication, division, and integer exponents, so Taylor Series are a natural way to express functions in a computer. The downside is that we can only get true equality in the Taylor Series if we have infinitely many terms in the series. A computer cannot do infinitely many computations. So, in practice, we **truncate** Taylor Series after many terms and think of the new polynomial function as being *close enough* to the actual function so long as we do not stray too far from the anchor x_0 .

Exercise 2.9. Do all of the calculations to show that the Taylor Series centred at $x_0 = 0$ for the function $f(x) = \sin(x)$ is indeed

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Exercise 2.10. Write a Python function `sin_approx(x, n)` that computes the n th order Taylor Series approximation of $\sin(x)$ centred at $x_0 = 0$. Test your function by comparing its output to `np.sin(x)` for a few values of x and n . Use it to make a plot of the first three approximations for x in the range $[-\pi, \pi]$ similar to the plots you made for approximations of e^x above.

Exercise 2.11. Let us compute a Taylor Series that is not centred at $x_0 = 0$. For example, let us approximate the function $f(x) = \log(x)$ near $x_0 = 1$. Near the point $x_0 = 1$, the Taylor Series approximation will take the form

$$f(x) = f(1) + f'(1)(x-1) + \frac{f''(1)}{2!}(x-1)^2 + \frac{f'''(1)}{3!}(x-1)^3 + \dots$$

Write the first several terms of the Taylor Series for $f(x) = \log x$ centred at $x_0 = 1$ until you get a feel for the pattern.

Exercise 2.12. Write a Python function `log_approx(x, n)` that computes the n th order Taylor Series approximation of $\log(x)$ centred at $x_0 = 1$. Use it to build the plot below showing the approximations.

2. Functions

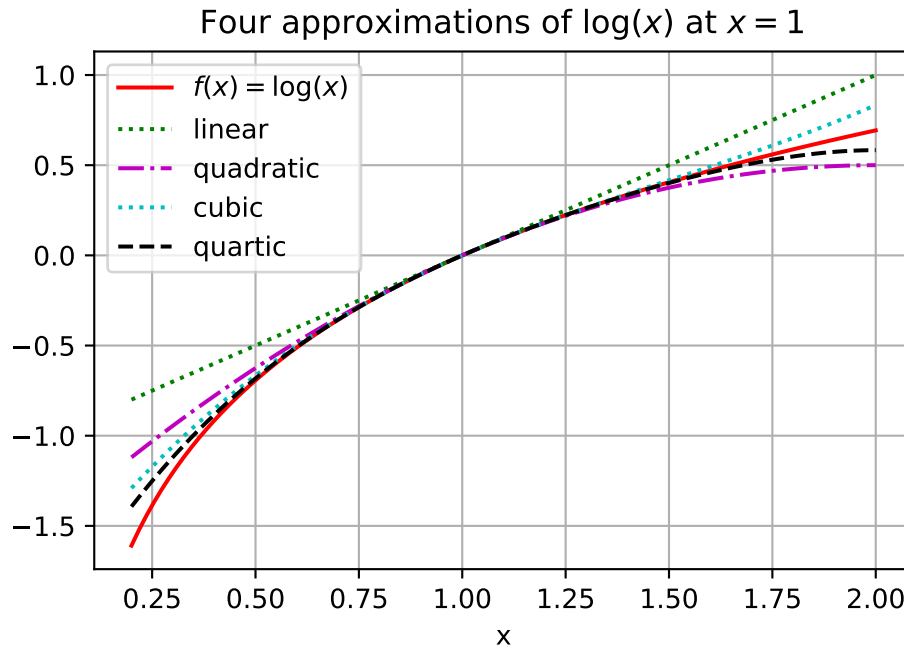


Figure 2.2.: Taylor series approximation of the logarithm.

Example 2.1. Let us conclude this brief section by examining an interesting example. Consider the function

$$f(x) = \frac{1}{1-x}.$$

If we build a Taylor Series centred at $x_0 = 0$ it is not too hard to show that we get

$$f(x) = 1 + x + x^2 + x^3 + x^4 + x^5 + \dots$$

(you should stop now and verify this!). However, if we plot the function $f(x)$ along with several successive approximations for $f(x)$ we find that beyond $x = 1$ we do not get the correct behaviour of the function (see Figure 2.3). More specifically, we cannot get the Taylor Series to change behaviour across the vertical asymptote of the function at $x = 1$. This example is meant to point out the fact that a Taylor Series will only ever make sense *near* the point at which you centre the expansion. For the function $f(x) = \frac{1}{1-x}$ centred at $x_0 = 0$ we can only get good approximations within the interval $x \in (-1, 1)$ and no further.

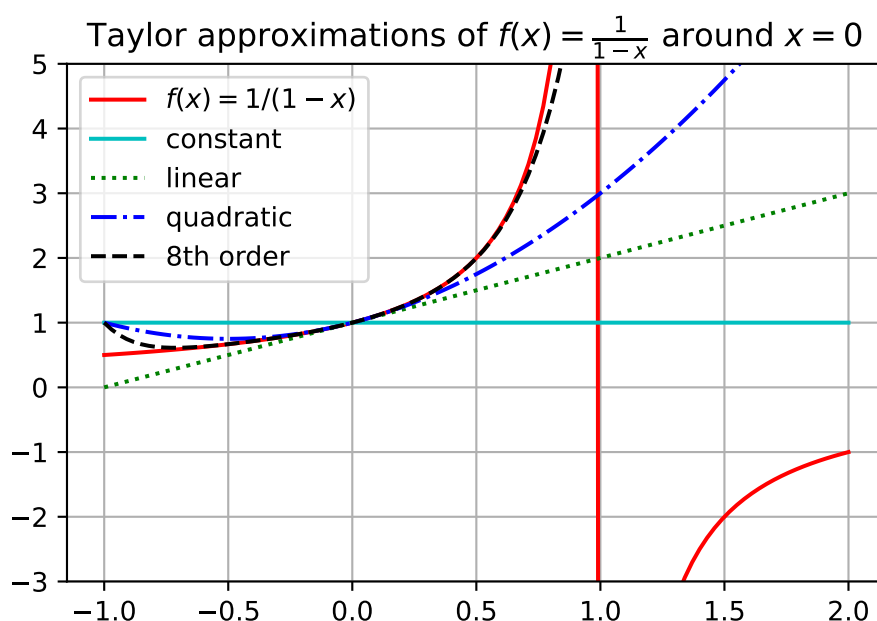


Figure 2.3.: Several Taylor Series approximations of the function $f(x) = 1/(1-x)$.

In the previous example we saw that we cannot always get approximations from Taylor Series that are good everywhere. For every Taylor Series there is a **domain of convergence** where the Taylor Series actually makes sense and gives good approximations. It is beyond the scope of this section to give all of the details for finding the domain of convergence for a Taylor Series. You have done that in your first-year Calculus module. However a good heuristic is to observe that a Taylor Series will only give reasonable approximations of a function from the centre of the series to the nearest asymptote. The domain of convergence is typically symmetric about the centre as well. For example:

- If we were to build a Taylor Series approximation for the function $f(x) = \log(x)$ centred at the point $x_0 = 1$ then the domain of convergence should be $x \in (0, 2)$ since there is a vertical asymptote for the natural logarithm function at $x = 0$.
- If we were to build a Taylor Series approximation for the function $f(x) = \frac{5}{2x-3}$ centred at the point $x_0 = 4$ then the domain of convergence should be $x \in (1.5, 6.5)$ since there is a vertical asymptote at $x = 1.5$ and the distance from $x_0 = 4$ to $x = 1.5$ is 2.5 units.
- If we were to build a Taylor Series approximation for the function $f(x) = \frac{1}{1+x^2}$ centred at the point $x_0 = 0$ then the domain of convergence should be $x \in (-1, 1)$. This may seem quite odd (and perhaps quite surprising!) but let us think about where the nearest asymptote might be. To find the asymptote we need to solve $1+x^2=0$ but this gives us the values $x = \pm i$. In the complex plane, the numbers i and $-i$ are 1 unit away from $x_0 = 0$, so the “asymptote” is not visible in a

2. Functions

real-valued plot but it is still only one unit away. Hence the domain of convergence is $x \in (-1, 1)$. You may want to pause now and build some plots to show yourself that this indeed appears to be true.

Of course you learned all this and more in your first-year Calculus but I hope it was fun to now rediscover these things yourself. In your Calculus module it was probably not stressed how fundamental Taylor series are to doing numerical computations.

2.2. Truncation Error

The great thing about Taylor Series is that they allow for the representation of potentially very complicated functions as polynomials, and polynomials are easily dealt with on a computer since they involve only addition, subtraction, multiplication, division, and integer powers. The downside is that the order of the polynomial is infinite. Hence, every time we use a Taylor series on a computer, what we are actually going to be using is a **Truncated Taylor Series** where we only take a finite number of terms. The idea here is simple in principle:

- If a function $f(x)$ has a Taylor Series representation it can be written as an infinite sum.
- Computers cannot do infinite sums.
- So stop the sum at some point n and throw away the rest of the infinite sum.
- Now $f(x)$ is approximated by some finite sum so long as you stay pretty close to $x = x_0$,
- and everything that we just chopped off of the end is called the **remainder** for the finite sum.

Let us be a bit more concrete about it. The Taylor Series for $f(x) = e^x$ centred at $x_0 = 0$ is

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

When we truncate this series at order n , we separate it into an approximation and a remainder:

$$e^x = \underbrace{1 + x + \frac{x^2}{2!} + \dots + \frac{x^n}{n!}}_{n^{th} \text{ order approximation}} + \underbrace{\frac{x^{n+1}}{(n+1)!} + \frac{x^{n+2}}{(n+2)!} + \dots}_{\text{remainder}}$$

For small values of x near $x_0 = 0$, the largest term in the remainder is $\frac{x^{n+1}}{(n+1)!}$, since higher powers of x become progressively smaller. We use **Big-O notation** to express this:

$$e^x \approx 1 + x + \frac{x^2}{2!} + \dots + \frac{x^n}{n!} + \mathcal{O}(x^{n+1}),$$

where the notation $\mathcal{O}(x^{n+1})$ (read “Big-O of x^{n+1} ”) signifies that the error is bounded by $C|x|^{n+1}$ for some constant C as $x \rightarrow 0$. This indicates that the error scales like x^{n+1} for values of x near the center $x_0 = 0$.

For example:

- 0th order: $e^x \approx 1 + \mathcal{O}(x)$
- 1st order: $e^x \approx 1 + x + \mathcal{O}(x^2)$
- 2nd order: $e^x \approx 1 + x + \frac{x^2}{2} + \mathcal{O}(x^3)$

Keep in mind that this sort of analysis is only good for values of x that are very close to the centre of the Taylor Series. If you are making approximations that are too far away then all bets are off.

Exercise 2.13. Now make the previous discussion a bit more concrete. You know the Taylor Series for $f(x) = e^x$ around $x = 0$ quite well at this point so use it to approximate the values of $f(0.1) = e^{0.1}$ and $f(0.2) = e^{0.2}$ by truncating the Taylor series at different orders. Because $x = 0.1$ and $x = 0.2$ are pretty close to the centre of the Taylor Series $x_0 = 0$, this sort of approximation is reasonable.

Then compare your approximate values to Python’s values $f(0.1) = e^{0.1} \approx \text{np.exp}(0.1) = 1.1051709180756477$ and $f(0.2) = e^{0.2} \approx \text{np.exp}(0.2) = 1.2214027581601699$ to calculate the truncation errors $\epsilon_n(0.1) = |f(0.1) - f_n(0.1)|$ and $\epsilon_n(0.2) = |f(0.2) - f_n(0.2)|$.

Fill in the blanks in the table. If you like, you can copy and paste the code and extend it to fill in the missing rows. For a bit of explanation of the syntax of the print commands see Example A.20 but for more detailed information ask Gemini.

Order n	$f_n(0.1)$	$\epsilon_n(0.1)$	$f_n(0.2)$	$\epsilon_n(0.2)$
0	1	0.1051709181	1	0.2214027582
1	1.1	0.005170918076	1.2	0.02140275816
2				
3				
4				
5				

You will find that, as expected, the truncation errors $\epsilon_n(x)$ decrease with n but increase with x .

2. Functions

Exercise 2.14. To investigate the dependence of the truncation error $\epsilon_n(x)$ on n and x a bit more, add an extra column to the table from the previous exercise with the ratio $\epsilon_n(0.2)/\epsilon_n(0.1)$.

Order n	$_{-n}(0.1)$	$_{-n}(0.2)$	$_{-n}(0.2) / _{-n}(0.1)$
0	0.1051709181	0.2214027582	2.105170918
1	0.005170918076	0.02140275816	4.139063479
2			
3			
4			
5			

Formulate a conjecture about how ϵ_n changes as x changes.

Exercise 2.15. To test your conjecture, examine the truncation error for the sine function near $x_0 = 0$. You know that the sine function has the Taylor Series centred at $x_0 = 0$ as

$$f(x) = \sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

So there are only approximations of odd order. Use the truncated Taylor series to approximate $f(0.1) = \sin(0.1)$ and $f(0.2) = \sin(0.2)$ and use Python's values `np.sin(0.1)` and `np.sin(0.2)` to calculate the truncation errors $\epsilon_n(0.1) = |f(0.1) - f_n(0.1)|$ and $\epsilon_n(0.2) = |f(0.2) - f_n(0.2)|$.

Complete the following table:

Order n	$_{-n}(0.1)$	$_{-n}(0.2)$	$_{-n}(0.2) / _{-n}(0.1)$
1	0.0001665833532	0.001330669205	7.988008283
3			
5			
7			
9			

To learn how you can loop over only odd integers in Python, see Example A.7.

Did these results force you to revise your conjecture of how ϵ_n changes as x changes?

The entry in the last row of the table will almost certainly not agree with your conjecture. That is okay! That discrepancy has a different explanation. Can you figure out what it is? Hint: Think about the discussion of machine precision in Chapter 1.

Exercise 2.16. Perform another check of your conjecture by approximating $\log(1.02)$ and $\log(1.1)$ from truncations of the Taylor series around $x = 1$:

$$\log(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \frac{x^5}{5} - \dots$$

Exercise 2.17. Write down your group's observations about how the truncation error changes as x changes. Explain this in terms of the form of the remainder of the truncated Taylor series.

2.3. Exam-Style Question

You know the Taylor Series expansion for $f(x) = \sin(x)$ centred at $x_0 = 0$:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

- (a) Explain why there is no x^2 term in this Taylor Series expansion. [2 marks]
- (b) Complete the Python code to calculate the Taylor expansion for $f(x) = \sin(x)$ centred at $x_0 = 0$ up to order n . [4 marks]

```
def sin_taylor(x, n):
    """
    Calculate the Taylor expansion for f(x) = sin(x) centred at x_0 = 0 up to order n.
    """
    ....
    for i in range(...):
        ....

    return result
```

- (c)
 - i) If we only keep terms up to x^3 in this Taylor Series expansion, what is the largest integer n so that the truncation error is $O(x^n)$? [2 marks]
 - ii) If we use the same approximation to calculate $\sin(0.05)$ instead of $\sin(0.1)$, by what factor do we expect the truncation error to decrease? [2 marks]
-

2.4. Neural Networks

So far we have discussed approximating functions with polynomials, and in particular with Taylor Series. However there are other families of functions that can be used to approximate an arbitrary function. One of them you have already met in your first year: Fourier series. Another one that is usually discussed in a course on Numerical Analysis is splines. In this module we will not discuss these families of functions. Our aim in this module is not to be exhaustive, but to get the fundamental ideas across, so that you will be well equipped to acquire further knowledge on the topic later on.

There is however a family of functions that has recently become very popular in machine learning: **neural networks**. This section guides you through exercises to obtain a good intuitive understanding of neural networks.

It may well be that by the time you reach this point in the learning guide, you will not have much time left this week. For that reason the material on neural networks is not examinable. You are under no pressure to work through this material. However, if you do have the time and interest, I would encourage you to do so.

Similar to how a polynomial $p(x)$ is determined by giving the coefficients in front of the powers of x , a neural network is determined by giving a set of parameters, called weights and biases. In this section you will explore how the weights and biases determine the neural network function.

2.4.1. A Single Neuron

Let us start by building up the components of a neural network one piece at a time. The fundamental building block is called a **neuron**.

Exercise 2.18. Consider the simple function

$$f(x) = \max(0, x).$$

The function $g(x) = \max(0, x)$ is called the **Rectified Linear Unit** or **ReLU** for short.

1. By hand, sketch the graph of this function for $x \in [-3, 3]$.
2. What is the derivative of $f(x)$ for $x > 0$? What about for $x < 0$? What happens at $x = 0$?

The ReLU function is an example of what is called an **activation function** in neural networks. The idea is that the neuron “activates” (produces a non-zero output) only when the input exceeds a certain threshold.

Exercise 2.19. Now let us make our neuron a bit more interesting by allowing it to have adjustable parameters.

Consider the function

$$h(x) = \max(0, wx + b)$$

where w and b are parameters that we can choose. These are called the **weight** and **bias** respectively.

1. By hand, sketch the graph of $h(x)$ for $x \in [-3, 3]$ for each of the following parameter values:
 - i. $w = 1, b = 0$
 - ii. $w = 2, b = 0$
 - iii. $w = 1, b = 1$
 - iv. $w = 1, b = -1$
 - v. $w = -1, b = 0$
2. Describe in words what the weight w controls about the function $h(x)$.
3. Describe in words what the bias b controls about the function $h(x)$.
4. For which values of x is the function “active” (i.e., non-zero) when $w = 1$ and $b = -1$?

Exercise 2.20. So far we have been working with a neuron that takes a single input x . In many applications, we want to work with functions of multiple variables.

Suppose we have two inputs x_1 and x_2 , and we define a neuron as

$$h(x_1, x_2) = \max(0, w_1x_1 + w_2x_2 + b)$$

where w_1, w_2 are weights and b is a bias.

1. What is the output of this neuron when $x_1 = 1, x_2 = 2$, using the parameters $w_1 = 1, w_2 = -1, b = 0$?
2. The expression $w_1x_1 + w_2x_2 + b = 0$ defines a line in the (x_1, x_2) plane. For the parameters in part 1, sketch this line in the region $x_1 \in [-2, 2], x_2 \in [-2, 2]$.
3. On which side of this line is the neuron “active” (produces non-zero outputs)? Shade that area in your sketch from part 2.

2. Functions

2.4.2. Combining Neurons into a Layer

A single neuron can detect when the input crosses a particular threshold. But to approximate more complex functions, we need to combine multiple neurons together. Let us explore this idea.

Exercise 2.21. Suppose we have two neurons, both taking the same input x , but with different weights and biases:

$$h_1(x) = \max(0, w_1x + b_1) \quad (2.1)$$

$$h_2(x) = \max(0, w_2x + b_2) \quad (2.2)$$

Consider the specific case where $w_1 = 1, b_1 = -1$ and $w_2 = -1, b_2 = 1$.

1. For what values of x is $h_1(x)$ active (non-zero)?
2. For what values of x is $h_2(x)$ active (non-zero)?
3. By hand, sketch the graphs of both $h_1(x)$ and $h_2(x)$ on the same axes for $x \in [-2, 2]$.
4. Now consider the sum $h_1(x) + h_2(x)$. By hand, sketch the graph of this function. Describe its shape.
5. What is the minimum value of $h_1(x) + h_2(x)$? At what value of x does this minimum occur?

Exercise 2.22. Rather than just adding two ReLU neurons, consider the weighted combination:

$$f(x) = c_1h_1(x) + c_2h_2(x)$$

where c_1 and c_2 are **output weights** that can be positive or negative.

Let $h_1(x) = \max(0, x)$ and $h_2(x) = \max(0, x - 1)$.

- What is $f(x) = h_1(x) - 2h_2(x)$ for $x < 0$?
- What is $f(x) = h_1(x) - 2h_2(x)$ for $0 \leq x < 1$?
- What is $f(x) = h_1(x) - 2h_2(x)$ for $x \geq 1$?

By hand, sketch the graph of this function.

Exercise 2.23. In the previous exercise you created a function with a triangular bump but the function continued decreasing to negative values beyond $x = 2$. Now see if you can find a way to combine three ReLU neurons to create a triangular bump function that:

- is zero for $x < 0$ and $x > 2$
 - equals x for $0 \leq x \leq 1$
 - equals $2 - x$ for $1 < x \leq 2$
1. Write down the weights $w_1, b_1, w_2, b_2, w_3, b_3$ and output weights c_1, c_2 and c_3 that produce the desired function. Hint: You need h_3 to “cancel out” the negative values after $x = 2$.
 2. Sketch the graph of your function to verify that it works.
 3. How could you modify the output weights to make the bump twice as tall?
 4. How could you modify the weights and biases to shift the bump so that it is centred at $x = 5$ instead of $x = 1$?
-

2.4.3. A Two-Layer Neural Network

In the previous exercises, you discovered that by combining multiple neurons (each with a ReLU activation), we can build flexible function approximations. Let us now formalise this into what is called a **two-layer neural network** or **single hidden layer network**.

The structure is as follows:

1. **Input:** We start with an input value x .
2. **Hidden Layer:** We apply n neurons to the input, creating n hidden values:

$$h_i(x) = \max(0, w_i x + b_i) \quad \text{for } i = 1, 2, \dots, n$$

The parameters w_i and b_i are called the **weights** and **biases** of the hidden layer.

3. **Output Layer:** We combine the hidden values using output weights:

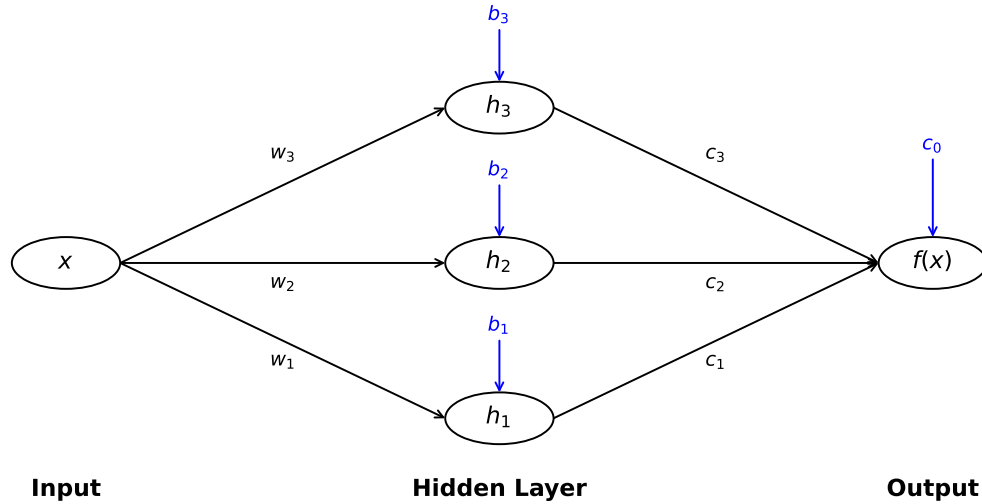
$$f(x) = c_1 h_1(x) + c_2 h_2(x) + \dots + c_n h_n(x) + c_0$$

where $c_0, c_1, \dots, c_n$ are the output layer parameters.

2. Functions

We can write this more compactly using summation notation:

$$f(x) = \sum_{i=1}^n c_i \max(0, w_i x + b_i) + c_0.$$



This function $f(x)$ is determined by the parameters:

- Hidden layer weights: $w_1, w_2, \dots, w_n$
- Hidden layer biases: $b_1, b_2, \dots, b_n$
- Output weights: $c_1, c_2, \dots, c_n$
- Output bias: c_0

That gives us a total of $3n + 1$ parameters to work with.

For the one-dimensional piecewise linear examples below, it is enough to use hidden neurons with weight 1. The biases place the knots, and the output weights change the slopes from one interval to the next. Thus we choose to work with

$$f_{nn}(x) = \sum_{i=1}^n c_i \max(0, x + b_i) + c_0$$

The following Python code implements such a neural network. Make sure you understand the code.

Exercise 2.24. Explain in your own words why a two-layer neural network is flexible enough to approximate many different functions. How do the output weights control the shape of the approximation?

How is this similar to polynomial approximation? How is it different?

Exercise 2.25. In this exercise we will approximate a smooth function with a neural network using a simple but powerful idea: we can create a piecewise linear approximation by using a sum of ReLU neurons, each of which activates at a new point. Each ReLU neuron that activates at a new point changes the slope of our piecewise linear approximation.

Consider approximating $f(x) = \sin(x)$ on the interval $[0, \pi]$ using a two-layer neural network. We'll place ReLU neurons at several “knot points” along the curve, and each neuron will adjust the slope.

1. Understanding slope changes: Suppose we use knot points at $x_1 = -b_1 = 0, x_2 = -b_2 = \pi/2$.
 - For $x \in [0, \pi/2)$: What is the slope of $f_{nn}(x)$? (Hint: which neurons are active?)
 - For $x \in [\pi/2, \pi)$: What is the slope?

Explain how the output weight c_2 controls the change in slope at $x = \pi/2$.

- Suggest a choice for the output bias c_0 and output weights c_1, c_2 that you think will give some kind of crude approximation of $\sin(x)$ on the interval $[0, \pi]$. Use the Python function `plot_neural_network` provided below to make a plot of your approximation.
2. Use the Python function `plot_neural_network` with 4 knot points at $x = 0, \pi/4, \pi/2, 3\pi/4$. Try to find output weights that create a good approximation.
 3. Calculate the maximum approximation error and discuss how it could be improved.
 4. Experiment with more knot points (try 9 or 17 points evenly spaced). How does the approximation improve?

Python helper function:

After your experimentation, the following result will no longer seem so surprising:

2. Functions

Theorem 2.1 (Universal Approximation Theorem (informal version)). *A two-layer neural network with a sufficient number of neurons can approximate any continuous function on a bounded interval to arbitrary accuracy.*

More precisely: for any continuous function $f : [a, b] \rightarrow \mathbb{R}$ and any $\epsilon > 0$, there exists a two-layer neural network f_{nn} such that

$$\max_{x \in [a, b]} |f(x) - f_{nn}(x)| < \epsilon.$$

Exercise 2.26. Reflect on what you have learned about neural networks and compare them to Taylor series:

1. What are the advantages of using neural networks for function approximation compared to Taylor series?
 2. What are the advantages of Taylor series compared to neural networks?
 3. For each of the following functions, which approximation method (Taylor series or neural network) do you think would be more appropriate, and why?
 - $f(x) = e^x$ near $x = 0$
 - $f(x) = |x|$ near $x = 0$
 - $f(x) = \sin(100x)$ on $[0, 2\pi]$
 - A function defined by experimental data points
 4. Neural networks are determined by their weights and biases. Taylor series are determined by the derivatives of the function at a point. Which do you think is easier to compute in practice, and why?
-

2.4.4. Deep Neural Networks

So far we have looked at **two-layer** neural networks (one hidden layer + one output layer). But we can also stack multiple layers on top of each other to create **deep neural networks**.

The idea is simple: instead of having the output layer directly combine the hidden neurons, we can feed the hidden neurons into another layer of neurons, and then another, and so on.

For example, a three-layer network would look like:

1. **Input:** x

2. **First Hidden Layer:**

$$h_i^{(1)}(x) = \max(0, w_i^{(1)}x + b_i^{(1)}) \quad \text{for } i = 1, \dots, n_1$$

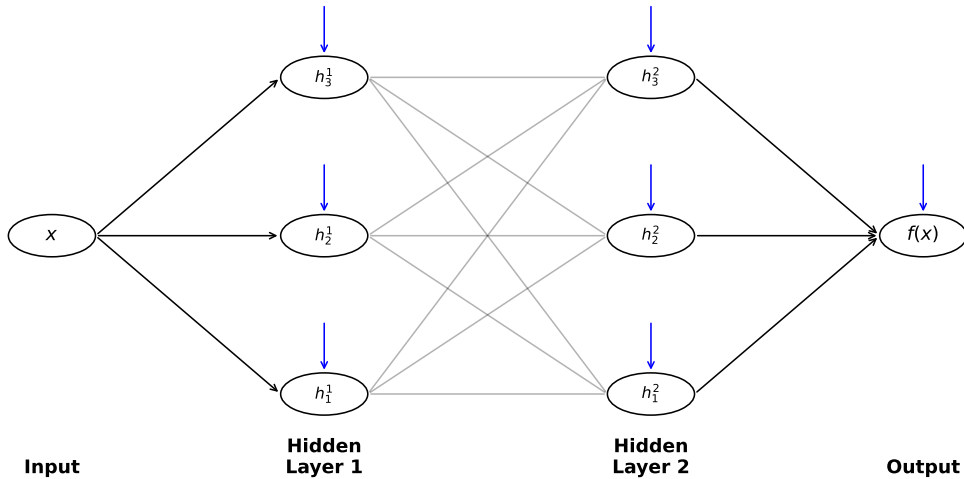
3. **Second Hidden Layer:** Each neuron in this layer takes all outputs from the first hidden layer:

$$h_j^{(2)} = \max \left(0, \sum_{i=1}^{n_1} w_{ji}^{(2)} h_i^{(1)} + b_j^{(2)} \right) \quad \text{for } j = 1, \dots, n_2$$

4. **Output Layer:**

$$f(x) = \sum_{j=1}^{n_2} c_j h_j^{(2)} + c_0$$

This is called a **deep neural network** because it has multiple hidden layers. The term “deep learning” comes from using such multi-layer architectures.



2.4.5. Why Depth Matters: The Sawtooth Example

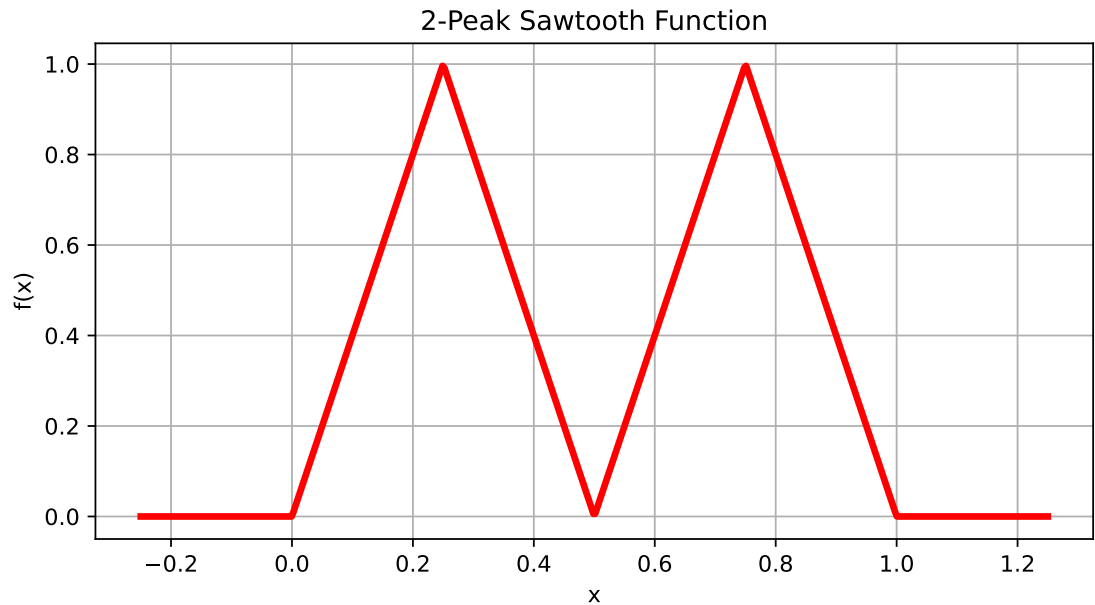
Why are deep neural networks, with many layers, often more powerful than shallow networks with just one large hidden layer? In the following exercises, you will explore a classic example, the **sawtooth wave**, to discover a concrete answer. You’ll build both a shallow and a deep network to represent the same function, and compare their efficiency as the complexity grows. Through this exploration, you’ll see how deep networks

2. Functions

can represent certain functions far more efficiently by exploiting their compositional structure.

We'll work with a sawtooth function that has 2 triangular peaks on the interval $[0, 1]$:

$$f(x) = \begin{cases} 4x & \text{if } 0 \leq x < 0.25 \\ 2 - 4x & \text{if } 0.25 \leq x < 0.5 \\ 4x - 2 & \text{if } 0.5 \leq x < 0.75 \\ 4 - 4x & \text{if } 0.75 \leq x \leq 1 \end{cases}$$



Exercise 2.27 (Building a Shallow Network for the Sawtooth). You already know how to represent piecewise linear functions with a two-layer neural network. Each of the 5 “hinge points” (where the slope changes) requires a ReLU neuron.

- a) For each hinge point at position x_i , you need a ReLU neuron with bias $b_i = -x_i$.
- b) What are the output weights with which you need to combine these neurons?

Use this to complete the code below and use the plot to check that the resulting function has the desired form.


```
def shallow_2_peak_network(x):
    biases = [.....]
    output_weights = [.....]
    return two_layer_network(x, biases, output_weights)

# Plot
plt.figure(figsize=(10, 6))
x = np.linspace(-0.1, 1.1, 500)
plt.plot(x, shallow_2_peak_network(x), 'r-', lw=3)
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('2-Peak Sawtooth Function')
plt.show()
```

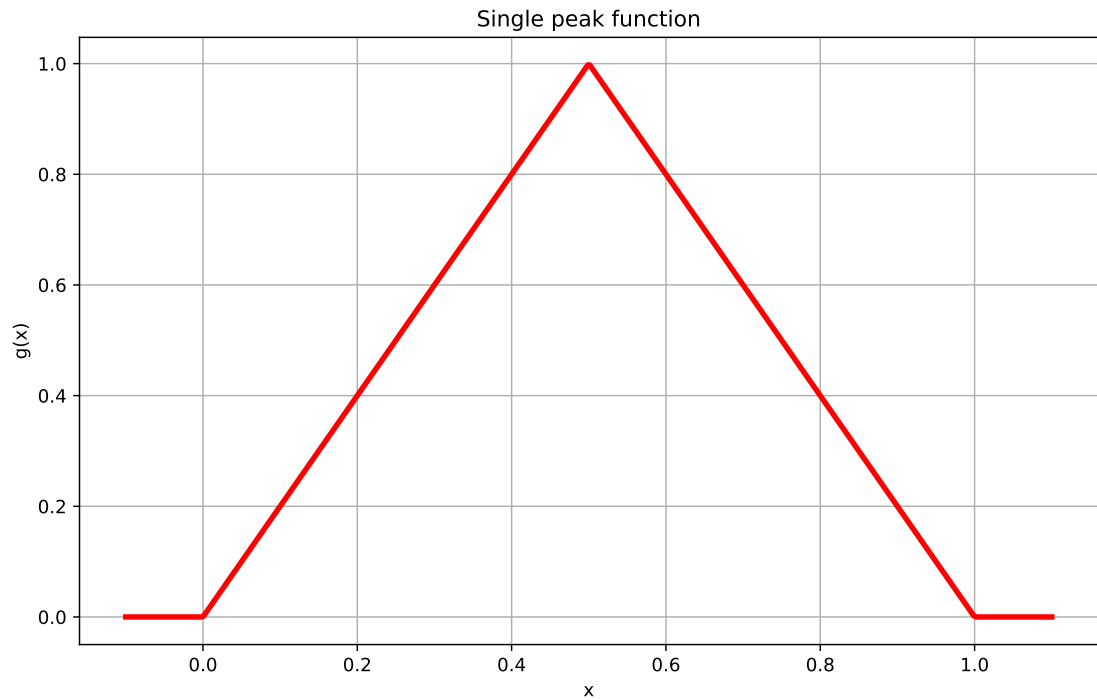
Exercise 2.28 (Building the Deep Network). A deeper network takes a more elegant approach. Instead of directly creating all the peaks, it uses one layer to create a single peak and then another layer to duplicate this peak.

The key insight is to create a single-peak function

$$g(x) = \begin{cases} 2x & 0 \leq x < 1/2 \\ 2(1-x) & 1/2 \leq x \leq 1 \\ 0 & x \notin [0, 1] \end{cases}$$

that produces one triangular peak at $x = 0.5$ with height 1, returning to 0 at $x < 0$ and $x > 1$.

2. Functions



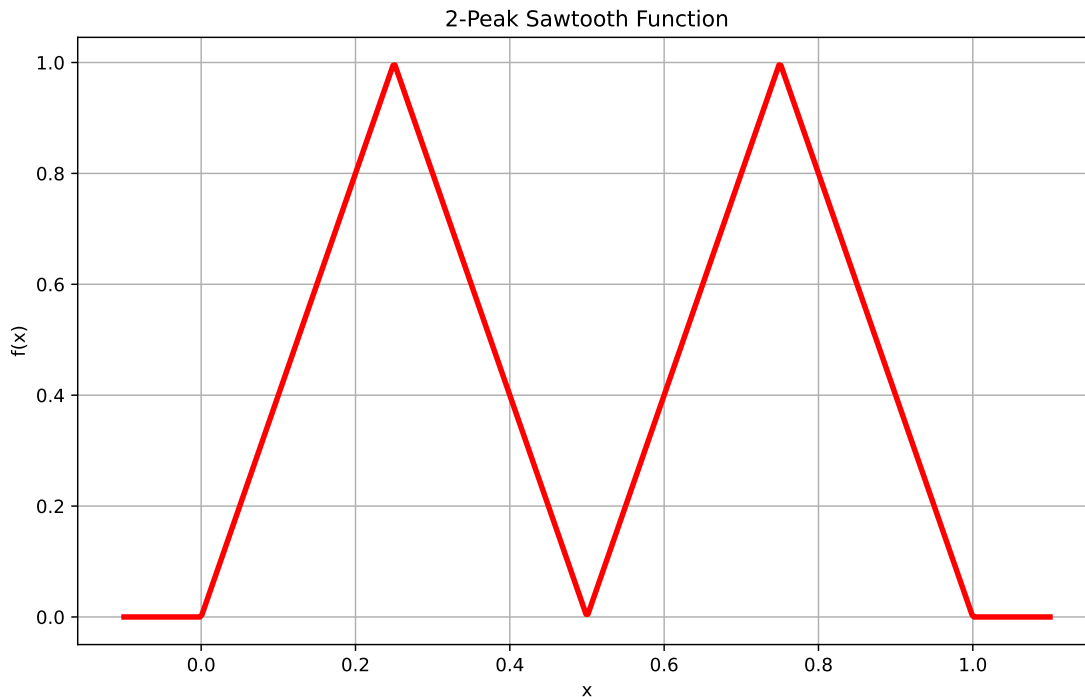
Then, our 2-peak sawtooth is simply $f(x) = g(g(x))$.

a) First, create a single-peak as a shallow network with three hidden neurons.

```
def g(x):
    """ Single triangular peak """
    biases = [.....]
    output_weights = [.....]
    return two_layer_network(x, biases, output_weights)

# Plot
plt.figure(figsize=(10, 6))
x = np.linspace(-0.1, 1.1, 500)
plt.plot(x, g(x), 'r-', lw=3)
plt.grid()
plt.xlabel('x')
plt.ylabel('g(x)')
plt.title('Single peak function')
plt.show()
```

b) Now comes the magic! Pass the result of your network $g(x)$ through the same network again: compute $g(g(x))$. Plot the result and compare it to the target.



c) Explain in your own words: Why does applying g twice produce two peaks? Think about what happens as the first application of g creates a peak that rises from 0 to 1 and back to 0. What does the second application of g see as its input?

d) Create a 4-peak sawtooth by composing g three times: $g(g(g(x)))$. Plot it. How many hidden neurons does this use? Compare to the shallow network approach from the previous exercise.

Exercise 2.29 (Scaling Analysis: The Exponential Advantage). Now let's compare how the two approaches scale as we increase the number of peaks.

a) Complete this table by implementing both shallow and deep networks for different numbers of peaks:

Number of Peaks	Shallow Network Neurons	Deep Network Neurons	Advantage (ratio)
2	5	6	0.83
4	?	?	?
8	?	?	?
16	?	?	?

2. Functions

b) Based on your table, write formulas for the number of neurons needed as a function of the number of peaks n for both approaches. What kind of growth do you observe (linear, logarithmic, exponential)?

Hint: For the deep network, if $n = 2^k$ peaks, how many layers do you need? How many neurons per layer?

c) Visualize the scaling behavior. Plot the number of neurons (y-axis) versus the number of peaks (x-axis) for both approaches:

```
# TODO: Create arrays for different numbers of peaks
peaks = [2, 4, 8, 16, 32, 64]
shallow_neurons = [...] # Fill in based on your formula
deep_neurons = [...]    # Fill in based on your formula

# Plot
plt.figure(figsize=(8, 5))
plt.plot(peaks, shallow_neurons, 'bo-', label='Shallow Network', lw=2)
plt.plot(peaks, deep_neurons, 'mo-', label='Deep Network', lw=2)
plt.xlabel('Number of Peaks')
plt.ylabel('Number of Hidden Neurons')
plt.title('Scaling: Shallow vs Deep Networks')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

d) Based on your analysis, explain why deep networks can be more efficient than shallow networks for certain types of functions. What specific property of the sawtooth function makes it particularly well-suited for a deep architecture?

e) The deep network achieves its efficiency by **reusing** the same learned “peak-making” module across layers through composition. Can you think of other real-world functions or patterns that might benefit from deep architectures in a similar way? Consider:

- Functions with repetitive structures at different scales (e.g., fractals, wavelets)
- Hierarchical patterns (e.g., language: letters → words → sentences)
- Functions that can be naturally expressed as compositions of simpler functions

f) What are the limitations of this advantage? Can you think of functions where a shallow network might be more efficient than a deep one? (Hint: What if the function has no repetitive or compositional structure?)

3. Roots 1

Success is the sum of small efforts, repeated day in and day out.
–Zeno of Elea

3.1. Introduction to Numerical Root Finding

In this chapter and the next we want to solve equations using a computer. Because it takes time to get used to doing numerical calculations we have split this material over two weeks.¹ In this chapter we will cover the bisection method and fixed point iteration. In the next chapter we will cover the Newton-Raphson and secant methods.

The goal of equation solving is to find the values of the independent variable that make the equation true. These are the sorts of equations that you learned to solve at school. For a very simple example, *solve for x* if $x + 5 = 2x - 3$. Or, for another example, the equation $x^2 + x = 2x - 7$ is an equation that could be solved with the quadratic formula. The equation $\sin(x) = \frac{\sqrt{2}}{2}$ is an equation that can be solved using some knowledge of trigonometry. The topic of numerical root finding really boils down to approximating the solutions to equations *without* using all of the by-hand techniques that you learned in high school. The downside to everything that we are about to do is that our answers are only ever going to be approximations.

The fact that we will only ever get approximate answers begs the question: *why would we want to do numerical algebra if by-hand techniques exist?* The answers are relatively simple:

- Most equations do not lend themselves to by-hand solutions. The reason you may not have noticed that is that we tend to show you only nice equations that arise in often very simplified situations. When equations arise naturally they are often not *nice*.
- By-hand algebra is often very challenging, quite time-consuming, and error-prone. You will find that the numerical techniques are quite elegant, work very quickly, and require very little overhead to actually implement and verify.

¹You may find that we are going too slowly for you and that you have the capacity and interest to learn more. In this case I can recommend the optional Appendix C on Numerical Linear Algebra.

3. Roots 1

Let us first take a look at equations in a more abstract way. Consider the equation $\ell(x) = r(x)$ where $\ell(x)$ and $r(x)$ stand for left-hand and right-hand expressions respectively. To begin solving this equation we can first rewrite it by subtracting the right-hand side from the left to get

$$\ell(x) - r(x) = 0. \quad (3.1)$$

Hence, we can define a function $f(x)$ as $f(x) = \ell(x) - r(x)$ and observe that **every** equation can be written as:

$$\text{Find } x \text{ such that } f(x) = 0. \quad (3.2)$$

This gives us a common language with which to frame all of our numerical algorithms. An x where $f(x) = 0$ is called a **root** of f and thus we have seen that solving an equation is always a root-finding problem.

For example, if we want to solve the equation $3\sin(x) + 9 = x^2 - \cos(x)$ then this is the same as solving $(3\sin(x) + 9) - (x^2 - \cos(x)) = 0$. We illustrate this idea in Figure 3.1. You should pause and notice that there is no way that you are going to apply by-hand techniques from algebra to solve this equation ... an approximate answer is pretty much our only hope.

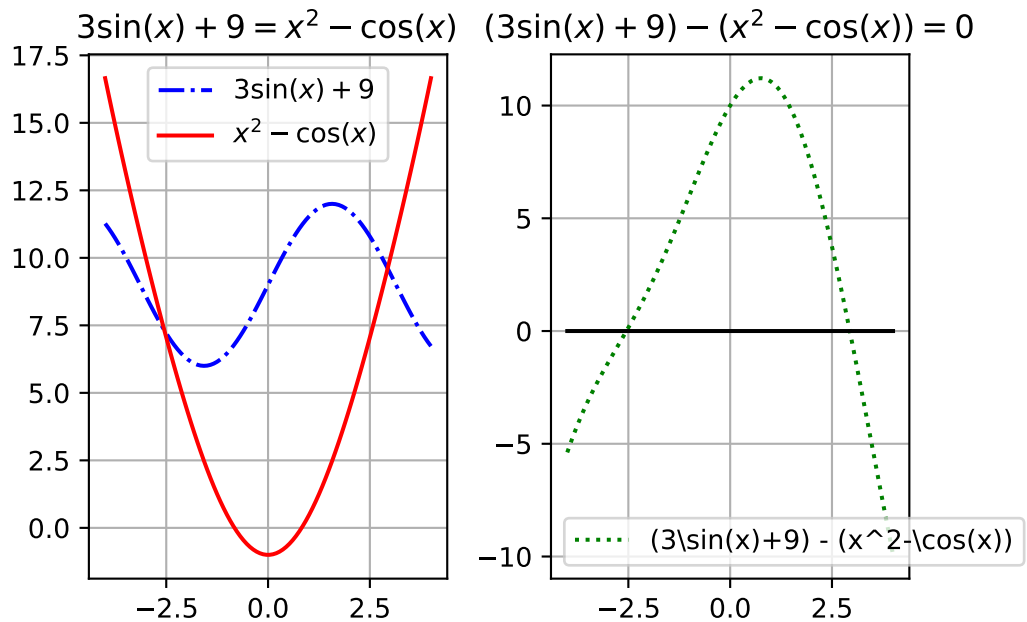


Figure 3.1.: Two ways to visualise the same root-finding problem

On the left-hand side of Figure 3.1 we see the solutions as the intersections of the graph of $3\sin(x) + 9$ with the graph of $x^2 - \cos(x)$, and on the right-hand side we see the solutions as the intersections of the graph of $(3\sin(x) + 9) - (x^2 - \cos(x))$ with the x -axis. From either plot we can read off the approximate solutions: $x_1 \approx -2.55$ and $x_2 \approx 2.88$.

Figure 3.1 should demonstrate what we mean when we say that solving equations of the form $\ell(x) = r(x)$ will give the same answer as finding the roots of $f(x) = \ell(x) - r(x)$.

We now have one way to view every equation-solving problem. As we will see in this chapter, if $f(x)$ has certain properties then different numerical techniques for solving the equation will apply, and some will be much faster and more accurate than others. In the following sections you will develop several different techniques for solving equations of the form $f(x) = 0$. You will start with the simplest techniques to implement and then move to the more powerful techniques that use some ideas from calculus to understand and analyse. Throughout this chapter you will also work to quantify the amount of error that one makes when using these techniques.

This chapter is split over two weeks. In the first week we will cover the bisection method and fixed point iteration. In the second week we will cover the Newton-Raphson and secant methods.

3.2. The Bisection Method

3.2.1. Intuition

This section contains several discussion exercises to get you thinking about the bisection method. You may not have anyone around to discuss with, in which case you should just discuss the questions with yourself, i.e., think through the questions. When you next meet with your group you can then discuss them with your peers.

Exercise 3.1. A friend tells you that she is thinking of a number between 1 and 100. She will allow you multiple guesses with some feedback for where the mystery number falls. How do you systematically go about guessing the mystery number? Is there an optimal strategy?

For example, the conversation might go like this.

- Sally: I am thinking of a number between 1 and 100.
- Joe: Is it 35?
- Sally: No, 35 is too low.
- Joe: Is it 99?
- Sally: No, 99 is too high.
- ...

3. Roots 1

Exercise 3.2. Imagine that Sally likes to formulate her answer not in the form “ x is too small” or “ x is too large” but in the form “ $f(x)$ is positive” or “ $f(x)$ is negative”. If she uses $f(x) = x - x_0$, where x_0 is Sally’s chosen number, do her new answers contain exactly the same information as her previous answers? Can you now explain how Sally’s game is a root-finding game?

Exercise 3.3. Now go and play the game with other functions $f(x)$. Choose someone from your group to be Sally and someone else to be Joe. Sally can choose a continuous function and Joe needs to guess its root. Does your strategy still allow Joe to find the root of $f(x)$ at least approximately? When should the game stop? Does Sally need to give Joe some extra information to give him a fighting chance?

Exercise 3.4. Was it necessary to say that Sally’s function was continuous? Does your strategy work if the function is not continuous?

Now let us get to the maths. We will start the mathematical discussion with a theorem from Calculus.

Theorem 3.1 (The Intermediate Value Theorem (IVT)). *If $f(x)$ is a continuous function on the closed interval $[a, b]$ and y_* lies between $f(a)$ and $f(b)$, then there exists some point $x_* \in [a, b]$ such that $f(x_*) = y_*$.*

Exercise 3.5. Draw a picture of what the intermediate value theorem says graphically.

Exercise 3.6. If $y_* = 0$ the Intermediate Value Theorem gives us important information about solving equations. What does it tell us? Fill in the blanks in the following corollary.

Corollary 3.1. *If $f(x)$ is a continuous function on the closed interval $[a, b]$ and if $f(a)$ and $f(b)$ have opposite signs then from the Intermediate Value Theorem we know that there exists some point $x_* \in [a, b]$ such that _____.*

The Intermediate Value Theorem (IVT) and its corollary are *existence theorems* in the sense that they tell us that some point exists. The annoying thing about mathematical existence theorems is that they typically do not tell us *how* to find the point that is guaranteed to exist. The method that you developed in Exercise 3.1 and Exercise 3.3 gives one possible way to find the root.

In those exercises you likely came up with an algorithm such as this:

- Say we know that a continuous function has opposite signs at $x = a$ and $x = b$.
- Guess that the root is at the midpoint $m = \frac{a+b}{2}$.
- By using the signs of the function, narrow the interval that contains the root to either $[a, m]$ or $[m, b]$.
- Repeat until the interval is small enough.

Now we will turn this strategy into computer code that will simply play the game for us. But first we need to pay careful attention to some of the mathematical details.

Exercise 3.7. Where is the Intermediate Value Theorem used in the root-guessing strategy?

Exercise 3.8. Why was it important that the function $f(x)$ is continuous when playing this root-guessing game? Provide a few sketches to demonstrate your answer.

3.2.2. Implementation

Exercise 3.9 (The Bisection Method). Goal: We want to solve the equation $f(x) = 0$ for x assuming that the solution x^* is in the interval $[a, b]$. To understand the bisection method, have someone in the group sketch a graph of the function $f(x)$ on the interval $[a, b]$. Then together, draw in the steps of the following algorithm:

The Algorithm: We assume that your group member has followed the instructions above and has sketched the graph of a function $f(x)$ that is continuous on the interval $[a, b]$. To make approximations of the solutions to the equation $f(x) = 0$, do the following:

1. Check to see if $f(a)$ and $f(b)$ have opposite signs. You can do this by taking the product of $f(a)$ and $f(b)$.
 - If $f(a)$ and $f(b)$ have different signs then what does the IVT tell you?
 - If $f(a)$ and $f(b)$ have the same sign then what does the IVT not tell you? What should you do in this case?
 - Why does the product of $f(a)$ and $f(b)$ tell us something about the signs of the two numbers?
2. Compute the midpoint of the closed interval, $m = \frac{a+b}{2}$. Mark the result in your plot so that you can see the value of $f(m)$.
 - Will m always be a better guess of the root than a or b ? Why?
 - What should you do here if $f(m)$ is really close to zero?
3. Compare the signs of $f(a)$ versus $f(m)$ and $f(b)$ versus $f(m)$.
 - What do you do if $f(a)$ and $f(m)$ have opposite signs?
 - What do you do if $f(m)$ and $f(b)$ have opposite signs?
4. Choose the new interval $[a, m]$ or $[m, b]$ and repeat steps 2 and 3 for this interval. Stop when the interval containing the root is small enough.

Exercise 3.10. We want to write a Python function for the bisection method. Instead of jumping straight into writing the code we should first come up with the structure of the code. It is often helpful to outline the structure as comments in your file. Use the template below and complete the comments. Note how the function starts with a so-called docstring that describes what the function does and explains the function parameters and its return value. This is standard practice and is how the help text is generated that you see when you hover over a function name in your code.

Don't write the code yet, just complete the comments. I recommend switching off the AI for this exercise because otherwise the AI will keep already suggesting the code while you write the comments.

```
def bisection(f, a, b, tol=1e-5):
    """
    Find a root of f(x) in the interval [a, b] using the bisection method.

    Parameters:
        f : function, the function for which we seek a root
        a : float, left endpoint of the interval
        b : float, right endpoint of the interval
        tol : float, stopping tolerance

    Returns:
        float: approximate root of f(x)
    """

    # check that a and b have opposite signs
    # if not, return an error and stop

    # calculate the midpoint m = (a+b)/2

    # start a while loop that runs while the interval is
    # larger than 2 * tol

    # if ...
    # elif ...
    # elif ...

    # Calculate midpoint of new interval

    # end the while loop
    # return the approximate root
```

Exercise 3.11. Now use the comments from Exercise 3.10 as a guide to complete a Python function for the bisection method. Test your bisection method code on the following equations. For each equation make a plot of the function on the interval to verify your result.

1. $x^2 - 2 = 0$ on $x \in [0, 2]$

3. Roots 1

2. $\sin(x) + x^2 = 2 \log(x) + 5$ on $x \in [1, 5]$

3. $3 \sin(x) + 9 = x^2 + \cos(x)$ on $x \in [1, 5]$

3.2.3. Analysis

After we build any root-finding algorithm we need to stop and think about how it will perform on new problems. The questions that we typically have for a root-finding algorithm are:

- Will the algorithm always converge to a solution?
 - How fast will the algorithm converge to a solution?
 - Are there any pitfalls that we should be aware of when using the algorithm?
-

Exercise 3.12. Discussion:

- What must be true in order to use the bisection method?
 - Does the bisection method work if the Intermediate Value Theorem does not apply? (Hint: what does it mean for the IVT to “not apply?”)
 - If there is a root of a continuous function $f(x)$ between $x = a$ and $x = b$, will the bisection method always be able to find it? Why or why not?
-

Next we will focus on a deeper mathematical analysis that will allow us to determine exactly how fast the bisection method actually converges to within a pre-set tolerance. Work through the next problem to develop a formula that tells you exactly how many steps the bisection method needs to take before it gets close enough to the true solution.

Exercise 3.13. Let $f(x)$ be a continuous function on the interval $[a, b]$ and assume that $f(a) \cdot f(b) < 0$. A recurring theme in Numerical Analysis is to approximate some mathematical thing to within some tolerance. For example, if we want to approximate the solution to the equation $f(x) = 0$ to within ε with the bisection method, we should be able to figure out how many steps it will take to achieve that goal.

3.2. The Bisection Method

1. Let us say that $a = 3$ and $b = 8$ and $f(a) \cdot f(b) < 0$ for some continuous function $f(x)$. The width of this interval is 5, so if we guess that the root is $m = (3 + 8)/2 = 5.5$ then our error is less than $5/2$. In the more general setting, if there is a root of a continuous function in the interval $[a, b]$ then how far off could the midpoint approximation of the root be? In other words, what is the error in using $m = (a + b)/2$ as the approximation of the root?
2. The bisection method cuts the width of the interval down to a smaller size at every step. As such, the approximation error gets smaller at every step. Fill in the blanks in the following table to see the pattern in how the approximation error changes with each iteration.

Iteration	Width of Interval	Maximal Error
1	$ b - a $	$\frac{ b-a }{2}$
2	$\frac{ b-a }{2}$	
3	$\frac{ b-a }{2^2}$	
$\vdots$	$\vdots$	$\vdots$
n	$\frac{ b-a }{2^{n-1}}$	

3. Now to the key question:

If we want to approximate the solution to the equation $f(x) = 0$ to within some tolerance ε then how many iterations of the bisection method do we need to take? Hint: Set the n^{th} approximation error from the table to be less than or equal to ε . What should you solve for from there?

In Exercise 3.13 you actually proved the following theorem.

Theorem 3.2 (Convergence Rate of the Bisection Method). *If $f(x)$ is a continuous function with a root in the interval $[a, b]$ and if the bisection method is performed to find the root then:*

- *The guaranteed upper bound on the error between the actual root and the approximate root decreases by a factor of 2 at every iteration.*
- *If we want the approximate root found by the bisection method to be within a tolerance of ε then it is enough to choose n such that*

$$\frac{|b - a|}{2^n} \leq \varepsilon \quad (3.3)$$

where n is the number of iterations that it takes to achieve that tolerance.

3. Roots 1

Solving for the number n of iterations we get

$$n = \log_2 \left(\frac{|b - a|}{\varepsilon} \right). \quad (3.4)$$

Rounding the value of n up to the next integer gives the number of iterations necessary to approximate the root to a precision less than ε .

Exercise 3.14. Apply what you have learned in Theorem 3.2 to determine how many iterations the bisection method will take to approximate the solution of the equation

$$3 \sin(x) + 9 = x^2 + \cos(x)$$

on $x \in [1, 5]$ to within a tolerance of 10^{-6} .

Now create a second version of your Python bisection method function that uses a **for** loop that takes the exact number of steps required to guarantee that the approximation to the root lies within a requested tolerance. This should be in contrast to your first version which likely used a **while** loop to decide when to stop. Is there an advantage to using one of these versions of the bisection method over the other?

The final type of analysis that we should do on the bisection method is to make plots of the error between the approximate solution that the bisection method gives you and the exact solution to the equation. This is a bit of a funny thing! Stop and think about this for a second: *if you know the exact solution to the equation then why are you solving it numerically in the first place!?!?* However, whenever you build an algorithm you need to test it on problems where you actually do know the answer so that you can be somewhat sure that it is not giving you nonsense. Furthermore, analysis like this tells us how fast the algorithm is expected to perform.

From Theorem 3.2 you know that the bisection method cuts the interval in half at every iteration. You proved in Exercise 3.13 that the guaranteed upper bound on the error is therefore cut in half at every iteration as well. The following example demonstrates this theorem graphically.

Example 3.1. Let us solve the very simple equation $x^2 - 2 = 0$ for x to get the solution $x = \sqrt{2}$ with the bisection method. Since we know the exact answer we can compare the exact answer to the value of the midpoint given at each iteration and calculate an absolute error:

$$\text{Absolute Error} = |\text{Approximate Solution} - \text{Exact Solution}|. \quad (3.5)$$

Let us write a Python function that implements the bisection method and collects the absolute errors at each iteration into a list.

We can now use this function to see the absolute error at each iteration when solving the equation $x^2 - 2 = 0$ with the bisection method.

```
[np.float64(0.08578643762690485),
 np.float64(0.16421356237309515),
 np.float64(0.039213562373095145),
 np.float64(0.023286437626904855),
 np.float64(0.007963562373095145),
 np.float64(0.0076614376269048545),
 np.float64(0.00015106237309514547),
 np.float64(0.0037551876269048545),
 np.float64(0.0018020626269048545),
 np.float64(0.0008255001269048545),
 np.float64(0.0003372188769048545),
 np.float64(9.307825190485453e-05),
 np.float64(2.8992060595145475e-05),
 np.float64(3.2043095654854525e-05),
 np.float64(1.5255175298545254e-06),
 np.float64(1.3733271532645475e-05),
 np.float64(6.103877001395475e-06),
 np.float64(2.2891797357704746e-06),
 np.float64(3.818311029579746e-07),
 np.float64(5.718432134482754e-07),
 np.float64(9.500605524515038e-08),
 np.float64(1.4341252385641212e-07),
 np.float64(2.4203234305630872e-08)]
```

Next we write a function to plot the absolute error on the vertical axis and the iteration number on the horizontal axis. We get Figure 3.2. As expected, the absolute error is controlled by an exponentially decreasing upper bound. Notice that it is not a completely smooth curve since we will have some jumps in the accuracy just due to the fact that sometimes the root will be near the midpoint of the interval and sometimes it will not be.

Without Theorem 3.2 it would be rather hard to tell what the exact behaviour is in the exponential plot above. We know from Theorem 3.2 that the error bound divides by 2 at

3. Roots 1

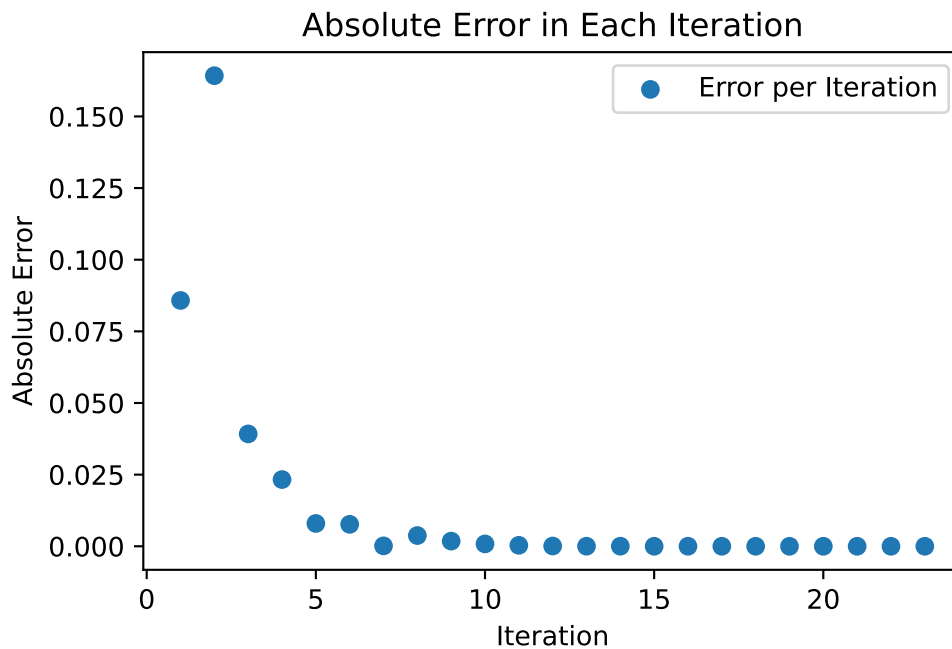


Figure 3.2.: The evolution of the absolute error when solving the equation $x^2 - 2 = 0$ with the bisection method.

every step, so if we instead plot the base-2 logarithm of the absolute error against the iteration number we should see a broadly linear trend as shown in Figure 3.3.

slope = -0.9888817473824996, intercept = -1.621939046956721.

There will be times later in this course where we will not have a nice theorem like Theorem 3.2 and instead we will need to deduce the relationship from plots like these.

1. The trend is linear since logarithms and exponential functions are inverses. Hence, applying a logarithm to an exponential will give a linear function.
2. The fitted slope should be close to -1 in this case since the error bound is divided by a factor of 2 each iteration. You can visually verify that the plot above follows this trend (the red fitted line in the plot is shown to help you see the slope).

Exercise 3.15. Carefully read and understand all of the details of the previous example and plots. Then create plots similar to that example to solve the equation

$$e^{x-3} + \sqrt{x+6} = 4$$

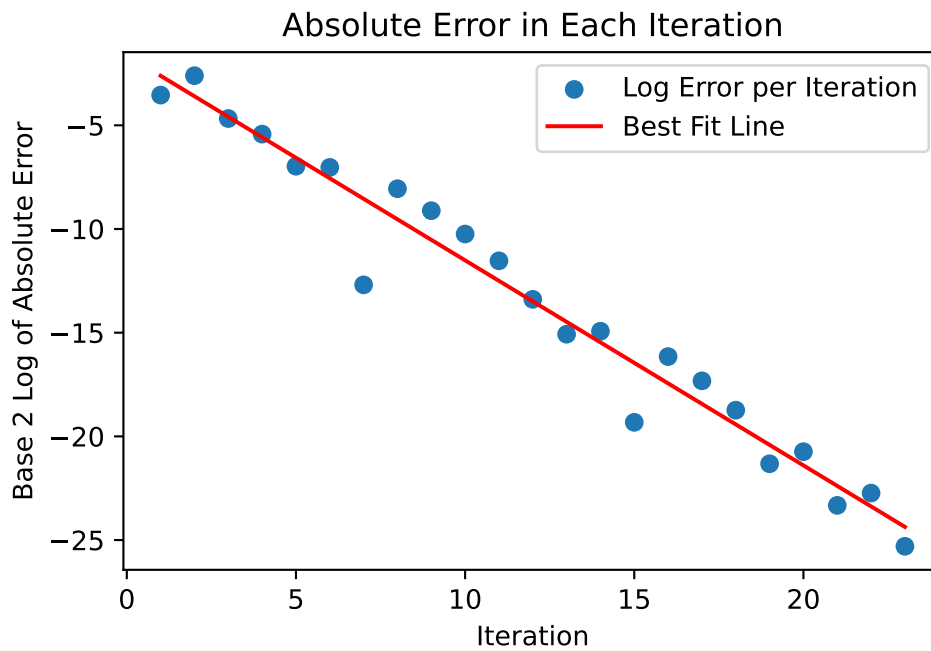


Figure 3.3.: Iteration number vs the base-2 logarithm of the absolute error. The fitted slope is close to -1 , reflecting the bisection error bound.

on the interval $x \in [0, 5]$, for which the exact solution is $x = 3$. You should see the same basic behaviour based on the theorem that you proved in Exercise 3.13. If you do not see the same basic behaviour then something has gone wrong. Also, the plot will look much more regular than in the previous example. Can you explain why?

Example 3.2. Another plot that numerical analysts use quite frequently for determining how an algorithm is behaving as it progresses is shown in Figure 3.4 and is defined by the following axes:

- The horizontal axis is the absolute error at iteration n .
- The vertical axis is the absolute error at iteration $n + 1$.

This type of plot takes a bit of explaining the first time you see it. Each point in the plot corresponds to an iteration of the algorithm. The x-coordinate of each point is the base-2 logarithm of the absolute error at step n and the y-coordinate is the base-2 logarithm of the absolute error at step $n + 1$. The initial iterations are on the right-hand side of the plot where the error is the largest (this will be where the algorithm starts). As the iterations progress and the error decreases the points move to the left-hand side of the plot. Examining the slope of the trend line in this plot shows how we expect the error

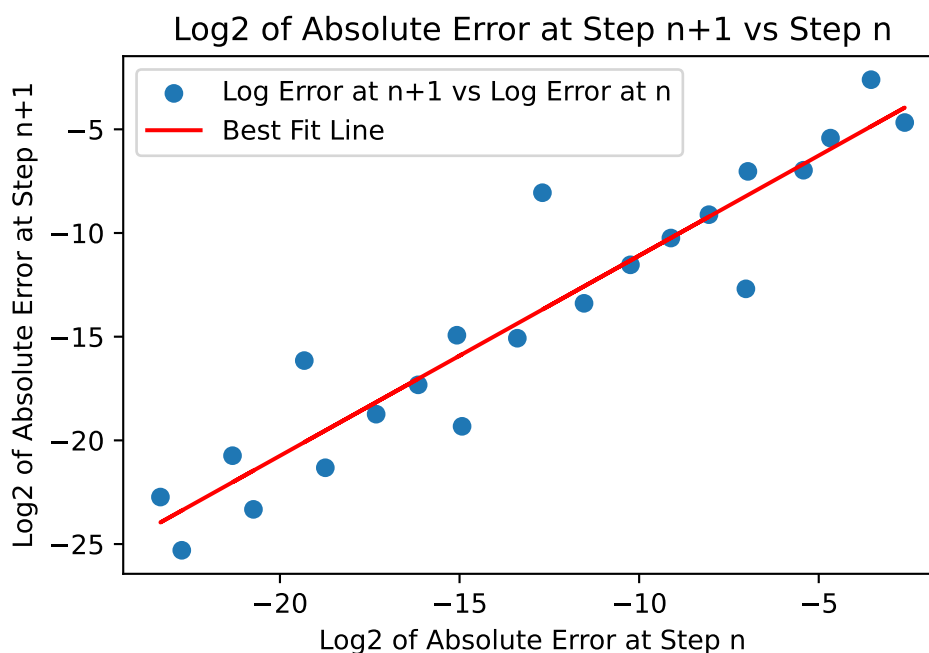


Figure 3.4.: The base-2 logarithm of the absolute error at iteration n vs the base-2 logarithm of the absolute error at iteration $n + 1$.

to progress from step to step. The slope appears to be close to 1 in Figure 3.4 and the intercept appears to be close to -1 . In this case we used a base-2 logarithm for each axis, so the fitted trend suggests that

$$\log_2(\text{absolute error at step } n + 1) \approx 1 \cdot \log_2(\text{absolute error at step } n) - 1. \quad (3.6)$$

Exponentiating both sides gives

$$(\text{absolute error at step } n + 1) \approx \frac{1}{2}(\text{absolute error at step } n). \quad (3.7)$$

This is consistent with the error bound proved for the bisection method. Pause and ponder this result for a second: we just empirically checked the convergence behaviour by examining Figure 3.4. That's what makes these types of plots so useful!

Exercise 3.16. Reproduce plots like that in the previous example but for the equation that you used in Exercise 3.15. Again check that the plots have the expected shape.

3.3. Fixed Point Iteration

We will now investigate a different problem that is closely related to root finding: the fixed-point problem. Given a function g (of one real argument with real values), we look for a number p such that

$$g(p) = p.$$

This p is called a **fixed point** of g .

Any root-finding problem $f(x) = 0$ can be reformulated as a fixed-point problem, and this can be done in many (in fact, infinitely many) ways. For example, given f , we can define $g(x) := f(x) + x$; then

$$f(x) = 0 \quad \Leftrightarrow \quad g(x) = x.$$

Just as well, we could set $g(x) := \lambda f(x) + x$ with any $\lambda \in \mathbb{R} \setminus \{0\}$, and there are many other possibilities.

The heuristic idea for approximating a fixed point of a function g is quite simple. We take an initial approximation x_0 and calculate subsequent approximations using the formula

$$x_n := g(x_{n-1}).$$

A graphical representation of this sequence when $g = \cos$ and $x_0 = 0.2$ is shown in Figure 3.5.

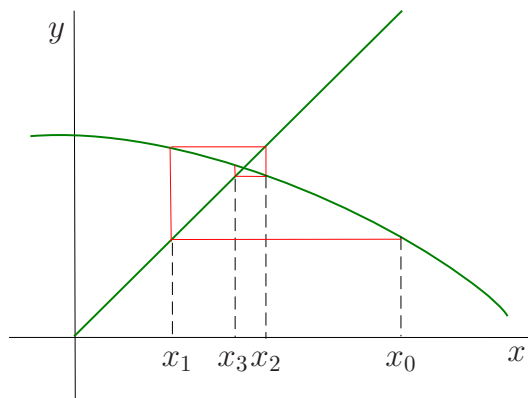


Figure 3.5.: Fixed point iteration

Exercise 3.17. The plot that emerges in Figure 3.5 is known as a cobweb diagram, for obvious reasons. Explain to others in your group what is happening in the animation in Figure 3.5 and how that animation is related to the fixed point iteration $x_n = \cos(x_{n-1})$.

3. Roots 1

Exercise 3.18. The animation in Figure 3.5 is a graphical representation of the fixed point iteration $x_n = \cos(x_{n-1})$. Use Python to calculate the first 10 iterations of this sequence with $x_0 = 0.2$. Use that to get an estimate of the solution to the equation $\cos(x) - x = 0$.

Why is the sequence (x_n) expected to approximate a fixed point? Suppose for a moment that the sequence (x_n) converges to some number p , and that g is continuous. Then

$$p = \lim_{n \rightarrow \infty} x_n = \lim_{n \rightarrow \infty} g(x_{n-1}) = g\left(\lim_{n \rightarrow \infty} x_{n-1}\right) = g(p). \quad (3.8)$$

Thus, *if* the sequence converges, then it converges to a fixed point. However, this resolves the problem only partially. One would like to know:

- Under what conditions does the sequence (x_n) converge?
- How fast is the convergence, i.e., can one obtain an estimate for the approximation error?

So there is much for you to investigate!

Exercise 3.19. Copy the two plots in Figure 3.6 to a piece of paper and draw the first few iterations of the fixed point iteration $x_n = g(x_{n-1})$. In the first plot, try the starting values $x_0 = 0.2$ and $x_0 = 1.5$. In the second plot, start with $x_0 = 0.9$. What do you observe about the convergence of the sequence in each case?

Can you make some conjectures about when the sequence (x_n) will converge to a fixed point and when it will not?

Exercise 3.20. Make similar plots as in the previous exercise but with different slopes of the blue line. Can you make some conjectures about how the speed of convergence is related to the slope of the blue line?

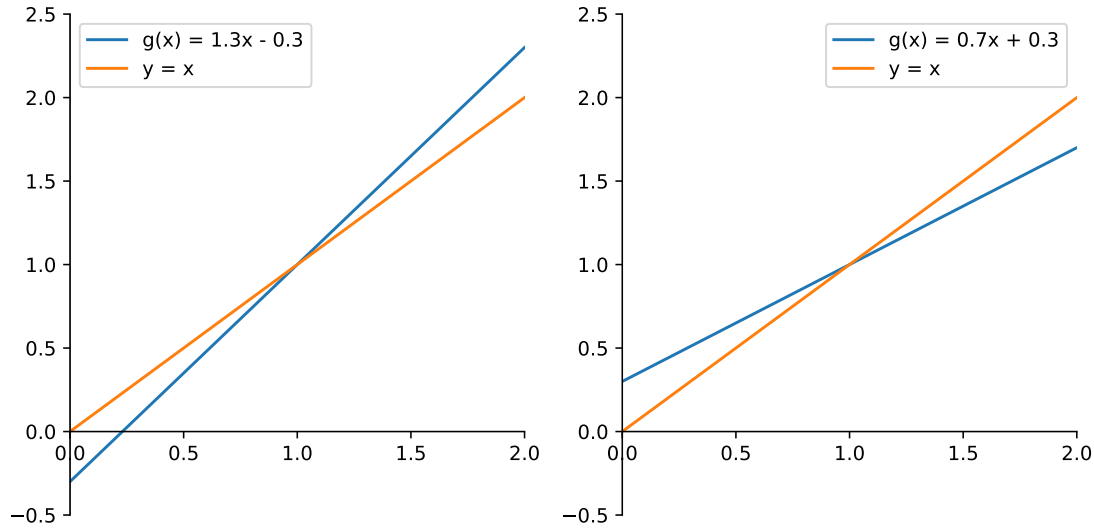


Figure 3.6.: Two plots for practicing your cobweb skills.

Now see if your observations are in agreement with the following theorem:

Theorem 3.3 (Fixed Point Theorem). *Suppose that $g : [a, b] \rightarrow [a, b]$ is differentiable, and that there exists $0 < k < 1$ such that*

$$|g'(x)| \leq k \quad \text{for all } x \in (a, b). \quad (3.9)$$

Then, g has a unique fixed point $p \in [a, b]$; and for any choice of $x_0 \in [a, b]$, the sequence defined by

$$x_n := g(x_{n-1}) \quad \text{for all } n \geq 1 \quad (3.10)$$

converges to p . The following estimate holds:

$$|p - x_n| \leq k^n |p - x_0| \quad \text{for all } n \geq 1. \quad (3.11)$$

Proof. The proof of this theorem is not difficult, but you can skip it and go directly to Exercise 3.21 if you feel that the theorem makes intuitive sense and you are not interested in proofs.

We first show that g has a fixed point p in $[a, b]$. If $g(a) = a$ or $g(b) = b$ then g has a fixed point at an endpoint. If not, then it must be true that $g(a) > a$ and $g(b) < b$. This means that the function $h(x) := g(x) - x$ satisfies

$$h(a) = g(a) - a > 0, \quad h(b) = g(b) - b < 0$$

and since h is continuous on $[a, b]$ the Intermediate Value Theorem guarantees the existence of $p \in (a, b)$ for which $h(p) = 0$, equivalently $g(p) = p$, so that p is a fixed point of g .

3. Roots 1

To show that the fixed point is unique, suppose that $q \neq p$ is a fixed point of g in $[a, b]$. The Mean Value Theorem implies that there is a number ξ in the interval $(\min\{p, q\}, \max\{p, q\}) \subset (a, b)$ such that

$$\frac{g(p) - g(q)}{p - q} = g'(\xi).$$

Then

$$|p - q| = |g(p) - g(q)| = |(p - q)g'(\xi)| = |p - q||g'(\xi)| \leq k|p - q| < |p - q|,$$

where the inequalities follow from Eq. 3.9. This is a contradiction, which must have come from the assumption $p \neq q$. Thus $p = q$ and the fixed point is unique.

Since g maps $[a, b]$ into itself, the sequence $\{x_n\}$ is well defined. For each $n \geq 0$ the Mean Value Theorem gives a ξ in the interval $(\min\{x_n, p\}, \max\{x_n, p\}) \subset (a, b)$ such that

$$\frac{g(x_n) - g(p)}{x_n - p} = g'(\xi).$$

Thus for each $n \geq 1$ by Eq. 3.9, Eq. 3.10

$$|x_n - p| = |g(x_{n-1}) - g(p)| = |(x_{n-1} - p)g'(\xi)| = |x_{n-1} - p||g'(\xi)| \leq k|x_{n-1} - p|.$$

Applying this inequality inductively, we obtain the error estimate Eq. 3.11. Moreover since $k < 1$ we have

$$\lim_{n \rightarrow \infty} |x_n - p| \leq \lim_{n \rightarrow \infty} k^n |x_0 - p| = 0,$$

which implies that (x_n) converges to p . □

Exercise 3.21. This exercise shows why the conditions of the Theorem 3.3 are important.

The equation

$$f(x) = x^2 - 2 = 0$$

has a unique root $\sqrt{2}$ in $[1, 2]$. There are many ways of writing this equation in the form $x = g(x)$; we consider two of them:

$$x = g(x) = x - (x^2 - 2), \quad x = h(x) = x - \frac{x^2 - 2}{3}.$$

Calculate the first few terms in the sequences generated by the fixed point iteration procedures $x_n = g(x_{n-1})$ and $x_n = h(x_{n-1})$ with start value $x_0 = 1$. Which of these fixed point problems generate a rapidly converging sequence? Calculate the derivatives of g and h and check if the conditions of the fixed point theorem are satisfied.

The previous exercise illustrates that one needs to be careful in rewriting root-finding problems as fixed-point problems—there are many ways to do so, but not all lead to a good approximation. In the next section about Newton’s method we will discover a very good choice.

Note at this point that Theorem 3.3 gives only sufficient conditions for convergence; in practice, convergence might occur even if the conditions are violated.

Exercise 3.22. In this exercise you will write a Python function to implement the fixed point iteration algorithm.

For implementing the fixed point method as a computer algorithm, there’s one more complication to be taken into account: how many steps of the iteration should be taken, i.e., how large should n be chosen, in order to reach the desired precision? The error estimate in Eq. 3.11 is often difficult to use for this purpose because it involves estimates on the derivative of g which may not be known.

Instead, one uses a different **stopping condition** for the algorithm. Since the sequence is expected to converge rapidly, one uses the difference $|x_n - x_{n-1}|$ to measure the precision reached. If this difference is below a specified limit, say τ , the iteration is stopped. Since it is possible that the iteration does *not* converge—see the example above—one would also stop the iteration (with an error message) if a certain number of steps is exceeded, in order to avoid infinite loops. In pseudocode the fixed point iteration algorithm is then implemented as follows:

Fixed point iteration

```

1 : function FixedPoint( $g, x_0, tol, N$ )            $\#$  function  $g$ , start point  $x_0$ ,
2 :    $x \leftarrow x_0$ ;                            $\#$  tolerance  $tol$ ,
3 :   for  $i \leftarrow 1$  to  $N$                       $\#$  max. num. of iterations  $N$ 
4 :      $x_{new} \leftarrow g(x)$ 
5 :     if  $|x_{new} - x| < tol$  then                $\#$  Desired tolerance reached
6 :       return  $x_{new}$ 
7 :     end if
8 :      $x \leftarrow x_{new}$ 
9 :   end for
10 :   exception(Iteration has not converged)
11 : end function

```

3. Roots 1

The following is an outline of a Python function to implement the fixed point iteration algorithm with “???” indicating where you need to fill in the code. You may want to disable Gemini in your Colab notebook before pasting the code snippet below to avoid Gemini trying to fill in the “???” for you.

```
def fixed_point_iteration(g, x0, tol, N):
    """
    Implements the fixed point iteration algorithm.

    Parameters:
        g (function): The function for which to find the fixed point
        x0 (float): The initial guess for the fixed point
        tol (float): The tolerance for convergence
        N (int): The maximum number of iterations

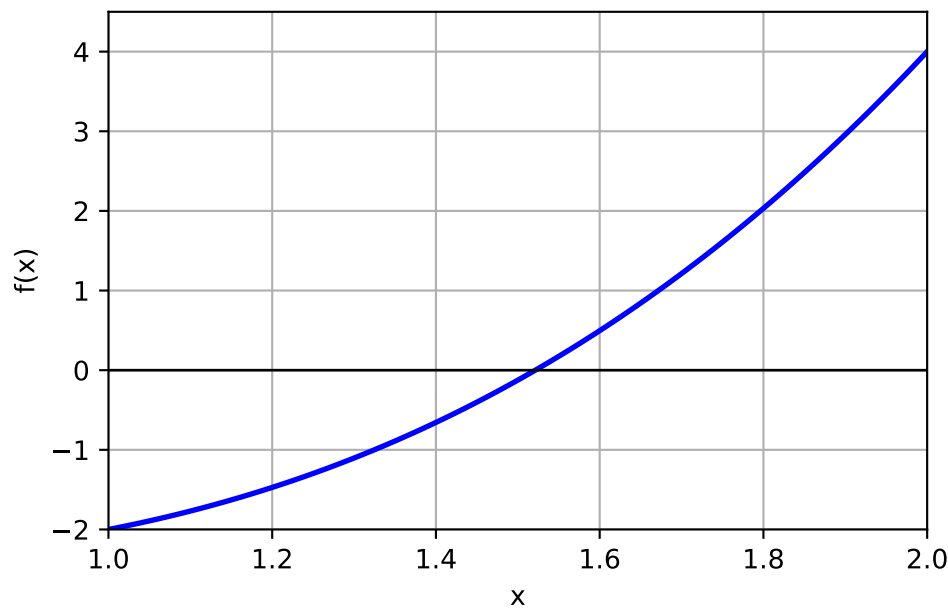
    Returns:
        float: The estimated fixed point if convergence is achieved within N iterations
    """
    x = x0
    for i in range(1, ???):
        x_new = ???
        if ???:
            return ???
        ???
    raise Exception("Iteration has not converged")
```

Fill in the missing parts of the code. Use the function to approximate the fixed point of the function $g(x) = \cos(x)$ with start value $x_0 = 2$ and tolerance $tol = 10^{-8}$.

3.4. Exam-Style Question

- (a) What are the conditions on a function f that guarantee that the bisection method on an interval $[a, b]$ converges to a root of f ? [2 marks]
- (b) How many iterations of the bisection method starting with the interval $[1, 2]$ would be required to guarantee that the approximation to the root is no further than a distance of $\varepsilon = 2^{-20}$ from the true root? [1 mark]

Consider the function $f(x) = x^3 - x - 2$. We want to find a root of this function in the interval $[1, 2]$. To help you, the graph of $f(x)$ on this interval is provided below.

Figure 3.7.: Graph of $f(x) = x^3 - x - 2$.

- (c) Using the graph to roughly estimate function values, carry out the first three iterations of the bisection method. [3 marks]
- (d) The equation $x^3 - x - 2 = 0$ can be rewritten as a fixed-point problem $x = g(x)$ in several ways. Consider the rearrangement $g(x) = \sqrt[3]{x+2}$. We have $g(1) \approx 1.44$ and $g(2) \approx 1.58$.
- Calculate the derivative $g'(x)$. Use it to show whether the fixed-point iteration $x_n = g(x_{n-1})$ is guaranteed to converge for any starting value $x_0 \in [1, 2]$. [3 marks]
 - Complete the following Python code to perform 10 iterations of the fixed-point iteration $x_n = g(x_{n-1})$ starting with $x_0 = 1.0$. [3 marks]

```
def fixed_point_iteration():
    x = 1.0
    for i in range(...):
        x = ...
    return x
```

Further reading: Section 2.2 of (Burden and Faires 2010).

4. Roots 2

This week we will continue our study of root-finding methods. We will start with Newton's method, then move on to the secant method and finally discuss the theory behind the order of convergence of these methods.

4.1. Newton's Method

In the bisection method (Section 3.2) we used only the sign of the function at the guessed points. We will now investigate how we can also use the value and the slope (derivative) of the function to get a much improved method.

4.1.1. Intuition

Root finding is really the process of finding the x -intercept of the function. If the function is complicated (e.g. highly nonlinear or does not lend itself to traditional by-hand techniques) then we can approximate the x -intercept by creating a Taylor series approximation of the function at a nearby point and then finding the x -intercept of that simpler Taylor series. The simplest non-trivial Taylor series is a linear function: a tangent line.

Exercise 4.1. A tangent line approximation to a function $f(x)$ near a point x_0 is

$$y = f(x_0) + f'(x_0)(x - x_0).$$

Set y to zero and solve for x to find the x -intercept of the tangent line.

x -intercept of tangent line is $x =$ _____

The idea of approximating the function by its tangent line gives us an algorithm for approximating the root of a function:

4. Roots 2

- Given a value of x that is a decent approximation of the root, draw a tangent line to $f(x)$ at that point.
- Find where the tangent line intersects the x -axis.
- Use this intersection as the new x value and repeat.

The first step has been shown for you in Figure 4.1. The tangent line to the function $f(x)$ at the point $(x_0, f(x_0))$ is shown in red. The x -intercept of the tangent line is the new x value, x_1 . The process is then repeated with x_1 as the new x_0 and so on.

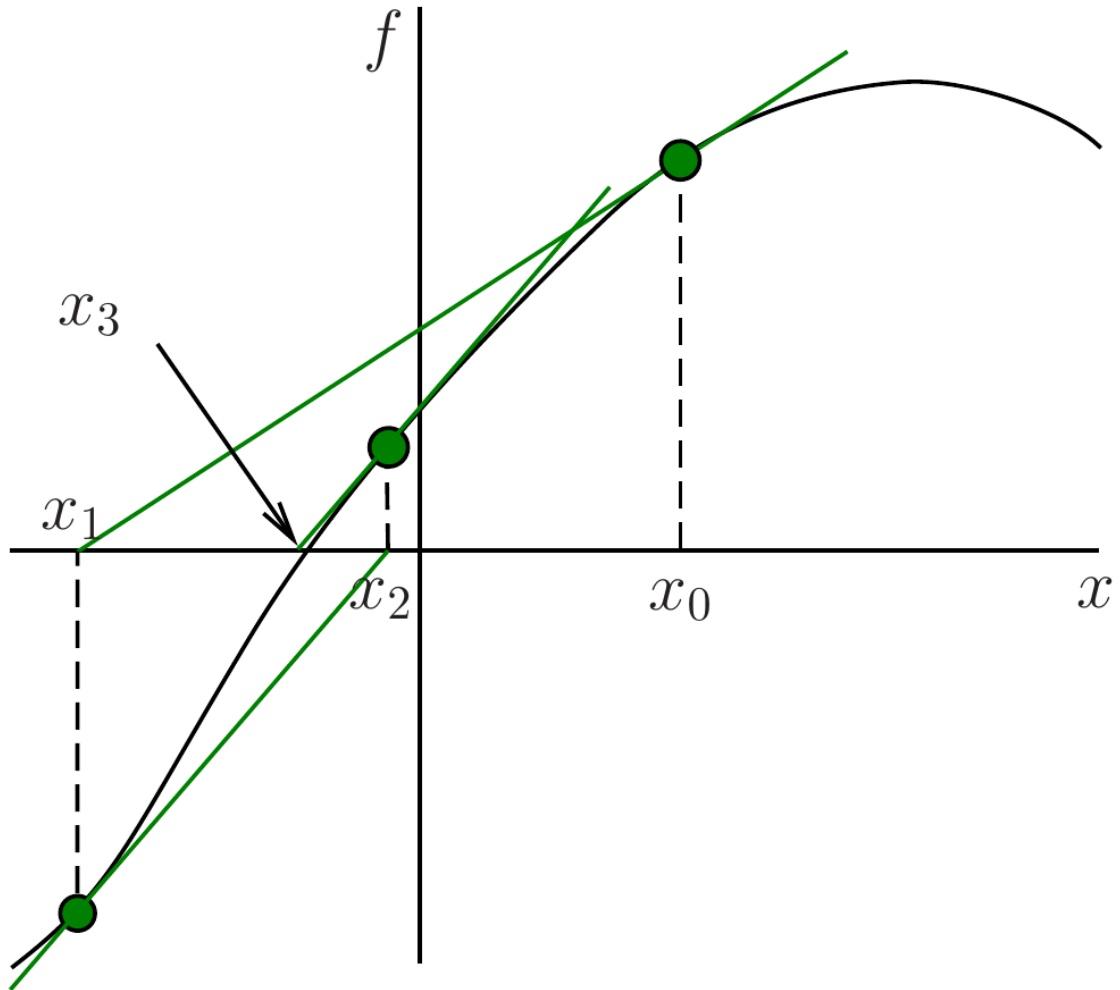


Figure 4.1.: Using successive tangent line approximations to find the root of a function

The graphical method illustrated in Figure 4.1 was introduced by Sir Isaac Newton and is therefore known as Newton's method. Joseph Raphson then gave the algebraic formulation and popularised the method, which is therefore also known as the Newton-Raphson method.

Exercise 4.2. If we had started not at x_0 in Figure 4.1 but instead at the very end of the x -axis in that figure, what would have happened? Would this initial guess have worked to eventually approximate the root?

Exercise 4.3. Sketch some other function $f(x)$ with a root and choose an initial point x_0 and graphically perform the Newton iteration a few times, similar to Figure 4.1. Does the algorithm appear to converge to the root in your example? Do you think that this will generally take more or fewer steps than the bisection method?

Exercise 4.4. Consider the function $f(x) = \sin(x) + 1$. It has roots at $x = 2n\pi + 3\pi/2$ for $n \in \mathbb{Z}$. However, at the roots we have $f'(x) = 0$. Make yourself a sketch to see why. What will happen when you apply Newton's method with a starting value of $x_0 = \pi$? You should be able to answer this just by looking at the sketch.

Exercise 4.5. Using your result from Exercise 4.1, write the formula for the x -intercept of the tangent line to $f(x)$ at the point $(x_n, f(x_n))$. This is the formula for the next guess x_{n+1} in Newton's method. Newton's method is a fixed-point iteration method of the form $x_{n+1} = g(x_n)$ with

$$g(x) = \dots$$

Fill in the blank in the above formula.

Exercise 4.6. Apply Newton's method to find the root of the function $f(x) = x^2 - 2$ with an initial guess of $x_0 = 1$. Calculate the first two iterations of the sequence by hand (you do not need a calculator or computer for this). Use a calculator or computer to calculate the next two iterations and fill in the following table:

n	x_n	$f(x_n)$	$f'(x_n)$
0	$x_0 = 1$	$f(x_0) = -1$	$f'(x_0) = 2$
1	$x_1 = 1 - \frac{-1}{2} = \frac{3}{2}$	$f(x_1) =$	$f'(x_1) =$

4. Roots 2

n	x_n	$f(x_n)$	$f'(x_n)$
2	$x_2 =$	$f(x_2) =$	$f'(x_2) =$
3	$x_3 =$	$f(x_3) =$	$f'(x_3) =$
4	$x_4 =$		

4.1.2. Implementation

Exercise 4.7. The following is an outline of a Python function called `newton()` for Newton's method. Complete the function definition by copying your code from your `fixed_point_iteration()` function and modifying it to implement Newton's method.

```
def newton(f, fprime, x0, tol=1e-10, N=30):
    """
    Find a root of f(x) using Newton's method.

    Parameters:
        f (function): Function whose root we want to find
        fprime (function): Derivative of f
        x0 (float): Initial guess for the root
        tol (float, optional): Error tolerance. Defaults to 1e-10
        N (int, optional): Maximum number of iterations. Defaults to 30.

    Returns:
        float: Approximate root of f(x) if convergence is achieved within N iterations
    """

    # Copy your code from your `fixed_point_iteration()` function and modify it to
```

Exercise 4.8. Use your implementation from Exercise 4.7 to approximate the root of the function $f(x) = x^2 - 2$ with an initial guess of $x_0 = 1$. Use a tolerance of 10^{-10} .

4.1.3. Failures

There are several ways in which Newton's method will behave unexpectedly, or downright fail. Some of these issues can be foreseen by examining the Newton iteration formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Some of the failures that we will see are a little more surprising.

Exercise 4.9. One of the failures of Newton's method is that it requires a division by $f'(x_n)$. If $f'(x_n)$ is zero then the algorithm completely fails. Go back to your Python function and put an `if` statement in the function that catches instances when Newton's method fails in this way.

Exercise 4.10. An interesting failure can occur with Newton's method that you might not initially expect. Consider the function $f(x) = x^3 - 2x + 2$. This function has a root near $x = -1.77$. Fill in the table below with pen and paper. You really do not need a computer for this. Then draw the tangent lines into Figure 4.2 for approximating the solution to $f(x) = 0$ with a starting point of $x = 0$.

n	x_n	$f(x_n)$	$f'(x_n)$
0	$x_0 = 0$	$f(x_0) = 2$	$f'(x_0) = -2$
1	$x_1 = 0 - \frac{f(x_0)}{f'(x_0)} = 1$	$f(x_1) = 1$	$f'(x_1) = 1$
2	$x_2 = 1 - \frac{f(x_1)}{f'(x_1)} =$	$f(x_2) =$	$f'(x_2) =$
3	$x_3 =$	$f(x_3) =$	$f'(x_3) =$
4	$x_4 =$	$f(x_4) =$	$f'(x_4) =$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Exercise 4.11. Repeat the previous exercise with the function $f(x) = x^3 - 5x$ with the starting point $x_0 = -1$. Again this is easy to do with pen and paper.

4. Roots 2

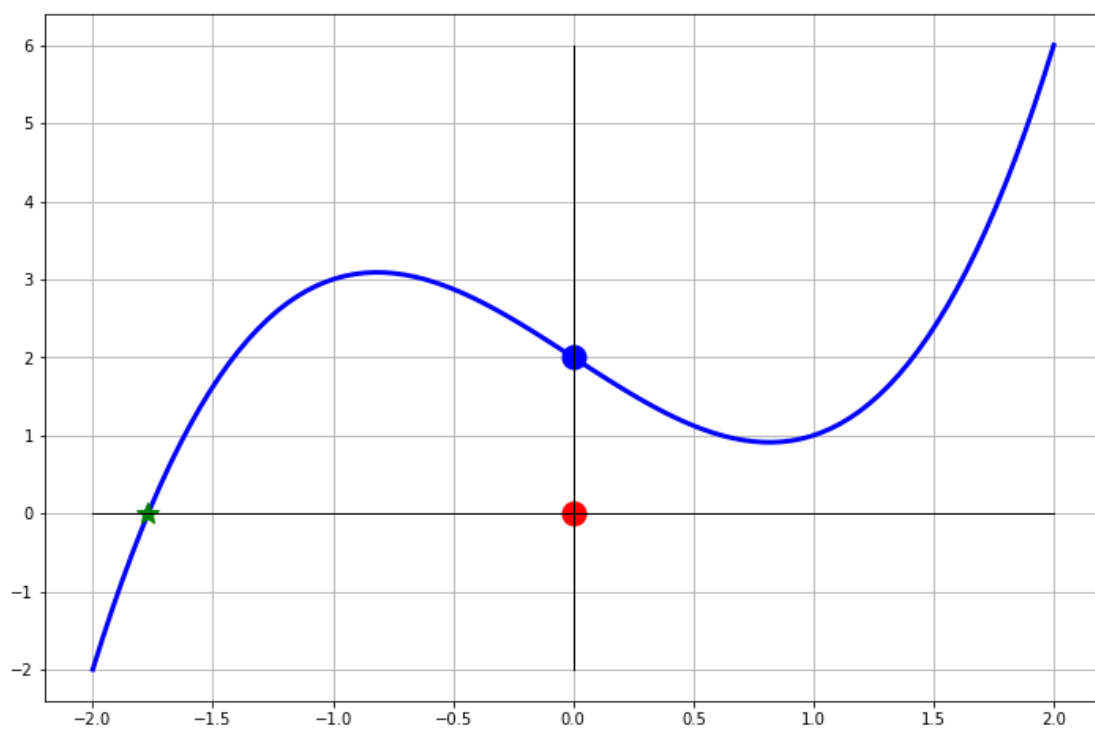


Figure 4.2.: An interesting Newton's method failure when $f(x) = x^3 - 2x + 2$.

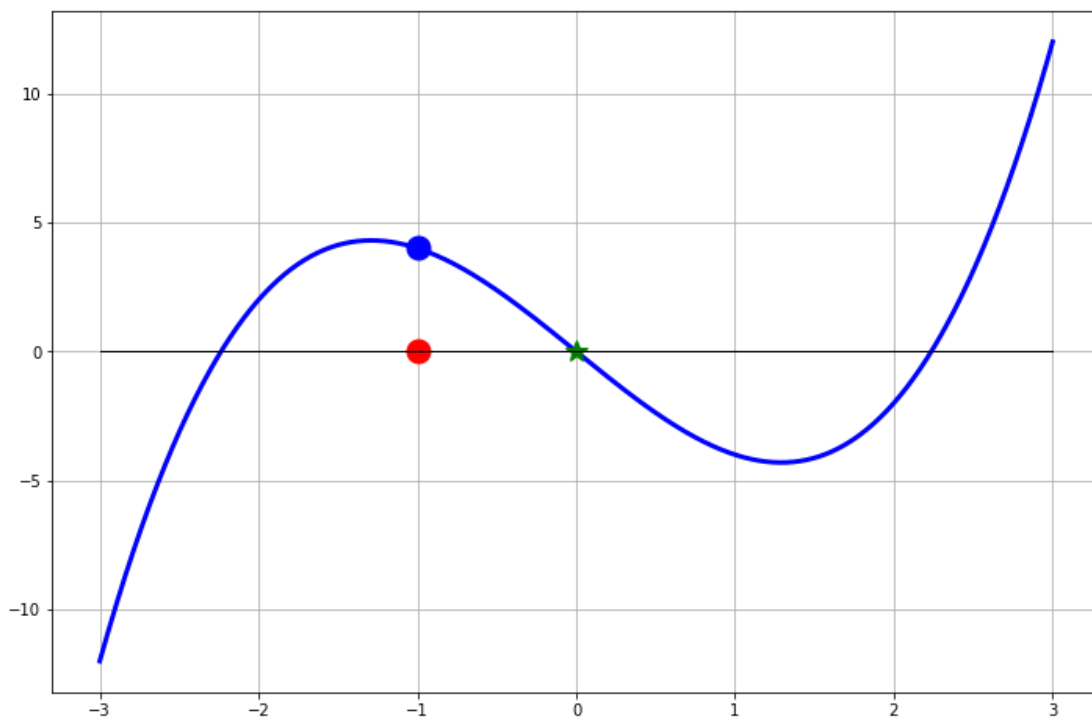


Figure 4.3.: Another surprising Newton's method failure when $f(x) = x^3 - 5x$.

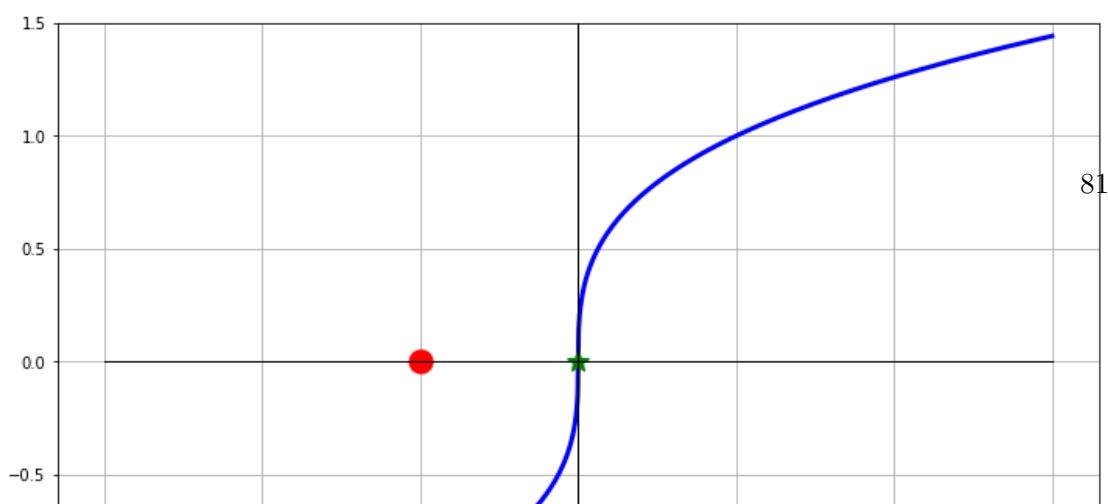
Exercise 4.12. Now let us consider the function $f(x) = \sqrt[3]{x}$. This function has a root $x = 0$. Furthermore, it is differentiable everywhere except at $x = 0$ since

$$f'(x) = \frac{1}{3}x^{-2/3} = \frac{1}{3x^{2/3}}.$$

The point of this exercise is to show what can happen when the point of non-differentiability is precisely the point that you are looking for.

1. Fill in the table of iterations starting at $x = -1$, draw the tangent lines on the plot, and make a general observation of what is happening with the Newton iterations.

n	x_n	$f(x_n)$	$f'(x_n)$
0	$x_0 = -1$	$f(x_0) = -1$	$f'(x_0) =$
1	$x_1 = -1 - \frac{f(-1)}{f'(-1)} =$	$f(x_1) =$	$f'(x_1) =$
2			
3			
4			
$\vdots$	$\vdots$	$\vdots$	$\vdots$



4. Roots 2

2. Now let us look at the Newton iteration in a bit more detail. Since $f(x) = x^{1/3}$ and $f'(x) = \frac{1}{3}x^{-2/3}$ the Newton iteration can be simplified as

$$x_{n+1} = x_n - \frac{x_n^{1/3}}{\left(\frac{1}{3}x_n^{-2/3}\right)} = x_n - 3\frac{x_n^{1/3}}{x_n^{-2/3}} = x_n - 3x_n = -2x_n.$$

What does this tell us about the Newton iterations?

Hint: You should have found the exact same thing in the numerical experiment in part 1.

3. Was there anything special about the starting point $x_0 = -1$? Will this problem exist for every starting point?

Exercise 4.13. You have now seen several reasons why Newton's method could fail. Work with your group to come up with a list of reasons. Support each of your reasons with a sketch or an example.

4.1.4. Rate of Convergence

In this section we will look at the convergence rate of Newton's method and we will show that we can greatly outperform the bisection method.

We saw in Example 3.2 how we could empirically determine the rate of convergence of the bisection method by plotting the error in the new iterate on the y -axis and the error in the old iterate on the x -axis of a log-log plot. Take a look at that example again and make sure you understand the code and the explanation of the plot. Then, to investigate the rate of convergence of Newton's method you can do the same thing.

Exercise 4.14. Write a Python function `newton_with_error_tracking()` that returns a list of absolute errors between the iterates and the exact solution. You can start with your code from `newton()` from Exercise 4.7 and add the collection of the errors in a list as in the `bisection_with_error_tracking()` function that we wrote in Example 3.1. Your new function should accept a Python function for $f(x)$, a Python function for $f'(x)$, the exact root, an initial guess, an optional error tolerance and an optional maximum number of iterations.

Then, use this function to list the error progression for Newton's method for finding the root of $f(x) = x^2 - 2$ with the initial guess $x_0 = 1$ and tolerance 10^{-10} . You should find that the error goes down extremely quickly and that for the final iteration Python can no longer detect a difference between the new iterate and its own value for $\sqrt{2}$.

Exercise 4.15.

1. Plot the errors you calculated in Exercise 4.14 using the `plot_error_progression()` function that we wrote in Example 3.2. Note that you will first need to remove the zeros from the error list, otherwise you will get an error. You can remind yourself of how to work with lists in Section A.2.2.
2. Give a thorough explanation for how to interpret the plot that you just made.

Exercise 4.16. You will have observed that the points in your log-log plot lie approximately on a straight line, i.e.,

$$\log(\epsilon_{n+1}) \approx m \log(\epsilon_n) + b$$

Extract the slope m and intercept b of the line that you fitted to the log-log plot of the error progression using the `np.polyfit` function.

Convert this into a statement about the ratio between ϵ_{n+1} and ϵ_n ,

$$\epsilon_{n+1} \approx ??? (\epsilon_n)^{??}$$

Exercise 4.17. Reproduce plots like those in the previous example but for the function

$$f(x) = e^{x-3} + \sqrt{x+6} - 4$$

that you used in Exercise 3.15 with the initial guess $x_0 = 1$ and tolerance 10^{-10} . Again check that the plots have the expected shape. What is the approximate relationship between ϵ_{n+1} and ϵ_n in this case?

4.2. The Secant Method

4.2.1. Intuition and Implementation

Newton's method has second-order (quadratic) convergence and, as such, will perform faster than the bisection method. However, Newton's method requires that you have a function and a derivative of that function. The conundrum here is that sometimes the derivative is cumbersome or impossible to obtain but you still want to have the great quadratic convergence exhibited by Newton's method.

4. Roots 2

Recall that Newton's method is

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

If we replace $f'(x_n)$ with an approximation of the derivative then we may have a method that is *close* to Newton's method in terms of convergence rate but is less troublesome to compute. Any method that replaces the derivative in Newton's method with an approximation is called a **quasi-Newton method**.

The first, and most obvious, way to approximate the derivative is just to use the slope of a secant line instead of the slope of the tangent line in the Newton iteration. If we choose two starting points that are quite close to each other then the slope of the secant line through those points will be approximately the same as the slope of the tangent line.

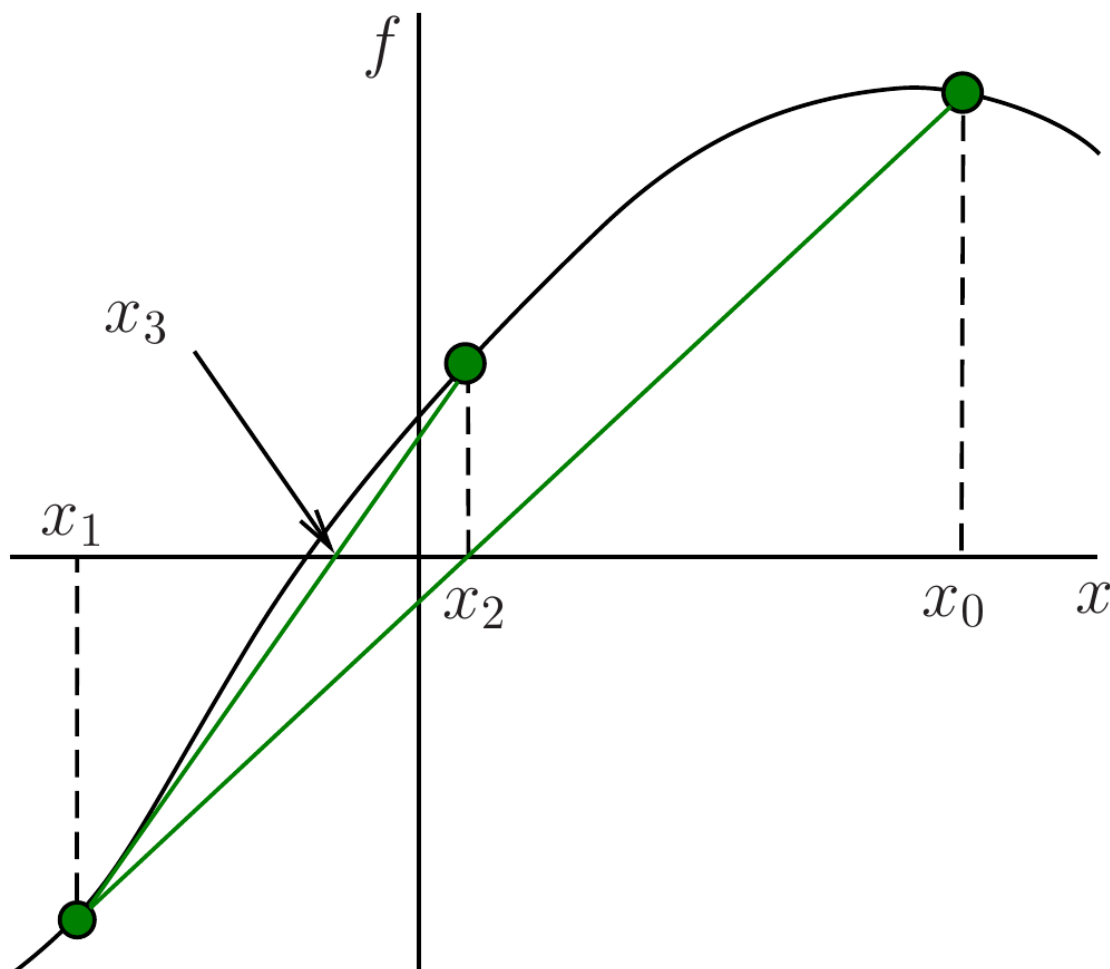


Figure 4.5.: Using successive secant line approximations to find the root of a function

Exercise 4.18. Use the backward difference

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

to approximate the derivative of f at x_n . Discuss why this approximates the derivative. Use this approximation of $f'(x_n)$ in the expression for x_{n+1} of Newton's method. Show that the result simplifies to

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

Exercise 4.19. Notice that the iteration formula for x_{n+1} that you derived depends on both x_n and x_{n-1} . So to start the iteration you need to choose two points x_0 and x_1 before you can calculate $x_2, x_3, \dots$. Draw several pictures showing what the secant method does pictorially. Discuss whether it is important to choose these starting points close to the root and close to each other.

Exercise 4.20. Write a Python function for solving equations of the form $f(x) = 0$ with the secant method. Your function should accept a Python function, two starting points, and an optional error tolerance. Also write a test script that clearly shows that your code is working.

4.2.2. Analysis

Up to this point we have done analysis work on the bisection method and Newton's method. We have found that the methods are first order and second order respectively. We end this chapter by doing the same for the secant method.

Exercise 4.21. Write a function `secant_with_error_tracking()` that returns a list of absolute errors between the iterates and the exact solution. You can start with your code from `secant()` from Exercise 4.20 and add the collection of the errors in a list as in the `bisection_with_error_tracking()` function from Example 3.1. Your new function

4. Roots 2

should accept a Python function, the exact root, two starting points, and an optional error tolerance.

Use this function to list the error progression for the secant method for finding the root of $f(x) = x^2 - 2$ with the starting points $x_0 = 1$ and $x_1 = 3$. Use a tolerance of 10^{-10} . Use the `plot_error_progression()` function that we wrote in Example 3.2 to plot the error progression.

Exercise 4.22. Using the `np.polyfit` function, extract the slope of the line that you had fit to the log-log plot of the error progression in Exercise 4.21. What does this slope tell you about how the error at each iteration is related to the error at the previous iteration?

Exercise 4.23. Make error progression plots for a few different functions and look at the slopes. What do you notice?

4.3. Order of Convergence

You will by now have noticed that Newton's method converges much faster than the bisection method. The secant method also converges faster than the bisection method, though not quite as fast as Newton's method. You will have observed that reflected in the different slopes of the error progression graphs. In this section we summarize your observations theoretically. Thus this section is untypical in that it does not contain further explorations for you to do but instead just consolidates what you have already done. In class this will be presented as a lecture.

4.3.1. Definition

Definition 4.1. Suppose that $x_n \rightarrow p$ as $n \rightarrow \infty$ and $x_n \neq p$ for all n . The sequence $\{x_n\}$ is said to have **order of convergence** $\alpha \geq 1$ if there exists a constant $\lambda > 0$ such that

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n^\alpha} = \lambda. \quad (4.1)$$

Here E_n denotes the absolute error in the n th approximation: $E_n = |x_n - p|$.

4.3. Order of Convergence

If $\alpha = 1, 2, 3, \dots$, the convergence is said to be *linear*, *quadratic*, *cubic*, ..., respectively. Note that if the convergence is linear, then the positive constant λ that appears in the above definition must be smaller than 1 ($0 < \lambda < 1$), because otherwise the sequence will not converge.

A sequence with a higher order of convergence converges much more rapidly than a sequence with a lower order of convergence. To see this, let us consider the following example:

Example 4.1. Let $\{x_n\}$ and $\{y_n\}$ be sequences converging to zero and let, for $n \geq 0$,

$$|x_{n+1}| = k|x_n| \quad \text{and} \quad |y_{n+1}| = k|y_n|^2, \quad (4.2)$$

where $0 < k < 1$. According to the definition, $\{x_n\}$ is linearly convergent and $\{y_n\}$ is quadratically convergent.

Also, we have

$$\begin{aligned} |x_n| &= k|x_{n-1}| = k^2|x_{n-2}| = \dots = k^n|x_0|, \\ |y_n| &= k|y_{n-1}|^2 = k(k|y_{n-2}|^2)^2 = k^3|y_{n-2}|^4 = k^7|y_{n-3}|^8 = \dots = k^{2^n-1}|y_0|^{2^n}. \end{aligned} \quad (4.3)$$

This illustrates that quadratic convergence is much faster than linear convergence.

We have defined the order of convergence of a converging sequence. We will also say that an iterative method has order of convergence α if the sequence of approximations that it produces on a generic problem has order of convergence α .

4.3.2. Fixed Point Iteration

Suppose that $g(x)$ satisfies the conditions of the Fixed Point Theorem on the interval $[a, b]$, so that the sequence $\{x_n\}$ generated by the formula $x_{n+1} = g(x_n)$ with $x_0 \in [a, b]$ converges to a fixed point p . Then, using the Mean Value Theorem, we obtain

$$\begin{aligned} E_{n+1} &= |x_{n+1} - p| = |g(x_n) - g(p)| \\ &= |g'(\xi_n)(x_n - p)| = E_n |g'(\xi_n)|, \end{aligned} \quad (4.4)$$

where ξ_n is a number between x_n and p . This implies that if $x_n \rightarrow p$, then $\xi_n \rightarrow p$ as $n \rightarrow \infty$. Therefore,

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n} = |g'(p)|. \quad (4.5)$$

In general, $g'(p) \neq 0$, so that the fixed-point iteration produces a linearly convergent sequence.

Can the fixed-point iteration produce convergent sequences with convergence of order 2, 3, etc.? It turns out that, under certain conditions, this is possible.

We will prove the following

4. Roots 2

Theorem 4.1. *Let $m > 1$ be an integer, and let $g \in C^m[a, b]$. Suppose that $p \in [a, b]$ is a fixed point of g , and suppose that there exists a point $x_0 \in [a, b]$ such that the sequence generated by the formula $x_{n+1} = g(x_n)$ converges to p . If $g'(p) = \dots = g^{(m-1)}(p) = 0$, then $\{x_n\}$ has the order of convergence m .*

Proof. Expanding $g(x_n)$ in a Taylor series at the point p , we obtain:

$$\begin{aligned} x_{n+1} = g(x_n) &= g(p) + (x_n - p)g'(p) + \dots \\ &\quad + \frac{(x_n - p)^{m-1}}{(m-1)!}g^{(m-1)}(p) \\ &\quad + \frac{(x_n - p)^m}{m!}g^{(m)}(\xi_n) \\ &= p + \frac{(x_n - p)^m}{(m)!}g^{(m)}(\xi_n), \end{aligned} \tag{4.6}$$

where ξ_n is between x_n and p and, therefore, in $[a, b]$ ($x_n \in [a, b]$ at least for sufficiently large n). Then we have

$$\begin{aligned} E_{n+1} = |x_{n+1} - p| &= |g(x_n) - p| = \left| \frac{(x_n - p)^m}{(m)!}g^{(m)}(\xi_n) \right| \\ &= E_n^m \frac{|g^{(m)}(\xi_n)|}{m!}. \end{aligned} \tag{4.7}$$

Therefore (using the fact that $\xi_n \rightarrow p$),

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n^m} = \frac{|g^{(m)}(p)|}{m!}, \tag{4.8}$$

which means that $\{x_n\}$ has convergence of order m . $\square$

4.3.3. Newton's Method

Newton's method for approximating the root p of the equation $f(x) = 0$ is equivalent to the fixed-point iteration $x_{n+1} = g(x_n)$ with

$$g(x) = x - \frac{f(x)}{f'(x)}. \tag{4.9}$$

Suppose that the sequence $\{x_n\}$ converges to p and $f'(p) \neq 0$. We have

$$g'(x) = \frac{f(x)f''(x)}{f'(x)^2} \Rightarrow g'(p) = \frac{f(p)f''(p)}{f'(p)^2} = 0. \tag{4.10}$$

It follows from the above theorem that the order of convergence of Newton's method is at least 2. We also have

$$g''(x) = \frac{f'(x)f''(x) + f(x)f'''(x)}{f'(x)^2} - \frac{2f(x)f''(x)^2}{f'(x)^3}$$

and hence, again using that $f(p) = 0$, we find

$$g''(p) = \frac{f''(p)}{f'(p)} \neq 0 \quad \text{if } f''(p) \neq 0. \quad (4.11)$$

If $f''(p) = 0$, then $g''(p) = 0$ and the order of convergence is at least 3.

4.3.4. Secant Method

The situation with the secant method is more complicated (since it cannot be reduced to a fixed-point iteration) and requires a separate treatment. The result is that the secant method has order of convergence $\alpha = \frac{1+\sqrt{5}}{2} \approx 1.618$.

Note that α is known as the *golden ratio*. If you are intrigued to see the golden ratio appear in this context, you can find a proof below. If you are happy to just accept the result, you can skip the proof.

Suppose that a sequence $\{x_n\}$, generated by the secant method

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}, \quad (4.12)$$

converges to p . Let

$$e_n = x_n - p, \quad (4.13)$$

so that $E_n = |e_n|$, and we assume that $E_n \ll 1$, which is definitely true for sufficiently large n (since the sequence $\{x_n\}$ is converging to p). Subtracting p from both sides of Eq. 4.12, we obtain

$$e_{n+1} = e_n - \frac{f(p + e_n)(e_n - e_{n-1})}{f(p + e_n) - f(p + e_{n-1})}, \quad (4.14)$$

Expanding $f(p + e_n)$ and $f(p + e_{n-1})$ in Taylor series about p and taking into account that $f(p) = 0$, we find that

$$\begin{aligned} f(p + e_n) &= e_n f'(p) + \frac{e_n^2}{2} f''(p) + \dots \\ &= e_n f'(p)(1 + e_n Q) + \dots, \\ f(p + e_{n-1}) &= e_{n-1} f'(p) + \frac{e_{n-1}^2}{2} f''(p) + \dots \\ &= e_{n-1} f'(p)(1 + e_{n-1} Q) + \dots, \end{aligned} \quad (4.15)$$

where

$$Q = \frac{f''(p)}{2f'(p)}. \quad (4.16)$$

4. Roots 2

Substitution of Eq. 4.15 into Eq. 4.14 yields

$$\begin{aligned} e_{n+1} &= e_n - \frac{e_n(e_n - e_{n-1})f'(p)(1 + e_n Q) + \dots}{f'(p)[e_n - e_{n-1} + Q(e_n^2 - e_{n-1}^2) + \dots]} \\ &= e_n \left(1 - \frac{1 + e_n Q + \dots}{1 + Q(e_n + e_{n-1}) + \dots} \right). \end{aligned} \quad (4.17)$$

Since, for small x ,

$$\frac{1}{1 + x + \dots} = 1 - x + \dots, \quad (4.18)$$

we obtain

$$\begin{aligned} e_{n+1} &= e_n (1 - (1 + e_n Q + \dots)(1 - Q(e_n + e_{n-1}) + \dots)) \\ &= Q e_n e_{n-1} + \dots. \end{aligned} \quad (4.19)$$

Thus, for sufficiently large n , we have

$$e_{n+1} \approx Q e_n e_{n-1}. \quad (4.20)$$

Hence,

$$E_{n+1} \approx |Q| E_n E_{n-1}. \quad (4.21)$$

Now we assume that (for all sufficiently large n)

$$E_{n+1} \approx \lambda E_n^\alpha, \quad (4.22)$$

where λ and α are positive constants. Substituting Eq. 4.22 into Eq. 4.21, we find

$$\lambda E_n^\alpha \approx |Q| E_n E_{n-1} \quad \text{or} \quad \lambda E_n^{\alpha-1} \approx |Q| E_{n-1}. \quad (4.23)$$

Applying Eq. 4.22 one more time (with n replaced by $n-1$), we obtain

$$\lambda (\lambda E_{n-1}^\alpha)^{\alpha-1} \approx |Q| E_{n-1} \quad (4.24)$$

or, equivalently,

$$\lambda^\alpha E_{n-1}^{\alpha(\alpha-1)} \approx |Q| E_{n-1}. \quad (4.25)$$

The last equation will be satisfied provided that

$$\lambda^\alpha = |Q|, \quad \alpha(\alpha-1) = 1, \quad (4.26)$$

which requires that

$$\lambda = |Q|^{1/\alpha}, \quad \alpha = (1 + \sqrt{5})/2 \approx 1.62. \quad (4.27)$$

Thus, we have shown that if $\{x_n\}$ is a convergent sequence generated by the secant method, then

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n^\alpha} = |Q|^{1/\alpha}. \quad (4.28)$$

Thus, the secant method has *superlinear* convergence.

Further reading: Section 2.4 of (Burden and Faires 2010).

4.4. Exam-Style Question

- (a) Briefly explain how the secant method modifies Newton's method. Give one advantage and one disadvantage of the secant method compared to Newton's method. [3 marks]
- (b) Define what it means for a sequence $\{x_n\}$ converging to p to have an *order of convergence* $\alpha \geq 1$. State the order of convergence for the bisection method, the secant method, and Newton's method. [4 marks]
- (c) Consider finding the root of $f(x) = x^2 - 5$.
- i) Apply Newton's method with an initial guess $x_0 = 2$ to find the next approximation x_1 . [2 marks]
 - ii) Apply the secant method with initial guesses $x_0 = 3$ and $x_1 = 2$ to find the next approximation x_2 . [2 marks]
- (d) Suppose a root-finding method produces a sequence $\{x_n\}$ with quadratic convergence, and the errors $E_n = |x_n - p|$ satisfy $E_{n+1} \approx E_n^2$. If the current error is $E_2 = 10^{-3}$, roughly what is the expected error E_4 ? [1 mark]
- (e) Show that if p is a simple root of $f(x)$ (that is, $f(p) = 0$ and $f'(p) \neq 0$) and $f(x)$ is sufficiently smooth, Newton's method has an order of convergence of at least $\alpha = 2$. You may use the theorem on the order of convergence for fixed-point iteration. [4 marks]
-

5. Calculus

The calculus was the first achievement of modern mathematics and it is difficult to overestimate its importance.

–Hungarian-American Mathematician John von Neumann

5.1. Intro to Numerical Calculus

In this chapter we build some of the common techniques for approximating the two primary computations in calculus: taking derivatives and evaluating definite integrals. The primary goal of this chapter is to build a solid understanding of the basic techniques for numerical differentiation and integration. These will be crucial in the later chapters of this book when we numerically integrate ordinary and partial differential equations.

Recall the typical techniques from differential calculus: the power rule, the chain rule, the product rule, the quotient rule, the differentiation rules for exponentials, inverses, and trig functions, implicit differentiation, etc. With these rules, and enough time and patience, we can find a derivative of any algebraically defined function. The truth of the matter is that not all functions are given to us algebraically, and even the ones that are given algebraically are sometimes really cumbersome.

Exercise 5.1. When a police officer fires a laser speed gun at a moving car it uses a laser to measure the distance from the officer to the car:

- The speed of light is constant.
- The round-trip time between when the laser is fired and when the light reflected off of the car is received can be measured very accurately.
- Using the formula $\text{distance} = \text{rate} \cdot \text{time}$, the round-trip time for the laser pulse to be sent and received can then be converted to a distance to the car by dividing by 2.

How does the speed gun then use that information to calculate the speed of the moving car?

Integration, on the other hand, is a more difficult situation. You may recall some of the techniques of integral calculus such as the power rule, u -substitution, and integration by parts. However, these tools are not enough to find an antiderivative for every given function. Furthermore, not every function can be written algebraically.

Exercise 5.2. In statistics the function known as *the normal distribution* (the bell curve) is defined as

$$N(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

One of the primary computations of introductory statistics is to find the area under a portion of this curve since this area gives the probability of some event

$$P(a < x < b) = \int_a^b \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx.$$

The trouble is that there is no elementary antiderivative of this function. Propose a method for approximating this area.

Exercise 5.3. A dam operator has control of the rate at which water is flowing out of a hydroelectric dam. He has records for the approximate flow rate through the dam over the course of a day. Propose a way for the operator to use his data to determine the total amount of water that has passed through the dam during that day.

What you have seen here are just a few examples of why you might need to use numerical calculus instead of the classical routines that you learned earlier in your mathematical career. Another typical need for numerical derivatives and integrals arises when we approximate the solutions to differential equations later in the module.

Throughout this chapter we will make heavy use of Taylor's Theorem to build approximations of derivatives and integrals. This has become a recurring scheme in this module: Taylor's Theorem is your all-purpose approximation tool. It suggests the approximation methods and it allows you to understand the error that comes with the approximation.

5.2. Differentiation with Finite Differences

5.2.1. The First Derivative

Exercise 5.4. Recall from your first-semester Calculus class that the derivative of a function $f(x)$ is defined as

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}.$$

A Calculus student proposes that it would just be much easier if we dropped the limit and instead just always choose Δx to be some small number, like 0.001 or 10^{-6} . Discuss the following questions:

1. When might the Calculus student's proposal actually work pretty well in place of calculating an actual derivative?
2. When might the Calculus student's proposal fail in terms of approximating the derivative?

In this section we will build several approximations of first and second derivatives. The primary idea for each of these approximations is:

- Partition the interval $[a, b]$ into N subintervals
- Define the distance between two points in the partition as Δx .
- Approximate the derivative at any point x in the interval $[a, b]$ by using linear combinations of $f(x - \Delta x)$, $f(x)$, $f(x + \Delta x)$, and/or other points in the partition.

Partitioning the interval into discrete points turns the continuous problem of finding a derivative at every real point in $[a, b]$ into a discrete problem where we calculate the approximate derivative at finitely many points in $[a, b]$.

This distance Δx between neighbouring points in the partition is often referred to as the **step size**. It is also common to denote the step size by the letter h . We will use both notations for the step size interchangeably, using mostly h in this section on differentiation and Δx in the next section on integration. Note that in general the points in the partition do not need to be equally spaced, but that is the simplest place to start. Figure 5.1 shows a depiction of the partition as well as making clear that h is the separation between each of the points in the partition.

5. Calculus

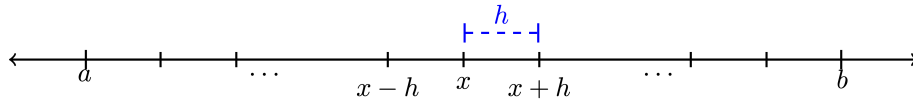


Figure 5.1.: A partition of the interval $[a, b]$.

Exercise 5.5. Let us take a close look at partitions before moving on to more details about numerical differentiation.

1. If we partition the interval $[0, 1]$ into 3 equal subintervals each with length h then:
 - a. $h = \underline{\hspace{2cm}}$
 - b. $[0, 1] = [0, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, 1]$
 - c. There are four total points that define the partition. They are 0, $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$, 1.
2. If we partition the interval $[3, 7]$ into 5 equal subintervals each with length h then:
 - a. $h = \underline{\hspace{2cm}}$
 - b. $[3, 7] = [3, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, \underline{\hspace{1cm}}] \cup [\underline{\hspace{1cm}}, 7]$
 - c. There are 6 total points that define the partition. They are 3, $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$, 7.
3. More generally, if a closed interval $[a, b]$ contains N equal subintervals where

$$[a, b] = \underbrace{[a, a+h] \cup [a+h, a+2h] \cup \cdots \cup [b-2h, b-h] \cup [b-h, b]}_{N \text{ total subintervals}}$$

then the length of each subinterval, h , is given by the formula

$$h = \frac{\hspace{1cm} - \hspace{1cm}}{\hspace{1cm}}.$$

Exercise 5.6. In Python's `numpy` library there is a nice tool called `np.linspace()` that partitions an interval in exactly the way that we want. The command takes the form `np.linspace(a, b, n)` where the interval is $[a, b]$ and n the number of points used to create the partition. For example, `np.linspace(0, 1, 5)` will produce the list of numbers 0, 0.25, 0.5, 0.75, 1. Notice that there are 5 total points, the first point is a , the last point is b , and there are 4 total subintervals in the partition. Hence, if we want to partition the interval $[0, 1]$ into 20 equal subintervals then we would use the command `np.linspace(0, 1, 21)` which would result in a list of numbers starting with 0, 0.05, 0.1, 0.15, etc. What command would you use to partition the interval $[5, 10]$ into 100 equal subintervals?

Exercise 5.7. Consider the Python command `np.linspace(0,1,50)`.

1. What interval does this command partition?
 2. How many points are going to be returned?
 3. How many equal length subintervals will we have in the resulting partition?
 4. What is the length of each of the subintervals in the resulting partition?
-

Now let us get back to the discussion of numerical differentiation. If we recall that the definition of the first derivative of a function is

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

our first approximation for the first derivative is naturally

$$\frac{df(x)}{dx} \approx \frac{f(x+h) - f(x)}{h} =: \Delta f(x).$$

In this approximation of the derivative we have simply removed the limit and instead approximated the derivative as the slope. It should be clear that this approximation is only good if the step size h is *small*. In Figure 5.2 we see a graphical depiction of what we are doing to approximate the derivative. The slope of the tangent line ($\Delta y / \Delta x$) is what we are after, and a way to approximate it is to calculate the slope of the secant line formed by looking h units forward from the point x .

While this is the simplest and most obvious approximation for the first derivative there is a much more elegant technique, using Taylor series, for arriving at this approximation. Furthermore, the Taylor series technique gives us information about the approximation error and later will suggest an infinite family of other techniques.

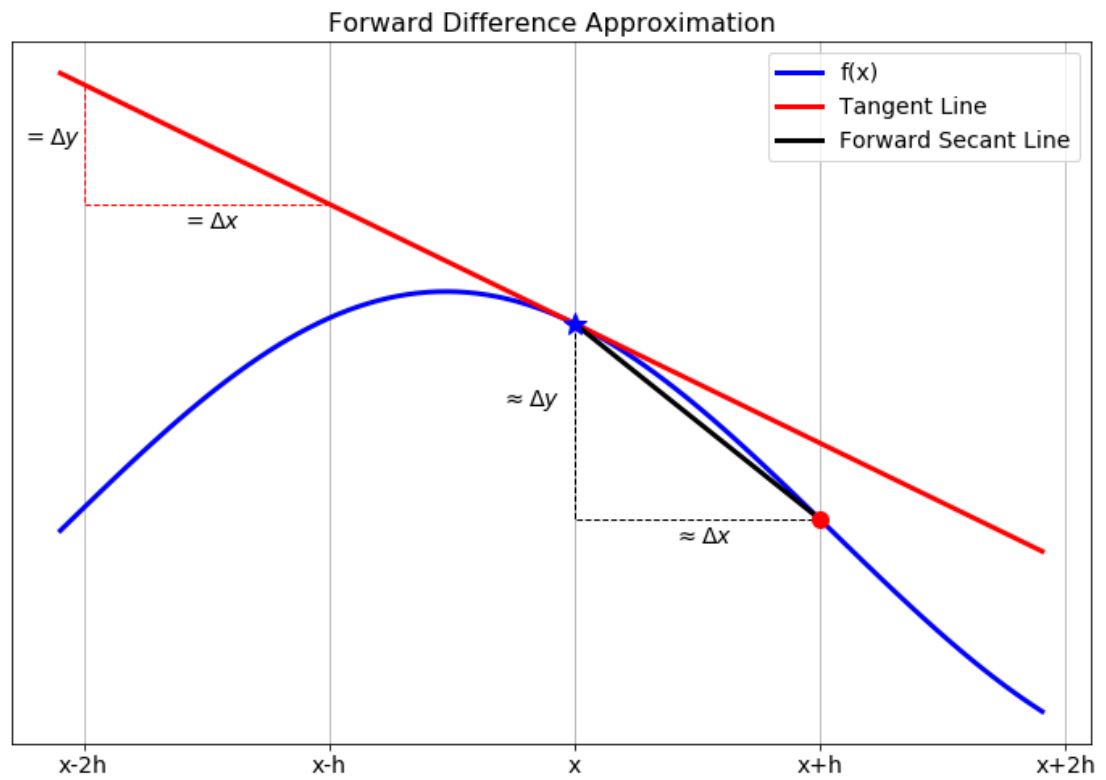


Figure 5.2.: The forward difference differentiation scheme for the first derivative.

5.2.2. Truncation Error

Exercise 5.8. From Taylor's Theorem we know that, for a sufficiently smooth function $f(x)$ whose Taylor series represents it locally,

$$f(x) = f(x_0) + \frac{f'(x_0)}{1!}(x-x_0)^1 + \frac{f''(x_0)}{2!}(x-x_0)^2 + \frac{f^{(3)}(x_0)}{3!}(x-x_0)^3 + \cdots.$$

What do we get if we replace every “ x ” in the Taylor Series with “ $x + h$ ” and replace every “ x_0 ” in the Taylor Series with “ x ”? In other words, in Figure 5.1 we want to centre the Taylor series at x and evaluate the resulting series at the point $x + h$.

$$f(x+h) = \underline{\hspace{10cm}}$$

Exercise 5.9. Solve the result from the previous exercise for $f'(x)$ to create an approximation for $f'(x)$ using $f(x+h)$, $f(x)$, and some higher-order terms. (Fill in the blanks and the question marks.)

$$f'(x) = \frac{f(x+h) - ???}{??} + \underline{\hspace{2cm}}$$

Exercise 5.10. In the formula that you developed in Exercise 5.9, if we were to truncate after the first fraction and drop everything else (called the *remainder*), we know that we would be introducing a truncation error into our derivative computation. If h is taken to be very small then the first term in the remainder is the largest and everything else in the remainder can be ignored (since all subsequent terms should be extremely small ... pause and ponder this fact). Therefore, the amount of error we make in the derivative computation by dropping the remainder depends on the power of h in that first term in the remainder.

What is the power of h in the first term of the remainder from Exercise 5.9?

Definition 5.1 (Order of a Numerical Differentiation Scheme). The **order** of a numerical derivative is the power of the step size in the first term of the remainder of the rearranged Taylor Series. For example, a first-order method will have “ h^1 ” in the first term of the remainder. A second-order method will have “ h^2 ” in the first term of the remainder. Etc.

For sufficiently small step size h , the error that you make by truncating the series is dominated by the first term in the remainder, which is proportional to the power of h

5. Calculus

in that term. Hence, the **order** of a numerical differentiation scheme tells you how the error you are making by using the approximation scheme decreases as you decrease the step size h .

Definition 5.2. (Big O Notation) We say that the error in a differentiation scheme is $\mathcal{O}(h)$ (read: “big O of h ”) if there is a positive constant M such that

$$|\text{Error}| \leq M \cdot h$$

when h is sufficiently small. If the leading nonzero error term is proportional to h , then the differentiation method is first order.

More generally, we say that the error in a differentiation scheme is $\mathcal{O}(h^k)$ (read: “big O of h^k ”) if there is a positive constant M such that

$$|\text{Error}| \leq M \cdot h^k.$$

when h is sufficiently small. If the leading nonzero error term is proportional to h^k , then the differentiation scheme is “ k^{th} order.”

Theorem 5.1. *The approximation you derived in Exercise 5.9 gives a first-order approximation of the first derivative:*

$$f'(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h).$$

*This is called the **forward difference approximation** of the first derivative.*

Exercise 5.11. Consider the function $f(x) = \sin(x)(1-x)$. The goal of this exercise is to make sense of the discussion of the “order” of the derivative approximation.

- Find $f'(x)$ analytically.
- Use your answer to part (a) to verify that $f'(1) = -\sin(1) \approx -0.8414709848$.

5.2. Differentiation with Finite Differences

- c. To approximate the first derivative at $x = 1$ numerically with the forward-difference approximation formula from Theorem 5.1 we calculate

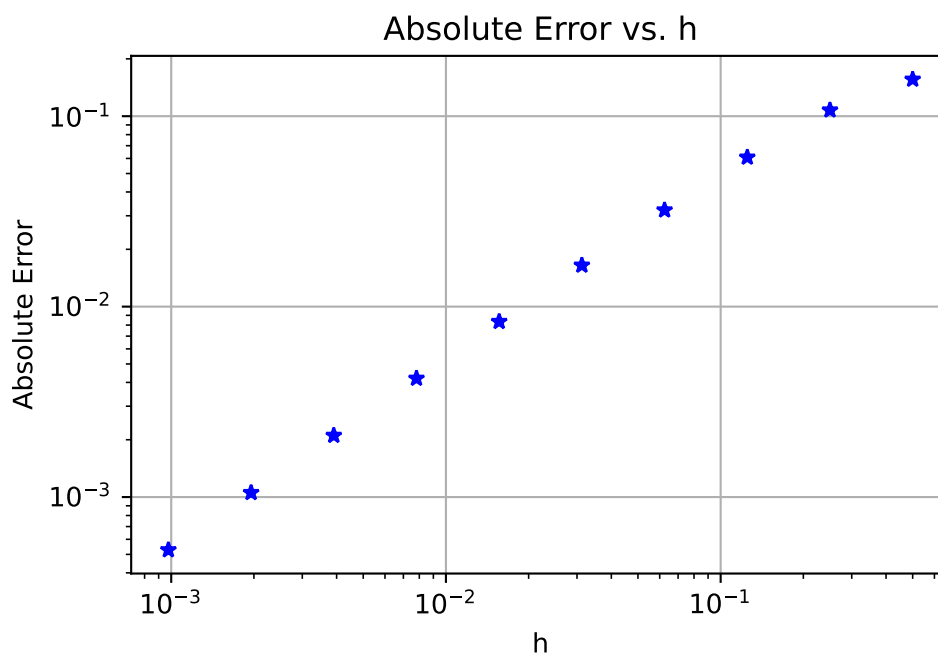
$$f'(1) \approx \frac{f(1+h) - f(1)}{h} =: \Delta f(1).$$

We want to see how the error in the approximation behaves as h is made smaller and smaller. The table below shows the approximation and the absolute error for $h = 2^{-1}$ and $h = 2^{-2}$. It also shows the error reduction factor, which is the ratio of two successive absolute errors. Extend the table to include $h = 2^{-3}$ up to $h = 2^{-10}$. Start with the code given below but use a loop to create the 10 rows.

h	$\Delta f(1)$	$f'(1) - \Delta f(1)$	Error reduction factor
2 ⁻¹ = 0.5	-0.997494986604054	0.156024001796158	
2 ⁻² = 0.25	-0.948984619355586	0.107513634547690	1.451202002913877

- d. Use the following function to plot the absolute error as a function of h .

Here is an example of how to use the function:



- e. Discuss the pattern in how the absolute error changes as h is changed? What does the log-log plot tell you? How does this agree with what the table tells you?

5. Calculus

- f. There was nothing really special in part (c) about powers of 2. Modify your code to build similar tables and plots for the following sequences of h :

$$h = 3^{-1}, 3^{-2}, 3^{-3}, \dots$$

$$h = 5^{-1}, 5^{-2}, 5^{-3}, \dots$$

$$h = 10^{-1}, 10^{-2}, 10^{-3}, \dots$$

Do you see any anomalies that could be caused by rounding errors?

- g. Create the table and the plot for several different choices of the function f and the point x at which you compute the derivative. Does the pattern you observe in part (e) continue to hold?
- h. What does your answer to part (e) have to do with the approximation order of the numerical derivative method that you used?

Exercise 5.12. Explain the phrase: *The forward difference approximation $f'(x) \approx \frac{f(x+h)-f(x)}{h}$ is first order.*

5.2.3. Efficient Coding

Now that we have a handle on how the forward-difference approximation scheme for the first derivative works and how the error depends on the step size, let us build some code that will take in a function and output the approximate first derivative on an entire interval instead of just at a single point.

Exercise 5.13. We want to build a Python function that accepts:

- a mathematical function,
- the bounds of an interval,
- and the number of subintervals.

The function will return the forward-difference approximation of the first derivative at every point in the interval except at the right-hand side. For example, we could send the function $f(x) = \sin(x)$, the interval $[0, 2\pi]$, and tell it to split the interval into 100 subintervals. We would then get back an approximate value of the derivative $f'(x)$ at all of the points except at $x = 2\pi$.

1. First of all, why can we not compute the forward-difference approximation of the derivative at the last point?
2. Next, fill in the blanks in the partially complete code below and update the comments to say what each line does.

```
import numpy as np
import matplotlib.pyplot as plt
def ForwardDiff(f,a,b,N):
    x = np.linspace(a,b,N+1) # What does this line of code do?
    # What's up with the N+1 in the previous line?
    h = x[1] - x[0] # What does this line of code do?
    df = [] # What does this line of code do?
    for j in np.arange(len(x)-1): # What does this line of code do?
        # What's up with the -1 in the definition of the loop?

        # Now we want to build the approximation
        # (f(x+h) - f(x)) / h.
        # Obviously "x+h" is just the next item in the list of
        # x values so when we do f(x+h) mathematically we should
        # write f(x[j+1]) in Python (explain this).
        # Fill in the question marks below
        df.append((f(???) - f(???)) / h )
    return df
```

3. Now we want to call upon this function to build the first-order approximation of the first derivative for some function. We will use the function $f(x) = \sin(x)$ on the interval $[0, 2\pi]$ with 100 subintervals (since we know what the answer should be). Complete the code below to call upon your `ForwardDiff()` function and to plot $f(x)$, $f'(x)$, and the approximation of $f'(x)$.

```
f = lambda x: np.sin(x)
exact_df = lambda x: np.cos(x)
a = ???
b = ???
N = 100 # What is this?
x = np.linspace(a,b,N+1)
# What does the previous line do?
# What's up with the N+1?

df = ForwardDiff(f,a,b,N) # What does this line do?

# In the next line we plot three curves:
# 1) the function f(x)
```

5. Calculus

```
# 2) the function f'(x)
# 3) the approximation of f'(x)
# However, we do something funny with the x in the last plot. Why?
plt.plot(x,f(x),'b',x,exact_df(x),'r--',x[0:-1], df, 'k-.')
plt.grid()
plt.legend(['f(x) = sin(x)',
           'exact first deriv',
           'approx first deriv'])
plt.show()
```

4. Implement your completed code and then test it in several ways:

- Test your code on functions where you know the derivative. Be sure that you get the plots that you expect.
- Test your code with a very large number of subintervals, N . What do you observe?
- Test your code with a small number of subintervals, N . What do you observe?

Exercise 5.14. Now let us build the first derivative function in a much *smarter* way – using NumPy arrays in Python. Instead of looping over all of the x values, we can take advantage of the fact that NumPy operations can act on all the elements of an array at once and hence we can do all of the function evaluations and all the subtractions and divisions at once without a loop.

- The line of code `x = np.linspace(a,b,N+1)` builds a numpy vector of $N + 1$ values of x starting at a and ending at b . Then `y = f(x)` builds a vector with the function values at all the elements in `x`. In the following questions remember that Python indexes all lists starting at 0. Also remember that you can call on the last element of a list using an index of `-1`. Finally, remember that if you do `x[p:q]` in Python you will get a list of `x` values starting at index `p` and ending at index `q-1`.
 - What will we get if we evaluate the code `y[1:]`?
 - What will we get if we evaluate the code `y[:-1]`?
 - What will we get if we evaluate the code `y[1:] - y[:-1]`?
 - What will we get if we evaluate the code `(y[1:] - y[:-1]) / h`?
- Use the insight from part (1) to simplify your first-order first derivative function to look like the code below.


```
def ForwardDiff(f,a,b,N):
    x = np.linspace(a,b,N+1)
    h = x[1] - x[0]
    y = f(x)
    df = # your line of code goes here?
    return df
```

3. Test your new function by checking that the plots from the previous exercise still look the same.

Exercise 5.15. Write code that finds a first-order approximation for the first derivative of $f(x) = \sin(x)(1 - x)$ on the interval $x \in (0, 15)$. Your script should output two plots (side-by-side).

- a. The left-hand plot should show the function in blue and the approximate first derivative as a red dashed curve. Sample code for this exercise is given below.

```
import matplotlib.pyplot as plt
import numpy as np

f = lambda x: np.sin(x) * (1 - x)
a = 0
b = 15
N = # make this an appropriately sized number of subintervals
x = np.linspace(a,b,N+1) # what does this line do?
y = f(x) # what does this line do?
df = ForwardDiff(f,a,b,N) # what does this line do?

fig, ax = plt.subplots(1,2) # what does this line do?
ax[0].plot(x,y,'b',x[0:-1],df,'r--') # what does this line do?
ax[0].grid()
```

- b. The right-hand plot should show the absolute error between the exact derivative and the numerical derivative. You should use a logarithmic y axis for this plot.

```
exact = lambda x: # write a function for the exact derivative
# There is a lot going on the next line of code ... explain it.
ax[1].semilogy(x[0:-1],abs(exact(x[0:-1]) - df))
ax[1].grid()
fig
```

5. Calculus

- c. You should see a rather spiky plot on the right, where the error drops to very low values at certain x values. Can you explain why the forward difference approximation is so much more precise at these points? What is special about the shape of the original function at these points?
- d. Play with the number of subintervals, N , and demonstrate the fact that we are using a first-order method to approximate the first derivative.

Exercise 5.16. You have seen in the previous exercise how the error is different at different x . It might be interesting to look at the largest error over the entire interval for different step sizes.

1. Find the numerical first derivative with your `ForwardDiff` function on the interval $[0, 15]$ with a step size of $h = 0.1$.
2. Find the absolute difference between your numerical first derivative and the actual first derivative. This is point-by-point subtraction so you should end up with a vector containing the errors at each point, just as in the previous exercise. Again be careful to take into account that the numerical derivative is not available at the right endpoint.
3. Find the maximum of your errors. If all is well, you should find that the maximum error is approximately 0.6731791497487658.
4. Now we want to see how this maximum error changes as we change the number of points used. Build a function `plot_max_forward_difference_errors` that produces a log-log plot showing the value of h on the horizontal axis and the maximum error on the vertical axis.

```
def plot_max_forward_difference_errors(f, f_prime, a, b, H):
    """
    Plot the maximum absolute error in the forward difference
    approximation for the first derivative of f on the interval [a,b]

    Parameters:
        f (function): Function whose derivative we approximate
        f_prime (function): Exact derivative of f
        a (float): Left endpoint of interval
        b (float): Right endpoint of interval
        H (vector): Vector of step sizes h

    Returns:
        A plot with the step sizes on the x axis and the
        absolute error on the y axis
```

```
"""
```

```
# Your code here
```

You will need to write a loop that gets the error for many different values of h . You can build on the code from your `plot_forward_difference_errors` function. You should find that the maximum errors again line up approximately on a straight line with a slope of 1.

5.2.4. A Better First Derivative

Next we will build a more accurate numerical first derivative scheme. The derivation technique is the same: play a little algebra game with the Taylor series and see if you can get the first derivative to simplify out. This time we will be hoping to get a second-order method.

Exercise 5.17. Consider again the Taylor series for a sufficiently smooth function $f(x)$ whose Taylor series represents it locally:

$$f(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0)^1 + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f^{(3)}(x_0)}{3!}(x - x_0)^3 + \dots$$

1. Replace the “ x ” in the Taylor Series with “ $x + h$ ” and replace the “ x_0 ” in the Taylor Series with “ x ” and simplify.

$$f(x + h) = \underline{\hspace{10em}}.$$

2. Now replace the “ x ” in the Taylor Series with “ $x - h$ ” and replace the “ x_0 ” in the Taylor Series with “ x ” and simplify.

$$f(x - h) = \underline{\hspace{10em}}.$$

3. Find the difference between $f(x + h)$ and $f(x - h)$ and simplify. Be very careful of your signs.

$$f(x + h) - f(x - h) = \underline{\hspace{10em}}.$$

5. Calculus

4. Solve for $f'(x)$ in your result from part (3). Fill in the question marks and blanks below once you have finished simplifying.

$$f'(x) = \frac{??? - ???}{2h} + \underline{\hspace{4cm}}.$$

5. Use your result from part (4) to verify that

$$f'(x) = \underline{\hspace{4cm}} + \mathcal{O}(h^2).$$

6. Draw a picture similar to Figure 5.2 showing what this scheme is doing graphically.

Exercise 5.18. Let us return to the function $f(x) = \sin(x)(1 - x)$ but this time we will approximate the first derivative at $x = 1$ using the formula

$$f'(1) \approx \frac{f(1+h) - f(1-h)}{2h} =: \delta f(1).$$

You should already have the first derivative and the exact answer from earlier exercises (if not, then go get them by hand again).

- a. We want to see how the error in the approximation behaves as h is made smaller and smaller. Adapt your code from Exercise 5.11 to extend the table to include $h = 2^{-3}$ up to $h = 2^{-10}$.

h	f(1)	f'(1) - f(1)	Error reduction fac
2 ⁻¹ = 0.5	-0.738460262604129	0.103010722203768	
2 ⁻² = 0.25	-0.815311689689460	0.026159295118436	3.937824843421275

- b. Adapt your code for the `plot_forward_difference_errors` function to create a `plot_central_difference_errors` function and use it to plot the errors for the sequences of h in part (a).
- c. Build similar tables for the following sequences of h :

$$h = 3^{-1}, 3^{-2}, 3^{-3}, \dots$$

$$h = 5^{-1}, 5^{-2}, 5^{-3}, \dots$$

$$h = 10^{-1}, 10^{-2}, 10^{-3}, \dots$$

- d. Discuss the pattern in how the absolute error changes as h is changed? What does the log-log plot tell you? How does this agree with what the table tells you? How does this compare to the error for the forward difference approximation?

- e. What does your answer to part (d) have to do with the approximation order of the numerical derivative method that you used?

Exercise 5.19. Write a Python function `CentralDiff(f, a, b, N)` that takes a mathematical function `f`, the start and end values of an interval `[a, b]` and the number `N` of subintervals to use. It should return a second-order numerical approximation to the first derivative on the interval. This should be a vector with $N - 1$ entries (why?). You should try to write this code without using any loops. (Hint: This should really be a minor modification of your first-order first derivative code.) Test the code on functions where you know the first derivative.

Exercise 5.20. The plot shown in Figure 5.3 shows the maximum absolute error between the exact first derivative of a function $f(x)$ and a numerical first derivative approximation scheme. At this point we know two schemes:

$$f'(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h)$$

and

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2).$$

1. Which curve in the plot matches with which method. How do you know?
 2. Recreate the plot with a function of your choosing.
-

5.2.5. The Second Derivative

Now we will search for an approximation of the second derivative. Again, the game will be the same: experiment with the Taylor series and some algebra with an eye toward getting the second derivative to pop out cleanly. This time we will do the algebra in such a way that the first derivative cancels.

From the previous exercises you already have Taylor expansions of the form $f(x+h)$ and $f(x-h)$. Let us summarise them here since you are going to need them for future computations.

$$\begin{aligned} f(x+h) &= f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f^{(3)}(x)}{3!}h^3 + \dots \\ f(x-h) &= f(x) - \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 - \frac{f^{(3)}(x)}{3!}h^3 + \dots \end{aligned}$$

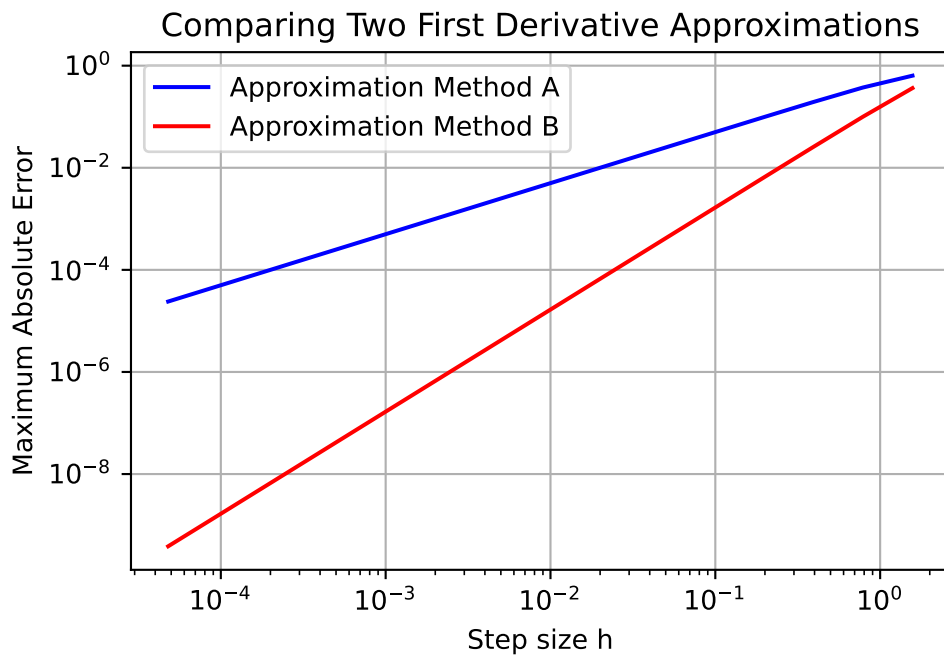


Figure 5.3.: Maximum absolute error between the first derivative and two different approximations of the first derivative.

Exercise 5.21. The goal of this exercise is to use the Taylor series for $f(x + h)$ and $f(x - h)$ to arrive at an approximation scheme for the second derivative $f''(x)$.

1. Add the Taylor series for $f(x + h)$ and $f(x - h)$ and combine all like terms. You should notice that several terms cancel.

$$f(x + h) + f(x - h) = \underline{\hspace{10cm}}.$$

2. Solve your answer in part (1) for $f''(x)$.

$$f''(x) = \frac{?? - 2 \cdot ?? + ??}{h^2} + \underline{\hspace{2cm}}.$$

3. If we were to drop all of the terms after the fraction on the right-hand side of the previous result we would be introducing some error into the derivative computation. What does this tell us about the order of the error for the second derivative approximation scheme we just built?

Exercise 5.22. Again consider the function $f(x) = \sin(x)(1 - x)$.

1. Calculate the derivative of this function and calculate the exact value of $f''(1)$.
2. If we calculate the second derivative with the central difference scheme that you built in the previous exercise using $h = 0.5$ then we get an absolute error of about 0.044466. Stop now and verify this error calculation.
3. Based on our previous work with the order of the error in a numerical differentiation scheme, what do you predict the error will be if we calculate $f''(1)$ with $h = 0.25$? With $h = 0.05$? With $h = 0.005$? Defend your answers.

Exercise 5.23. Write a Python function `SecondDiff(f, a, b, N)` that takes a mathematical function `f`, the start and end values of an interval `[a, b]` and the number `N` of subintervals to use. It should return a second-order numerical approximation to the second derivative on the interval. This should be a vector with $N - 1$ entries (why?). As before, you should write your code without using any loops.

Exercise 5.24. Test your second derivative code on the function $f(x) = \sin(x)(1 - x)$ by calculating the second derivative at $x = 1$ and comparing it to the exact answer you determined in Exercise 5.22.

Exercise 5.25. You have seen in the previous exercise how the error is different at different x . It might be interesting to look at the largest error over the entire interval for different step sizes.

1. Find the numerical second derivative with your `SecondDiff` function on the interval $[0, 15]$ with a step size of $h = 0.1$.
2. Find the absolute difference between your numerical second derivative and the actual second derivative. This is point-by-point subtraction so you should end up with a vector containing the errors at each interior point. Again be careful to take into account that the numerical second derivative is not available at the endpoints.
3. Find the maximum of your errors.

5. Calculus

4. Now we want to see how this maximum error changes as we change the number of points used. Build a function `plot_max_second_difference_errors` that produces a log-log plot showing the value of h on the horizontal axis and the maximum error on the vertical axis. You can simply adapt the code from your `plot_max_forward_difference_errors` function. You should find that the maximum errors again line up approximately on a straight line.
5. Discuss what you see? How do you see the fact that the numerical second derivative is second-order accurate?

The table below summarises the formulas that we have for derivatives thus far.

Derivative	Formula	Error	Name
1^{st}	$f'(x) \approx \frac{f(x+h)-f(x)}{h}$	$\mathcal{O}(h)$	Forward Difference
1^{st}	$f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$	$\mathcal{O}(h^2)$	Central Difference
2^{nd}	$f''(x) \approx \frac{f(x+h)-2f(x)+f(x-h)}{h^2}$	$\mathcal{O}(h^2)$	Central Difference

Exercise 5.26. Let $f(x)$ be a twice differentiable function. We are interested in the first and second derivative of the function f at the point $x = 1.74$. Use what you have learned in this section to answer the following questions. (For clarity, you can think of “ f ” as a different function in each of the following questions ...it does not really matter exactly what function f is.)

1. Johnny used a numerical first derivative scheme with $h = 0.1$ to approximate $f'(1.74)$ and found an absolute error of 3.28. He then used $h = 0.01$ and found an absolute error of 0.328. What was the order of the error in his first derivative scheme? How can you tell?
2. Betty used a numerical first derivative scheme with $h = 0.2$ to approximate $f'(1.74)$ and found an absolute error of 4.32. She then used $h = 0.1$ and found an absolute error of 1.08. What numerical first derivative scheme did she likely use?
3. Harry wants to compute $f''(1.74)$ to within 1% using a central difference scheme. He tries $h = 0.25$ and gets an absolute percentage error of 3.71%. What h could he try next so that his absolute percentage error is close to 1%?

Exercise 5.27. We said at the start of this section that the spacing of the steps does not have to be constant. Instead of a constant step size h we could have variable step sizes $\Delta x_i := x_{i+1} - x_i$. However, this complicates the expression for the second derivative. In this exercise you can check that you fully understood the derivation of the formula for the second derivative by repeating it for these variable step sizes. So you will need to start with the Taylor expansions of $f(x_{i+1}) = f(x_i + \Delta x_i)$ and $f(x_{i-1}) = f(x_i - \Delta x_{i-1})$ around x_i .

5.3. Integration

Now that we understand how to calculate derivatives, we begin our work on the second principal computation of calculus: evaluating a definite integral. You will be able to transfer much of what you did in the previous section, where you investigated the errors in numerical differentiation, to the investigation of the errors in numerical integration.

Exercise 5.28. Remember that a single-variable definite integral can be interpreted as the signed area between the curve and the x axis. Consider the shaded area of the region under the function plotted in Figure 5.4 between $x = 0$ and $x = 2$.

1. What rectangle with area 6 gives an upper bound for the area under the curve? Can you give a better upper bound?
 2. Why must the area under the curve be greater than 3?
 3. Is the area greater than 4? Why/Why not?
 4. Work with your group to give an estimate of the area and provide an estimate for the amount of error that you are making.
-

In this section we will study three different techniques for approximating the value of a definite integral: Riemann sums, the trapezoidal rule, and Simpson's rule.

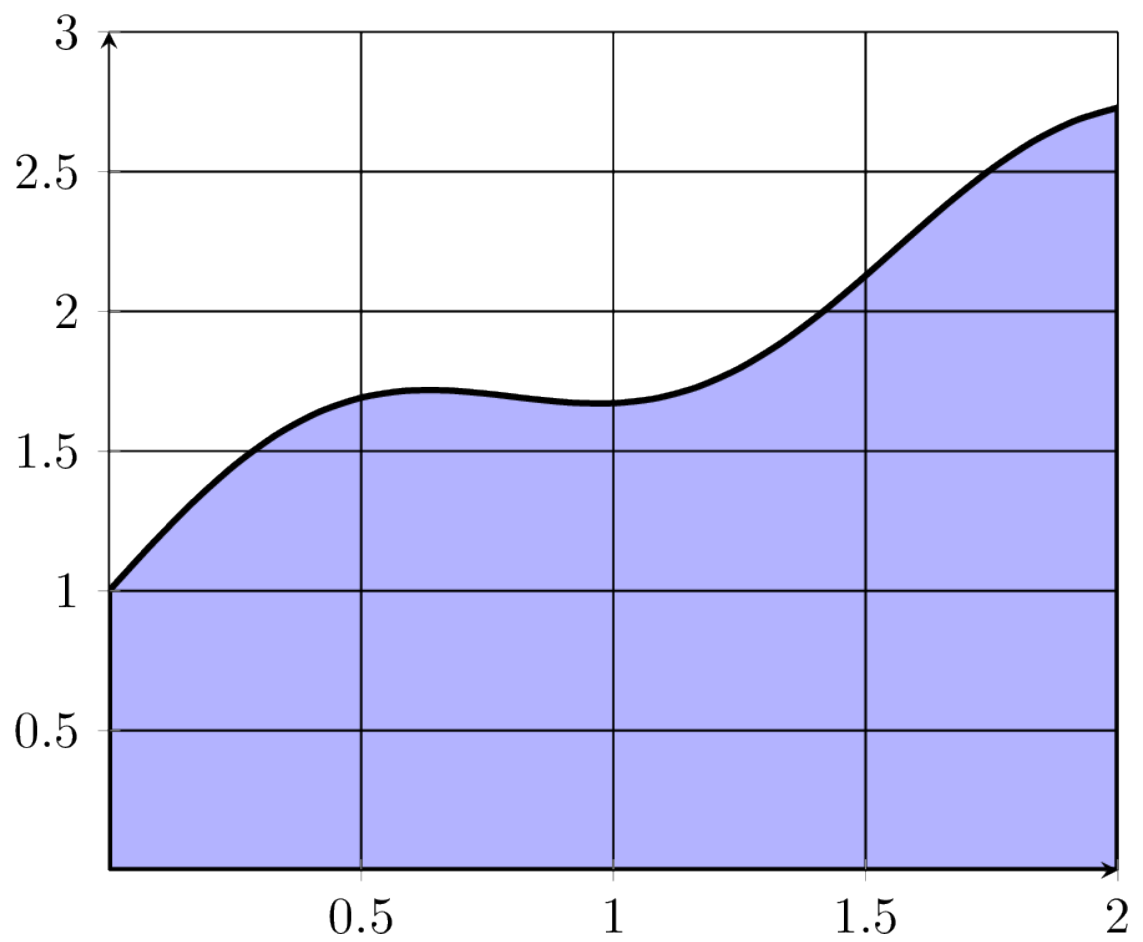


Figure 5.4.: A sample integration

5.3.1. Riemann Sums

In this subsection we will build our first method for approximating definite integrals. Recall from Calculus that the definition of the Riemann integral is

$$\int_a^b f(x)dx = \lim_{\Delta x \rightarrow 0} \sum_{j=1}^N f(x_j^*)\Delta x,$$

where N is the number of subintervals on the interval $[a, b]$, Δx is the width of each subinterval¹, and x_j^* is a sample point in the j^{th} subinterval. As with differentiation, we can remove the limit and have a decent approximation of the integral so long as N is large (or equivalently, as long as Δx is small).

$$\int_a^b f(x)dx \approx \sum_{j=1}^N f(x_j^*)\Delta x.$$

You are likely familiar with this approximation of the integral from Calculus. The value of x_j^* can be chosen anywhere within the subinterval, and three common choices are to use the left endpoint, the right endpoint, and the midpoint. We see a depiction of the resulting approximations in Figure 5.5.

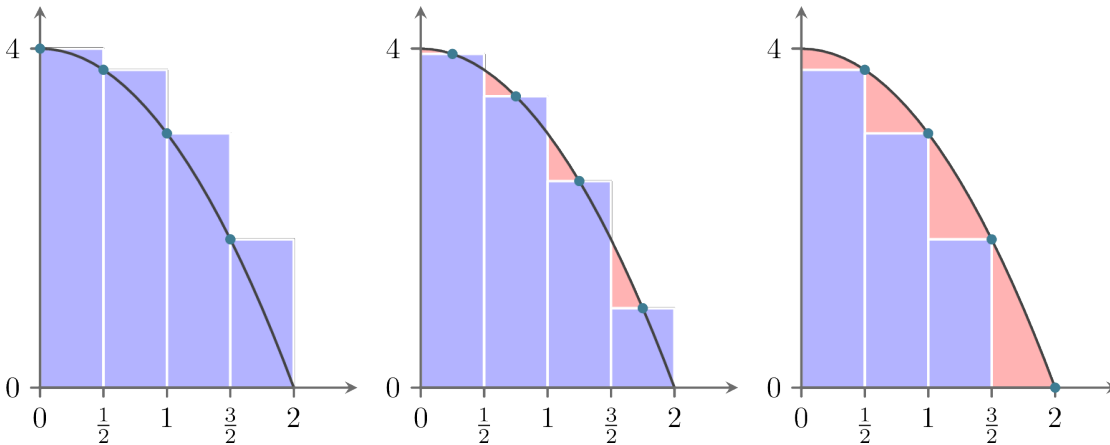


Figure 5.5.: Left-aligned Riemann sum, midpoint-aligned Riemann sum, and right-aligned Riemann sum

Clearly, the more rectangles we choose the closer the sum of the areas of the rectangles will get to the integral.

Assuming the boundaries of the N subintervals are given by $x_0, x_1, \dots, x_N$, these three choices define the following approximations:

¹In this module we will give all subintervals the same length. This simplifies things but is not necessary.

5. Calculus

Left Riemann sum:

$$\int_a^b f(x)dx \approx \sum_{j=0}^{N-1} f(x_j)\Delta x$$

Right Riemann sum:

$$\int_a^b f(x)dx \approx \sum_{j=0}^{N-1} f(x_{j+1})\Delta x$$

Midpoint Riemann sum:

$$\int_a^b f(x)dx \approx \sum_{j=0}^{N-1} f\left(\frac{x_j + x_{j+1}}{2}\right)\Delta x$$

Exercise 5.29. Write a Python function `RiemannSum(f, a, b, N, method='left')` that approximates an integral with a Riemann sum. Your Python function should accept a Python function `f`, a lower bound `a`, an upper bound `b`, the number of subintervals `N`, and an optional input `method` that allows the user to designate whether they want ‘left’, ‘right’, or ‘midpoint’ rectangles. You should write your code without any loops. Test your code on several functions for which you know the integral.

Exercise 5.30. Consider the function $f(x) = \sin(x)$. We know the antiderivative for this function, $F(x) = -\cos(x) + C$. In this question we are going to get a sense of the order of the error when doing Riemann sum integration.

- a. Find the exact value of

$$I = \int_0^1 f(x)dx.$$

- b. To approximate the integral numerically using the left Riemann sum we calculate

$$I \approx \sum_{j=0}^{N-1} f(x_j)\Delta x =: A_{\text{left}}(\Delta x).$$

We want to see how the error in the approximation behaves as Δx is made smaller and smaller. The table below shows the approximation, the absolute error, and the error reduction factor, which is the ratio of two successive absolute errors, for the sequence

$$\Delta x = 3^{-1}, 3^{-2}, 3^{-3}, \dots$$

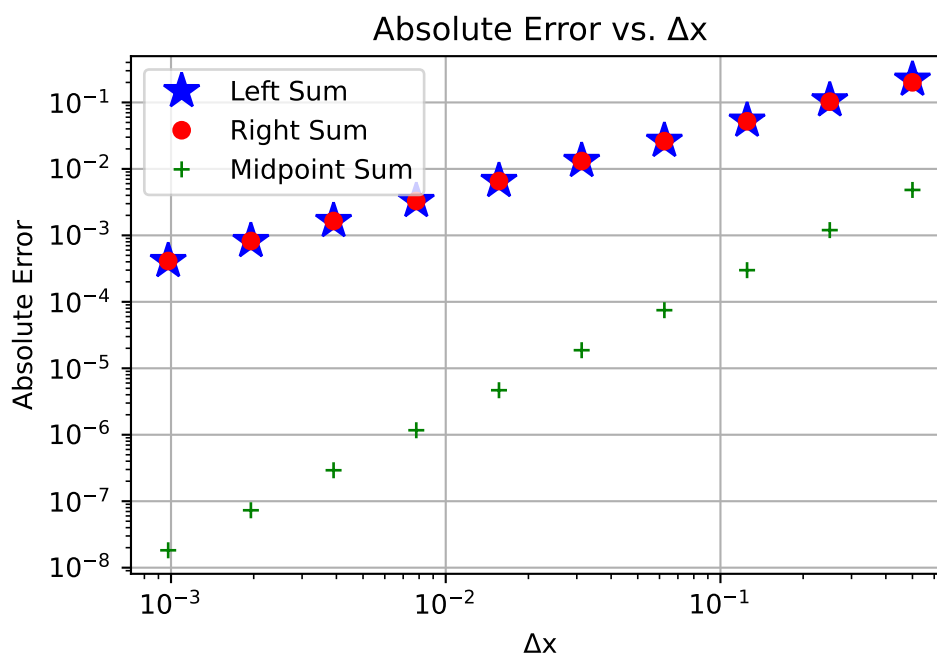
Δx	A_left(Δx)	I - A_left(Δx)	Error reduction factor
3 ⁻¹ = 0.333333	0.315188166621963	0.144509527509897	
3 ⁻² = 0.111111	0.412476268731065	0.047221425400796	3.060253397337351
3 ⁻³ = 0.037037	0.444062348024963	0.015635346106897	3.020171416606265
3 ⁻⁴ = 0.0123457	0.454497590013096	0.005200104118764	3.006737124835329
3 ⁻⁵ = 0.00411523	0.457965623600503	0.001732070531358	3.002247324586603
3 ⁻⁶ = 0.00137174	0.459120481455185	0.000577212676675	3.000749292852054
3 ⁻⁷ = 0.000457247	0.459505305924864	0.000192388206996	3.000249784988630
3 ⁻⁸ = 0.000152416	0.459633566509368	0.000064127622492	3.000083263971756
3 ⁻⁹ = 5.08053e-05	0.459676318455456	0.000021375676404	3.000027754918068
3 ⁻¹⁰ = 1.69351e-05	0.459690568928366	0.000007125203495	3.000009251680006

- c. Formulate your conjecture for how the error depends on the step size.
- d. Repeat part (b) to create the tables and answer the questions using the right Riemann sum and the midpoint Riemann sum instead of the left Riemann sum. What happens to the error reduction factor in each case? What do you conjecture is the order of the three Riemann sum methods? Compare your conjecture with the other members of your group.

Exercise 5.31. This exercise continues examining the absolute integration errors from the previous problem, this time with a plot.

Use the following function to plot the absolute error as a function of Δx .

Here is an example of how to use the function:



Discuss the pattern in how the absolute error changes as Δx is changed. What does the log-log plot tell you? How does this agree with what the tables you created in the previous exercise told you? (Again notice the similarity between this exercise and Exercise 5.20 where you created a similar plot for approximating a derivative.)

5.3.2. Trapezoidal Rule

Now let us turn our attention to some slightly better algorithms for calculating the value of a definite integral: The Trapezoidal Rule and Simpson's Rule. There are many others, but in practice these two are relatively easy to implement and have reasonably good error approximations. To motivate the idea of the trapezoidal rule consider Figure 5.6. It is plain to see that trapezoids will make better approximations than rectangles at least in this particular case. Another way to think about using trapezoids, however, is to see the top side of the trapezoid as a secant line connecting two points on the curve. As Δx gets arbitrarily small, the secant lines become better and better approximations for tangent lines and are hence arbitrarily good approximations for the curve. For these reasons it seems like we should investigate how to systematically approximate definite integrals via trapezoids.

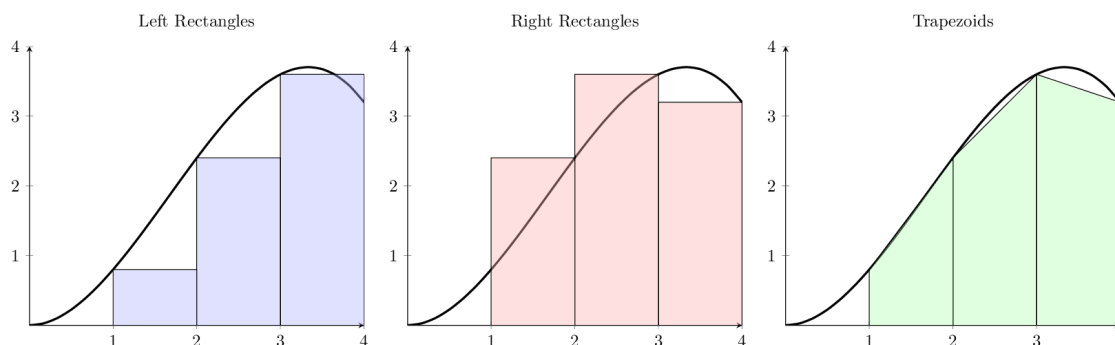


Figure 5.6.: Motivation for using trapezoids to approximate a definite integral.

Exercise 5.32. Consider a single trapezoid approximating the area under a curve. From geometry we recall that the area of a trapezoid is

$$A = \frac{f(a) + f(b)}{2} (b - a).$$

The function shown in Figure 5.7 is $f(x) = \frac{1}{5}x^2(5 - x)$. Find the area of the shaded region as an approximation to

$$\int_1^4 \left(\frac{1}{5}x^2(5 - x) \right) dx.$$

Now use the same idea with $h = \Delta x = 1$ to approximate the area under the function using three trapezoids, as illustrated in the last panel of Figure 5.6.

Exercise 5.33 (Trapezoidal Rule). Now generalise the idea from Exercise D.5 to divide the interval $[a, b]$ into N subintervals with boundaries $\{x_0 = a, x_1, x_2, \dots, x_{N-1}, x_N = b\}$. Fill in the missing bits in the equations below. The area of the trapezoid on the subinterval from x_{j-1} to x_j is

$$A_j = \frac{1}{2} [f(\text{???}) + f(\text{???})] (\text{???} - \text{???}).$$

Then the approximation of the integral is

$$\int_a^b f(x) dx \approx \sum_{\text{???}}^{\text{???}} A_j = A.$$

This simplifies to

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(x_0) + f(x_N) + 2 \sum_{\text{???}}^{\text{???}} f(\text{???}) \right].$$

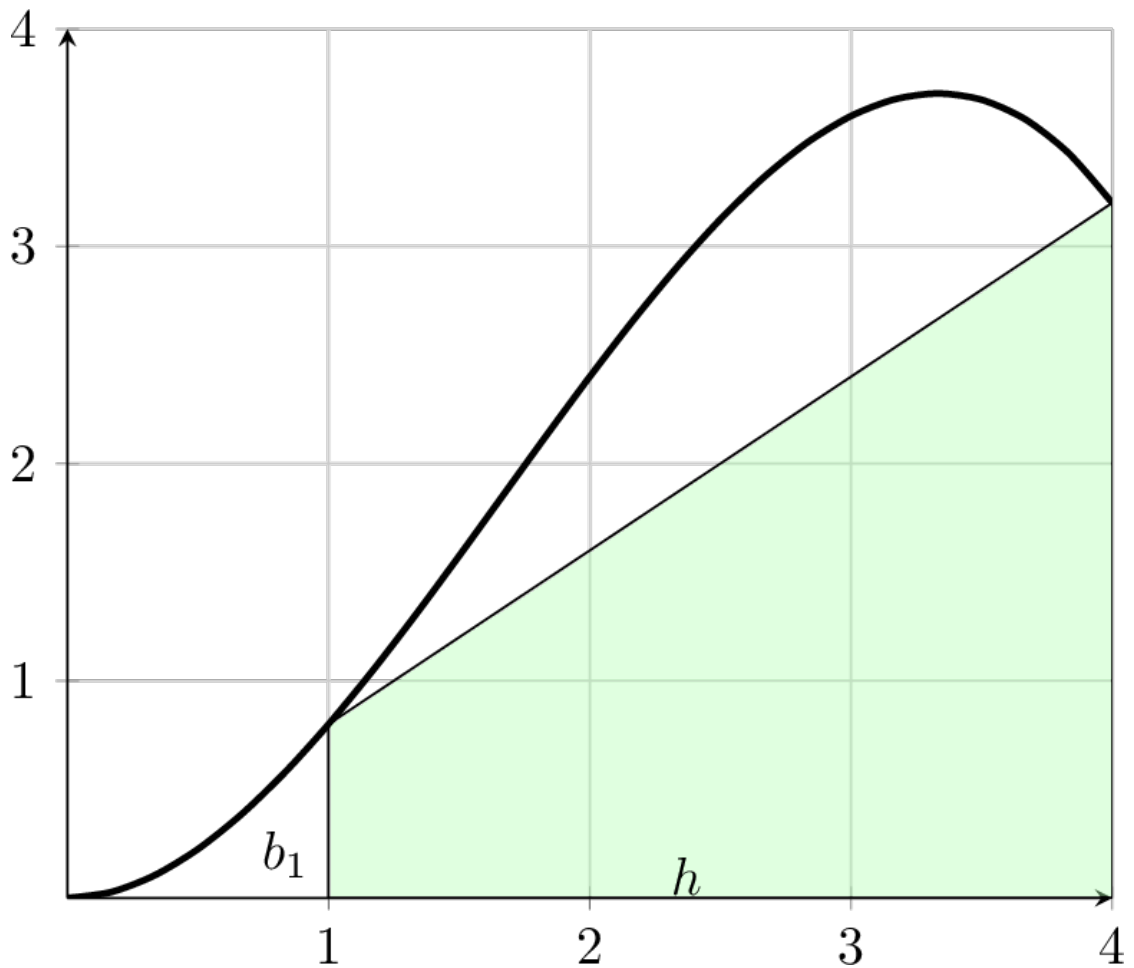


Figure 5.7.: A single trapezoid to approximate area under a curve.

Exercise 5.34. Write a Python function `Trapezoidal(f, a, b, N)` that approximates an integral with the trapezoidal method you derived in the previous exercise. Your Python function should accept a Python function `f`, a lower bound `a`, an upper bound `b` and the number of subintervals `N`. You should write your code without any loops. Test your code on several functions for which you know the integral.

Exercise 5.35. You have by now developed and repeatedly used ways to investigate how the errors for numerical integration and differentiation schemes depend on the step size. It is now up to you to do the same for the trapezoidal rule. The goal is to answer the question:

If I approximate the integral with a fixed Δx and find an absolute error of P , then what will the absolute error be using a width of $\Delta x/M$?

You can either do this with a table as in Exercise D.2 or with a graph as in Exercise D.3. What is the order of the error for the trapezoidal rule?

5.3.3. Simpson's Rule

The trapezoidal rule does a decent job approximating integrals, but ultimately you are using linear functions to approximate $f(x)$ and the accuracy may suffer if the step size is too large or the function too nonlinear. You likely notice that the trapezoidal rule will give an exact answer if you were to integrate a linear or constant function. A potentially better approach would be to get a method that integrates quadratic functions exactly. We call this method Simpson's Rule after Thomas Simpson (1710-1761) who, by the way, was a basket weaver in his day job so he could pay the bills and keep doing math.

Three points are needed to uniquely determine a quadratic function, where two points were enough to uniquely determine a linear function. So for Simpson's method we need to evaluate the function at three points (not two as for the trapezoidal rule). To approximate the integral of a function $f(x)$ on the interval $[a, b]$ we will use the three points $(a, f(a))$, $(m, f(m))$, and $(b, f(b))$ where $m = \frac{a+b}{2}$ is the midpoint of the two boundary points.

We want to find constants A_1 , A_2 , and A_3 in terms of a , b , $f(a)$, $f(b)$, and $f(m)$ such that

$$\int_a^b f(x)dx = A_1f(a) + A_2f(m) + A_3f(b)$$

5. Calculus

is exact for all constant, linear, and quadratic functions. This guarantees that we have an exact integration method for all polynomials of degree 2 or less. In fact, because of the symmetry of the points, the resulting formula is also exact for cubic polynomials, and it should serve as a decent approximation if the function is not a low-degree polynomial.

To make sure that the formula is exact for all constant, linear, and quadratic functions we can plug in $f(x) = 1$, $f(x) = x$, and $f(x) = x^2$ into the equation and solve the resulting linear system of equations for A_1 , A_2 , and A_3 .

$$\begin{aligned}\int_a^b 1dx &= b - a = A_1 + A_2 + A_3, \\ \int_a^b xdx &= \frac{b^2 - a^2}{2} = A_1a + A_2\left(\frac{a+b}{2}\right) + A_3b, \\ \int_a^b x^2dx &= \frac{b^3 - a^3}{3} = A_1a^2 + A_2\left(\frac{a+b}{2}\right)^2 + A_3b^2,\end{aligned}$$

is solved by

$$A_1 = \frac{b-a}{6}, \quad A_2 = \frac{4(b-a)}{6}, \quad \text{and} \quad A_3 = \frac{b-a}{6}.$$

Exercise 5.36. At this point we can see that an integral can be approximated as

$$\int_a^b f(x)dx \approx \left(\frac{b-a}{6}\right) \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right)$$

and the technique will give an exact answer for any polynomial of degree 3 or below.

Verify the previous sentence by integrating $f(x) = 1$, $f(x) = x$, $f(x) = x^2$, and $f(x) = x^3$ by hand on the interval $[0, 1]$ and using the approximation formula.

To make the punchline of the previous exercises a bit more clear: Using the formula

$$\int_a^b f(x)dx \approx \left(\frac{b-a}{6}\right) (f(a) + 4f(m) + f(b))$$

is the same as fitting a parabola to the three points $(a, f(a))$, $(m, f(m))$, and $(b, f(b))$ and finding the area under the parabola exactly. That is exactly the step up from the trapezoidal rule and Riemann sums that we were after:

- Riemann sums approximate the function with constant functions,

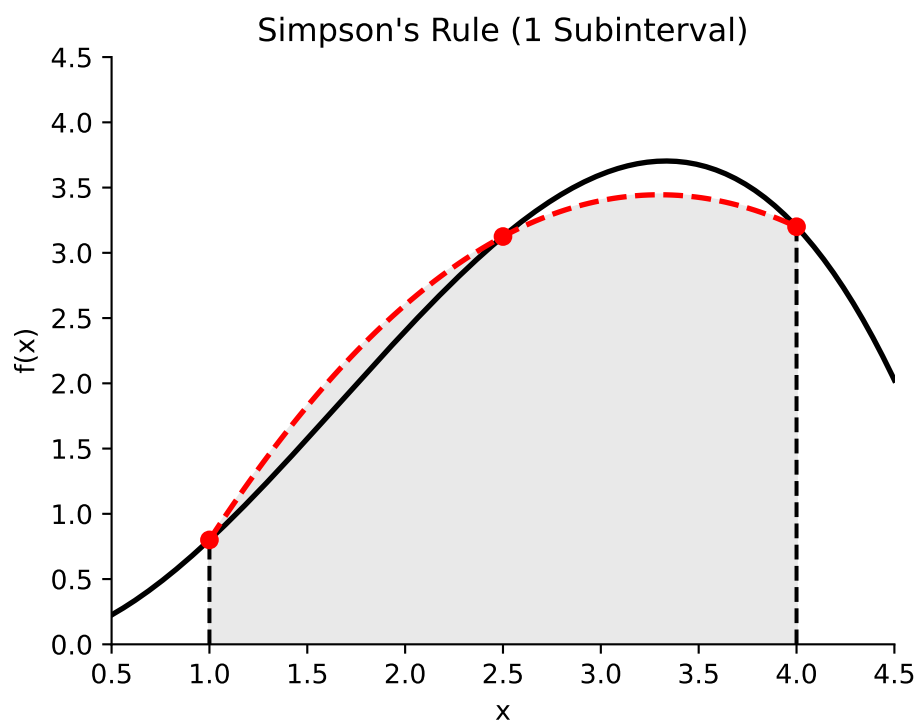


Figure 5.8.: Simpson's rule approximation with a single parabola.

5. Calculus

- the trapezoidal rule uses linear functions, and
- now we have a method for approximating with parabolas.

To improve upon this idea we now examine the problem of partitioning the interval $[a, b]$ into small pieces and running this process on each piece. This is called Simpson's Rule for integration.

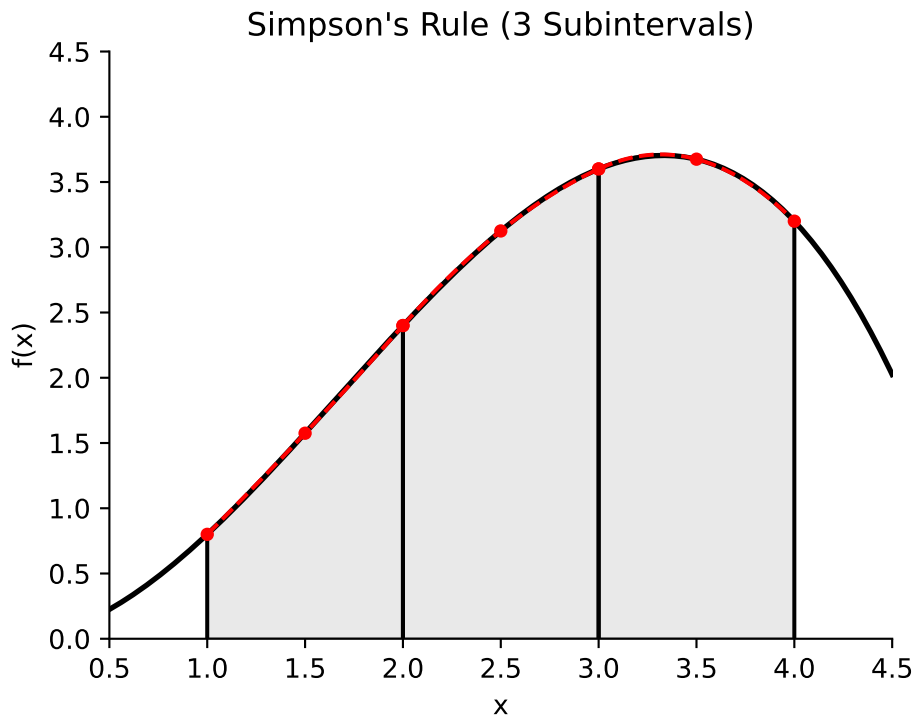


Figure 5.9.: Simpson's rule approximation with three parabolas.

Exercise 5.37 (Simpson's Rule). We divide the interval $[a, b]$ into N subintervals with boundaries $\{x_0 = a, x_1, x_2, \dots, x_{N-1}, x_N = b\}$. Fill in the missing bits in the equations below. We approximate the integral on the subinterval from x_{j-1} to x_j by

$$A_j = \frac{??? - ???}{???} [f(???) + ???f(???) + f(???)].$$

Then the approximation of the integral is

$$\int_a^b f(x)dx \approx \sum_{???}^{???} A_j.$$

This can be simplified to

$$\int_a^b f(x)dx \approx \frac{h}{6} \left[f(x_0) + f(x_N) + 2 \sum_{j=1}^{N-1} f(x_j) + 4 \sum_{j=1}^{N-1} f(x_{j+1/2}) \right].$$

Exercise 5.38. Write a Python function `Simpsons(f, a, b, N)` that approximates an integral with Simpson's rule that you derived in the previous exercise. Your Python function should accept a Python function `f`, a lower bound `a`, an upper bound `b` and the number of subintervals `N`. You should write your code without any loops. Test your code on several functions for which you know the integral.

Exercise 5.39. As in Exercise 5.35, use your favourite method to determine how the absolute error in Simpson's rule depends on the step size and hence determine the order of the error for Simpson's rule?

Exercise 5.40. Use the integration problem and exact answer

$$\int_0^{\pi/4} e^{3x} \sin(2x) dx = \frac{3}{13} e^{3\pi/4} + \frac{2}{13}$$

and produce a log-log error plot with Δx on the horizontal axis and the absolute error on the vertical axis. Include one graph for each of our integration methods. Fully explain how the error rates show themselves in your plot.

Thus far we have three numerical approximations for definite integrals: Riemann sums (with rectangles), the trapezoidal rule, and Simpson's rule. There are many other approximations for integrals and we leave the further research to the curious reader.

Further reading: Sections 4.3 to 4.9 of (Burden and Faires 2010).

5.4. Algorithm Summaries

Exercise 5.41. Starting from Taylor series prove that

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

is a first-order approximation of the first derivative of $f(x)$. Clearly describe what “first-order approximation” means in this context.

Exercise 5.42. Starting from Taylor series prove that

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

is a second-order approximation of the first derivative of $f(x)$. Clearly describe what “second-order approximation” means in this context.

Exercise 5.43. Starting from Taylor series prove that

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

is a second-order approximation of the second derivative of $f(x)$. Clearly describe what “second-order approximation” means in this context.

Exercise 5.44. Explain how to approximate the value of a definite integral with Riemann sums. When will the Riemann sum approximation be exact? Distinguish between left, right and midpoint Riemann sums. State how the error of these approximations depends on the step size, i.e., give the order of the error for each of the three Riemann sums.

Exercise 5.45. Explain how to approximate the value of a definite integral with the trapezoidal rule. When will the trapezoidal rule approximation be exact? What is the order of the trapezoidal rule?

Exercise 5.46. Explain how to approximate the value of a definite integral with Simpson's rule. Give the full mathematical details for where Simpson's rule comes from. When will the Simpson's rule approximation be exact? What is the order of Simpson's rule?

5.5. Truncation Errors

Many of the approximation methods in Numerical Analysis are based on Taylor series expansions and then dropping higher-order terms in the series. The error that is introduced by truncating a Taylor series in this way is called the truncation error. The material in this section will be covered by lectures. You will not need to have read this section before the assessment quiz for this week.

Let us recall Taylor's formula for a function f that is p times differentiable on the interval between x and x_0 :

$$\begin{aligned} f(x) = & f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots \\ & + \frac{f^{(p-1)}(x_0)}{(p-1)!}(x - x_0)^{p-1} + \frac{f^{(p)}(\xi)}{p!}(x - x_0)^p \end{aligned}$$

for some ξ between x and x_0 . The error introduced by truncating the series after the term of degree $p-1$ is thus

$$|R_p(x)| = \left| \frac{f^{(p)}(\xi)}{p!}(x - x_0)^p \right|.$$

We had first discussed this in Chapter 2.

5.5.1. Truncation Error in Finite-Difference Formulae

In the finite-difference formulae for derivatives that we discussed in Section 5.2, we approximated the derivative of a function f at a point x by a linear combination of function values at points close to x . For example, the forward difference formula for the first derivative is

$$\begin{aligned} f'(x) &\approx \frac{f(x+h) - f(x)}{h} \\ &= \frac{f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \cdots - f(x)}{h} \\ &= f'(x) + \frac{h}{2}f''(\xi). \end{aligned}$$

5. Calculus

for some ξ between x and $x + h$. We see that the truncation error is proportional to the step size h . We say that the truncation error is of order h and that this forward-difference approximation is a first-order approximation. This means that in order to halve the size of the error we need to halve the step size.

You can now do a similar analysis for the other finite-difference formulae that we discussed in Section 5.2.

5.5.2. Truncation Error in Numerical Integration

In numerical integration we approximate the integral of a function f over an interval $[a, b]$ by splitting the interval into many subintervals. So we introduce a grid of points $x_j = x_0 + jh$ for $j = 0, \dots, N$ with $x_0 = a$ and $x_N = b$ and $h = (b - a)/N$ and write the integral as a sum

$$\int_a^b f(x)dx = \sum_{j=1}^N \int_{x_{j-1}}^{x_j} f(x)dx.$$

5.5.2.1. Right Riemann Sum

For the right Riemann sum we have

$$I_j = \int_{x_{j-1}}^{x_j} f(x)dx \approx \int_{x_{j-1}}^{x_j} f(x_j)dx = f(x_j)h = A_j.$$

The **local truncation error** is defined as the absolute error made in each subinterval, divided by the width of the subinterval:

$$\tau_j = \frac{1}{h}|I_j - A_j|.$$

For the right Riemann sum we have

$$\begin{aligned} I_j - A_j &= \int_{x_{j-1}}^{x_j} (f(x) - f(x_j)) dx \\ &= \int_{x_{j-1}}^{x_j} (f(x_j) + f'(\xi_j)(x - x_j) - f(x_j)) dx \\ &= f'(\xi_j) \int_{x_{j-1}}^{x_j} (x - x_j) dx \\ &= -f'(\xi_j) \frac{h^2}{2} \end{aligned}$$

for some ξ_j between x_{j-1} and x_j . So the local truncation error is

$$\tau_j = \frac{1}{h} \left| -f'(\xi_j) \frac{h^2}{2} \right| = \frac{h}{2} |f'(\xi_j)|.$$

We see that the local truncation error is of order h and that the right Riemann sum is a first-order approximation. The reason for dividing by h in the definition of the local truncation error is that the power of h in the local truncation error is the same as in the **global truncation error**. The global truncation error τ is obtained by summing over the errors from all subintervals. If $M = \max_{a \leq x \leq b} |f'(x)|$, then

$$\begin{aligned}\tau &= \left| \sum_{j=1}^N (I_j - A_j) \right| \leq \sum_{j=1}^N |I_j - A_j| \\ &\leq \sum_{j=1}^N \frac{h^2}{2} M = N \frac{h^2}{2} M \\ &= \frac{b-a}{2} h M.\end{aligned}$$

5.5.2.2. Midpoint Riemann Sum

For the midpoint Riemann sum we approximate the function on each subinterval by the value of the function at the midpoint $m_j = (x_{j-1} + x_j)/2$. So we have

$$I_j = \int_{x_{j-1}}^{x_j} f(x) dx \approx \int_{x_{j-1}}^{x_j} f(m_j) dx = f(m_j)h = A_j.$$

For the error we expand $f(x)$ in a Taylor series around the midpoint m_j :

$$\begin{aligned}I_j - A_j &= \int_{x_{j-1}}^{x_j} (f(x) - f(m_j)) dx \\ &= \int_{x_{j-1}}^{x_j} \left(f(m_j) + f'(m_j)(x - m_j) + \frac{1}{2}f''(\xi_j)(x - m_j)^2 - f(m_j) \right) dx \\ &= f'(m_j) \int_{x_{j-1}}^{x_j} (x - m_j) dx + \frac{1}{2}f''(\xi_j) \int_{x_{j-1}}^{x_j} (x - m_j)^2 dx\end{aligned}$$

for some ξ_j between x_{j-1} and x_j . Since $\int_{x_{j-1}}^{x_j} (x - m_j) dx = 0$ and $\int_{x_{j-1}}^{x_j} (x - m_j)^2 dx = \frac{h^3}{12}$, we have

$$I_j - A_j = \frac{1}{2}f''(\xi_j)\frac{h^3}{12} = f''(\xi_j)\frac{h^3}{24}.$$

So the local truncation error is

$$\tau_j = \frac{1}{h} \left| f''(\xi_j) \frac{h^3}{24} \right| = \frac{h^2}{24} |f''(\xi_j)|.$$

We see that the local truncation error is of order h^2 and that the midpoint Riemann sum is a second-order approximation.

5. Calculus

5.5.2.3. Trapezoidal Rule

For the trapezoidal rule we approximate the integral on each subinterval by the area of a trapezoid:

$$A_j = \frac{h}{2} (f(x_{j-1}) + f(x_j)).$$

To find the truncation error we expand $f(x)$ in a Taylor series around the midpoint $m_j = (x_{j-1} + x_j)/2$. From our previous calculation for the midpoint Riemann sum, we know the exact value of the integral is given up to the second derivative term by:

$$I_j = \int_{x_{j-1}}^{x_j} f(x) dx \approx f(m_j)h + f''(m_j)\frac{h^3}{24}.$$

For the trapezoidal approximation A_j , we evaluate the Taylor series of $f(x)$ at the endpoints $x_{j-1} = m_j - h/2$ and $x_j = m_j + h/2$:

$$\begin{aligned} A_j &\approx \frac{h}{2} \left[\left(f(m_j) - f'(m_j)\frac{h}{2} + \frac{1}{2}f''(m_j)\frac{h^2}{4} \right) + \left(f(m_j) + f'(m_j)\frac{h}{2} + \frac{1}{2}f''(m_j)\frac{h^2}{4} \right) \right] \\ &= \frac{h}{2} \left[2f(m_j) + f''(m_j)\frac{h^2}{4} \right] \\ &= f(m_j)h + f''(m_j)\frac{h^3}{8}. \end{aligned}$$

Taking the difference gives the error for the subinterval:

$$\begin{aligned} I_j - A_j &\approx \left(f(m_j)h + f''(m_j)\frac{h^3}{24} \right) - \left(f(m_j)h + f''(m_j)\frac{h^3}{8} \right) \\ &= f''(m_j)h^3 \left(\frac{1}{24} - \frac{3}{24} \right) = -f''(m_j)\frac{h^3}{12}. \end{aligned}$$

More rigorously, one can show that the exact local error is

$$I_j - A_j = -\frac{1}{12}f''(\xi_j)h^3$$

for some ξ_j between x_{j-1} and x_j . So the local truncation error is

$$\tau_j = \frac{1}{h} \left| -\frac{h^3}{12}f''(\xi_j) \right| = \frac{h^2}{12}|f''(\xi_j)|.$$

We see that the local truncation error is of order h^2 and that the trapezoidal rule is a second-order approximation.

5.5.2.4. Simpson's Rule

For Simpson's rule we approximate the integral on each subinterval $[x_{j-1}, x_j]$ of width h using a parabola that passes through the endpoints and the midpoint $m_j = (x_{j-1} + x_j)/2$:

$$A_j = \frac{h}{6} (f(x_{j-1}) + 4f(m_j) + f(x_j)).$$

To find the truncation error we expand $f(x)$ in a Taylor series around the midpoint m_j . Because Simpson's rule matches polynomials up to degree 3 exactly, the lower-order error terms will cancel out, so we must expand up to the fourth derivative:

$$f(x) \approx f(m_j) + f'(m_j)(x - m_j) + \frac{1}{2}f''(m_j)(x - m_j)^2 + \frac{1}{6}f'''(m_j)(x - m_j)^3 + \frac{1}{24}f^{(4)}(m_j)(x - m_j)^4.$$

Integrating this expansion over the subinterval gives the integral I_j . The odd powers of $(x - m_j)$ integrate to zero, leaving:

$$I_j = \int_{x_{j-1}}^{x_j} f(x)dx \approx f(m_j)h + f''(m_j)\frac{h^3}{24} + f^{(4)}(m_j)\frac{h^5}{1920}.$$

For the Simpson's approximation A_j , we evaluate the Taylor series of $f(x)$ at the endpoints $x_{j-1} = m_j - h/2$ and $x_j = m_j + h/2$:

$$f(m_j \pm h/2) \approx f(m_j) \pm f'(m_j)\frac{h}{2} + f''(m_j)\frac{h^2}{8} \pm f'''(m_j)\frac{h^3}{48} + f^{(4)}(m_j)\frac{h^4}{384}.$$

Adding these endpoint values gives:

$$f(x_{j-1}) + f(x_j) \approx 2f(m_j) + f''(m_j)\frac{h^2}{4} + f^{(4)}(m_j)\frac{h^4}{192}.$$

Substituting this into our expression for A_j yields:

$$\begin{aligned} A_j &= \frac{h}{6} [4f(m_j) + (f(x_{j-1}) + f(x_j))] \\ &\approx \frac{h}{6} \left[4f(m_j) + 2f(m_j) + f''(m_j)\frac{h^2}{4} + f^{(4)}(m_j)\frac{h^4}{192} \right] \\ &= f(m_j)h + f''(m_j)\frac{h^3}{24} + f^{(4)}(m_j)\frac{h^5}{1152}. \end{aligned}$$

Taking the difference $I_j - A_j$, we see that the $f(m_j)$ and $f''(m_j)$ terms cancel out:

$$\begin{aligned} I_j - A_j &\approx \left(f(m_j)h + f''(m_j)\frac{h^3}{24} + f^{(4)}(m_j)\frac{h^5}{1920} \right) - \left(f(m_j)h + f''(m_j)\frac{h^3}{24} + f^{(4)}(m_j)\frac{h^5}{1152} \right) \\ &= f^{(4)}(m_j)h^5 \left(\frac{1}{1920} - \frac{1}{1152} \right) = -f^{(4)}(m_j)\frac{h^5}{2880}. \end{aligned}$$

5. Calculus

More rigorously, one can show that the exact local error is

$$I_j - A_j = -\frac{1}{2880}f^{(4)}(\xi_j)h^5$$

for some ξ_j between x_{j-1} and x_j . So the local truncation error is

$$\tau_j = \frac{1}{h} \left| -\frac{h^5}{2880}f^{(4)}(\xi_j) \right| = \frac{h^4}{2880}|f^{(4)}(\xi_j)|.$$

We see that the local truncation error is of order h^4 and that Simpson's rule is a fourth-order approximation.

5.6. Exam-Style Question

- (a) Write down the forward difference formula and the central difference formula for the first derivative of $f(x)$ using a step size h . State the order of accuracy for each method. [3 marks]
- (b) By using the Taylor series expansion for $f(x+h)$ and $f(x-h)$ around the point x , derive the central difference formula and show that its truncation error is $\mathcal{O}(h^2)$. [3 marks]
- (c) Suppose you use the central difference scheme with a step size $h = 0.2$ to approximate $f'(2)$ for some function f , and the absolute error in your approximation is 0.016. If you reduce the step size to $h = 0.05$, roughly what would you expect the new absolute error to be? Briefly justify your answer. [2 marks]
- (d) Draw a graph that illustrates how the trapezoidal rule approximates a definite integral. [3 marks]
- (e) What will the trapezoidal rule give as an approximation to the integral $\int_0^5 (5x+3)dx$ when using 5 subintervals? You may use any shortcuts you like to arrive at the answer. [3 marks]
- (f) State the order of accuracy of the trapezoidal rule and explain clearly what this means for how the approximation error changes when the number of subintervals is doubled. [3 marks]
- (g) The following incomplete Python code computes the approximation of an integral using the trapezoidal rule on the interval $[a, b]$ with N subintervals. Provide the missing code indicated by `...`. Do not use loops. [3 marks]

```
import numpy as np

def trapezoidal(f, a, b, N):
    """
    Approximate the integral of f(x) from a to b using the trapezoidal rule.
    """
    # Create vector of equally-spaced x values bounding the N subintervals
    x = ...

    # Calculate width of subintervals
    dx = ...

    # Create vector of approximate integrals for all subintervals
    A = ...

    # Return the sum over the subintervals
    return ...
```

6. Optimisation

It is not enough to do your best; you must know what to do, and then do your best.

–W. Edwards Deming

In applied mathematics we are often not interested in all solutions of a problem but in the optimal solution. Optimisation therefore permeates many areas of mathematics and science. In this section we will look at a few examples of optimisation problems and the numerical methods that can be used to solve them.

While working through this chapter on finding optima you will often be reminded of your work in Chapter 3 and Chapter 4 on finding roots. So I hope this chapter will also consolidate your understanding of those earlier chapters.

6.1. Single Variable Optimisation

Let us start with solving optimisation problems where the quantity to be optimised is a function of a single variable. We'll later optimise over multiple variables in Section 6.2.

Exercise 6.1. Here is an atypically easy optimisation problem that you can quickly do by hand:

A piece of cardboard measuring 20 cm by 20 cm is to be cut so that it can be folded into a box without a lid (see Figure 6.1). We want to find the size of the cut, x , that maximises the volume of the box.

1. Write a function $V(x)$ for the volume of the box resulting from a cut of size x . What is the domain of your function?
2. We know that we want to maximise this function so go through the full Calculus exercise to find the maximum:
 - take the derivative $V'(x)$,
 - set it to zero to find the critical points,

6. Optimisation

- determine the critical point that gives the maximum volume.

Check that your answer is $x = 20/3$.

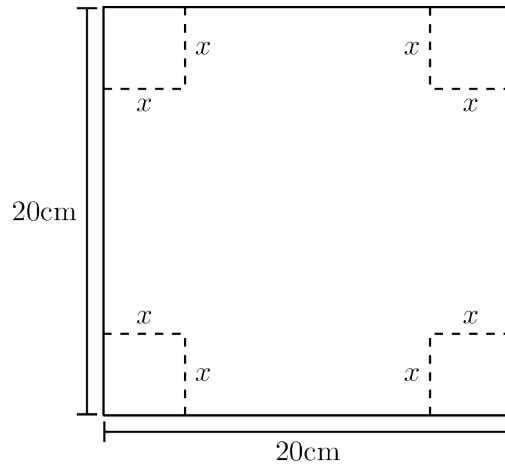


Figure 6.1.: Folds to make a cardboard box

An optimisation problem is approached by first writing the quantity you want to optimise as a function of the parameters of the model. In the previous exercise that was the function $V(x)$ that gives the volume of the box as a function of the parameter x , which was the length of the cut. That function then needs to be maximised (or minimised, depending on what is optimal).

Clearly any method for finding the minimum of a function can be used to find the maximum of a function because the maximum of a function f is exactly at the same point as the minimum of the function $-f$. For concreteness we will usually default to methods for finding the minimum of a function.

In the above example it was easy to find the value of x that maximised the function analytically. However, in many cases it is not so easy. The equation for the parameters that arises from setting the derivatives to zero is usually not solvable analytically. In these cases we need to use numerical methods to find the extremum.

6.1.1. Brute Force Search

Exercise 6.2. Consider the function

$$f(x) = -e^{-x^2} - \sin(x^2)$$

on the domain $0 \leq x \leq 1.5$. The minimum of this function on this domain cannot be determined analytically.

Use Python to make a plot of this function over this domain. You should get something similar to the graph shown in Figure 6.2. What is the x that minimises the function on this domain? What is the x that maximises the function on this domain? You can simply read off the approximate values from the graph, but this requires sampling the function across the interval.

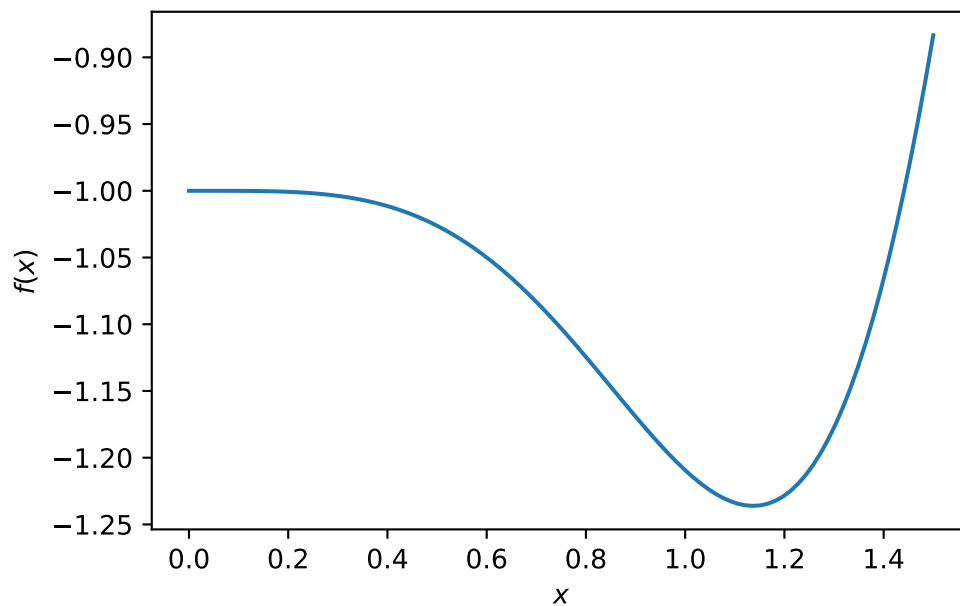


Figure 6.2.: Graph of the function $f(x) = -e^{-x^2} - \sin(x^2)$.

You can also find the minimum directly in Python: simply evaluate the function at many points and choose the smallest value:

```
import numpy as np

f = lambda x: -np.exp(-x**2) - np.sin(x**2)
x = np.linspace(0, 1.5, 1000)
y = f(x)
print(x[np.argmin(y)])
```

6. Optimisation

Check that this agrees with what you obtained visually. How many of the digits returned by the above code should you believe?

This is called a *brute force* search. This method is not very efficient. If evaluating the function is expensive, this is not a good approach. Just think about how often you would need to evaluate the function for the above approach to give the answer to 12 decimal places. As in the case of root finding we want a method that only requires a small number of function evaluations.

The advantage of this brute force method is that it is guaranteed to find the global minimum in the interval. Other, more efficient methods can get stuck in local minima.

6.1.2. Golden Section Search

Here is an idea for a method that is similar to the bisection method for root finding.

In the bisection method we needed a starting interval so that the function values had opposite signs at the endpoints. You were therefore guaranteed that there would be at least one root in that interval. Then you chose a point in the middle of the interval and by looking at the function value at that new point were able to choose an appropriate smaller interval that was still guaranteed to contain a root. By repeating this you homed in on the root.

Unfortunately by just looking at the function values at two points there is no way of knowing whether there is a minimum between them. However, if you were to look at the function values at three points and found that the value at the middle point was less than the values at the endpoints then you would know that there was a minimum between the endpoints.

Exercise 6.3. Make a sketch of a function and choose three points on the function such that the middle point is lower than the two outer points. Use this to illustrate that there must be at least a local minimum between the two outer points.

The idea now is to choose a new point between the two outer points, compare the function value there to those at the previous three points, and then choose a new triplet of points that is guaranteed to contain a minimum. By repeating this process you would hone in on the minimum.

Exercise 6.4. You want to find a minimum of a continuous function f using the golden section search method. You start with the three points $a = 1, c = 3, b = 5$ where the function takes the values $f(1) = 5, f(3) = 2, f(5) = 3$. For the next step you decided to add the point $d = 2.5$ and find that $f(2.5) = 1$. Which three points should you choose to continue the search?

Exercise 6.5. Complete the following function to implement this method. If you have done the previous two exercises you will have a good idea of what to do and this is mostly a coding task. So it would be alright to delegate this to just the group members who enjoy coding the most.

```
def golden_section(f, a, b, c, tol = 1e-12):
    """
    Find an approximation of a local minimum of a function f within the
    interval [a, b] using a bracketing method.

    The function narrows down the interval [a, b] by maintaining a
    triplet (a, c, b) where  $f(c) < f(a)$  and  $f(c) < f(b)$ .
    The process iteratively updates the triplet to home in on the minimum,
    stopping when the interval is smaller than `tol`.

    Parameters:
    f (function): A function to minimise.
    a, b (float): The initial interval bounds where the minimum is to be
                  searched. It is assumed that  $a < b$ .
    c (float): An initial point within the interval (a, b) where
                $f(c) < f(a)$  and  $f(c) < f(b)$ .
    tol (float): The tolerance for the convergence of the algorithm.
                 The function stops when  $b - a < tol$ .

    Returns:
    float: An approximation of a point where f achieves a local minimum.
    """
```

6. Optimisation

```
# Check that the points are ordered a < c < b

# Check that the function value at `c` is lower than at both `a` and `b`

# Loop until you have an interval smaller than the tolerance
while b-a >= tol:

    # Choose a new point `d` between `a` and `b`
    # Think about what is the most efficient choice

    # Compare f(d) with f(c) and use the result
    # to choose a new triplet `a`, `b`, `c` in such a way that
    # b-a has decreased but f(c) is still lower than both f(a) and f(b)

    # While debugging, include a print statement to let you know what
    # is happening within your loop

return c
```

Then try out your code on the function

$$f(x) = -e^{-x^2} - \sin(x^2) \quad (6.1)$$

from Figure 6.2 to see if it can find the local minimum near $x \approx 1.14$.

The reason the method is called the golden section search is that, by a clever choice of the interior point, the size of the interval is reduced by the golden ratio at each step. This is the ratio that satisfies the equation

$$\frac{\text{longer subinterval}}{\text{shorter subinterval}} = \frac{\text{whole interval}}{\text{longer subinterval}} = \phi \approx 1.618 \quad (6.2)$$

This is the second time the golden ratio has shown up unexpectedly in this module (see Section 4.3.4). One simply has to love Maths.

So the golden section method converges a little slower than bisection method, because in the latter the interval is reduced by a factor of 2 at each step. And both methods converge only linearly.

Besides the slow convergence, the main shortcoming of the golden section search method is that it works only for functions of a single argument.

The intuition behind more efficient numerical optimisation schemes is typically to visualise the function as representing a landscape on which you are trying to walk or jump to the highest or lowest point. You can, however, only sense your immediate neighbourhood and need to use that information to make decisions about where to jump or walk to next.

Exercise 6.6. If you were blindfolded and standing on a hillside, could you find the top of the hill? (assume no trees and no cliffs ...this is not supposed to be dangerous) How would you do it? Explain your technique clearly.

Exercise 6.7. If you were blindfolded and standing in a crater on the moon could you find the lowest point? How would you do it? Remember that you can hop as far as you like ... because gravity ... but sometimes that's not a great thing because you could hop too far.

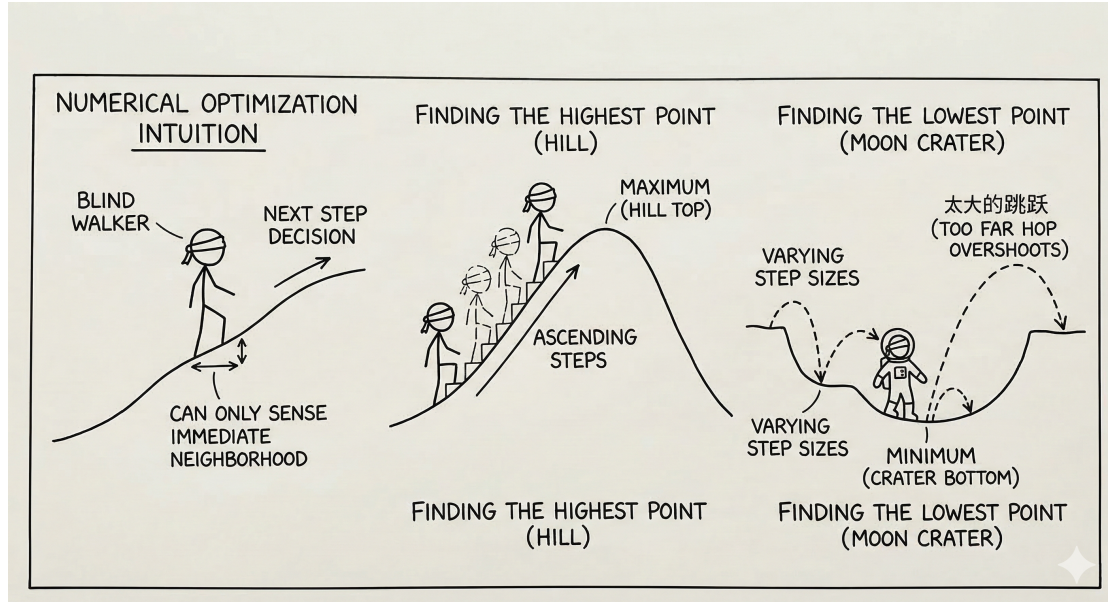


Figure 6.3.: Intuition behind numerical optimisation (Visualised by Gemini)

6.1.3. Gradient Descent

Let us explore the intuitive method of simply taking steps in the downhill direction. That should eventually bring us to a local minimum. The problem is only to know how to choose the step size and the direction. The gradient descent method is a simple and effective way to do this. By making the step size proportional to the negative gradient of the function we are moving in a local downhill direction and also automatically reducing the step size as we get closer to a minimum where the gradient gets smaller.

Let $f(x)$ be the objective function which you are seeking to minimise.

- Find the derivative of your objective function, $f'(x)$.
- Pick a starting point, x_0 .
- Pick a small control parameter, α (in machine learning this parameter is called the “learning rate” for the gradient descent algorithm).
- Use the iteration $x_{n+1} = x_n - \alpha f'(x_n)$.
- Iterate (decide on a good stopping rule).

Exercise 6.8. What is the Gradient Descent algorithm doing geometrically? Draw a picture and be prepared to explain to your peers. Discuss what would be a good stopping rule.

Exercise 6.9. Write code to implement the 1D gradient descent algorithm. Note that gradient descent is a fixed-point iteration method. So you can adapt the code for Newton’s method from Exercise 4.7. Just be sure to change the iteration formula.

```
def gradient_descent_1d(df, x0, alpha,
                       tol = 1e-12, max_iter=1000):
    """
    Find an approximation of a local minimum of a function f
    using the gradient descent method.

    The function iteratively updates the current guess `x0`
    by stepping in the direction of the negative gradient
    of `f` at `x0` multiplied by `alpha`.
    The process stops when the step taken is smaller than `tol`.
```

```

Parameters:
df (function): The derivative of the function you
                want to minimise.
x0 (float): The initial guess for the minimum.
alpha (float): The learning rate multiplies the
                gradient to give the step size.
tol (float): The tolerance for the convergence.
max_iter (int): The maximum number of iterations to perform.

Returns:
float: An approximation of a point where f achieves
       a local minimum.
"""
# Initialize `x` with the starting value

# Loop for a maximum of `max_iter` iterations

    # Calculate the next approximation `x_new`

    # If the step taken was smaller than `tol` then
    # return `x_new`

    # Update `x = x_new`

# If the loop finishes without returning then
#     return a message that method did not converge

```

Use your function to solve Exercise 6.1. Compare your answer to the analytic solution. Also test your function to find the minimum of the function in Figure 6.2

Exercise 6.10. Use your gradient descent function to find the minimum of the function

$$f(x) = (\sin(4x) + 1)((x - 5)^2 - 25)$$

on the domain $0 \leq x \leq 8$. Figure 6.4 shows a graph of the function with its multiple local minima.

Check that the derivative is

$$f'(x) = 4 \cos(4x)((x - 5)^2 - 25) + 2(x - 5)(\sin(4x) + 1)$$

6. Optimisation

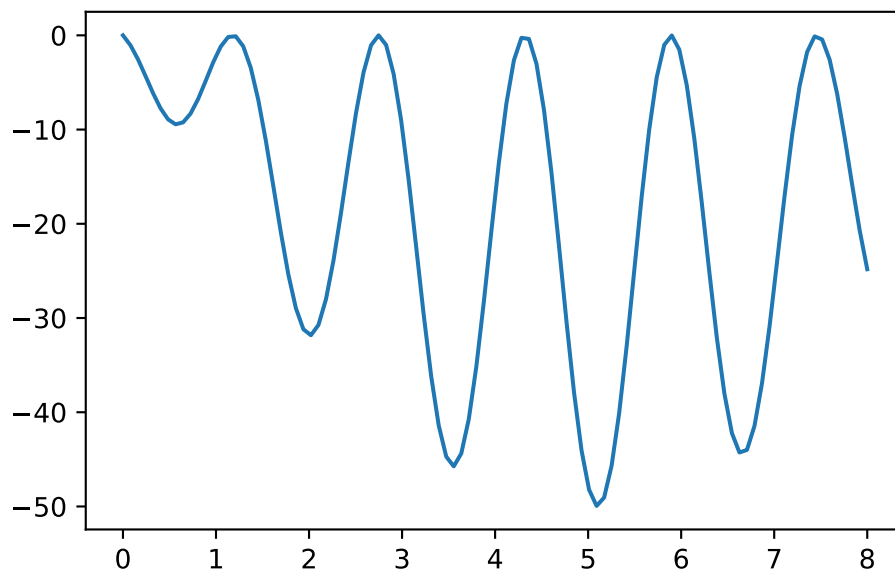


Figure 6.4.: Graph of the function $f(x) = (\sin(4x) + 1)((x - 5)^2 - 25)$.

If you choose $x_0 = 3$ as your starting point and a learning rate of 0.001, what approximation do you get for x at the minimum? Did gradient descent find the global minimum?

Exercise 6.11. Experiment with different values of the learning rate for the previous question, assuming that you can only specify it with up to 3 digits after the decimal point. Which choice of learning rate requires the smallest number of steps to reach the required tolerance of 10^{-12} ?

Exercise 6.12. You will now explore how the error decreases as the number of gradient descent steps increases.

1. Write a Python function `gradient_descent_with_error_tracking()` that returns a list of absolute errors between the iterates and the exact minimum. You can adapt your `newton_with_error_tracking()` function from Exercise 4.14.
2. Use this to print the errors when applied to the function $f(x) = \cos(x)$ with a starting value of $x_0 = 3$, a learning rate of $\alpha = 0.1$ and a tolerance of 10^{-12} . Make sure you use the correct exact minimum in your error calculations, namely the one that you expect gradient descent to find.

3. Use your `plot_error_progression()` function that we wrote in Example 3.2 to make a plot of the log of the absolute error at iteration $n + 1$ against the log of the absolute error at iteration n . You should observe that the points line up approximately along a straight line.
4. Read off the slope of the straight line fitted to the points.
5. What does this slope tell you about how the error ϵ_{n+1} is related to the error at the previous iteration ϵ_n ?

$$\epsilon_{n+1} \approx ???\epsilon_n^{??}$$

6. Experiment with other starting points, other learning rates, other functions.

6.1.4. Order of Convergence of Gradient Descent

You will by now have observed in Exercise 6.12 that the error progression plots for gradient descent show a straight line with slope approximately equal to 1 in the log-log plot of E_{n+1} against E_n . This means that gradient descent has **linear** convergence. In this section we consolidate your observations theoretically. As in Chapter 4, this section is presented as a lecture rather than containing further explorations.

Recall from Definition 4.1 that a convergent sequence $\{x_n\}$ has **order of convergence** α if there exists a constant $\lambda > 0$ such that

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n^\alpha} = \lambda,$$

where $E_n = |x_n - p|$ is the absolute error. Convergence with $\alpha = 1$ is called *linear* and requires $0 < \lambda < 1$; convergence with $\alpha = 2$ is called *quadratic*.

The gradient descent iteration for minimising a function $f(x)$ is

$$x_{n+1} = x_n - \alpha f'(x_n).$$

This is a fixed-point iteration $x_{n+1} = g(x_n)$ with

$$g(x) = x - \alpha f'(x). \tag{6.3}$$

The fixed points of g are the stationary points of f , i.e., the points p where $f'(p) = 0$.

We can now apply the theory of the order of convergence of fixed-point iteration from Chapter 4. We compute

$$g'(x) = 1 - \alpha f''(x) \quad \Rightarrow \quad g'(p) = 1 - \alpha f''(p). \tag{6.4}$$

6. Optimisation

If p is a local minimum then $f''(p) > 0$. In general $g'(p) \neq 0$, so it follows from Theorem 4.1 (with $m = 1$, i.e., using Eq. 4.5) that the gradient descent method produces a **linearly convergent** sequence with

$$\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n} = |1 - \alpha f''(p)|. \quad (6.5)$$

For this to converge at all we need $|1 - \alpha f''(p)| < 1$, which requires $0 < \alpha < \frac{2}{f''(p)}$. This explains why the learning rate α must not be too large — if it is, the method diverges.

The convergence is fastest when $|1 - \alpha f''(p)|$ is as small as possible, ideally zero. This happens when $\alpha = \frac{1}{f''(p)}$, corresponding to the “optimal” learning rate. However, we usually do not know $f''(p)$ in advance (since we do not know p), so we cannot simply choose this optimal value.

Notice that when $\alpha = \frac{1}{f''(p)}$ we have $g'(p) = 0$. In this case, the linear term vanishes and we can apply Theorem 4.1 to determine whether higher-order convergence is achieved. We compute

$$g''(x) = -\alpha f'''(x) \quad \Rightarrow \quad g''(p) = -\alpha f'''(p).$$

If $f'''(p) \neq 0$, then $g''(p) \neq 0$ and the convergence is quadratic. This mirrors the situation with Newton’s method for root-finding, where $g'(p) = 0$ also leads to quadratic convergence.

Note the interesting parallel: Newton’s method applied to $f'(x) = 0$ gives the iteration

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)},$$

which is precisely gradient descent with the “learning rate” $\alpha = \frac{1}{f''(x_n)}$ adapted at each step. This explains why Newton’s method for optimisation can converge quadratically while gradient descent with a fixed learning rate generically converges only linearly.

6.2. Multivariable Optimisation

Now let us look at multivariable optimisation.

Example 6.1. Here is a two-variable example: Find the minimum of the function

$$f(x, y) = \sin(x) \exp(-\sqrt{x^2 + y^2})$$

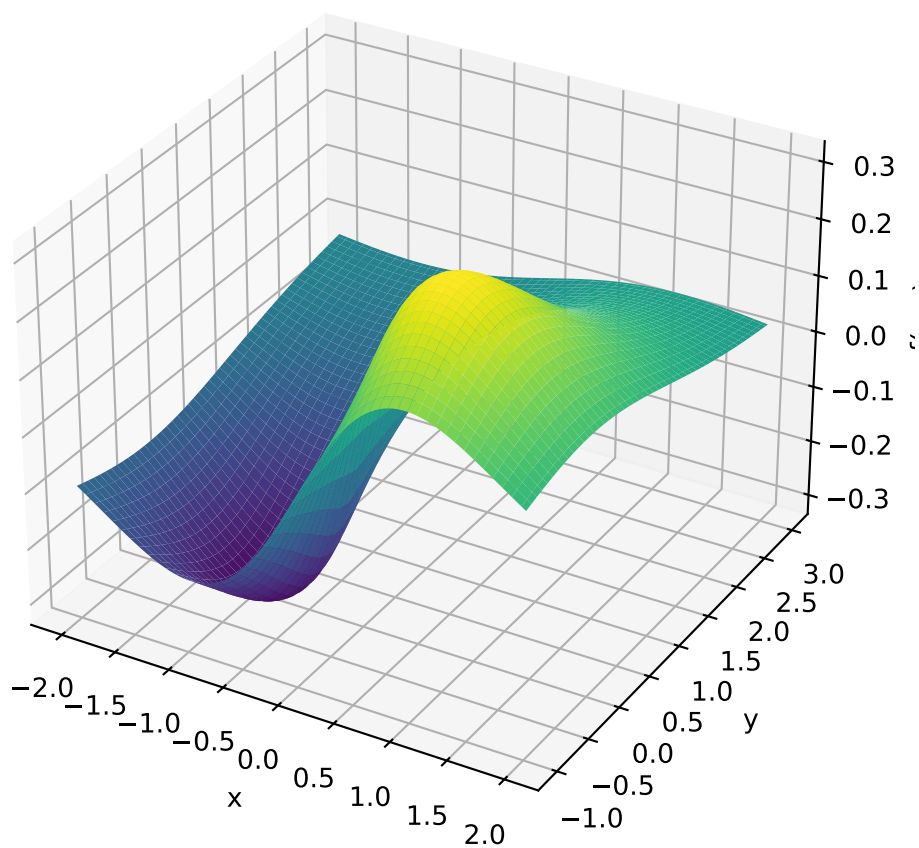


Figure 6.5.: Graph of the function $\sin(x) \exp(-\sqrt{x^2 + y^2})$.

6. Optimisation

Finding the minima of multivariable functions is a bit more complicated than finding the minima of single-variable functions. The reason is that there are many more directions in which to move. But the basic intuition that we want to move downhill to move towards a minimum of course still works. The gradient descent method is still a good choice for finding the minimum of a multivariable function. The only difference is that the gradient is now a vector and the step is in the direction of the negative gradient.

Exercise 6.13. In your group, answer each of the following questions to remind yourselves of multivariable calculus.

1. What is a partial derivative (explain geometrically). For the function $f(x, y) = \sin(x) \exp(-\sqrt{x^2 + y^2})$ what is $\frac{\partial f}{\partial x}$ and what is $\frac{\partial f}{\partial y}$? (When writing the answer you need to specify the value of the partial derivatives at $(x, y) = (0, 0)$ separately.)
 2. What is the gradient of a function? What does it tell us physically or geometrically? If $f(x, y) = \sin(x) \exp(-\sqrt{x^2 + y^2})$ then what is ∇f ?
-

But we could have a lot more than two variables x and y . If we have many variables, we quickly run out of letters. So we change notation a bit and call our variables $x_0, x_1, \dots, x_{k-1}$. We want to find the minimum of a function $f(x_0, x_1, \dots, x_{k-1})$. Such higher-dimensional problems are very common and the dimension k can be very large in practical problems. A good example is the loss function of a neural network which is a function of the weights and biases of the network. In a large language model the loss function is a function of many billions of variables and the training of the model is a large optimisation problem.

Note that when we write $f(x_0, x_1, \dots, x_{k-1})$, the indices on the x distinguish the different variables. Do not confuse that with our use of the index in Chapter 5 for the grid points when discretising a single variable.

We can now combine all the arguments into a vector $x = (x_0, x_1, \dots, x_{k-1})$.

Example 6.2. For the function $f(x, y) = \sin(x) \exp(-\sqrt{x^2 + y^2})$ here is a Python function that takes a NumPy array with two elements with values for x and y as input and returns a NumPy array with two elements containing the values of the gradient.

6.2.1. Gradient Descent Algorithm

We want to find the minimum of a multivariable function $f(x) = f(x_0, x_1, \dots, x_{k-1})$. We proceed as follows:

1. Choose an arbitrary starting point $x_0 = (x_{0,0}, x_{0,1}, \dots, x_{0,k-1})$. Note the potential notational confusion. The index on x is the iteration number, not to be confused with the indices of the components of the vector.
2. Use this iteration equation to calculate a new guess for the optimal value:

$$x_{n+1} = x_n - \alpha \nabla f(x_n).$$

This iteration equation says to follow the negative gradient a certain distance from your present point (why are we doing this?). Note that choosing the value of α is up to you, so experiment with a few values.

3. Repeat the iterative process in step 2 until two successive points are *close enough* to each other, in the sense that their Euclidean distance is smaller than a given tolerance `tol`.

$$\|x_{n+1} - x_n\|_2 = \sqrt{\sum_{i=0}^{k-1} (x_{n+1,i} - x_{n,i})^2} < \text{tol}$$

Exercise 6.14. Write code to implement the gradient descent algorithm for a function $f(x)$.

```
def gradient_descent(df, x0, alpha, tol=1e-12, max_iter=1000):
    """
    Finds an approximation of a local minimum of a multivariable function using
    gradient descent.

    Parameters:
    df (function): A function that returns the gradient of the function to
                   minimise. It should take a NumPy array with k elements
                   as input and return a NumPy array with k elements
                   representing the gradient.
    x0 (NumPy array): The initial guess for the minimum
                      (a NumPy array with k elements).
    alpha (float): The learning rate (step size multiplier).
    tol (float): Tolerance for convergence
                 (stops when the magnitude of the step is below this).
    max_iter (int): Maximum number of iterations.

    Returns:
    NumPy array: The approximated minimum point
                 (a NumPy array with k elements).
    """

    # Here comes your code.
```

6. Optimisation

You can build on your code for the single-variable gradient descent from Exercise 6.9. To calculate the Euclidean distance between two points x and y you can use `np.linalg.norm(x-y)`.

Use your function to find the minimum of the function

$$f(x, y) = \sin(x) \exp(-\sqrt{x^2 + y^2}).$$

with a starting point $(x_0, y_0) = (-1, 1)$, a learning rate of 1 and a tolerance of 10^{-6} . You can use your gradient function from Example 6.2.

Exercise 6.15. The gradient descent method works well for some functions but struggles with others. In this exercise you will explore two functions that gradient descent struggles with.

1. Modify your `gradient_descent` function from Exercise 6.14 to create a new function `gradient_descent_path` that returns the list of all iterates, not just the final one.
2. Consider the function

$$f(x, y) = x^2 + 100y^2.$$

This function has a clear minimum at the origin but is much steeper in the y direction than in the x direction. Try the starting point $(x_0, y_0) = (1, 1)$ and experiment with different learning rates. The code below will get you started. Can you find a learning rate that gives fast convergence in both variables simultaneously?

3. Use the `plot_path()` function defined below to make a plot of the function and the path of the gradient descent iterates on top of it. Discuss with your group why gradient descent struggles with this function. Discuss why having very different curvatures along different directions makes gradient descent inefficient.
4. In the previous example one could avoid the problem by simply rescaling the y variable. However, for some functions this is not possible. Consider Rosenbrock's banana function:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

Choose a starting point $(x_0, y_0) = (-1, 1)$. The code below sets this up for you. The global minimum is at $(1, 1)$ where $f(1, 1) = 0$. Try several different learning rates. You will find that small learning rates lead to extremely slow convergence and large learning rates lead to divergence. Can you find a learning rate that converges in a reasonable number of iterations?

To use the gradient descent algorithm one first needs to work out the gradient function. Doing this by hand can be annoying when the function is very complicated. A good alternative then is to use automatic differentiation as described in Appendix D.

Of course there are many other methods for finding the minimum of a multivariable function. An important method that does not need the gradient of the function is the Nelder-Mead method. This method is a direct search method that only needs the function values at the points it is evaluating. The method is very robust and is often used when the gradient of the function is too difficult to calculate. There are also clever variants of the gradient descent method that are more efficient than the basic method. The Adam and RMSprop algorithms are two such methods that are used in machine learning. This subject is a large and active area of research and we will not go into more detail here.

6.3. Machine Learning

A very important application of optimisation is in machine learning, where one wants to learn from data. Of course we won't go into the details of machine learning here, it is a huge topic. We will just introduce a very simple learning task:

If we have some data points and a reasonable guess for the type of function fitting the points, how would we determine the actual function?

You may recognise this as the basic question of nonlinear regression from statistics. But it captures the basic idea behind supervised learning.

What we will do here is pose this machine learning problem as an optimisation problem. Then we will use the tools that we have built so far to solve the optimisation problem.

Exercise 6.16. Consider the function $f(x)$ that goes exactly through the points $(0, 2)$, $(1, 4)$, and $(2, 12)$.

1. Find a function that goes through these points exactly. Be able to defend your work.
 2. Is your function unique? That is to say, is there another function out there that also goes exactly through these points?
-

6. Optimisation

Exercise 6.17. Now let us make a minor tweak to the previous exercise. Let us say that we have the data points $(0, 2.37)$, $(1, 4.14)$, $(2, 12.22)$, and $(3, 23.68)$. Notice that these points are *close* to the points we had in the previous exercise, but all of the y values have a little noise in them and we have added a fourth point. If we suspect that a function $f(x)$ that *best* fits this data is quadratic then $f(x) = ax^2 + bx + c$ for some constants a , b , and c .

1. Work with your group to choose a , b , and c so that you get a good visual match to the data. The Python code below will help you get started.

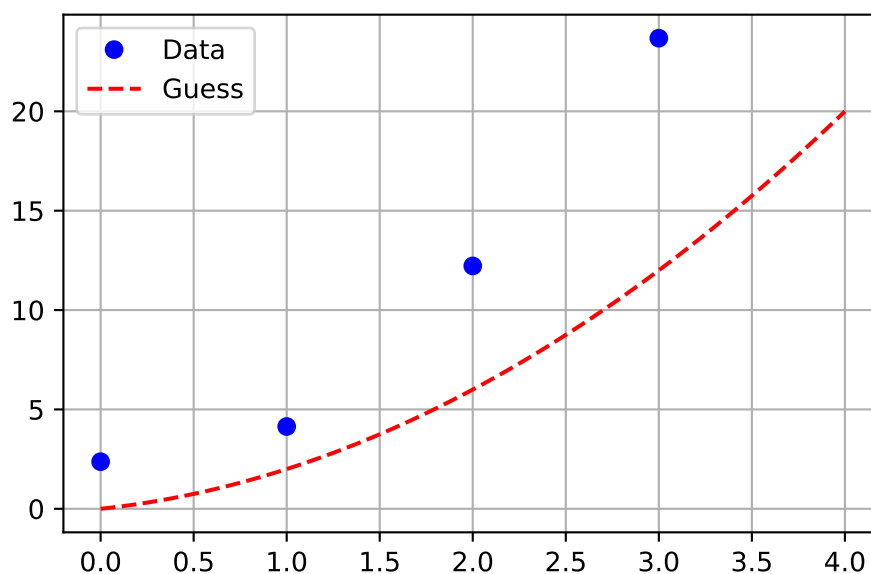


Figure 6.6.: Initial attempt at matching data with a quadratic.

Exercise 6.18. Now let us be a bit more systematic about things from the previous exercise. Let us say that you have a pretty good guess that $b \approx 2$ and $c \approx 0.7$. We need to get a good estimate for a .

1. Pick an arbitrary starting value for a then for each of the four points find the error between the predicted y value and the actual y value. These errors are called the *residuals*.
2. Square all four of your errors and add them up. (Pause, ponder, and discuss: why are we squaring the errors before we sum them?)
3. Now change your value of a to several different values and record the sum of the squared errors for each of your values of a .

4. Make a plot with the value of a on the horizontal axis and the value of the sum of the squared errors on the vertical axis. Use your plot to defend the optimal choice for a .

Clearly the above is calling for some Python code to automate this exploration. Write a loop that tries many values of a in very small increments and calculates the sum of the squared errors. The following partial Python code should help you get started. In the resulting plot you should see a clear local minimum. What does that minimum tell you about solving this problem?

```
import numpy as np
import matplotlib.pyplot as plt
xdata = np.array([0, 1, 2, 3])
ydata = np.array([2.37, 4.14, 12.22, 23.68])
b = 2
c = 0.75
A = # give a numpy array of values for a
SumSqRes = [] # this is storage for the sum of the sq. residuals
for a in A:
    guess = a*xdata**2 + b*xdata + c
    residuals = # write code to calculate the residuals
    SumSqRes.append( ??? ) # calculate the sum of the squ. residuals
plt.plot(A, SumSqRes)
plt.grid()
plt.xlabel('Value of a')
plt.ylabel('Sum of squared residuals')
plt.show()
```

Now let us formalise the process that we have described in the previous exercises.

Least Squares Regression

Let

$$S = \{(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)\} \quad (6.6)$$

be a set of $n + 1$ ordered pairs in $\mathbb{R}^2$. If we guess that a function $f(x)$ is a best choice to fit the data and if $f(x)$ depends on parameters $a_0, a_1, \dots, a_m$ then

1. Pick initial values for the parameters $a_0, a_1, \dots, a_m$ so that the function $f(x)$ *looks like* it is close to the data (this is strictly a visual step ...take care that it may take some playing around to guess the initial values of the parameters)

6. Optimisation

2. Calculate the squared error between each data point and the prediction from the function $f(x)$

$$\text{error for the point } x_i: e_i = (y_i - f(x_i))^2. \quad (6.7)$$

Note that squaring the error has the advantages of removing the sign, accentuating errors larger than 1, and decreasing errors that are less than 1.

3. As a measure of the total error between the function and the data, sum the squared errors

$$\text{sum of squared errors} = \sum_{i=0}^n (y_i - f(x_i))^2 =: L(a). \quad (6.8)$$

This function $L(a)$ is called the *loss function*.

4. Change the parameters $a_0, a_1, \dots$ so as to minimise the loss function $L(a)$.

Exercise 6.19. The last step above is a bit vague. That was purposeful since there are many techniques that could be used to minimise the sum of the squared errors. However, if we just think about the sum of the squared residuals as a function then we can apply `scipy.optimize.minimize()` to that function in order to return the values of the parameters that best minimise the sum of the squared residuals. The following blocks of Python code implement the idea in a very streamlined way. Go through the code and comment each line to describe exactly what it does.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
xdata = np.array([0, 1, 2, 3])
ydata = np.array([2.37, 4.14, 12.22, 23.68])

def SSRes(parameters):
    # In the next line of code we want to build our
    # quadratic approximation y = ax^2 + bx + c
    # We are sending in a list of parameters so
    # a = parameters[0], b = parameters[1], and c = parameters[2]
    yapprox = parameters[0]*xdata**2 + \
               parameters[1]*xdata + \
               parameters[2]
    residuals = np.abs(ydata-yapprox)
    return np.sum(residuals**2)

BestParameters = minimize(SSRes, [1,1,1])
```

```
print("The best values of a, b, and c are: \n",BestParameters.x)
# If you want to print the diagnostics then use the line below:
# print("The minimisation diagnostics are: \n",BestParameters)
```

```
plt.plot(xdata,ydata,'bo')
x = np.linspace(0,4,100)
y = BestParameters.x[0]*x**2 + \
    BestParameters.x[1]*x + \
    BestParameters.x[2]
plt.plot(x,y,'r--')
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
plt.title('Best Fit Quadratic')
plt.show()
```

Exercise 6.20. Now I'll let you in on a little secret: the data that you have fitted the quadratic function to above was not real data. I created it with the following script.

```
[0.  1.  2.  3.]
[ 2.37  4.14 12.22 23.68]
```

Such fake data that is generated from a known function with a known error distribution is referred to as “synthetic data” and is very useful for testing parameter fitting methods.

You will notice that the parameters that we estimated with the least squares method are not at all the same as the parameters that I used to generate the data. This is because we had only 4 noisy data points to estimate 3 parameters.

Adjust the above script so that it generates 100 values for x between 0 and 3 and corresponding noisy y values. Then try to estimate the parameters of the quadratic function that generated the data. Does the estimate get closer to the original parameter values?

Least squares fitting to data is of course just a very simple example of machine learning. But much of machine learning is based on the same idea:

6. Optimisation

1. Choose a family of functions $f(x)$ that you think might be able to describe the data. Often this is a deep neural network as introduced in Section 2.4, sometimes with billions of parameters.
2. Choose a loss function that measures how well your function $f(x)$ describes the data. The sum of squared residuals is a common choice.
3. Implement a fast way to calculate the gradient of the loss function with respect to the parameters of $f(x)$. This is often done with automatic differentiation, introduced in Appendix D. In the case of deep neural networks this is known as backpropagation.
4. Use a gradient descent algorithm to find the parameters of $f(x)$ that minimise the loss function. This is usually done with stochastic gradient descent, where one uses random subsets of the data to calculate the gradient of the loss function.

This is a large and active area of research and we will not go into more detail here.

6.4. Algorithm Summaries

Exercise 6.21. Explain in clear language how the Golden Section Search method works.

Exercise 6.22. Explain in clear language how the Gradient Descent method works.

Exercise 6.23. Explain in clear language how you use the method of Least Squares to fit a function to data.

6.5. Exam-Style Question

- (a) Briefly explain the logic behind the Golden Section Search method for finding a local minimum of a continuous function $f(x)$. What is the necessary condition for a triplet of points (a, c, b) with $a < c < b$ to guarantee that a minimum exists within the interval $[a, b]$? [3 marks]
- (b) When using the Gradient Descent method to find the minimum of a single-variable function $f(x)$, the iteration formula is $x_{n+1} = x_n - \alpha f'(x_n)$. Explain the role of the parameter α (the learning rate). What happens if α is chosen to be too large, and what happens if α is chosen to be too small? [3 marks]
- (c) Consider finding the minimum of a multivariable function $f(x, y)$. Write down the iteration formula for the Gradient Descent method in this case. Explain geometrically why moving in the direction of the negative gradient is an effective strategy for finding a minimum point. [3 marks]
- (d) The following incomplete Python code implements the Gradient Descent method for a single-variable function. Provide the missing code indicated by `....`. Assume `df` is the derivative function, `x0` is the starting guess, `alpha` is α , and `tol` is the tolerance for stopping when the gradient is sufficiently close to zero. [2 marks]

```
def gradient_descent_1d(df, x0, alpha, tol=1e-12, max_iter=10000):
    x = x0
    for i in range(max_iter):
        grad = ...
        if abs(grad) < tol:
            return x
        x = ...
    return x
```

7. ODE 1

The mathematical discipline of differential equations furnishes the explanation of all those elementary manifestations of nature which involve time.

–Norwegian Mathematician Sophus Lie

The topic of this chapter is to find *approximate solutions* to *ordinary differential equations*.

Let us briefly recall what an ordinary differential equation (ODE) is. A rather arbitrarily chosen example for an ODE (here, of second order) is

$$y''(x) + 4y'(x) + \sqrt[3]{y(x)} + \cos(x) = 0. \quad (7.1)$$

Equations like this are normally satisfied by many functions $y(x)$: the problem has many solutions. In order to specify a uniquely solvable problem, one needs to fix *initial values*, i.e., the value of y and its first derivative at some point, say, at $x = 0$:

$$y''(x) + 4y'(x) + \sqrt[3]{y(x)} + \cos(x) = 0, \quad y(0) = 1, \quad y'(0) = -2. \quad (7.2)$$

This is a so-called *initial-value problem* (IVP). Another variant is to specify the value of $y(x)$, but not of its derivative, at two different points:

$$y''(x) + 4y'(x) + \sqrt[3]{y(x)} + \cos(x) = 0, \quad y(0) = 2, \quad y(1) = 1. \quad (7.3)$$

This is called a *boundary value problem* (BVP).

Both IVPs and BVPs have a unique solution (under certain mathematical conditions). However, while one can show on abstract grounds that these solutions exist, it is often not practicable to find an explicit expression for them. The best one can hope for is to approximate the solution numerically.

So what *is* a numerical solution to a differential equation?

When solving a differential equation with analytic techniques the goal is to find an expression for the solution in terms of known functions. In a numerical solution the goal is typically to divide the domain for the solution function into a fine partition by introducing a grid of points, just like we did with numerical differentiation and integration, and then to approximate the solution to the differential equation at each point in that partition. Hence, the end result will be a list of approximate solution values at the grid points.

7. ODE 1

In this chapter we will examine some of the more common ways to create approximations of solutions to initial value problems. Moreover, we will lean heavily on Taylor Series to give us ways to accurately measure the order of the errors that we make in the process.

In this chapter we will often think of the argument of the function described by the ODE as time, but of course the methods are agnostic to the interpretation of the independent variable.

We will first concentrate on first-order differential equations of the form $x' = f(t, x)$. The geometric interpretation of this is that we are looking for a function $x(t)$ whose slope at every point (t, x) is known to us, given by $f(t, x)$.

7.1. Euler's Method

Exercise 7.1. Consider the differential equation $x' = -0.5x$ with the initial condition $x(0) = 6$.

- a. Since we know that $x(0) = 6$ and we know that $x'(0) = -0.5x(0)$ we can approximate the value of x at some future time step. Let us go 1 unit forward in time. That is, approximate $x(1)$ knowing that $x(0) = 6$ and $x'(0) = -3$.
Hint: We know a value, a slope, and the size of the step that we would like to move in the t direction.

$$x(1) \approx \underline{\hspace{2cm}} \quad (7.4)$$

- b. Use your answer from part (a) for time $t = 1$ to approximate the x value at time $t = 2$. Then use that value to approximate the value at time $t = 3$. Repeat the process to approximate the value of x at times $t = 2, 3, 4$. Record your answers in the table below. Then find the analytic solution to this differential equation and record the x values at the appropriate times.

t	0	1	2	3	4
Approximation of $x(t)$	6				
Exact value of $x(t)$	6				

- c. The “approximations of x ” that you found in part (b) are a **numerical approximation** of the solution to the differential equation. You should notice that your numerical solution is pretty far off from the actual solution for most values of t . Why? What could be the sources of this error and how could we fix it?
- d. In Figure 7.1 you see a slope field and the exact solution to the differential equation $x' = -0.5x$ with $x(0) = 6$. Mark your approximate solutions at times $t = 1, t = 2, \dots, t = 4$ on the plot and connect them with straight lines.

1. Why are we using straight lines to connect the points?

2. What do you notice about your approximate solutions?
3. Why is it helpful to have the slope field in the background on this plot?

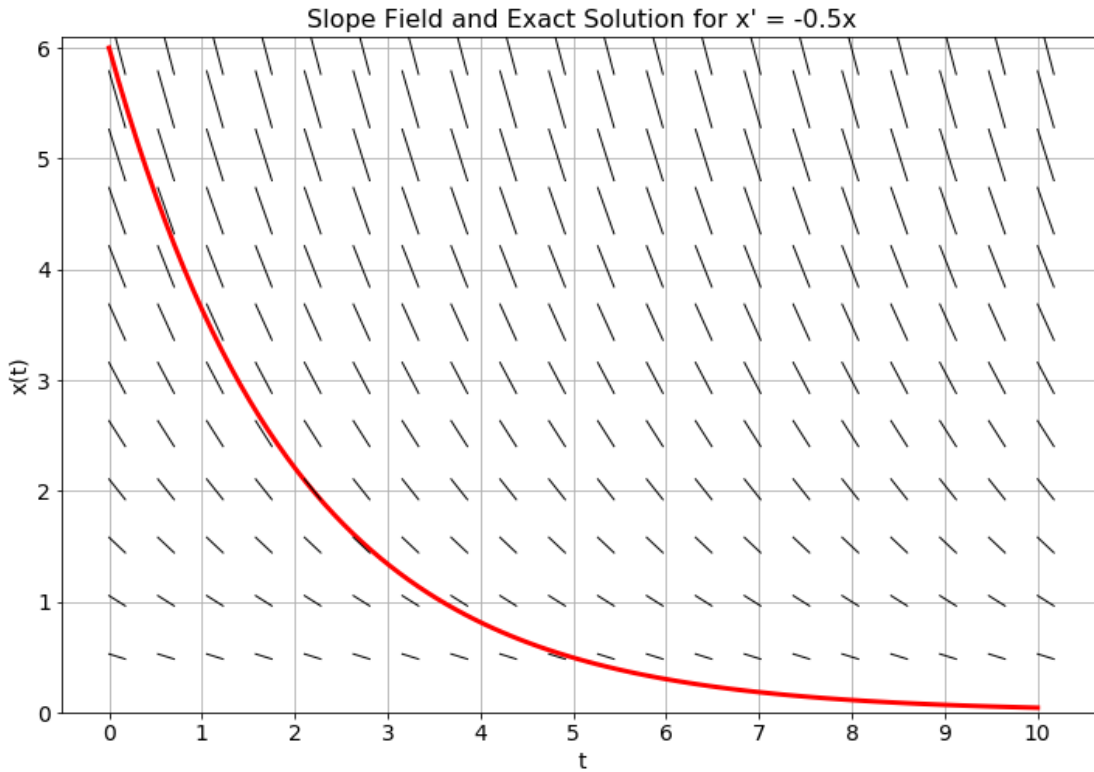


Figure 7.1.: Plot your approximate solution on top of the slope field and the exact solution.

Exercise 7.2. In Figure 7.2 you see the analytic solution with initial condition $x(0) = 5$ and a slope field for an unknown differential equation.

- a. Use the slope field and a step size of $\Delta t = 1$ to plot approximate solution values at $t = 1, t = 2, \dots, t = 10$. Connect your points with straight lines. The collection of line segments that you just drew is an approximation to the solution of the unknown differential equation.
- b. Use the slope field and a step size of $\Delta t = 0.5$ to plot approximate solution values at $t = 0.5, t = 1, t = 1.5, \dots, t = 10$. Again, connect your points with straight lines to get an approximation of the solution to the unknown differential equation.

7. ODE 1

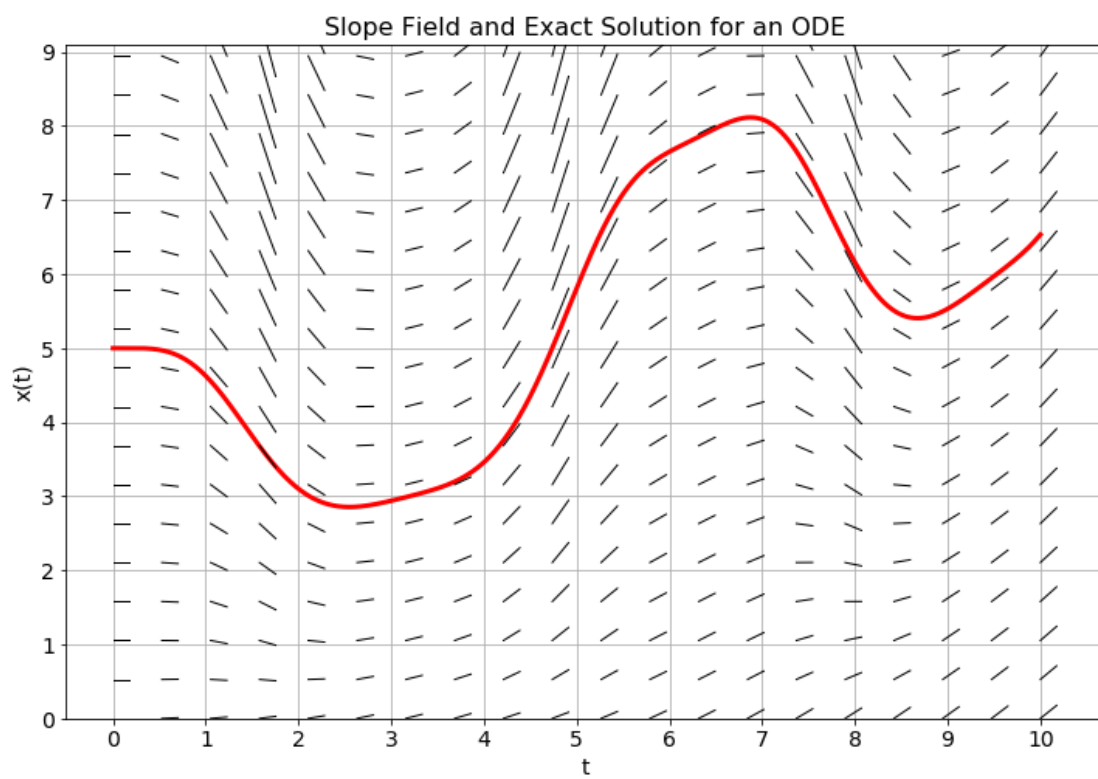


Figure 7.2.: Plot your approximate solution on top of the slope field and the exact solution.

- c. If you could take Δt to be very very small, what difference would you see graphically between the exact solution and your collection of line segments? Why?

The notion of approximating solutions to differential equations is simple in principle:

- make a discrete approximation to the derivative and
- step forward through time as a difference equation.

The challenging part is making the approximation to the derivative(s). There are many methods for approximating derivatives, and that is exactly where we will start.

Definition 7.1 (Euler's Method). Euler's Method is a technique for approximating the solution to the differential equation $x'(t) = f(t, x(t))$. Recall from Exercise 5.9 that the first derivative of a function can be discretized as

$$x'(t) = \frac{x(t+h) - x(t)}{h} + \mathcal{O}(h) \quad (7.5)$$

where $h = \Delta t$ is the step size (or the size of each partition in the domain), so the differential equation $x'(t) = f(t, x(t))$ becomes

$$\frac{x(t+h) - x(t)}{h} \approx f(t, x(t)). \quad (7.6)$$

Rewriting as a difference equation, letting $x_{n+1} = x(t_n + h)$ and $x_n = x(t_n)$, we get

$$x_{n+1} = x_n + hf(t_n, x_n) \quad (7.7)$$

A way to think about Euler's method is that at a given point, the slope is approximated by the value of the right-hand side of the differential equation and then we step forward h units in time following that slope. Figure 7.3 shows a depiction of the idea. Notice in the figure that in regions of high curvature Euler's method will deviate a lot from the exact solution to the differential equation. However, taking the limit as h tends to 0 theoretically gives the exact solution at the trade off of needing infinite computational resources.

7. ODE 1

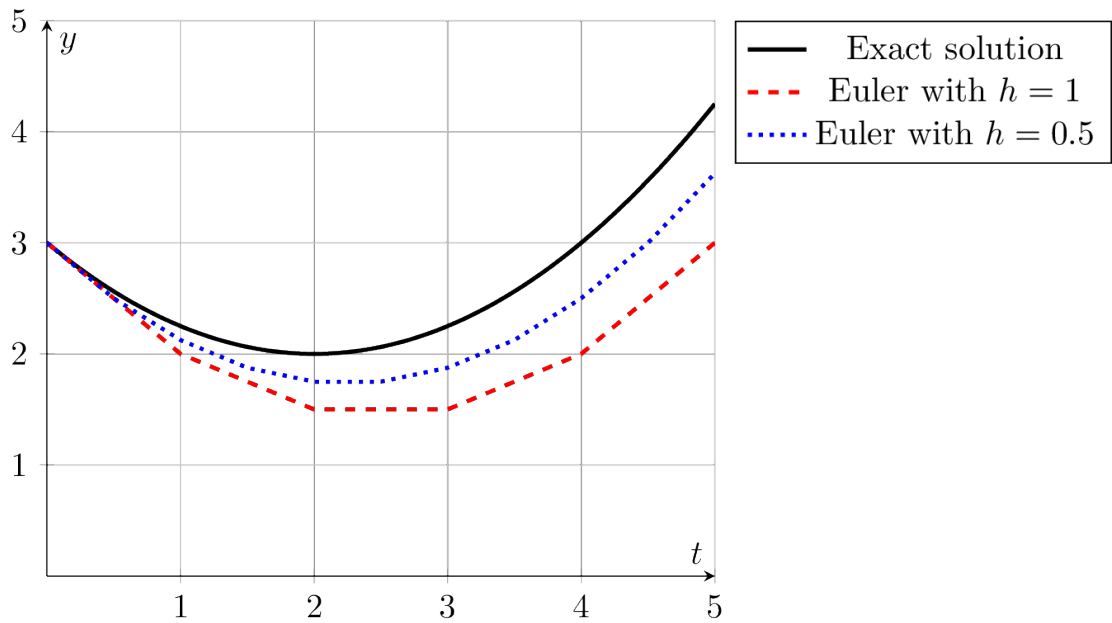


Figure 7.3.: Numerical solutions to a differential equation using Euler's method.

Exercise 7.3. Why would Euler's method do poorly in regions where the solution exhibits high curvature?

Exercise 7.4. Consider the differential equation $x'(t) = -2x(t)/3 + 4t$ with initial condition $x(0) = 6$. By hand perform four steps of the Euler method with stepsize $h = 1/2$ to obtain an approximation for $x(1/2), x(1), x(3/2)$ and $x(2)$.

Exercise 7.5. Write code to implement Euler's method for initial value problems. Your function should accept as input a Python function $f(t, x)$, an initial condition, a start time, an end time, and the value of $h = \Delta t$. The output should be vectors for t and x that you can easily plot to show the numerical solution. The code below will get you started.

```
def euler_1d(f, x0, t0, tmax, dt):  
    """  
    Solves a first-order ordinary differential equation using the Euler method.  
  
    Parameters:
```

```

    f      : function, the function defining the differential equation. It should
              take two arguments, the independent variable t and the dependent
              variable x, and return the derivative of x with respect to t.
    x0     : float, the initial value of the dependent variable.
    t0     : float, the initial value of the independent variable.
    tmax   : float, the maximum value of the independent variable.
    dt     : float, the time step.

Returns:
    tuple containing two numpy arrays:
        - t : vector of time values.
        - x : vector of solution values at each time value.
"""
# We need an integer number of steps so we round the number of steps
# and then adjust dt to match.
N = round((tmax - t0)/dt)
dt = (tmax - t0)/N

# Set up the time grid
t = np.linspace(t0, tmax, N+1)
# set up an array for x that is the same size as t
x = np.zeros(len(t))

# fill in the initial condition
x[0] = ???

for n in range(???): # think about how far we should loop
    # advance the solution forward in time with Euler
    x[n+1] = ???
return t, x

```

Exercise 7.6. Test your code from the previous exercise on a first order differential equation where you know an analytic solution. For example you could use the differential equation

$$x' = -\frac{1}{3}x + \sin(t) \quad \text{where} \quad x(0) = 1. \quad (7.8)$$

This has the analytic solution

$$x(t) = \frac{1}{10} (19e^{-t/3} + 3\sin(t) - 9\cos(t)). \quad (7.9)$$

Make a plot of the approximate solution and the exact solution on the same plot for $t \in [0, 10]$ The partial code below should get you started.

7. ODE 1

```
import numpy as np
import matplotlib.pyplot as plt

# Define the function giving x' in terms of t and x
f = lambda t, x: ???
x0 = ??? # initial condition
t0 = ??? # initial time
tmax = ??? # final time
dt = ??? # time step (your choice, but make it small)
t, x = euler_1d(f, x0, t0, tmax, dt)
plt.plot(t, x, 'b-', label='Euler')

# Define a function giving the analytic solution
x_exact = lambda t: ???
# We will plot the exact solution at a higher resolution
t_highres = np.linspace(t0, tmax, 100)
plt.plot(t_highres, x_exact(t_highres), 'r--', label='Exact')
plt.grid()
plt.show()
```

Experiment with different values for Δt and see how the numerical solution changes.

Exercise 7.7. The goal of this problem will be to compare the maximum error when you solve the differential equation from the previous exercise on the interval $t \in [0, 10]$ with the Euler method for various values of Δt .

Write a function that produces a plot with the value of Δt on the horizontal axis and the value of the associated absolute error on the vertical axis. You should use a log-log plot. Obviously you will need to run your code many times at many different values of Δt to build your data set. The following incomplete code will get you started.

```
# Create vector with different step sizes
dt = 10**(-np.linspace(0, 4, 50))
# Create vector with the same size as dt for holding the errors
errors = np.zeros_like(dt)
# Loop over the different step sizes to calculate the errors
for i in range(len(dt)):
    # Approximate the solution with Euler's method
    t, x = euler_1d(f, x0, t0, tmax, dt[i])
    errors[i] = ??? # Calculate maximum absolute error
```

```
# Plot the errors with logarithmic axes
plt.loglog(dt, errors)
plt.xlabel('Step size')
plt.ylabel('Maximum error')
plt.grid()
```

What does the plot tell you? In general, if you were to divide your value of Δt by 10, how much approximately does that reduce the error?

Exercise 7.8. Shelby solved a first order ODE $x' = f(t, x)$ using Euler's method with a step size of $dt = 0.1$ on a domain $t \in [0, 3]$. To test her code she used a differential equation where she knew the exact analytic solution and she found the maximum absolute error on the interval to be 0.15. Jackson then solves the exact same differential equation, on the same interval, with the same initial condition using Euler's method and a step size of $dt = 0.01$. What would be your best estimate of Jackson's maximum absolute error?

Theorem 7.1. *Euler's method is a first order method for approximating the solution to the differential equation $x' = f(t, x)$. Hence, if the step size h of the partition of the domain were to be divided by some positive constant M then the maximum absolute error between the numerical solution and the exact solution would ???*
(Complete the last sentence.)

7.2. Higher-order equations and systems of equations

Exercise 7.9 (Harmonic Oscillator). If a mass is hanging from a spring then Newton's second law, $\sum F = ma$, gives us the differential equation $mx'' = F_{restoring} + F_{damping}$ where x is the displacement of the mass from equilibrium, m is the mass of the object hanging from the spring, $F_{restoring}$ is the force pulling the mass back to equilibrium, and $F_{damping}$ is the force due to friction or air resistance that slows the mass down.

1. Which of the following is a good candidate for a restoring force in a spring? Defend your answer.
 - a. $F_{restoring} = -kx$: The restoring force is proportional to the displacement away from equilibrium.

7. ODE 1

- a. $F_{restoring} = -kx'$: The restoring force is proportional to the velocity of the mass.
 - b. $F_{restoring} = -kx$: The restoring force is proportional to the displacement of the mass.
 - c. $F_{restoring} = -kx''$: The restoring force is proportional to the acceleration of the mass.
2. Which of the following is a good candidate for a damping force in a spring? Defend your answer.
 - a. $F_{damping} = -bx$: The damping force is proportional to the displacement away from equilibrium.
 - b. $F_{damping} = -bx'$: The damping force is proportional to the velocity of the mass.
 - c. $F_{damping} = -bx''$: The damping force is proportional to the acceleration of the mass.
3. Put your answers to parts (1) and (2) together and simplify to form a second-order differential equation for position:

$$\underline{\hspace{1cm}} x'' = \underline{\hspace{1cm}} x' + \underline{\hspace{1cm}} x \tag{7.10}$$

4. If we want to solve a second order differential equation numerically we need to convert it to first order differential equations (Euler's method is only designed to deal with first order differential equations, not second order). To do so we can introduce a new variable, x_1 , such that $x_1 = x'$. For the sake of notational consistency we define $x_0 = x$. The result is a system of first-order differential equations.

$$\begin{aligned} x'_0 &= x_1 \\ x'_1 &= \end{aligned} \quad (7.11)$$

5. The code and Euler’s method algorithm that we have created thus far in this chapter are only designed to work with a single differential equation instead of a system, so we need to make some modifications. We can discretize the system of differential equations using Euler’s method so that

$$x' = F(t, x) \quad (7.12)$$

where F is a function that accepts a vector of inputs, plus time, and returns a vector of outputs. In the context of this particular problem,

$$F(t, x) = \begin{pmatrix} x'_0 \\ x'_1 \end{pmatrix} = \begin{pmatrix} x_1 \\ \hline \end{pmatrix} \quad (7.13)$$

7.2. Higher-order equations and systems of equations

6. We now need to discretize the derivatives in the system. As with 1D Euler's method, we will use a first-order approximation of the first derivative so that

$$\frac{x_{n+1} - x_n}{h} = F(t_n, x_n) + \mathcal{O}(h). \quad (7.14)$$

Rearranging and solving for x_{n+1} gives

$$x_{n+1} = \text{_____} + hF(\text{____}, \text{____}). \quad (7.15)$$

7. The following Python function will implement Euler's method. Complete the code.

```
def euler(F, x0, t0, tmax, dt):
    """
    Solves a system of first-order ordinary differential equations
    using the Euler method.

    Parameters:
        F : function, the function defining the system of differential equations.
            It should take two arguments, the independent variable t and the
            dependent variable x (as a 1D numpy array), and return the
            derivative of x with respect to t (as a 1D numpy array).
        x0 : numpy vector, the initial values of the dependent variables.
        t0 : float, the initial value of the independent variable.
        tmax : float, the maximum value of the independent variable.
        dt : float, the time step.

    Returns:
        tuple containing two numpy arrays:
            - t : vector of time values.
            - x : array of solutions at each time value. Each column of x
              corresponds to a different dependent variable.
    """
    # We need an integer number of steps so we round the number of steps
    # and then adjust dt to match.
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    # Set up the time grid
    t = ???
    # set up an array for x with one row for each dependent variable and one column
    # for each grid point
    x = np.zeros((len(t), len(x0)))
    # store the initial condition in the first row of x
    x[0, :] = x0
    # loop over the different time steps
```

7. ODE 1

```
for n in range(???):
    x[n+1, :] = x[???, ???] + dt * F(t[???], x[???, ???])
return t, x
```

8. To use the `euler()` function to solve the system of equations from part (4), complete the following code. Use a mass of $m = 2\text{kg}$, a damping force of $b = 40\text{kg/s}$, and a spring constant of $k = 128\text{N/m}$. Consider an initial position of $x = 0$ (equilibrium) and an initial velocity of $x_1 = 0.6\text{m/s}$.

```
m = 2
b = 40
k = 128
F = lambda t, x: np.array([x[1], ???])
x0 = [???, ???] # initial conditions
t0 = 0
tmax = 5 # pick something reasonable here
dt = 0.01 # your choice. pick something small
t, x = euler(F, x0, t0, tmax, dt)
```

9. Complete the following code to make a plot that shows both position and velocity versus time.

```
plt.plot(t, x[???, ???], 'b-', t, x[???, ???], 'r--')
plt.grid()
plt.title('Time Evolution of Position and Velocity')
plt.legend(['which legend entry here', 'which legend entry here'])
plt.xlabel('time')
plt.ylabel('position and velocity')
plt.show()
```

10. Complete the following code to make a second plot, called a phase plot, that shows position versus velocity. In a phase plot, time is implicit (not one of the axes).

```
plt.plot(x[???, ???], x[???, ???])
plt.grid()
plt.title('Phase Plot')
plt.xlabel('???')
plt.ylabel('???')
plt.show()
```

Exercise 7.10 (A Lotka-Volterra Predator-Prey Model). Let $x_0(t)$ denote the number of rabbits (prey) and $x_1(t)$ denote the number of foxes (predator) at time t . The relationship between the species can be modelled by the classic 1920's Lotka-Volterra Model:

$$\begin{cases} x'_0 &= \alpha x_0 - \beta x_0 x_1 \\ x'_1 &= \delta x_0 x_1 - \gamma x_1 \end{cases} \quad (7.16)$$

where α, β, γ , and δ are positive constants. For this problems take $\alpha = 1$, $\beta = 0.1$, $\gamma = 1$, and $\delta = 0.1$.

1. First rewrite the system of ODEs in the form $x' = F(t, x)$ so you can use your `euler()` code.
2. Modify your code from the previous problem so that it works for this problem. Use `tmax = 20` and `dt = 0.05`. Start with initial conditions $x_0(0) = 10$ rabbits and $x_1(0) = 5$ foxes.
3. Create the time evolution plot. What does this plot tell you in context?
4. Create a phase plot. What does this plot tell you in context?
5. If you decrease the step size by a factor of 10, what do you see in the two plots? Why?

7.3. The Midpoint Method

Now we get to improve upon Euler's method. There is a long history of wonderful improvements to the classic Euler's method – some that work in special cases, some that resolve areas where the error is going to be high, and some that are great for general purpose numerical solutions to ODEs with relatively high accuracy. In this section we will make a simple modification to Euler's method that has a surprisingly great pay-off in the error rate.

Exercise 7.11. In Euler's method, if we are at the point t_n then we approximate the slope $x'(t_n) = f(t_n, x_n)$ and use the slope to propagate forward one time step. As you have seen, this method can lead to an overshooting of the exact solution in regions of high curvature. It would be nice to be able to look into the future and get a better approximation of the slope so that we did not miss upcoming curvature. If you could build such a method that looks in to the future, finds a slope in the future, and then uses that slope (instead of the slope from Euler's method) to advance forward in time, how far into the future would you look? Why?

Exercise 7.12. Let us return to the simple differential equation $x' = -0.5x$ with $x(0) = 6$ that we saw in Exercise 7.1. Now we will propose a slightly different method for approximating the solution.

- a. At $t = 0$ we know that $x(0) = 6$. If we use the slope at time $t = 0$ to step forward in time then we will get the Euler approximation of the solution. Consider this alternative approach:

- Use the slope at time $t = 0$ and move *half* a step forward.
- Find the slope at the half-way point
- Then use the slope from the half way point to go a full step forward from time $t = 0$.

Perhaps a bit confusing ...let us build this idea together:

- What is the slope at time $t = 0$? $x'(0) = \underline{\hspace{2cm}}$
 - Use this slope to step a half step forward and find the x value: $x(0.5) \approx \underline{\hspace{2cm}}$
 - Now use the differential equation to find the slope at time $t = 0.5$. $x'(0.5) = \underline{\hspace{2cm}}$
 - Now take your answer from the previous step, and go one full step forward from time $t = 0$. What x value do you end up with?
 - Your answers to the previous bullets should be: $x'(0) = -3$, $x(0.5) \approx 4.5$, $x'(0.5) = -2.25$, so if we take a full step forward with slope $m = -2.25$ starting from $t = 0$ we get $x(1) \approx 3.75$.
- b. Repeat the process outlined in part (a) to approximate the solution to the differential equation at times $t = 2, 3, 4$. Also record the exact answer at each of these times by noting that the exact solution is $x(t) = 6e^{-0.5t}$.

t	0	1	2	3	4
Euler approx of $x(t)$	6				
New approx of $x(t)$	6				
Exact value of $x(t)$	6				

- c. Draw a clear picture of what this method is doing in order to approximate the slope at each individual step.
- d. How does your approximation compare to the Euler approximation that you found in Exercise 7.1?

Definition 7.2 (The Midpoint Method). The midpoint method is defined by first taking a half step with Euler's method to approximate a solution at time $t_{n+1/2}$. There is no grid point at $t_{n+1/2}$ so we define this as

$$t_{n+1/2} = (t_n + t_{n+1})/2.$$

We then take a full step using the value of f at $t_{n+1/2}$ and the approximate $x_{n+1/2}$.

$$\begin{aligned} x_{n+1/2} &= x_n + \frac{h}{2} f(t_n, x_n) \\ x_{n+1} &= x_n + h f(t_{n+1/2}, x_{n+1/2}) \end{aligned}$$

Exercise 7.13. As in in Exercise 7.4, consider the differential equation $x'(t) = -2x(t)/3 + 4t$ with initial condition $x(0) = 6$. By hand perform one step of the midpoint method with stepsize $h = 1$ to obtain an approximation for $x(1)$.

Exercise 7.14. Complete the code below to implement the midpoint method in one dimension.

```
def midpoint_1d(f, x0, t0, tmax, dt):
    """
    Solves a first-order ordinary differential equation using
    the midpoint method.

    Parameters:
        f      : function, the function defining the differential equation. It should
                  take two arguments, the independent variable t and the dependent
                  variable x, and return the derivative of x with respect to t.
        x0     : float, the initial value of the dependent variable.
        t0     : float, the initial value of the independent variable.
        tmax   : float, the maximum value of the independent variable.
        dt     : float, the time step.

    Returns:
        tuple containing two numpy arrays:
        - t : vector of time values.
        - x : vector of solution values at each time value.
```

7. ODE 1

```
"""
N = round((tmax - t0)/dt)
dt = (tmax - t0)/N
t = ??? # build the times
x = ??? # build an array for the x values
x[0] = ??? # save the initial condition
# On the next line: be careful about how far you're looping
for n in range(???):
    slope = ??? # get the slope at the current point
    x_halfstep = ??? # take a half step forward
    x[n+1] = ??? # take a full step forward
return t, x
```

Test your code on several differential equations where you know the solution (just to be sure that it is working).

```
f = lambda t, x: # your ODE right hand side goes here
x0 = # initial condition
t0 = 0
tmax = # ending time (up to you)
dt = # pick something small
t, x = midpoint_1d(???, ???, ???, ???, ???)
plt.plot(???, ???, ???)
x_exact = lambda t: # your exact solution goes here
plt.plot(???, ???, ???)
plt.legend(['Midpoint', 'Exact'])
plt.grid()
plt.show()
```

Also apply your method to the question in the feedback quiz.

Exercise 7.15. The goal in building the midpoint method was to hopefully capture some of the upcoming curvature in the solution before we overshot it. Consider the differential equation $x' = -\frac{1}{3}x + \sin(t)$ with initial condition $x(0) = 1$ on the domain $t \in [0, 10]$ as in Exercise 7.6. First get a numerical solution with Euler's method using $\Delta t = 1$. Then get a numerical solution with the midpoint method using the same value for $\Delta t = 1$. Plot the two solutions on top of each other along with the exact solution

$$x(t) = \frac{1}{10} (19e^{-t/3} + 3\sin(t) - 9\cos(t)). \quad (7.17)$$

What do you observe?

Exercise 7.16. Repeat Exercise 7.7 with the midpoint method. Compare your results to what you found with Euler’s method. Then use what you have discovered to answer the following, similar to Exercise 7.8:

Shelby solved a first order ODE $x' = f(t, x)$ using the midpoint method with a step size of $dt = 0.1$ on a domain $t \in [0, 3]$. To test her code she used a differential equation where she knew the exact analytic solution and she found the maximum absolute error on the interval to be 0.15. Jackson then solves the exact same differential equation, on the same interval, with the same initial condition using the midpoint method and a step size of $dt = 0.01$. What would be your best estimate of Jackson’s maximum absolute error?

Exercise 7.17. We have studied two methods thus far: Euler’s method and the Midpoint method. In Figure 7.4 we see a graphical depiction of how each method works on the differential equation $y' = y$ with $\Delta t = 1$ and $y(0) = 1$. The exact solution at $t = 1$ is $y(1) = e^1 \approx 2.718$ and is shown in red in each figure. The methods can be summarized in the table below.

Discuss what you observe as the pros and cons of each method based on the table and on the Figure.

Euler’s Method	Midpoint Method
1. Get the slope at time t_n	1. Get the slope at time t_n
2. Follow the slope for time Δt	2. Follow the slope for time $\Delta t/2$
	3. Get the slope at the point $t_n + \Delta t/2$
	4. Follow the new slope from time t_n for time Δt

When might you want to use Euler’s method instead of the midpoint method? When might you want to use the midpoint method instead of Euler’s method?

Exercise 7.18 (Midpoint Method in Several Dimensions). Write a function `midpoint()` that can solve a system of first-order ordinary differential equations using the midpoint method. Base your code on the `euler()` code from Exercise 7.9 . You should only have to add one line of code and then be careful about the size of the arrays that are in play. Test your code on several problems. Compare and contrast what you see with your Euler solutions and with your Midpoint solutions.

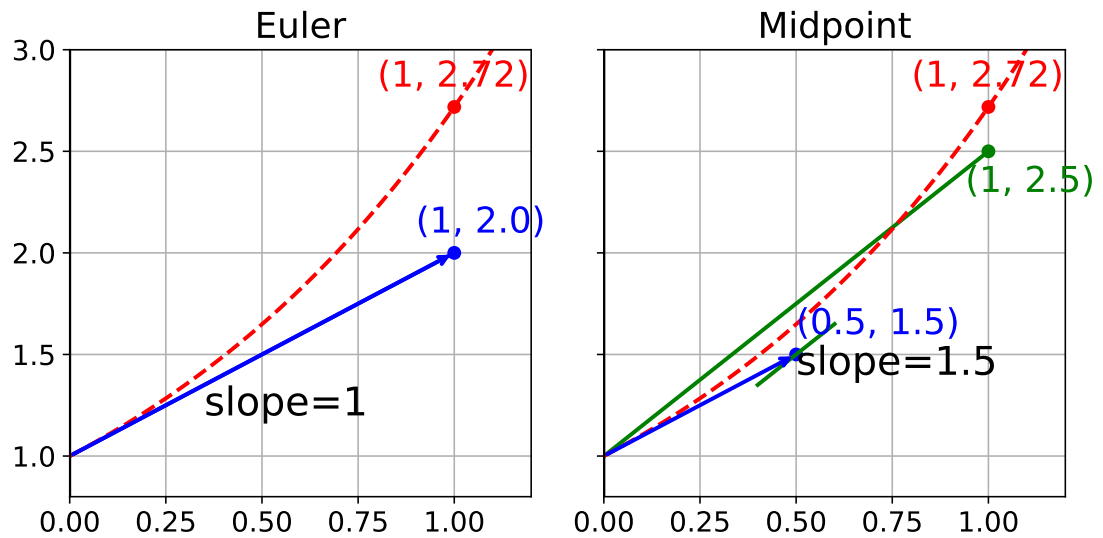


Figure 7.4.: Graphical depictions of two numerical methods. Euler (left) and Midpoint (right). The exact solution is shown in red.

7.4. Numerical Instabilities

This section is for discovery only. The theory behind what you are about to discover will be presented in a later chapter.

Exercise 7.19. Consider the differential equation $x' = -3x$ with initial condition $x(0) = 1$. The exact solution is the exponentially decaying function $x(t) = e^{-3t}$, which quickly approaches 0 as t gets larger.

- Use Euler's method by hand to approximate $x(0.1)$, $x(0.2)$, and $x(0.3)$ using a stepsize of $\Delta t = 0.1$. Does the numerical solution appear to be decaying towards 0?
- Now use Euler's method by hand to approximate $x(1)$, $x(2)$, and $x(3)$ using a stepsize of $\Delta t = 1$. What happens to the values of x ? Does it look like the exact solution?

Exercise 7.20. The phenomenon you just discovered is called *numerical instability*. It occurs when the stepsize is too large, causing the numerical solution to oscillate or grow exponentially even when the exact solution decays.

- Use your `euler_1d` function to solve the same differential equation $x' = -3x$ with $x(0) = 1$ on the interval $t \in [0, 10]$. Run it for different values of the stepsize Δt , for example $\Delta t \in \{0.1, 0.5, 0.7, 1.0\}$.

- b. Plot the numerical solutions along with the exact solution $x(t) = e^{-3t}$ on the same graph. (You might want to limit the y-axis using `plt.ylim(-5, 5)` to see the details before it blows up completely.)
- c. What is the limit on the stepsize Δt before the numerical solution completely loses its mind and grows exponentially? (Experiment with `dt` between 0.5 and 0.7 to find the exact threshold).
- d. Try the same experiment using your `midpoint_1d` function. Does the midpoint method also suffer from numerical instability? If so, what is the critical stepsize for the midpoint method on this problem?

Exercise 7.21. Based on your observations in the previous exercises, what is the danger of blindly trusting a numerical solution without considering the stepsize? How might you check if your numerical solution is stable in practice if you don't know the exact solution?

7.5. Algorithm Summaries

Exercise 7.22. Consider the first-order differential equation $x' = f(t, x)$. What is Euler's method for approximating the solution to this differential equation? What is the order of accuracy of Euler's method? Explain the meaning of the order of the method in the context of solving a differential equation.

Exercise 7.23. Explain in clear language what Euler's method does geometrically.

Exercise 7.24. Consider the first-order differential equation $x' = f(t, x)$. What is the Midpoint method for approximating the solution to this differential equation? What is the order of accuracy of the Midpoint method?

Exercise 7.25. Explain in clear language what the Midpoint method does geometrically.

7.6. Truncation Errors

Many of the approximation methods in Numerical Analysis are based on Taylor series expansions and then dropping higher-order terms in the series. The error that is introduced by truncating a Taylor series in this way is called the truncation error. We have seen that in Section 5.5.

The material in this section will be covered by lectures. You will not need to have read this section before the assessment quiz for this week.

We recall from Section 5.5 Taylor's formula: for a function x that is p times differentiable,

$$x(t+h) = x(t) + hx'(t) + \frac{h^2}{2!}x''(t) + \cdots + \frac{h^{p-1}}{(p-1)!}x^{(p-1)}(t) + \frac{h^p}{p!}x^{(p)}(\xi)$$

for some ξ between t and $t+h$.

Here we consider the initial-value problem $x'(t) = f(t, x(t))$ with exact solution $x(t)$. The **local truncation error** at step n is defined as the error made in a single step starting from the exact solution, divided by the step size h :

$$\tau_n = \frac{1}{h} |x(t_{n+1}) - x_{n+1}|,$$

where x_{n+1} is the value produced by the numerical method when started from the exact value $x(t_n)$.

7.6.1. Truncation Error in Euler's Method

Euler's method produces the update

$$x_{n+1} = x_n + hf(t_n, x_n).$$

Starting from the exact solution value $x(t_n)$, the method gives

$$x_{n+1} = x(t_n) + hf(t_n, x(t_n)) = x(t_n) + hx'(t_n).$$

We compare this to the exact value at the next grid point by expanding $x(t_{n+1}) = x(t_n+h)$ in a Taylor series around t_n :

$$x(t_{n+1}) = x(t_n) + hx'(t_n) + \frac{h^2}{2}x''(\xi_n)$$

for some ξ_n between t_n and t_{n+1} . The local error in one step is therefore

$$x(t_{n+1}) - x_{n+1} = \frac{h^2}{2}x''(\xi_n),$$

so the local truncation error is

$$\tau_n = \frac{1}{h} \left| \frac{h^2}{2} x''(\xi_n) \right| = \frac{h}{2} |x''(\xi_n)|.$$

We see that the local truncation error is of order h and that Euler's method is a **first-order** approximation. This means that halving the step size halves the error.

The **global truncation error** is obtained by summing the local errors over all $N = (b - a)/h$ steps. For some ξ between a and b ,

$$\tau = \left| \sum_{n=0}^{N-1} \frac{h^2}{2} x''(\xi_n) \right| \leq N \frac{h^2}{2} \max_{t \in [a, b]} |x''(t)| = \frac{b-a}{2} h \max_{t \in [a, b]} |x''(t)|.$$

The global truncation error is also of order h , confirming that Euler's method is a first-order method.

7.6.2. Truncation Error in the Midpoint Method

The midpoint method produces the update in two stages:

$$\begin{aligned} x_{n+1/2} &= x_n + \frac{h}{2} f(t_n, x_n), \\ x_{n+1} &= x_n + h f\left(t_n + \frac{h}{2}, x_{n+1/2}\right). \end{aligned}$$

Starting from the exact value $x(t_n)$, the half-step gives

$$x_{n+1/2} = x(t_n) + \frac{h}{2} x'(t_n).$$

We now expand f at the half-step point using a Taylor series in two variables. Since $f_t + f_x x' = x''$ (differentiating $x' = f(t, x(t))$ with respect to t), we have

$$\begin{aligned} f\left(t_n + \frac{h}{2}, x_{n+1/2}\right) &= f\left(t_n + \frac{h}{2}, x(t_n) + \frac{h}{2} x'(t_n)\right) \\ &= f(t_n, x(t_n)) + \frac{h}{2} f_t(t_n, x(t_n)) + \frac{h}{2} x'(t_n) f_x(t_n, x(t_n)) + \mathcal{O}(h^2) \\ &= x'(t_n) + \frac{h}{2} x''(t_n) + \mathcal{O}(h^2). \end{aligned}$$

Substituting into the full-step update gives

$$x_{n+1} = x(t_n) + h \left(x'(t_n) + \frac{h}{2} x''(t_n) \right) + \mathcal{O}(h^3) = x(t_n) + h x'(t_n) + \frac{h^2}{2} x''(t_n) + \mathcal{O}(h^3).$$

We compare this to the Taylor expansion of the exact solution:

$$x(t_{n+1}) = x(t_n) + h x'(t_n) + \frac{h^2}{2} x''(t_n) + \frac{h^3}{6} x'''(\xi_n)$$

7. ODE 1

for some ξ_n between t_n and t_{n+1} . The local error in one step is

$$x(t_{n+1}) - x_{n+1} = \frac{h^3}{6} x'''(\xi_n),$$

so the local truncation error is

$$\tau_n = \frac{1}{h} \left| \frac{h^3}{6} x'''(\xi_n) \right| = \frac{h^2}{6} |x'''(\xi_n)|.$$

We see that the local truncation error is of order h^2 and that the midpoint method is a **second-order** approximation. This means that halving the step size reduces the error by a factor of four.

The **global truncation error** is obtained by summing over all $N = (b - a)/h$ steps:

$$\tau \leq N \frac{h^3}{6} \max_{t \in [a, b]} |x'''(t)| = \frac{b - a}{6} h^2 \max_{t \in [a, b]} |x'''(t)|,$$

which is of order h^2 , confirming that the midpoint method is a second-order method.

7.7. Exam-Style Question

- (a) Explain what is meant by *numerical instability* when solving an ordinary differential equation using an explicit finite-difference scheme. What happens to the numerical solution, and how can it be avoided? [3 marks]
- (b) Consider the ordinary differential equation $x' = -2x + t$ with initial condition $x(0) = 1$. Perform one step of Euler's method with a step size of $h = 0.5$ to find an approximation for $x(0.5)$. [2 marks]
- (c) Now perform one step of the Midpoint method with a step size of $h = 0.5$ to find an approximation for $x(0.5)$ for the same initial value problem. [3 marks]
- (d) Consider forming a system of first-order differential equations from the second-order autonomous differential equation for a nonlinear pendulum:

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin(\theta) = 0,$$

where g and L are constants. By introducing a new variable $\omega = \frac{d\theta}{dt}$, write down the corresponding system of two first-order differential equations. [2 marks]

- (e) The following incomplete Python code computes the numerical solution to the differential equation $x' = f(t, x)$ using the Midpoint method. Provide the missing code indicated by `...` [3 marks]

```
import numpy as np

def midpoint_1d(f, x0, t0, tmax, dt):
    """
    Solves a first-order ordinary differential equation using the midpoint method.
    """
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    t = np.linspace(t0, tmax, N+1)
    x = np.zeros(len(t))
    x[0] = x0

    for n in range(len(t) - 1):
        slope = ...
        x_halfstep = ...
        x[n+1] = ...

    return t, x
```


8. ODE 2

8.1. Searching for a Better Method

OK. Ready for some experimentation? We are going to build a few experiments that eventually lead us to a more powerful method for finding numerical solutions to first order differential equations than the midpoint method. It is up to you to decide how much time to spend on experimentation in this section. If you are pressed for time, you can skip forward to Section 8.2.

Exercise 8.1. Let us talk about the Midpoint Method for a moment. The geometric idea of the midpoint method is outlined in the bullets below. Draw a picture along with the bullets.

- You are sitting at the point (t_n, x_n) .
- The slope of the solution curve to the ODE where you are standing is

$$\text{slope at the point } (t_n, x_n) \text{ is: } m_n = f(t_n, x_n)$$

- You take a half a step forward using the slope where you are standing. The new point, denoted $x_{n+1/2}$, is given by

$$\text{location a half step forward is: } x_{n+1/2} = x_n + \frac{\Delta t}{2} m_n.$$

- Now you are standing at $(t_n + \frac{\Delta t}{2}, x_{n+1/2})$ so there is a new slope here given by

$$\text{slope after a half of an Euler step is: } m_{n+1/2} = f(t_n + \Delta t/2, x_{n+1/2}).$$

- Go back to the point (t_n, x_n) and step a full step forward using slope $m_{n+1/2}$. Hence the new approximation is

$$x_{n+1} = x_n + \Delta t \cdot m_{n+1/2}$$

8. ODE 2

Exercise 8.2. The midpoint method only uses the slope at the midpoint. Perhaps one can find a better method by taking some combination of slopes calculated at different points. Here we will build four slopes:

- $m_n = f(t_n, x_n)$: the slope at the start of the step.
- A half-step forward using m_n gives $x_{n+1/2} = x_n + \frac{\Delta t}{2} m_n$, and we evaluate the slope there:

$$m_{n+1/2} = f\left(t_n + \frac{\Delta t}{2}, x_{n+1/2}\right).$$

- A half-step forward using $m_{n+1/2}$ gives a second estimate of the midpoint,

$$x_{n+1/2}^* = x_n + \frac{\Delta t}{2} m_{n+1/2},$$

and we evaluate the slope there:

$$m_{n+1/2}^* = f\left(t_n + \frac{\Delta t}{2}, x_{n+1/2}^*\right).$$

- A full step forward using $m_{n+1/2}^*$ gives an estimate of the endpoint,

$$x_{n+1}^* = x_n + \Delta t m_{n+1/2}^*,$$

and we evaluate the slope there:

$$m_{n+1}^* = f(t_n + \Delta t, x_{n+1}^*).$$

1. Draw a picture showing all four points where slopes are calculated.
2. Experiment with different ways to combine these four slopes as a linear combination

$$\text{estimate of slope} = c_n m_n + c_{n+1/2} m_{n+1/2} + c_{n+1/2}^* m_{n+1/2}^* + c_{n+1}^* m_{n+1}^*,$$

and then advance with

$$x_{n+1} = x_n + \Delta t \cdot \text{estimate of slope}.$$

The code below implements all four slope calculations and compares your method against Euler and midpoint on the differential equation $x'(t) = -\frac{1}{3}x + \sin(t)$ with $x(0) = 1$, whose exact solution is

$$x(t) = \frac{1}{10} (19e^{-t/3} + 3\sin(t) - 9\cos(t)).$$

The plots show the maximum absolute error vs. step size on a log-log scale; the slope in that plot gives the order of the method.


```

import numpy as np
import matplotlib.pyplot as plt

# *****
# You should copy your 1d euler and midpoint functions here.
# We will be comparing to these two existing methods.
# *****

def ode_new_method(f, x0, t0, tmax, dt):
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    t = np.linspace(t0, tmax, N+1) # set up the times
    x = np.zeros(len(t))           # set up the x
    x[0] = x0                       # initial condition
    for n in range(len(t)-1):
        m_n = f(t[n], x[n])
        x_n_plus_half = x[n] + dt/2 * m_n
        m_n_plus_half = f(t[n] + dt/2, x_n_plus_half)
        x_n_plus_half_star = x[n] + dt/2 * m_n_plus_half
        m_n_plus_half_star = f(t[n] + dt/2, x_n_plus_half_star)
        x_n_plus_1_star = x[n] + dt * m_n_plus_half_star
        m_n_plus_1_star = f(t[n] + dt, x_n_plus_1_star)
        estimate_of_slope = ??? # This is where you get to play
        x[n+1] = x[n] + dt * estimate_of_slope
    return t, x

f = lambda t, x: -(1/3.0) * x + np.sin(t)
exact = lambda t: (1/10.0)*(19*np.exp(-t/3) + \
    3*np.sin(t) - \
    9*np.cos(t))

x0 = 1 # initial condition
t0 = 0 # initial time
tmax = 3 # max time
dt = 2.0*(-np.arange(1, 8, 1))
err_euler = np.zeros(len(dt))
err_midpoint = np.zeros(len(dt))
err_ode_new_method = np.zeros(len(dt))
for n in range(len(dt)):
    t, xeuler = euler_1d(f, x0, t0, tmax, dt[n])
    err_euler[n] = np.max(np.abs(xeuler - exact(t)))
    t, xmidpoint = midpoint_1d(f, x0, t0, tmax, dt[n])

```

8. ODE 2

```
err_midpoint[n] = np.max(np.abs(xmidpoint - exact(t)))
t, xtest = ode_new_method(f, x0, t0, tmax, dt[n])
err_ode_new_method[n] = np.max(np.abs(xtest - exact(t)))

plt.loglog(dt, err_euler, 'r*- ',
           dt, err_midpoint, 'b*- ',
           dt, err_ode_new_method, 'k*- ')
plt.grid()
plt.legend(['euler', 'midpoint', 'new method'])
plt.show()
```

3. Work with your group to fill in the following summary table. If you tried combinations not listed below, add them.

	c_n	$c_{n+1/2}$	$c_{n+1/2}^*$	c_{n+1}^*	Order of Error	Name
1	1	0	0	0	$\mathcal{O}(\Delta t)$	Euler method
2	0	1	0	0	$\mathcal{O}(\Delta t^2)$	Midpoint Method
3	1/2	1/2	0	0		
4	1/4	2/4	0	1/4		
5	0	0	1	0		
6	0	1/2	1/2	0		
7	1/4	1/4	1/4	1/4		
8	1/5	2/5	1/5	1/5		
9	1/5	1/5	2/5	1/5		
10	1/6	2/6	2/6	1/6		
11	1/8	3/8	3/8	1/8		
12						
13						

Many of the resulting numerical ODE methods likely had the same order of accuracy, but some may have been much better or much worse. State which linear combination of slopes seems to have been the best, draw a picture of what this method does, and clearly state what the order of the error means about this method.

8.2. Runge-Kutta Method

Definition 8.1 (The Runge-Kutta 4 Method). The Runge-Kutta 4 (RK4) method for approximating the solution to the differential equation $x' = f(t, x)$ approximates the

slope m at the point t_n by using the following calculations:

$$\begin{aligned}k_1 &= f(t_n, x_n) \\k_2 &= f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1\right) \\k_3 &= f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2\right) \\k_4 &= f(t_n + h, x_n + hk_3) \\m &= \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

It then uses that slope to advance by one time step:

$$x_{n+1} = x_n + hm = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

The order of the error in the RK4 method is $\mathcal{O}(\Delta t^4)$.

Exercise 8.3. Of course there is a price to pay for the improvement provided by the RK4 method over our earlier methods. How many evaluations of the function $f(t, x)$ do we need to make at every time step of the RK4 method? Compare this to the Euler method and the midpoint method.

Exercise 8.4. Let us step back for a second and just see what the RK4 method does from a nuts-and-bolts point of view. Consider the differential equation $x' = x$ with initial condition $x(0) = 1$. The solution to this differential equation is clearly $x(t) = e^t$. Take $\Delta t = h$ and perform 1 step of the RK4 method BY HAND to approximate the value $x(h)$. You should find that $x(h)$ is given by a polynomial in h . Does that polynomial look familiar? Is this consistent with the fact that the RK4 method is a 4th order method?

Exercise 8.5. Write a Python function that implements the Runge-Kutta 4 method in one dimension.

8. ODE 2

```
import numpy as np
import matplotlib.pyplot as plt

def rk4_1d(f, x0, t0, tmax, dt):
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    t = np.linspace(t0, tmax, N+1)
    x = np.zeros(len(t))
    x[0] = x0
    for n in range(len(t)-1):
        # the interesting bits of the code go here
    return t, x
```

Test the problem on several differential equations where you know the solution.

Exercise 8.6 (RK4 Error Plot). Make a log-log plot of the error in the RK4 method for a differential equation whose solution you know exactly (you could use $x' = -\frac{1}{3}x + \sin(t)$ with $x(0) = 1$ on the domain $t \in [0, 10]$, similar to what you did for the Euler method and the midpoint method earlier). Check that the error plot shows that the error is $\mathcal{O}(\Delta t^4)$.

Exercise 8.7 (RK4 in Several Dimensions). Modify your Runge-Kutta 4 code to work for any number of dimensions. You may want to start from your `euler()` and `midpoint()` functions that already do this. You will only need to make minor modifications from there. Then test your new generalized RK4 method on all of the same problems which you used to test your `euler()` and `midpoint()` functions.

8.3. The Backward Euler Method

We have now built up a variety of numerical ODE solvers. All of the solvers that we have built thus far are called **explicit** numerical differential equation solvers since they try to advance the solution explicitly forward in time. Wouldn't it be nice if we could literally just say, *what slope is going to work best in the future time steps ... let us use that?* Seems like an unrealistic hope, but let's see.

Definition 8.2 (Backward Euler Method). We want to solve $x' = f(t, x)$ so:

- Approximate the derivative by looking forward in time(!)

$$\frac{x_{n+1} - x_n}{h} \approx f(t_{n+1}, x_{n+1})$$

- Rearrange to get the difference equation

$$x_{n+1} = x_n + hf(t_{n+1}, x_{n+1}).$$

- We will always know the value of t_{n+1} and we will always know the value of x_n , but we do not know the value of x_{n+1} . In fact, that is exactly what we want. The major trouble is that x_{n+1} shows up on both sides of the equation. Can you think of a way to solve for it? ...you have code that does this step!
 - This method is called the **Backward Euler** method and is known as an **implicit method** since it does not explicitly give an expression for x_{n+1} but instead it gives an equation that still needs to be solved for x_{n+1} . We will see the main advantage of implicit methods in Section 8.4.
-

Exercise 8.8. Let us take a few steps through the backward Euler method on a problem that we know well: $x' = -0.5x$ with $x(0) = 6$.

Let us take $h = 1$ for simplicity, so the backward Euler iteration scheme for this particular differential equation is

$$x_{n+1} = x_n - \frac{1}{2}x_{n+1}.$$

Notice that x_{n+1} shows up on both sides of the equation. A little bit of rearranging gives

$$\frac{3}{2}x_{n+1} = x_n \quad \implies \quad x_{n+1} = \frac{2}{3}x_n.$$

1. Complete the following table.

t	0	1	2	3	4	5	6
Euler Approx. of x	6	3	1.5	0.75			
Back. Euler Approx. of x	6	4	2.667	1.778			
Exact value of x	6	3.64	2.207	1.339			

2. Compare now to what we found for the midpoint method on this problem as well.

Exercise 8.9. By hand, apply the backward Euler method with stepsize $h = 1/2$ to the differential equation $x' = xt$ with initial condition $x(0) = 1$ to obtain an approximation for $x(1/2)$. Do your calculations in terms of fractions.

Example 8.1. The previous problem could potentially lead you to believe that the backward Euler method will always result in some other nice difference equation after some algebraic rearranging. That is not true! Let us consider a slightly more complicated differential equation and see what happens

$$x' = -\frac{1}{2}x^2.$$

1. Recall that the backward Euler approximation is

$$x_{n+1} = x_n + hf(t_{n+1}, x_{n+1}).$$

Let us take $h = 1$ for simplicity (we will make it smaller later). What is the backward Euler formula for this particular differential equation?

2. You should notice that your backward Euler formula is now a quadratic function in x_{n+1} . That is to say, if you are given a value of x_n then you need to solve a quadratic polynomial equation to get x_{n+1} .
3. Let us take $x_0 = 6$ and $h = 1$. What is the value of x_1 using the backward Euler method?

Did you get $x_1 = \sqrt{13} - 1$? (The quadratic has a second root $-\sqrt{13} - 1$, which we discard because $x_0 > 0$ and we expect $x_1 > 0$.)

The complication with the backward Euler method is that you have a nonlinear equation to solve at every time step

$$x_{n+1} = x_n + hf(t_{n+1}, x_{n+1}).$$

Notice that this is the same as solving the equation

$$G(x_{n+1}) := x_{n+1} - hf(t_{n+1}, x_{n+1}) - x_n = 0.$$

You know the values of $h = \Delta t$, t_{n+1} and x_n , and you know the function f , so, in a practical sense, you should use some sort of numerical method to solve that equation – at each time step. Since $G'(x_{n+1}) = 1 - h\partial f/\partial x$, applying Newton's method would require computing the partial derivative of f with respect to x , which is not always available or convenient. So our next best option is to use the secant method. You can use your code from Exercise 4.20 for the function `secant_1d()`.

Exercise 8.10. Complete the function `backward_euler_1d()` below. How do you define the function `G` inside the `for` loop and what starting values do you use for the secant method?

```
import numpy as np
def backward_euler_1d(f, x0, t0, tmax, dt):
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    t = np.linspace(t0, tmax, N+1)
    x = np.zeros(len(t))
    x[0] = x0
    for n in range(len(t)-1):
        G = lambda y: ??? # define this function
        # give appropriate starting values for the secant method
        x[n+1] = secant_1d(G, ???)
    return t, x
```

Test your `backward_euler_1d()` function on several differential equations where you know the solution.

Exercise 8.11. You can test your `backward_euler_1d()` function on the differential equation

$$x' = -\frac{1}{3}x + \sin(t) \quad \text{with } x(0) = 1$$

like we have for many of our past algorithms since we know that the solution is

$$x(t) = \frac{1}{10} (19e^{-t/3} + 3\sin(t) - 9\cos(t))$$

The following plot lets you compare the backward Euler method to the forward Euler method and the exact solution. You just need to first define the `euler_1d()` function (which you have done in Exercise 7.5) and the `backward_euler_1d()` function (which you have just completed in Exercise 8.10).

```
import numpy as np
import matplotlib.pyplot as plt
f = lambda t, x: -x/3+np.sin(t)
x_exact = lambda t: (19*np.exp(-t/3)+3*np.sin(t)-9*np.cos(t)) / 10
x0 = 1
```

```

t0 = 0
tmax = 4
dt = 0.1
t, x_euler = euler_1d(f,x0,t0,tmax,dt)
t, x_backward = backward_euler_1d(f,x0,t0,tmax,dt)
plt.plot(t, x_euler, 'g-.', label = "Euler")
plt.plot(t, x_backward, 'b--', label = "Backward Euler")
plt.plot(t, x_exact(t), 'r-', label = "Exact")
plt.legend()

```

8.4. Numerical Instabilities

Exercise 8.12. It may not be obvious at the outset, but the Backward Euler method will actually behave better than the (forward) Euler method in some sense. Let us take a look. Consider, for example, the really simple linear differential equation $x' = -10x$ with $x(0) = 1$ on the interval $t \in [0, 2]$. The analytic solution is $x(t) = e^{-10t}$. Write Python code that plots the analytic solution, the Euler approximation, and the Backward Euler approximation on top of each other. Use a time step that is larger than you normally would (such as $\Delta t = 0.25$ or $\Delta t = 0.5$ or larger). What do you notice? Why do you think this is happening?

Exercise 8.13. Consider the linear differential equation $x' = -cx$ with $c > 0$ a constant. If you solve this with the forward Euler method with a step size h , then each step can be written in the form

$$x_{n+1} = ??? \cdot x_n.$$

Because the exact solution to this differential equation $x(t) = e^{-ct}$ goes towards 0 as t goes to infinity, the Euler method should also go towards 0 as n goes to infinity. What is the condition on the ??? in the equation above that ensures that the Euler method has x_n going towards 0 as n goes to infinity? What condition does this impose on the step size h ?

If you now solve this same ODE with the same step size with the backward Euler method, you can solve the equation for x_{n+1} exactly to find that

$$x_{n+1} = ??? \cdot x_n.$$

Again you would like this to go towards 0 as n goes to infinity. What condition does this impose on the step size h now?

Because the observations you made in the previous two exercises are so important, I lectured about them. Here are the lecture notes:

The stability of a numerical method is the property that, for a given step size, the numerical solution behaves qualitatively like the exact solution. In particular, if the exact solution decays to zero, the numerical solution should also decay to zero rather than grow without bound. If the numerical solution grows when the exact solution decays, we say the method is **unstable** for that step size.

This is a different concern from convergence (which asks whether the numerical solution approaches the exact solution as $h \rightarrow 0$). Stability asks: for a fixed step size h , does the numerical method respect the qualitative behaviour of the exact solution? A method can be unstable for large h even when it is perfectly accurate for small h , so choosing the step size carefully is essential.

The test equation that is always used for studying the stability of a numerical method is the linear ODE

$$x' = \lambda x.$$

The exact solution to this equation is $x(t) = x_0 e^{\lambda t}$. So if the real part of λ is negative the solution decays, and if the real part of λ is positive the solution grows.

As we will see, applying a numerical method to this equation produces a recurrence of the form

$$x_{n+1} = Q(h\lambda) x_n$$

for some **amplification factor** $Q(h\lambda)$, so

$$x_n = Q(h\lambda)^n x_0.$$

Setting $z = h\lambda$, the numerical solution decays to zero if and only if $|Q(z)| < 1$. The set of all $z \in \mathbb{C}$ for which $|Q(z)| < 1$ is called the **region of absolute stability** of the method.

Since we want the numerical solution to decay whenever the exact solution does, i.e. whenever $\text{Re}(\lambda) < 0$ (equivalently $\text{Re}(z) < 0$), we want the region of absolute stability to cover as much of the left half-plane as possible. Note that λ may be complex, corresponding to oscillatory decaying solutions.

8. ODE 2

Example 8.2 (Stability of the Euler Method). The Euler method uses the formula $x_{n+1} = x_n + hf(t_n, x_n)$. Applying this to the test equation $x' = \lambda x$ gives

$$x_{n+1} = x_n + h\lambda x_n = (1 + h\lambda)x_n.$$

So

$$x_{n+1} = (1 + h\lambda)x_n = (1 + h\lambda)^2x_{n-1} = \dots = (1 + h\lambda)^{n+1}x_0.$$

Because we know the exact solution, we can calculate the absolute error that the Euler method produces:

$$E_n := |x_n - x(t_n)| = |x_0(1 + h\lambda)^n - x_0e^{\lambda t_n}|.$$

In the case where $\lambda < 0$ we have that the exact solution decays to zero as $n \rightarrow \infty$. So the error will be determined by the behaviour of the first term:

$$\begin{aligned} \lim_{n \rightarrow \infty} E_n &= \lim_{n \rightarrow \infty} |x_0(1 + h\lambda)^n| \\ &= \begin{cases} 0 & \text{if } |1 + h\lambda| < 1 \\ \infty & \text{if } |1 + h\lambda| > 1. \end{cases} \end{aligned}$$

The error grows exponentially unless $|1 + h\lambda| < 1$. This is the stability condition for the Euler method. It requires us to choose a step size

$$h < \frac{2}{|\lambda|}$$

So the amplification factor for the Euler method is $Q(z) = 1 + z$, and its region of absolute stability is the disk $|1 + z| < 1$, centred at $z = -1$ with radius 1. For real negative λ this recovers the step-size constraint $h < 2/|\lambda|$ derived above.

Example 8.3 (Stability of the Midpoint Method). The midpoint method uses the formula

$$x_{n+1} = x_n + hf\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}f(t_n, x_n)\right).$$

Applying this to the test equation $x' = \lambda x$ gives

$$x_{n+1} = x_n + h\lambda \left(x_n + \frac{h}{2}\lambda x_n\right) = x_n \left(1 + h\lambda + \frac{(h\lambda)^2}{2}\right).$$

So the amplification factor is $Q(z) = 1 + z + \frac{z^2}{2}$, and the region of absolute stability is the set of $z \in \mathbb{C}$ for which

$$\left|1 + z + \frac{z^2}{2}\right| < 1.$$

Implicit methods tend to have a larger region of absolute stability, often including the entire left half plane, in which case they are called “unconditionally stable”.

Example 8.4 (Stability of the Backward Euler Method). We demonstrate this in the case of the backward Euler method, which uses the formula

$$x_{n+1} = x_n + hf(t_{n+1}, x_{n+1}).$$

Applying this to the test equation $x' = \lambda x$ gives

$$x_{n+1} = x_n + h\lambda x_{n+1}.$$

So

$$x_{n+1} = \frac{x_n}{1 - h\lambda}.$$

The region of absolute stability is the set of $z \in \mathbb{C}$ for which

$$\left| \frac{1}{1 - z} \right| < 1,$$

or, equivalently,

$$|z - 1| > 1.$$

This is almost the entire complex plane, except for the disk of radius 1 centred at $z = 1$. In particular it contains the entire left half-plane, so no upper bound on the step size is needed for stability, even when $|\lambda|$ is very large (as arises in stiff equations, discussed in Section 8.5). We therefore say that the backward Euler method is **unconditionally stable**.

8.5. Stiff Equations

A differential equation is called **stiff** if the exact solution contains a rapidly decaying transient term. This is a problem for explicit numerical methods because the stability condition forces the step size to be very small — small enough to track the transient — even after the transient has disappeared and the remaining solution varies slowly. The step-size restriction therefore persists for the entire computation, making it very slow.

An example of such a rapidly decaying term would be a term of the form $e^{-\gamma t}$ with $\gamma \gg 1$. As an example consider the ODE

$$x' = -100x + \sin t.$$

8. ODE 2

This is similar to equations we solved in Exercise 7.6, just with a larger negative coefficient in front of x . The solution to this equation is

$$x(t) = Ce^{-100t} + \frac{100}{10001} \sin t - \frac{1}{10001} \cos t.$$

The term e^{-100t} decays very rapidly. We refer to such a term in the solution as a “transient” because it becomes irrelevant after a short time. In spite of the fact that this transient term is irrelevant, it forces us to take a very small step size in order to avoid an instability if we are using an explicit method. Thus for a stiff equation it is advisable to use an implicit method instead.

Exercise 8.14. Verify by substitution that

$$x(t) = Ce^{-100t} + \frac{100}{10001} \sin t - \frac{1}{10001} \cos t$$

is indeed the general solution to $x' = -100x + \sin t$. (The particular solution can be found by the method of undetermined coefficients; the analytic solution to $x' = -\frac{1}{3}x + \sin t$ was stated in Exercise 7.6.) Then:

- What value of C gives the solution satisfying the initial condition $x(0) = 0$?
- The transient Ce^{-100t} has decayed to 5% of its initial value by time t^* . Find the approximate value of t^* . You can use that $e^{-3} \approx 0.05$.
- What is the maximum step size that forward Euler can use on this equation and still be stable?

Exercise 8.15. Investigate the stiffness of $x' = -100x + \sin t$ with $x(0) = 0$ numerically.

- Solve the equation using forward Euler with step sizes $h = 0.1$, $h = 0.01$, and $h = 0.001$. Plot all three numerical solutions together with the exact solution on the interval $t \in [0, 1]$. What do you observe?
- Repeat with the backward Euler method using the same step sizes. What do you observe?
- Repeat with the RK4 method. What is the largest step size for which RK4 remains stable on this problem? (Hint: the stability region of RK4 reaches approximately $z \approx -2.8$ along the negative real axis.) Compare with what you found for forward Euler.
- Explain in your own words why an implicit method is preferable for stiff equations.

Stiffness also arises naturally in systems of differential equations. In a system, different components of the solution can decay at very different rates. If one component decays on a timescale of $1/\gamma_{\text{fast}}$ while another decays on a timescale of $1/\gamma_{\text{slow}}$, an explicit method must use a step size dictated by γ_{fast} , even if we only care about the slow component.

Example 8.5. A simple example comes from the overdamped spring. Consider the second-order ODE

$$x'' + (\gamma + 1)x' + \gamma x = 0,$$

which describes a spring-mass system with damping. This is the system from Exercise 7.9 with $m = 1$, $b = \gamma + 1$ and $k = \gamma$. Writing $u = x$ and $v = x'$ converts this to the first-order system

$$\begin{pmatrix} u \\ v \end{pmatrix}' = \begin{pmatrix} 0 & 1 \\ -\gamma & -(\gamma + 1) \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix}.$$

The characteristic equation of the matrix is $\lambda^2 + (\gamma + 1)\lambda + \gamma = (\lambda + 1)(\lambda + \gamma) = 0$, so the eigenvalues are $\lambda_1 = -1$ and $\lambda_2 = -\gamma$. The general solution is

$$x(t) = C_1 e^{-t} + C_2 e^{-\gamma t}.$$

For large γ the term $e^{-\gamma t}$ is a rapidly decaying transient and the stiffness ratio is $\gamma/1 = \gamma$. An explicit method must take $h < 2/\gamma$ to remain stable, even though the long-term behaviour of x is governed by the much slower rate e^{-t} .

Exercise 8.16. Consider the system from Example 8.5 with initial conditions $u(0) = 1$ and $v(0) = x'(0) = 0$:

$$\begin{pmatrix} u \\ v \end{pmatrix}' = \begin{pmatrix} 0 & 1 \\ -\gamma & -(\gamma + 1) \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix}, \quad \begin{pmatrix} u(0) \\ v(0) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

- Use the initial conditions $u(0) = 1$ and $v(0) = x'(0) = 0$ to find C_1 and C_2 .
- Write down the exact solution $x(t)$ for these initial conditions and plot it.
- For $\gamma = 100$, how small must the step size h be for the forward Euler method to be stable? How does this compare to the timescale on which the solution $u(t) = x(t)$ actually changes significantly?

Exercise 8.17. Solve the stiff system from Example 8.5 numerically with $\gamma = 100$ on the interval $t \in [0, 5]$.

8. ODE 2

- a. Solve the system with the forward Euler method and step sizes $h = 0.1$, $h = 0.01$, and $h = 0.001$. You can use your code from Exercise 7.9. At which step size does forward Euler first produce a stable solution?
- b. Solve the system with the backward Euler method using the same step sizes. Does backward Euler remain stable even for $h = 0.1$? You can use the following function for the higher-dimensional backward Euler method:

```
import numpy as np
from scipy import optimize

def backward_euler(F,x0,t0,tmax,dt):
    N = round((tmax - t0)/dt)
    dt = (tmax - t0)/N
    t = np.linspace(t0, tmax, N+1)
    x = np.zeros((len(t), len(x0)))
    x[0,:] = x0
    for n in range(len(t)-1):
        # G takes a vector y and returns a vector
        G = lambda y: y - dt*F(t[n+1],y) - x[n,:]
        # fsolve with initial guess as the previous time step's vector
        x[n+1,:] = optimize.fsolve(G, x[n,:])
    return t, x
```

- c. By what factor can backward Euler take larger steps than forward Euler and still be stable? How does this factor relate to γ ?

Warning

Note that while Backward Euler is stable for large h , it is still only a first-order method. Large steps might be stable but could be very inaccurate!

8.6. Algorithm Summaries

Exercise 8.18. Consider the first-order differential equation $x' = f(t, x)$. What is the Runge Kutta 4 method for approximating the solution to this differential equation? What is the order of accuracy of the Runge Kutta 4 method?

Exercise 8.19. Explain in clear language what the Runge Kutta 4 method does geometrically.

Exercise 8.20. Consider the first-order differential equation $x' = f(t, x)$. What is the Backward Euler method for approximating the solution to this differential equation? What is the order of accuracy of the backward Euler method? What is the region of absolute stability for the backward Euler method?

Exercise 8.21. Explain in clear language what the Backward Euler method does geometrically.

8.7. Exam-Style Question

- (a) Explain geometrically the difference between the explicit (forward) Euler method and the implicit backward Euler method. What is the fundamental property of backward Euler that makes it more suited for stiff equations? [3 marks]
 - (b) Show that the backward Euler method is unconditionally stable for the linear test equation $x' = \lambda x$ where $\text{Re}(\lambda) < 0$. [3 marks]
 - (c) Show that the backward Euler method has a local truncation error of order $O(h)$. [2 marks]
 - (d) State, but don't derive, the order of accuracy for the Runge-Kutta 4 (RK4) method. How many evaluations of the function $f(t, x)$ are required at each time step? [2 marks]
-

9. PDE 1

When you open the toolkit of differential equations you see the hammers and saws of engineering and physics for the past two centuries and for the foreseeable future.

–Benoit Mandelbrot

9.1. Intro to PDEs

Partial differential equations (PDEs) are differential equations involving the partial derivatives of an unknown multivariable function. In most of this chapter and the next we will examine two classical problems from physics: heat transport phenomena and wave phenomena. Do not think, however, that just because we are focusing on these two primary examples that this is the extent of the utility of PDEs. Basically, every scientific field has been impacted by (or has directly impacted) the study of PDEs. Any phenomenon that can be modelled via the change in multiple continuous variables (not restricted to space and time) is likely governed by a PDE model.

There is a wealth of wonderful theory for finding analytic solutions to many special classes of PDEs. However, most PDEs simply do not lend themselves to analytic solutions that we can write down in terms of the regular mathematical operations of sums, products, powers, roots, trigonometric functions, logarithms, etc. For these PDEs we must turn to numerical methods to approximate the solution.

Recall that numerical solutions to ODEs were approximations of the value of the unknown function at a discrete set of times. Similarly, numerical solutions to PDEs are going to be approximations of the value of the unknown function at a discrete set of points in time AND space.

What we will cover in this chapter will include one primary and powerful technique for approximating solutions to PDEs: **the finite difference method**. There are many other techniques for approximating solutions to PDEs, and the field of numerical PDEs is still an active area of mathematical and scientific research.

9.2. The Heat Equation

You have probably met the heat equation, also known as the diffusion equation, in a previous module. The heat equation is a partial differential equation that describes how heat diffuses through a material. The heat equation is a parabolic PDE and is given by

$$\frac{\partial u}{\partial t} = D \nabla^2 u$$

where $u(t, x)$ is the temperature of the material at time t and position x and D is the diffusion coefficient. The heat equation is a simple model for heat diffusion but also describes diffusion in general, like the diffusion of a solute in a solvent or of plants in a field or, well, you get the idea.

You of course know that the heat equation is easy to solve analytically, given that it is a linear homogeneous PDE with constant coefficients. However, the finite difference method is a powerful tool for solving similar PDEs that do not have simple analytic solutions. The advantage of using the heat equation as a test case for the finite difference method is that we can easily verify the accuracy of our numerical solutions by comparing them to the known analytic solutions.

9.2.1. In One Spatial Dimension

For the sake of simplicity we will start by considering the heat equation in 1 spatial dimension:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}.$$

We will also use the alternative notation

$$u_t = D u_{xx}, \tag{9.1}$$

where the subscripts denote partial derivatives.

Exercise 9.1. Just as we did in Chapter 7 to approximate solutions to ODEs, we will start by partitioning the spatial domain into finitely many pieces and we will partition time into finitely many pieces. We do this by introducing a grid of points (t_n, x_i) where $t_n = t_0 + n \Delta t$ and $x_i = x_0 + i \Delta x$. Then we want to build a numerical approximation to the function $u(t, x)$ at these grid points.

First we need to introduce some notation for the numerical solution. As you will see in a moment, there is a lot to keep track of in numerical PDEs so careful indexing and well-chosen notation is essential. Let U_i^n be the approximation of the solution to $u(t, x)$ at the point $t = t_n = t_0 + n \Delta t$ and $x = x_i = x_0 + i \Delta x$ (since we have two variables we need two indices). For example, U_4^1 is the value of the approximation at time t_1 and at the spatial point x_4 .

9.2. The Heat Equation

Next we need to approximate both derivatives u_t and u_{xx} in the PDE using methods that we have used before. Now would be a good time to go back to Section 5.2 and refresh your memory for how we build approximations of derivatives.

- (a) Use the forward-difference formula to approximate the time derivative u_t at the point $t = t_n$ and $x = x_i$.

$$u_t(t_n, x_i) \approx \frac{??? - ???}{???}.$$

- (b) Use the centred-difference formula to approximate the second spatial derivative u_{xx} at the point $t = t_n$ and $x = x_i$.

$$u_{xx}(t_n, x_i) \approx \frac{??? - ??? + ???}{???}.$$

- (c) Put your answers from parts (a) and (b) together using the 1D heat equation (Eq. 9.1)

$$\frac{??? - ???}{\Delta t} = D \left(\frac{??? - ??? + ???}{\Delta x^2} \right).$$

Be sure that your indexing is correct: the superscript n is the index for time and the subscript i is the index for space.

- (d) Rearrange your result from part (c) to solve for U_i^{n+1} :

$$U_i^{n+1} = ??? + \frac{D\Delta t}{\Delta x^2} (??? - ??? + ???).$$

The iterative scheme which you just derived is called the **forward difference scheme** for the heat equation. Notice that the term on the left is the only term at the next time step $n + 1$. So, for every spatial point x_i we can build U_i^{n+1} by evaluating the right-hand side of the finite difference scheme.

- (e) The numerical errors made by using the forward difference scheme we just built come from two sources: from the approximation of the time derivative and from the approximation of the second spatial derivative. Fill in the question marks in the powers of the following expression:

$$\text{Numerical Error} = \mathcal{O}(\Delta t^{???}) + \mathcal{O}(\Delta x^{???}).$$

- (f) Explain what the result from part (e) means in plain English.

9. PDE 1

There are many different finite difference schemes due to the fact that there are many different ways to approximate derivatives (See Section 5.2). One convenient way to keep track of which information you are using and what you are calculating in a finite difference scheme is to use a **finite difference stencil image**. Figure 9.1 shows the finite difference stencil for the approximation to the heat equation that you built in the previous exercise. In this figure we are showing that the function values U_{i-1}^n , U_i^n , and U_{i+1}^n at the points x_{i-1} , x_i , and x_{i+1} at time step t_n are used to calculate U_i^{n+1} . We will build similar stencil diagrams for other finite difference schemes throughout this chapter.

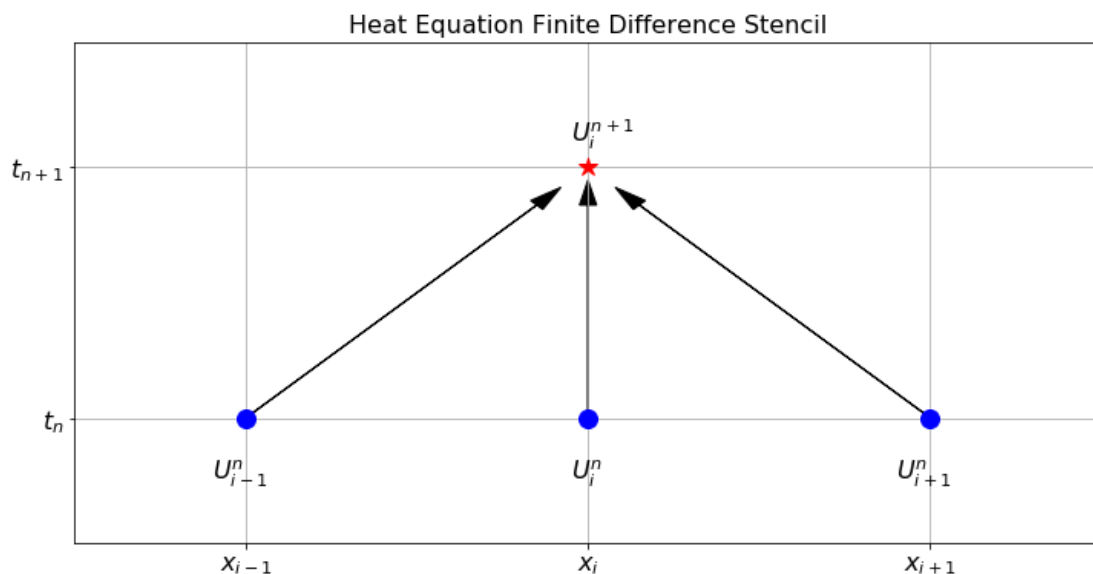


Figure 9.1.: The forward difference stencil for the 1D heat equation.

Exercise 9.2. Now we want to implement your answer to part (d) of the previous exercise to approximate the solution to the following problem:

$$u_t = 0.1u_{xx}$$

on the domain $0 < x < 1$ and $0 < t < 1$ with the initial condition

$$u(0, x) = \sin(2\pi x)$$

and boundary conditions

$$u(t, 0) = 0, \text{ and } u(t, 1) = 0.$$

For this purpose divide the x domain into 20 equal pieces and the t domain into 100 equal pieces.

Some partial code is given below to get you started.

- First we import the proper libraries, set up the time domain, and set up the spatial domain.

```
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interactive

# Write code to give a vector of times starting at t=0 and ending
# at t=1 that divides the interval into 100 equal pieces.

# Calculate the step size `dt`.

# Write code to give a vector of x values starting at x=0 and
# ending exactly at x=1 that divides the interval into 20 equal pieces.

# Calculate the step size `dx`.

# Specify the diffusion coefficient
D = 0.1
# The coefficient "a" appears in the forward difference scheme.
a = D*dt / dx**2

print("dt=", dt, ", dx=", dx, " and D dt/dx^2=", a)
```

- Next we build the array U so we can store all of the approximations at all times and at all spatial points. The array will have the dimensions $\text{len}(t)$ by $\text{len}(x)$. We then need to enforce the boundary conditions so for all times we fill the proper portions of the array with the proper boundary conditions. Lastly, we will build the initial condition for all spatial steps in the first time step.

```
U = np.zeros((len(t),len(x)))
U[:,0] = # left boundary condition
U[:,-1] = # right boundary condition
U[0,:] = # the function for the init. condition (should depend on x)
```

- Now we step through a loop that fills the U array one row at a time. Keep in mind that we want to leave the boundary conditions fixed so we will only fill indices 1 through -2 (stop and explain this). Be careful to get the indexing correct. For example, if we want U_i^n we use $U[n,1:-1]$, if we want U_{i+1}^n we use $U[n,2:]$, if we want U_i^{n+1} we use $U[n+1,1:-1]$, etc.

```
for n in range(len(t)-1):
    U[n+1,1:-1] = U[n,1:-1] + a*( U[n,2:] - 2*U[n,1:-1] + U[n,0:-2])
```

9. PDE 1

- We can now make a plot of the solution at the final time step to see what the solution looks like at $t = 1$.

```
plt.plot(x, U[:,?])
plt.xlabel("x")
plt.ylabel("u")
plt.grid()
```

- Let us also calculate an approximate value for $u(0.2, 0.25)$ by finding the indices in the `t` and `x` vectors that correspond to $t = 0.2$ and $x = 0.25$ and then looking up the value in the `U` array at those indices.

```
U[:,?]
```

Exercise 9.3. Now visualise the whole solutions over the entire time range. You can either make a plot or an animation to illustrate the time evolution of u . For each of these there are various Python packages you could use. Below is a function `plot_solution_1d()` using `plotly` to make a plot and a function `animate_solution_1d()` using `matplotlib` to make an animated 2D plot. You can use either of these or you can use your own plotting code.

```
import plotly.graph_objects as go

def plot_solution_1d(t, x, U):
    """Plots the numerical approximation to a function u(t,x).

    Args:
        t: A vector of time values.
        x: A vector of spatial values.
        U: A 2D array approximating the solution u(t,x) at each grid point.
    """
    fig = go.Figure(data=[go.Surface(z=U, x=x, y=t)])
    fig.update_layout(
        width=800, height=600,
        scene=dict(
            xaxis_title='x',
            yaxis_title='t',
            zaxis_title='u'
        )
    )
    return fig
```

```

import matplotlib.pyplot as plt
from matplotlib import animation, rc
from IPython.display import HTML

def animate_solution_1d(t, x, U):
    """Animates the numerical approximation to a function u(t,x).

    Args:
        t: A vector of time values.
        x: A vector of spatial values.
        U: A 2D array approximating the solution u(t,x) at each grid point.
    """
    fig, ax = plt.subplots()
    plt.close()
    ax.grid()
    ax.set_xlabel("x")
    ax.set_ylabel("u")
    ax.set_xlim((np.min(x), np.max(x)))
    ax.set_ylim((np.min(U), np.max(U)))
    frame, = ax.plot([], [], linewidth=2)

    # Don't display every time
    step = int(len(t)/30)+1
    frames = range(0, int(len(t)/step), 1)

    def animator(i):
        n = i*step
        ax.set_title(f"t = {t[n]:.2f}")
        frame.set_data(x, U[n,:])
        return (frame, )

    ani = animation.FuncAnimation(fig, animator, frames=frames, interval=100)
    rc('animation', html='jshtml') # embed in the HTML for Google Colab
    return ani

```

Exercise 9.4. Now wrap up your code for solving the one-dimensional heat equation as a function so that you can easily call it with different parameters.

```

def heat1d(u_0, D=0.1, t_0=0, t_max=1, N_t=100, x_left=0, x_right=1, N_x=20):
    """Solves the 1D heat equation using the forward difference method.

```

9. PDE 1

```
This function solves the 1D heat equation with given initial and
boundary conditions. It also prints a diagnostic message stating
the step sizes `dt` and `dx` used and the value of `a = D*dt/dx**2`.

Args:
    u_0: A function giving the initial condition u(0,x).
    D: The diffusion coefficient. Defaults to 0.1.
    t_0: The initial time. Defaults to 0.
    t_max: The maximum time. Defaults to 1.
    N_t: The number of time steps. Defaults to 100.
    x_left: The left boundary of the spatial domain. Defaults to 0.
    x_right: The right boundary of the spatial domain. Defaults to 1.
    N_x: The number of spatial steps. Defaults to 20.

Returns:
    A tuple containing the following:
        t: A vector of time values.
        x: A vector of spatial values.
        U: A 2D array approximating the solution u(t,x) at each grid point.
"""
# Your code goes here
```

Check that your function works by using it to calculate the same solution as in Exercise 9.2 and plotting the solution with `plot_solution_1d()` or `animate_solution_1d()`.

Exercise 9.5. Now run your `heat1d` function from Exercise 9.4 with the same diffusion coefficient $D = 0.1$, the same number $N_t = 100$ of time steps and the same initial and boundary conditions but double the number of spatial steps to $N_x=40$. Plot the solution on the domain $t \in [0, 1]$ and $x \in [0, 1]$. Do you believe what you see? What is happening to the solution?

Exercise 9.6. You will have found that you did not get a sensible solution from your method for the previous problem. The point of this exercise is to show that value of $a = D \frac{\Delta t}{\Delta x^2}$ controls the stability of the forward difference solution to the heat equation, and furthermore that there is a threshold for a above which the forward difference scheme will be unstable. Experiment with values of Δt and Δx and conjecture the values of $a = D \frac{\Delta t}{\Delta x^2}$ that give a stable result. Your conjecture should take the form:

If $a = D \frac{\Delta t}{\Delta x^2} < \underline{\hspace{1cm}}$ then the forward difference solution for the 1D heat equation is stable. Otherwise it is unstable.

Hint: the threshold is a simple fraction. If you think you have found a value for a at which the method is stable, run the simulation for longer (by increasing both `t_max` and `N_t` to keep the same Δt) to check that it is really stable. Close to the threshold the errors grow more slowly (albeit still exponentially).

Exercise 9.7. Consider the one dimensional heat equation with diffusion coefficient $D = 1$:

$$u_t = u_{xx}.$$

We want to solve this equation on the domain $x \in [0, 1]$ and $t \in [0, 0.1]$ subject to the initial condition $u(0, x) = \sin(\pi x)$ and the boundary conditions $u(t, 0) = u(t, 1) = 0$.

- Show that the function $u(t, x) = e^{-\pi^2 t} \sin(\pi x)$ is a solution to this heat equation, satisfies the initial condition, and satisfies the boundary conditions.
- Pick values of Δt and Δx so that you can get a stable forward difference solution to this heat equation. Then make a plot of your numerical solution.
- Compare your plot to the plot of the exact solution that you can get with

```
X, T = np.meshgrid(x, t)
u_exact = np.exp(-np.pi**2*T)*np.sin(np.pi*X)
plot_solution_1d(t, x, u_exact)
```

Exercise 9.8. Now let us change the initial condition to $u(0, x) = \sin(\pi x) + \sin(3\pi x)$. We will keep the same boundary conditions as before: $u(t, 0) = u(t, 1) = 0$.

- Show that the function $u(t, x) = e^{-\pi^2 t} \sin(\pi x) + e^{-9\pi^2 t} \sin(3\pi x)$ is a solution to this heat equation, matches this new initial condition, and matches the boundary conditions.
- Pick values of Δt and Δx so that you can get a stable forward difference solution to this heat equation. Make a 3d plot of your numerical solution.
- Compare your plot to the plot of the exact solution.

9.2.2. Different Boundary Conditions

In any initial and boundary value problem such as the heat equation, the boundary conditions are often of Dirichlet or Neumann type. In Dirichlet boundary conditions the values of the solution at the boundary are specified. In contrast, Neumann boundary conditions specify the flux at the boundary instead of the value of the solution.

Exercise 9.9 (Time-dependent Dirichlet Boundary Condition). Modify your 1D heat equation code to plot and to animate an approximate solution of the diffusion equation $u_t = 0.5u_{xx}$ with $x \in (0, 1)$, $u(0, x) = \sin(2\pi x)$, $u(t, 0) = 0$ and $u(t, 1) = \sin(5\pi t)$. See if everyone in your group gets the same plot and animation.

Exercise 9.10 (Neumann Boundary Condition). Consider the 1D heat equation $u_t = u_{xx}$ with boundary conditions $u_x(t, 0) = 0$ and $u(t, 1) = 0$ with initial condition $u(0, x) = \cos(\pi x/2)$. So at $x = 0$ we are imposing a Neumann boundary condition and at $x = 1$ we are imposing a Dirichlet boundary condition. Check that the initial condition satisfies these boundary conditions. As the heat profile evolves in time the Neumann boundary condition $u_x(t, 0) = 0$ says that the slope of the solution needs to be fixed at 0 at the left-hand boundary.

- (a) The Neumann boundary condition $u_x(t, 0) = 0$ can be approximated with the first order approximation

$$u_x(t_n, 0) \approx \frac{U_1^n - U_0^n}{\Delta x} \text{ for all } n > 0.$$

If we set this approximation to 0 (since $u_x(t, 0) = 0$) and solve for U_0^n we get an additional constraint at every time step of the numerical solution to the heat equation:

$$U_0^n = ??? \text{ for all } n > 0.$$

- (b) Modify your 1D heat equation code to implement this Neumann boundary condition, plot the numerical solution and verify visually that the Neumann boundary is satisfied.
-

9.3. Other Parabolic PDEs

The heat equation is a parabolic PDE and the forward-difference method that we have developed can be adapted to work for other parabolic PDEs.

9.3.1. Reaction-Diffusion Equations

For example, the heat equation can be modified to include a reaction term. The reaction-diffusion equation is a PDE that models the diffusion of a substance in space and time with a reaction term that describes the rate of change of the substance due to some reaction. While it has its origin in chemistry, it shows up in many other fields as well, for example in ecology and epidemiology.

Exercise 9.11 (Fisher-KPP Equation). Modify your 1D heat equation code to calculate an approximate solution of the Fisher-KPP equation

$$u_t = u_{xx} + u(1 - u)$$

with $t \in [0, 10]$, $x \in (0, 50)$, boundary conditions

$$u(t, 0) = 0, \quad u(t, 50) = 1$$

and initial condition

$$u(0, x) = \frac{1 + \tanh\left(\frac{x - 40}{2}\right)}{2}.$$

Use `animate_solution_1d()` to visualize the solution. How does the solution change as time evolves?

9.3.2. Advective-Diffusion Equations

The diffusion term usually arises from random spatial motion of particles. However, in some cases the particles are advected by a flow field. In this case we need to add an advection term to the diffusion equation. The advection-diffusion equation is a PDE that models the diffusion of a substance in space and time with an advection term that describes the rate of change of the substance due to some flow field.

Exercise 9.12. Modify your 1D heat equation code to plot an approximate solution of the following simple advection-diffusion equation:

$$u_t = 0.1u_{xx} - u_x$$

Use the forward difference formula for the u_x term and the centred difference formula for the u_{xx} term. Use the initial condition $u(0, x) = \sin(\pi x)$, Dirichlet boundary conditions $u(t, 0) = 0$ and $u(t, 1) = 0$, and $t \in [0, 1]$. Use 20 spatial steps and 100 time steps. Make a plot and an animation of the solution.

9.4. Stability of Forward Difference Method

While exploring the explicit finite-difference method for solving the 1d heat equation $u_t = Du_{xx}$ we encountered the stability condition

$$a = \frac{D\Delta t}{(\Delta x)^2} \leq \frac{1}{2}.$$

We now want to understand where this condition comes from.

We start by setting up the notation for solving the heat equation using the explicit finite-difference method. We discretise the spatial variable x into N_x intervals with grid points $x_0, \dots, x_{N_x}$ with stepsize $\Delta x = (x_{N_x} - x_0)/N_x$. We discretise the time variable t into N_t intervals with grid points $t_0, \dots, t_{N_t}$ with stepsize $\Delta t = (t_{N_t} - t_0)/N_t$. We denote the approximation of $u(t_n, x_i)$ by U_i^n . The initial condition sets $U_i^0 = u(t_0, x_i)$. We work with homogeneous Dirichlet boundary conditions, so $U_0^n = U_{N_x}^n = 0$ for all n .

The approximations at the remaining points is then calculated by the formula

$$U_i^{n+1} = U_i^n + a(U_{i+1}^n - 2U_i^n + U_{i-1}^n)$$

for $i = 1, \dots, N_x - 1$. You derived this in Section 9.2.1 using the finite-difference formulae for the derivatives from Section 5.2. We rewrite this in matrix notation:

$$\begin{pmatrix} U_1^{n+1} \\ \vdots \\ U_{N_x-1}^{n+1} \end{pmatrix} = \begin{pmatrix} 1-2a & a & & \\ a & 1-2a & a & \\ & \ddots & \ddots & \ddots \\ & & a & 1-2a \end{pmatrix} \begin{pmatrix} U_1^n \\ \vdots \\ U_{N_x-1}^n \end{pmatrix}.$$

The matrix is tridiagonal, with $1 - 2a$ on the diagonal and a on the two off-diagonals. We denote the matrix by A and the two vectors by $\mathbf{U}_{n+1}$ and $\mathbf{U}_n$ respectively. So we have the formula

$$\mathbf{U}_{n+1} = A\mathbf{U}_n.$$

The solution at time t_n is then given by

$$\mathbf{U}_n = A^n \mathbf{U}_0.$$

We want to understand how errors evolve over time. If errors grow exponentially over time then we call the method unstable and the method is not useful.

Let us assume that at some step, for convenience let us choose step 0, an error is introduced:

$$\tilde{\mathbf{U}}_0 = \mathbf{U}_0 + \mathbf{e}.$$

Then after n steps we have

$$\tilde{\mathbf{U}}_n = A^n \tilde{\mathbf{U}}_0 = A^n(\mathbf{U}_0 + \mathbf{e}) = A^n \mathbf{U}_0 + A^n \mathbf{e} = \mathbf{U}_n + A^n \mathbf{e}.$$

9.4. Stability of Forward Difference Method

So the error at time t_n is

$$^{(n)} = A^n.$$

To see if the error grows exponentially over time, we expand the initial error in terms of the eigenvectors of the matrix A :

$$= \sum_i \epsilon_i \mathbf{v}_i,$$

where

$$A\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

and the sum is over all eigenvectors $\mathbf{v}_i$ of A . Then

$$^{(n)} = \sum_i \epsilon_i \lambda_i^n \mathbf{v}_i.$$

This shows that if all eigenvalues have absolute value less than 1, then the method is stable. If at least one eigenvalue has absolute value greater than 1, then the corresponding component of the error will grow exponentially with time and the method is unstable.

There is a nice method to determine the eigenvalues of the matrix A , using Fourier analysis. We will not discuss this in this module and instead just give the result and then look at a simple example. The result (which you do not need to remember) is that the eigenvalues of the matrix A are given by

$$\lambda_k = 1 - 4a \left(\sin \left(\frac{k\pi}{2N_x} \right) \right)^2$$

for $k = 1, \dots, N_x - 1$. For stability we need $|\lambda_k| \leq 1$ for all k , i.e.,

$$0 \leq a \left(\sin \left(\frac{k\pi}{2N_x} \right) \right)^2 \leq \frac{1}{2}$$

for all k . The most stringent condition is that coming from $k = N_x - 1$, so the stability condition is

$$a \leq \frac{1}{2} \left(\sin \left(\frac{(N_x - 1)\pi}{2N_x} \right) \right)^{-2}.$$

In the limit $N_x \rightarrow \infty$ this gives the condition $a \leq 1/2$.

Example 9.1. Consider the heat equation on the spatial domain $x \in [0, 1]$ and divide this into three subintervals, so that our spatial grid consists of $x_0 = 0, x_1 = 1/3, x_2 = 2/3$ and $x_3 = 1$. The matrix A is then

$$A = \begin{pmatrix} 1 - 2a & a \\ a & 1 - 2a \end{pmatrix}.$$

9. PDE 1

The characteristic polynomial is

$$\begin{aligned}\det(A - \lambda I) &= \begin{vmatrix} 1 - 2a - \lambda & a \\ a & 1 - 2a - \lambda \end{vmatrix} \\ &= (1 - 2a - \lambda)^2 - a^2 \\ &= \lambda^2 - 2(1 - 2a)\lambda + (1 - 2a)^2 - a^2.\end{aligned}$$

The roots of this polynomial are

$$\lambda_{\pm} = 1 - 2a \pm a.$$

We need both of these to have a magnitude less than 1 for the method to be stable. This gives us an upper bound on the allowed a . The eigenvalue whose magnitude will increase above 1 first as a increases is $\lambda_- = 1 - 3a$, which equals -1 when $a = 2/3$ (meaning the error in that mode alternates sign every time step). So the stability condition is $a \leq 2/3$.

In this chapter we have developed the **explicit forward difference scheme** for the 1D heat equation, seen how to handle Dirichlet and Neumann boundary conditions, and explored variations such as reaction-diffusion and advection-diffusion equations. We have also seen that the scheme is only stable when $a = D \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}$, which places a severe constraint on the time step when the spatial resolution is fine. In the next chapter we will overcome this limitation by introducing **implicit schemes** such as the backward difference scheme and the Crank-Nicolson scheme, which are unconditionally stable and allow much larger time steps.

9.5. Exam-Style Question

- (a) Consider the 1D heat equation $u_t = Du_{xx}$. Using a forward difference approximation for the time derivative and a centred difference approximation for the second spatial derivative, derive the explicit finite difference scheme for U_i^{n+1} . Use step sizes Δt and Δx , and let $U_i^n \approx u(t_n, x_i)$. [3 marks]
- (b) What are the truncation errors associated with the forward difference approximation for u_t and the centred difference approximation for u_{xx} ? State the overall order of the numerical scheme derived in part (a). [3 marks]
- (c) What condition on $a = D \frac{\Delta t}{\Delta x^2}$ must be satisfied for this explicit forward difference scheme to be numerically stable? [1 mark]

- (d) Suppose you have a Neumann boundary condition at the left boundary, $u_x(t, 0) = 0$. Using a first-order forward difference approximation for the spatial derivative, express U_0^n in terms of other grid values. [2 marks]
- (e) The following incomplete Python code solves the 1D heat equation $u_t = Du_{xx}$ using the explicit scheme derived in part (a), subject to Dirichlet boundary conditions $u(t, 0) = 0$ and $u(t, 1) = 0$. Provide the missing code indicated by `...` [4 marks]

```
import numpy as np

def solve_heat1d(u_0, D, tmax, dt, xmax, dx):
    """
    Solves the 1D heat equation.
    u_0 is a function giving the initial condition.
    """
    t = np.arange(0, tmax + dt, dt)
    x = np.arange(0, xmax + dx, dx)

    a = ...

    U = np.zeros((len(t), len(x)))

    # Apply initial condition
    U[0, :] = u_0(x)

    # Boundary conditions are naturally 0 from np.zeros

    for n in range(len(t) - 1):
        # Update interior spatial points
        U[n+1, 1:-1] = ...

    return t, x, U
```

- (f) The explicit finite difference scheme from part (a) can be written in matrix form as $\mathbf{U}_{n+1} = A\mathbf{U}_n$, where $\mathbf{U}_n = (U_1^n, \dots, U_{N-1}^n)^T$ is the vector of interior grid values at time step n and A is a $(N-1) \times (N-1)$ matrix.
- Write down the matrix A explicitly for the case $N = 4$ (i.e., three interior grid points), expressing all entries in terms of $a = D\Delta t/(\Delta x)^2$. [2 marks]
 - Using the matrix form, write down an expression for $\mathbf{U}_n$ in terms of A , n , and the initial vector $\mathbf{U}_0$. [1 mark]
 - Suppose that at time step 0 an error ϵ is introduced, so that the computed vector is $\tilde{\mathbf{U}}_0 = \mathbf{U}_0 + \epsilon$. Show that the error at time step n is $\epsilon_n = A^n \epsilon$. [2 marks]

9. *PDE 1*

- (iv) Explain why and how the stability condition can be expressed in terms of the eigenvalues of A . [2 marks]

10. PDE 2

10.1. Two-dimensional Heat Equation

Now we transition to the two dimensional heat equation. Instead of thinking of this as heating a long metal rod we can think of heating a thin plate of metal (like a flat cookie sheet). The heat equation models the propagation of the heat energy throughout the 2D surface. In two spatial dimensions the heat equation is

$$\frac{\partial u}{\partial t} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right),$$

or, using subscript notation for the partial derivatives,

$$u_t = D(u_{xx} + u_{yy}).$$

Exercise 10.1. Let us build a numerical solution to the 2D heat equation. We need to make a minor modification to our notation since there is now one more spatial dimension to keep track of. Let $U_{i,j}^n$ be the approximation to u at the point (t_n, x_i, y_j) . For example, $U_{2,3}^4$ will be the approximation to the solution at the point (t_4, x_2, y_3) .

(a) We already know how to approximate the time derivative in the heat equation:

$$u_t(t_n, x_i, y_j) \approx \frac{U_{i,j}^{n+1} - U_{i,j}^n}{\Delta t}.$$

The new challenge now is that we have two spatial partial derivatives: one in x and one in y . Use what you learned in Chapter 5.2 to write the approximations of u_{xx} and u_{yy} .

$$u_{xx}(t_n, x_i, y_j) \approx \frac{??? - ??? + ???}{\Delta x^2}$$

$$u_{yy}(t_n, x_i, y_j) \approx \frac{??? - ??? + ???}{\Delta y^2}$$

Take careful note that the index i is the only one that changes for the x derivative. Similarly, the index j is the only one that changes for the y derivative.

- (b) Put your answers to part (a) together with the 2D heat equation

$$\frac{U_{i,j}^{n+1} - U_{i,j}^n}{\Delta t} = D \left(\frac{??? - ??? + ???}{\Delta x^2} + \frac{??? - ??? + ???}{\Delta y^2} \right).$$

- (c) Let us make one simplifying assumption. Choose the partition of the domain so that $\Delta x = \Delta y$. Note that we can usually do this in square domains. In more complicated domains we will need to be more careful. Simplify the right-hand side of your answer to part (b) under this assumption.

$$\frac{U_{i,j}^{n+1} - U_{i,j}^n}{\Delta t} = D \left(\frac{??? + ??? - ??? + ??? + ???}{???} \right).$$

- (d) Now solve your result from part (c) for $U_{i,j}^{n+1}$. Your answer is the explicit forward difference scheme for the 2D heat equation.

$$U_{i,j}^{n+1} = U_{???,???}^{???} + \frac{D \cdot ???}{???} (??? + ??? - ??? + ??? + ???)$$

The finite difference stencil for the 2D heat equation is a bit more complicated since we now have three indices to track. Hence, the stencil is naturally three dimensional. Figure 10.1 shows the stencil for the forward difference scheme that we built in the previous exercise. The left-hand subplot in the figure shows the five points used in time step t_n , and the right-hand subplot shows the one point that is calculated at time step t_{n+1} .

Exercise 10.2. Now we need to implement the finite difference scheme that you developed in the previous problem. As a model problem, consider the 2D heat equation $u_t = D(u_{xx} + u_{yy})$ on the domain $(x, y) \in [0, 1] \times [0, 1]$ with the initial condition $u(0, x, y) = \sin(\pi x) \sin(\pi y)$, Dirichlet boundary conditions $u(t, x, 0) = u(t, x, 1) = u(t, 0, y) = u(t, 1, y) = 0$, and $D = 1$. Fill in the holes in the following code chunks.

- First we import the proper libraries and set up the domains for x , y , and t .

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm # this allows for color maps
from ipywidgets import interactive
```

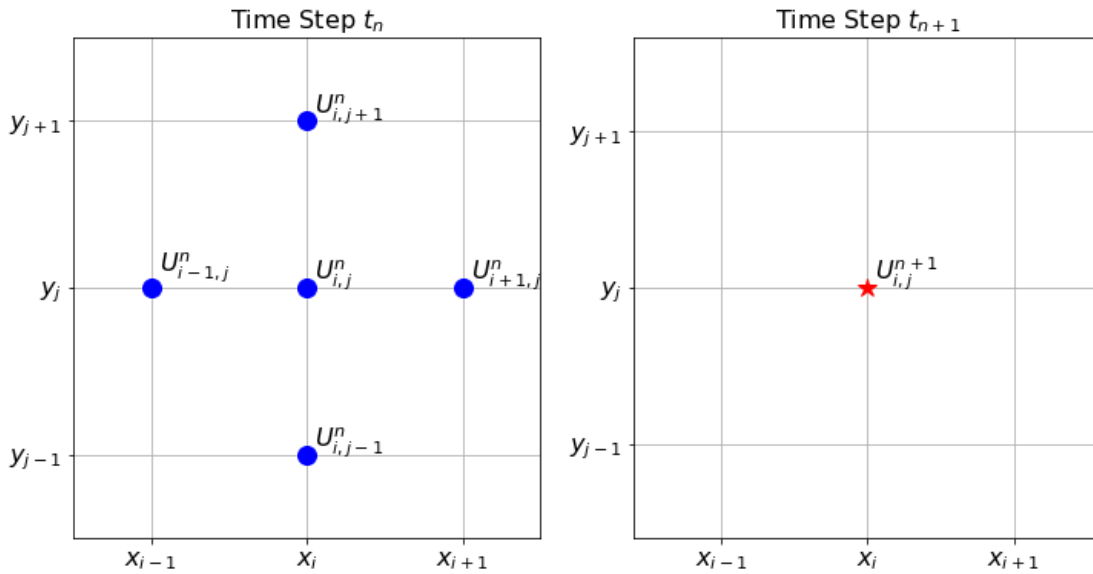


Figure 10.1.: The finite difference stencil for the 2D heat equation.

```
# Write code to build a linearly spaced array of x values
# starting at 0 and ending at exactly 1
x = # your code here
y = x # this could be generalised later
# The consequence of the previous line is that dy = dx.
dx = # Extract dx from your array of x values.
# Now write code to build a linearly spaced array of time values
# starting at 0 and ending at 0.1.
# You will want to use many more values for time than for space
# (think about the stability conditions from the 1D heat equation).
t = # your code here
dt = # Extract dt from your array of t values

# Next we will use the np.meshgrid() command to turn the arrays of
# x and y values into 2D grids of x and y values.
# If you match the corresponding entries of X and Y then you get
# every ordered pair in the domain.
Y, X = np.meshgrid(y, x)

# Next we set up a 3 dimensional array of zeros to store all of
# the time steps of the solutions.
U = np.zeros((len(t), len(x), len(y)))
```

- Next we have to set up the boundary and initial conditions for the given problem.

10. PDE 2

```
U[0,:,:] = # initial condition depending on X and Y
U[:,0,:] = # boundary condition for x=0
U[:,1,:] = # boundary condition for x=1
U[:, :, 0] = # boundary condition for y=0
U[:, :, 1] = # boundary condition for y=1
```

- We know that the value of $D\Delta t/\Delta x^2$ controls the stability of the forward difference method. Therefore, the next step in our code is to calculate this value and print it.

```
D = 1
a = D*dt/dx**2
print(a)
```

- Next for the part of the code that actually calculates all of the time steps. Be sure to keep the indexing straight. Also be sure that we are calculating all of the spatial indices *inside* the domain since the boundary conditions dictate what happens on the boundary.

```
for n in range(len(t)-1):
    U[n+1,1:-1,1:-1] = U[n,1:-1,1:-1] + \
        a*(U[n, 1:-1, 1:-1] + \
            U[n, 1:-1, 1:-1] - \
            4*U[n, 1:-1, 1:-1] + \
            U[n, 1:-1, 1:-1] + \
            U[n, 1:-1, 1:-1])
```

- Finally, we just need to visualize the solution. We can no longer make a plot of u against t, x and y because that would require four dimensions. So we will animate the solution. You can use the following function:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation
from mpl_toolkits.mplot3d import Axes3D
from IPython.display import HTML

def animate_solution_2d(t, x, y, U):
    Y, X = np.meshgrid(y, x)

    # Set up the figure and axis
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
```

```

# Initialize the surface plot
surface = [ax.plot_surface(X, Y, U[0, :, :], cmap='viridis')]

# Don't display every time
step = int(len(t)/30)+1
frames = int(len(t)/step)

def animate(i):
    n = i*step
    # Update the data of the surface plot for each frame
    ax.clear() # Clear the previous frame
    surface[0] = ax.plot_surface(X, Y, U[n, :, :], cmap='viridis')
    ax.set_zlim(np.min(U), np.max(U))
    ax.set_title(f"Time: {t[n]:.2f}")

# Create animation
ani = animation.FuncAnimation(fig, animate, frames=frames, repeat=True)
plt.close()

# Display the animation
return HTML(ani.to_jshtml())

```

Exercise 10.3. Time to do some experimentation with your new 2D heat equation code! Numerically solve the 2D heat equation with different boundary conditions (both Dirichlet and Neumann).

Exercise 10.4. In order for the forward difference solution to the 2D heat equation on a square domain to be stable we need $D\Delta t/\Delta x^2 < \underline{\hspace{1cm}}$.

Experiment with several parameters to empirically determine the bound. It is again a simple fraction that you should be able to guess.

Exercise 10.5. Now solve the 2D heat equation on a rectangular domain. You will need to make some modifications to your code since it is unlikely that assuming that $\Delta x = \Delta y$ is a good assumption any longer.

Let us summarize the stability criteria for the forward difference solutions to the heat equation.

- In the 1D heat equation the forward difference solution is stable if $D\Delta t/\Delta x^2 < \underline{\hspace{1cm}}$.
- In the 2D heat equation the forward difference solution is stable if $D\Delta t/\Delta x^2 < \underline{\hspace{1cm}}$ (assuming a square domain where $\Delta x = \Delta y$)

Exercise 10.6 (Sawtooth Errors). We have already seen that the 1D heat equation is stable if $D\Delta t/\Delta x^2 < 0.5$. The goal of this problem is to show what, exactly, occurs when we choose parameters in the unstable region. We will solve the PDE $u_t = u_{xx}$ on the domain $x \in [0, 1]$ with initial conditions $u(0, x) = \sin(\pi x)$ and homogeneous Dirichlet boundary conditions $u(t, 0) = u(t, 1) = 0$ for all $t \in [0, 0.25]$. The analytic solution is $u(t, x) = e^{-\pi^2 t} \sin(\pi x)$. To build the spatial and temporal grid use 20 spatial steps and 100 time steps. This means that $\Delta x = 0.05$ and $\Delta t = 0.0025$ so the ratio $D\Delta t/\Delta x^2 = 1 > 0.5$ (certainly in the unstable region). Solve the heat equation with your `heat1d()` function using these parameters. Make plots of the approximate solution on top of the exact solution at time steps 0, 10, 20, 30, 31, 32, 33, 34, etc. Describe what you observe.

Exercise 10.7 (Hedgehog Errors). Solve the 2D heat equation on the unit square with homogeneous Dirichlet boundary conditions with the following parameters: diffusion coefficient $D = 1$, step sizes $\Delta x = \Delta y = 0.05$ and $\Delta t = 0.001$ and initial condition $u(0, x, y) = \sin(\pi x) \sin(\pi y)$.

Make a 3D plot of the solution at the final time step. Describe what you observe.

10.2. Implicit Methods

It is actually possible to beat the stability criteria given in the previous exercises! What follows are two implicit methods that use a forward-looking scheme to help completely avoid unstable solutions. The primary advantage to these schemes is that we will not need to pay as close attention to the ratio of the time step to the square of the spatial step. Instead, we can take time and spatial steps that are appropriate for the application we have in mind.

10.2.1. Backward Difference Method

Exercise 10.8. For the 1D heat equation $u_t = Du_{xx}$ we have been finding the numerical solution using the explicit finite difference scheme

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} = D \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}$$

where we approximate the spatial derivative with the centred difference and the time derivative with the usual forward difference. If, however, we use the backward difference formula for the time derivative we get the finite difference scheme

$$\frac{U_i^n - U_i^{n-1}}{\Delta t} = D \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}.$$

or, shifting to the next timestep,

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} = D \frac{U_{i+1}^{n+1} - 2U_i^{n+1} + U_{i-1}^{n+1}}{\Delta x^2}.$$

This may seem completely ridiculous since we do not yet know the information at time step $n + 1$ but some algebraic rearrangement shows that we can treat this as a system of linear equations which can be solved (using something like `np.linalg.solve()`) for the $(n + 1)^{st}$ time step.

We again introduce the coefficient $a = D\Delta t/\Delta x^2$. This will save a little bit of writing in the coming steps.

1. Rearrange the new finite difference scheme so that all of the terms at the $(n + 1)^{st}$ time step are on the left-hand side and all of the term at the n^{th} time step are on the right-hand side.

$$(\text{---})U_{i-1}^{n+1} + (\text{---})U_i^{n+1} + (\text{---})U_{i+1}^{n+1} = U_i^n$$

2. Now we are going to build a very small example with only 6 spatial points so that you can clearly see the structure of the resulting linear system.
 - a. If we have 6 total points in the spatial grid $(x_0, x_1, \dots, x_5)$ then we have the following equations (fill in the blanks):

$$\begin{array}{lcl} \text{(for } x_1: \text{)} & \text{---} U_0^{n+1} + \text{---} U_1^{n+1} + \text{---} U_2^{n+1} = U_1^n \\ \text{(for } x_2: \text{)} & \text{---} U_1^{n+1} + \text{---} U_2^{n+1} + \text{---} U_3^{n+1} = U_2^n \\ \text{(for } x_3: \text{)} & \text{---} U_2^{n+1} + \text{---} U_3^{n+1} + \text{---} U_4^{n+1} = U_3^n \\ \text{(for } x_4: \text{)} & \text{---} U_3^{n+1} + \text{---} U_4^{n+1} + \text{---} U_5^{n+1} = U_4^n \end{array}$$

- b. Notice that we already know U_0^{n+1} and U_5^{n+1} since these are dictated by the boundary conditions (assuming Dirichlet boundary conditions). Hence we can move these known quantities to the right-hand side of the equations and hence rewrite the system of equations as:

$$\begin{aligned}
 (\text{for } x_1:) \quad & \frac{\quad}{\quad} U_1^{n+1} + \frac{\quad}{\quad} U_2^{n+1} = U_1^n + \frac{\quad}{\quad} U_0^{n+1} \\
 (\text{for } x_2:) \quad & \frac{\quad}{\quad} U_1^{n+1} + \frac{\quad}{\quad} U_2^{n+1} + \frac{\quad}{\quad} U_3^{n+1} = U_2^n \\
 (\text{for } x_3:) \quad & \frac{\quad}{\quad} U_2^{n+1} + \frac{\quad}{\quad} U_3^{n+1} + \frac{\quad}{\quad} U_4^{n+1} = U_3^n \\
 (\text{for } x_4:) \quad & \frac{\quad}{\quad} U_3^{n+1} + \frac{\quad}{\quad} U_4^{n+1} = U_4^n + \frac{\quad}{\quad} U_5^{n+1}
 \end{aligned}$$

- c. Now we can write this as a matrix equation:

$$\begin{pmatrix} \frac{\quad}{\quad} & \frac{\quad}{\quad} & 0 & 0 \\ \frac{\quad}{\quad} & \frac{\quad}{\quad} & \frac{\quad}{\quad} & 0 \\ 0 & \frac{\quad}{\quad} & \frac{\quad}{\quad} & \frac{\quad}{\quad} \\ 0 & 0 & \frac{\quad}{\quad} & \frac{\quad}{\quad} \end{pmatrix} \begin{pmatrix} U_1^{n+1} \\ U_2^{n+1} \\ U_3^{n+1} \\ U_4^{n+1} \end{pmatrix} = \begin{pmatrix} U_1^n \\ U_2^n \\ U_3^n \\ U_4^n \end{pmatrix} + \begin{pmatrix} \frac{\quad}{\quad} U_0^{n+1} \\ 0 \\ 0 \\ U_5^{n+1} \end{pmatrix}$$

3. At this point the structure of the coefficient matrix on the left and the vector sum on the right should be clear (even for more spatial points). It is time for us to start writing some code. We will start with the basic setup of the problem.

```

import numpy as np
import matplotlib.pyplot as plt

D = 1
x = # set up a linearly spaced spatial domain
t = # set up a linearly spaced temporal domain
dx = x[1]-x[0]
dt = t[1]-t[0]
a = D*dt/dx**2
IC = lambda x: # write a function for the initial condition
BCleft = lambda t: 0*t # left boundary condition
BCright = lambda t: 0*t # right boundary condition

U = np.zeros((len(t), len(x))) # set up a blank array for U
U[0,:] = IC(x) # set up the initial condition
U[:,0] = BCleft(t) # set up the left boundary condition
U[:,-1] = BCright(t) # set up the right boundary condition

```

4. Next we write a function that takes in the number of spatial points and returns the coefficient matrix for the linear system. Take note that the first and last rows take a little more care than the rest.


```
def coeffMatrix(M,a): # we are using M=len(x) as the first input
    A = np.zeros((M-2, M-2))
    # why are we using M-2 x M-2 for the size?
    A[0,0] = # top left entry
    A[0,1] = # entry in the first row second column
    A[-1,-1] = # bottom right entry
    A[-1,-2] = # entry in the last row second to last column
    for i in range(1,M-3): # now loop through all of the other rows
        A[i,i] = # entry on the main diagonal
        A[i,i-1] = # entry on the lower diagonal
        A[i,i+1] = # entry on the upper diagonal
    return A

A = coeffMatrix(len(x),a)
print(A)
plt.spy(A)
# spy is a handy plotting tool that shows the structure
# of a matrix (optional)
plt.show()
```

5. Next we write a loop that iteratively solves the system of equations for each new time step.

```
for n in range(len(t)-1):
    b1 = U[n,???]
    # b1 is a vector of U at step n for the inner spatial nodes
    b2 = np.zeros(len(b1)) # set up the second right-hand vector
    b2[0] = ???*BCleft(t[n+1]) # fill in the correct first entry
    b2[-1] = ???*BCright(t[n+1]) # fill in the correct last entry
    b = b1 + b2 # The vector "b" is the right side of the equation
    #
    # finally use a linear algebra solver to fill in the
    # inner spatial nodes at step n+1
    U[n+1,???] = ???
```

6. All of the hard work is now done. It remains to plot the solution. Try this method on several sets of initial and boundary conditions for the 1D heat equation. Be sure to demonstrate that the method is stable no matter the values of Δt and Δx .
7. What are the primary advantages and disadvantages to the implicit method described in this problem?

Exercise 10.9. Solve the equation $u_t = 0.2u_{xx}$ with boundary conditions $u(t, 0) = u(t, 1) = 0$ and initial condition $u(0, x) = \sin(\pi x)$ using the backward difference scheme with stepsizes $\Delta x = 0.05$ and $\Delta t = 0.5$ to determine an approximation of $u(1, 0.6)$.

Exercise 10.10. Solve the equation $u_t = 0.5u_{xx}$ with boundary conditions $u(t, 0) = t$, $u(t, 2) = 4 - t$ and initial condition $u(0, x) = x^2$ using the backward difference scheme with stepsizes $\Delta x = 0.1$ and $\Delta t = 0.4$ to determine an approximation of $u(4, 1)$.

10.2.2. Stability of the Backward Difference Method

The backward difference scheme can be written in matrix form. Rearranging so that all terms at time step $n + 1$ are on the left gives

$$B\mathbf{U}_{n+1} = \mathbf{U}_n,$$

where B is the $(N_x - 1) \times (N_x - 1)$ tridiagonal matrix with $1 + 2a$ on the diagonal and $-a$ on the two off-diagonals:

$$B = \begin{pmatrix} 1 + 2a & -a & & & \\ -a & 1 + 2a & -a & & \\ & \ddots & \ddots & \ddots & \\ & & -a & 1 + 2a & \end{pmatrix}.$$

Solving for $\mathbf{U}_{n+1}$ gives

$$\mathbf{U}_{n+1} = B^{-1}\mathbf{U}_n,$$

so after n steps, $\mathbf{U}_n = (B^{-1})^n \mathbf{U}_0$. Exactly as in the analysis of the forward difference method in Section 9.4, if an error is introduced at some step it evolves to $^{(n)} = (B^{-1})^n$. For stability we need all eigenvalues of B^{-1} to have absolute value at most 1, which is equivalent to requiring all eigenvalues of B to have absolute value at least 1.

Using the same Fourier analysis as for the forward difference method, the matrix B shares the same eigenvectors as the forward difference matrix A , and its eigenvalues are

$$\mu_k = 1 + 4a \left(\sin \left(\frac{k\pi}{2N_x} \right) \right)^2, \quad k = 1, \dots, N_x - 1.$$

Since $a > 0$ and $\sin^2(\cdot) \geq 0$, we have $\mu_k \geq 1$ for all k and all $a > 0$. Therefore $|1/\mu_k| \leq 1$ for every k , and the backward difference method is **unconditionally stable**: no restriction on the ratio $a = D\Delta t/(\Delta x)^2$ is needed to keep the numerical solution bounded.

10.2.3. The Crank-Nicolson Method

Exercise 10.11. We conclude this section with one more implicit scheme: the **Crank-Nicolson Method**. In this method we take the average of the forward and backward difference schemes:

$$\frac{U_i^{n+1} - U_i^n}{\Delta t} = \frac{1}{2} \left[D \left(\frac{U_{i-1}^n - 2U_i^n + U_{i+1}^n}{\Delta x^2} \right) + D \left(\frac{U_{i-1}^{n+1} - 2U_i^{n+1} + U_{i+1}^{n+1}}{\Delta x^2} \right) \right].$$

Letting $r = D\Delta t/(2\Delta x^2)$ we can rearrange to get

$$-U_{i-1}^{n+1} + U_i^{n+1} - U_{i+1}^{n+1} = -U_{i-1}^n + U_i^n - U_{i+1}^n.$$

This can now be viewed as a system of equations. Let us build this system carefully and then write code to solve the heat equation from the previous problems with the Crank-Nicolson method. For this problem we will assume fixed Dirichlet boundary conditions on both the left- and right-hand sides of the domain.

1. First let us write the equations for several values of i .

$$\begin{aligned} (x_1) : \quad & -U_0^{n+1} + U_1^{n+1} - U_2^{n+1} = -U_0^n + U_1^n - U_2^n \\ (x_2) : \quad & -U_1^{n+1} + U_2^{n+1} - U_3^{n+1} = -U_1^n + U_2^n - U_3^n \\ (x_3) : \quad & -U_2^{n+1} + U_3^{n+1} - U_4^{n+1} = -U_2^n + U_3^n - U_4^n \\ & \vdots \\ (x_{M-2}) : \quad & -U_{M-3}^{n+1} + U_{M-2}^{n+1} - U_{M-1}^{n+1} = -U_{M-3}^n + U_{M-2}^n - U_{M-1}^n \end{aligned}$$

where M is the number of spatial points (enumerated $x_0, x_1, x_2, \dots, x_{M-1}$).

2. The first and last equations can be simplified since we are assuming that we have Dirichlet boundary conditions. Therefore for x_1 we can rearrange to move the U_0^{n+1} term to the right-hand side since it is given for all time. Similarly for x_{M-2} we can move the U_{M-1}^{n+1} term to the right-hand side since it is fixed for all time. Rewrite these two equations.
3. Verify that the left-hand side of the equations that we have built in parts (1) and (2) can be written as the following matrix-vector product:

$$\begin{pmatrix} (1+2r) & -r & 0 & 0 & \dots & 0 \\ -r & (1+2r) & -r & 0 & \dots & 0 \\ 0 & -r & (1+2r) & -r & \dots & 0 \\ \vdots & & & & & 0 \\ 0 & \dots & & 0 & -r & (1+2r) \end{pmatrix} \begin{pmatrix} U_1^{n+1} \\ U_2^{n+1} \\ U_3^{n+1} \\ \vdots \\ U_{M-2}^{n+1} \end{pmatrix}$$

10. PDE 2

4. Verify that the right-hand side of the equations that we built in parts (1) and (2) can be written as

$$\begin{pmatrix} (1-2r) & r & 0 & 0 & \cdots & 0 \\ r & (1-2r) & r & 0 & \cdots & 0 \\ 0 & r & (1-2r) & r & & 0 \\ \vdots & & & & & \\ & & & r & (1-2r) & \end{pmatrix} \begin{pmatrix} U_1^n \\ U_2^n \\ U_3^n \\ \vdots \\ U_{M-2}^n \end{pmatrix} + \begin{pmatrix} r(U_0^{n+1} + U_0^n) \\ 0 \\ \vdots \\ 0 \\ r(U_{M-1}^n + U_{M-1}^{n+1}) \end{pmatrix}$$

5. Now for the wonderful part! The entire system of equations from part (a) can be written as

$$\mathcal{A}\mathbf{U}_{n+1} = \mathcal{B}\mathbf{U}_n + \mathbf{d},$$

What are the matrices $\mathcal{A}$ and $\mathcal{B}$ and what are the vectors $\mathbf{U}_{n+1}$, $\mathbf{U}_n$, and $\mathbf{d}$?

6. To solve for $\mathbf{U}_{n+1}$ at each time step we simply need to do a linear solve:

$$\mathbf{U}_{n+1} = \mathcal{A}^{-1}(\mathcal{B}\mathbf{U}_n + \mathbf{d}).$$

Of course, we will never do a matrix inverse on a computer. Instead we can lean on tools such as `np.linalg.solve()` to do the linear solve for us.

7. Finally. Write code to solve the 1D Heat Equation implementing the Crank Nicolson method described in this problem. The setup of your code should be largely the same as for the implicit method from Exercise 10.8. You will need to construct the matrices $\mathcal{A}$ and $\mathcal{B}$ as well as the vector $\mathbf{d}$. Then your time stepping loop will contain the code from part 6 of this problem.

Exercise 10.12. Solve the equation $u_t = 0.2u_{xx}$ with boundary conditions $u(t, 0) = u(t, 1) = 0$ and initial condition $u(0, x) = \sin(\pi x)$ using the Crank-Nicolson scheme with stepsizes $\Delta x = 0.05$ and $\Delta t = 0.5$ to determine an approximation of $u(1, 0.6)$.

Exercise 10.13. Solve the equation $u_t = 0.5u_{xx}$ with boundary conditions $u(t, 0) = t$, $u(t, 2) = 4 - t$ and initial condition $u(0, x) = x^2$ using the Crank-Nicolson scheme with stepsizes $\Delta x = 0.1$ and $\Delta t = 0.4$ to determine an approximation of $u(4, 1)$.

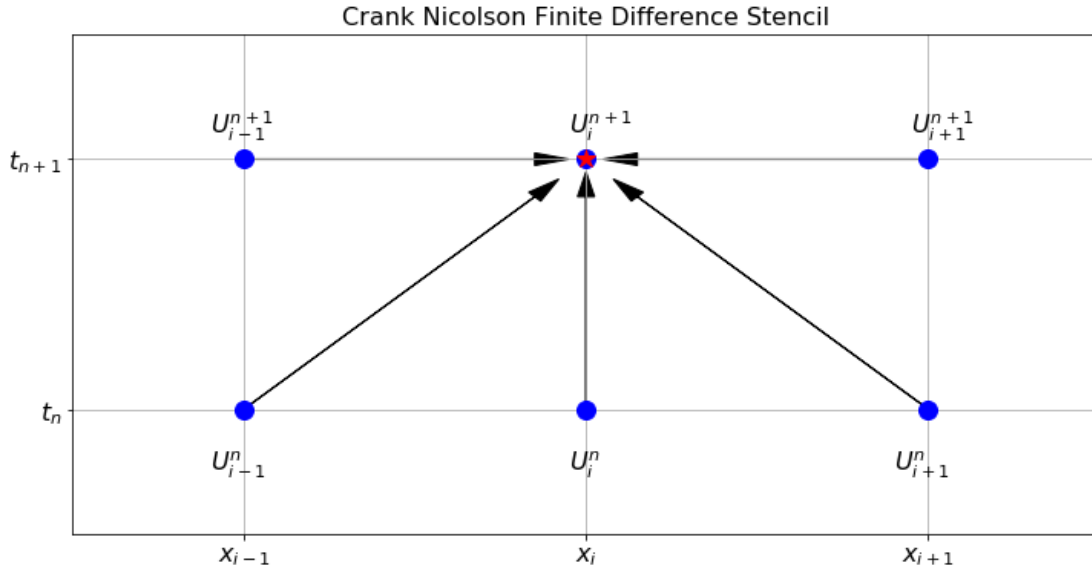


Figure 10.2.: The finite difference stencil for the Crank Nicolson method.

Exercise 10.14. To graphically represent the Crank Nicolson method we can again use a finite difference stencil to show where the information is coming from and where it is going to. In Figure 10.2 notice that there are three points at the new time step that are used to calculate the value of U_i^{n+1} at the new time step. Sketch a similar image for the original implicit scheme from Exercise 10.8

It turns out that the error terms for the forward and backward difference methods have the form $C\Delta t + O(\Delta t^2)$ and $-C\Delta t + O(\Delta t^2)$. Taking the average cancels the $\pm C\Delta t$ terms and leaves an error of order $O(\Delta t^2)$; in combination with the space variable, we have an error of order $O(\Delta t^2) + O(\Delta x^2)$ for the whole method, as compared with $O(\Delta t) + O(\Delta x^2)$ for the forward and backward difference methods. The Crank-Nicolson method is therefore a second-order method in time, as well as in space, and is often preferred for this reason.

10.2.4. Stability of the Crank-Nicolson Method

Ignoring the boundary terms (which are fixed and do not grow), the Crank-Nicolson scheme for homogeneous Dirichlet conditions reads

$$\mathcal{A}\mathbf{U}_{n+1} = \mathcal{B}\mathbf{U}_n,$$

where, using $r = D\Delta t/(2\Delta x^2)$, the matrices $\mathcal{A}$ and $\mathcal{B}$ are both tridiagonal:

$$\mathcal{A} = \begin{pmatrix} 1+2r & -r & & & \\ -r & 1+2r & -r & & \\ & \ddots & \ddots & \ddots & \\ & & -r & 1+2r & \end{pmatrix}, \quad \mathcal{B} = \begin{pmatrix} 1-2r & r & & & \\ r & 1-2r & r & & \\ & \ddots & \ddots & \ddots & \\ & & r & 1-2r & \end{pmatrix}.$$

The update formula is $\mathbf{U}_{n+1} = \mathcal{A}^{-1}\mathcal{B}\mathbf{U}_n$, so the amplification matrix is $\mathcal{A}^{-1}\mathcal{B}$. Using the same Fourier analysis as in Section 9.4, $\mathcal{A}$ and $\mathcal{B}$ share the same eigenvectors, with eigenvalues

$$\mu_k^{\mathcal{A}} = 1 + 4r \left(\sin \left(\frac{k\pi}{2N_x} \right) \right)^2, \quad \mu_k^{\mathcal{B}} = 1 - 4r \left(\sin \left(\frac{k\pi}{2N_x} \right) \right)^2,$$

for $k = 1, \dots, N_x - 1$. The eigenvalues of the amplification matrix $\mathcal{A}^{-1}\mathcal{B}$ are therefore

$$g_k = \frac{\mu_k^{\mathcal{B}}}{\mu_k^{\mathcal{A}}} = \frac{1 - 4r \sin^2 \left(\frac{k\pi}{2N_x} \right)}{1 + 4r \sin^2 \left(\frac{k\pi}{2N_x} \right)}.$$

Setting $s = 4r \sin^2(k\pi/(2N_x)) \geq 0$, we have $g_k = (1-s)/(1+s)$. Since $|1-s| \leq 1+s$ for all $s \geq 0$, we conclude $|g_k| \leq 1$ for every k and every $r > 0$. Hence the Crank-Nicolson method is **unconditionally stable**, with no restriction on $\Delta t/(\Delta x)^2$.

10.3. Exam-Style Question

- Consider the 2D heat equation $u_t = D(u_{xx} + u_{yy})$. Let $U_{i,j}^n$ denote the numerical approximation to $u(t_n, x_i, y_j)$. Using a forward difference for the time derivative and centred differences for the spatial derivatives, derive the explicit finite difference formula for $U_{i,j}^{n+1}$. Assume $\Delta x = \Delta y = h$. [3 marks]
- What is the finite difference stencil for the scheme derived in part (a)? Briefly explain how the points in the stencil are used to compute $U_{i,j}^{n+1}$. [2 marks]
- What is the condition on $a = D\frac{\Delta t}{h^2}$ for the explicit forward difference solution to the 2D heat equation to be stable? [1 mark]
- Consider instead the 1D wave equation $u_{tt} = c^2 u_{xx}$. Write down the explicit finite difference scheme for this equation using $U_i^n \approx u(t_n, x_i)$, and state the initial data required to start the numerical computation. [3 marks]
- The following incomplete Python code computes the numerical solution to the 2D heat equation using the explicit scheme derived in part (a) on a square domain $[0, 1] \times [0, 1]$ with Dirichlet boundary conditions of zero. Provide the missing code indicated by `....` [4 marks]

```

import numpy as np

def solve_heat2d(u_0, D, tmax, dt, L, dx):
    """
    Solves the 2D heat equation on [0,L]x[0,L].
    u_0 is a function u_0(X, Y) giving the initial condition.
    """
    t = np.arange(0, tmax + dt, dt)
    x = np.arange(0, L + dx, dx)
    y = np.arange(0, L + dx, dx)

    Y, X = np.meshgrid(y, x)

    a = ...

    U = np.zeros((len(t), len(x), len(y)))

    # Apply initial condition
    U[0, :, :] = ...

    # Boundary conditions are naturally 0

    for n in range(len(t) - 1):
        # Update interior spatial points
        U[n+1, 1:-1, 1:-1] = U[n, 1:-1, 1:-1] + a * (
            ... +
            ... -
            4 * U[n, 1:-1, 1:-1] +
            ... +
            ...
        )

    return t, x, y, U

```


A. Python

Simple is better than complex.

–Guido van Rossum (creator of Python)

In this appendix we will walk through some of the basics of using Python — the popular general-purpose programming language that we will use throughout this module.

For most of you this material will not be new. For example, you have probably seen it in your first year “Mathematical Programming & Skills” module. But for some of you this may be entirely new. You will have some notion of what a programming language “is” and “does”, but you may never have written any code. That is all right.

If you are new to Python, don’t feel that you need to work through this appendix in one go. Instead, spread the work over the first two weeks of the course and interlace it with your work on the first two chapters.

There is a lot of material in this appendix. Do not feel that you need to learn it all by hard. The idea is just that you should have seen the various language constructs once. Your familiarity with them will come automatically later when you use them throughout the course.

A.1. Why Python?

We are going to be using Python since

- Python is free,
- Python is very widely used,
- Python is flexible,
- Python is relatively easy to learn,
- and Python is quite powerful.

It is important to keep in mind that Python is a general purpose language that we will be using for Scientific Computing. The purpose of Scientific Computing is *not* to build apps, build software, manage databases, or develop user interfaces. Instead, Scientific Computing is the use of a computer programming language (like Python) along with mathematics to solve scientific and mathematical problems. For this reason it is definitely

A. Python

not our purpose to write an all-encompassing guide for how to use Python. We will only cover what is necessary for our computing needs. You will learn more as the course progresses, so use this appendix just to get going with the language. To keep things as simple as possible, we will for example not use object oriented programming, so will not introduce classes and methods.

We are also definitely not saying that Python is the best language for scientific computing under all circumstances. The reason there are so many scientific programming languages coexisting is that each has particular strengths that make it the best option for particular applications. But we are saying that Python is so widely used that everyone should know Python.

There is an overwhelming abundance of information available about Python and the suite of tools that we will frequently use.

- Python <https://www.python.org/>,
- **numpy** (numerical Python) <https://www.numpy.org/>,
- **matplotlib** (a suite of plotting tools) <https://matplotlib.org/>,
- **scipy** (scientific Python) <https://www.scipy.org/>.

These tools together provide all of the computational power that we will need. And they are free!

A.2. Python Programming Basics

If you are already very practised in using Python then you can jump straight to **?@sec-python_exercises** with the coding exercises. But if you are new to Python or your Python skills are a bit rusty, then you will benefit from working through all the examples and exercises below, making sure you copy and paste all the code into your Colab notebook and run it there, and then critically evaluate and understand the output.

A.2.1. Variables

Variable names in Python can contain letters (lower case or capital), numbers 0-9, and some special characters such as the underscore. Variable names must start with a letter. There are a bunch of reserved words that you can not use for your variable names because they have a special meaning in the Python syntax. Python will let you know with a syntax error if you try to use a reserved word for a variable name.

You can do the typical things with variables. Assignment is with an equal sign (be careful R users, we will not be using the left-pointing arrow here!).

Warning: When defining numerical variables you do not always get floating point numbers. In some programming languages, if you write `x=1` then automatically `x` is saved as 1.0; a floating point number, not an integer. In Python however, if you assign `x=1` it is defined as an integer (with no decimal digits) but if you assign `x=1.0` it is assigned as a floating point number.

```
# assign some variables
x = 7 # integer assignment of the integer 7
y = 7.0 # floating point assignment of the decimal number 7.0
print("The variable x has the value", x, " and has type", type(x), ". \n")
print("The variable y has the value", y, " and has type", type(y), ". \n")
```

Remember to copy each code block to your own notebook, execute it and look at the output. To copy the code from this guide to your notebook you can use the “Copy to Clipboard” icon that pops up in the top right corner of a code block when you hover over that code block.

```
# multiplying by a float will convert an integer to a float
x = 7 # integer assignment of the integer 7
print("Multiplying x by 1.0 gives", 1.0*x)
print("The type of this value is", type(1.0*x), ". \n")
```

The allowed mathematical operations are:

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /
- Integer Division (modular division): // and %
- Exponents: **

That’s right, the caret key, `^`, is NOT an exponent in Python (sigh). Instead we have to get used to `**` for exponents.

```
x = 7.0
y = x**2 # square the value in x
y
```

A. Python

Exercise A.1. Write code to define positive integers a, b and c of your own choosing. Then calculate a^2, b^2 and c^2 . When you have all three values computed, check to see if your three values form a Pythagorean Triple so that $a^2 + b^2 = c^2$. Have Python simply say True or False to verify that you do, or do not, have a Pythagorean Triple defined. **Hint:** You will need to use the `==` Boolean check just like in other programming languages.

A.2.2. Indexing and Lists

Lists are a key component to storing data in Python. Lists are exactly what the name says: lists of things (in our case, usually the entries are floating point numbers).

Warning: Python indexing starts at 0 whereas some other programming languages have indexing starting at 1. In other words, the first entry of a list has index 0, the second entry as index 1, and so on. We just have to keep this in mind.

We can extract a part of a list using the syntax `name[start:stop]` which extracts elements between index `start` and `stop-1`. Take note that Python stops reading at the second to last index. This often catches people off guard when they first start with Python.

Example A.1 (Lists and Indexing). Let us look at a few examples of indexing from lists. In this example we will use the list of numbers 0 through 8. This list contains 9 numbers indexed from 0 to 8.

- Create the list of numbers 0 through 8

```
my_list = [0,1,2,3,4,5,6,7,8]
```

- Output the list

```
my_list
```

- Select only the element with index 0.

```
my_list[0]
```

- Select all elements up to, but not including, the third element of `my_list`.

```
my_list[:2]
```

- Select the last element of `my_list` (this is a handy trick!).

```
my_list[-1]
```

- Select the elements indexed 1 through 4. Beware! This is not the first through fifth element.

```
my_list[1:5]
```

- Select every other element in the list starting with the first.

```
my_list[0::2]
```

- Select the last three elements of `my_list`

```
my_list[-3:]
```

In Python, elements in a list do not need to be the same type. You can mix integers, floats, strings, lists, etc.

Example A.2. In this example we see a list of several items that have different data types: float, integer, string, and complex. Note that the imaginary number i is represented by `1j` in Python. This use of j instead of i is common in some scientific disciplines and is just another thing that we Mathematicians will need to get used to in Python.

```
MixedList = [1.0, 7, 'Bob', 1-1j]
print(MixedList)
print(type(MixedList[0]))
print(type(MixedList[1]))
print(type(MixedList[2]))
print(type(MixedList[3]))
# Notice that we use 1j for the imaginary number "i".
```

Exercise A.2. In this exercise you will put your new list skills into practice.

A. Python

1. Create the list of the first several Fibonacci numbers:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89. \quad (\text{A.1})$$

2. Print the first four elements of the list.
 3. Print every third element of the list starting from the first.
 4. Print the last element of the list.
 5. Print the list in reverse order.
 6. Print the list starting at the last element and counting backward by every other element.
-

A.2.3. List Operations

Python is awesome about allowing you to do things like appending items to lists, removing items from lists, and inserting items into lists. Note in all of the examples below that we are using the code

`variable.method`

where you put the variable name, a dot, and the thing that you would like to do to that variable. For example, `my_list.append(7)` will append the number 7 to the list `my_list`. We say that `append` is a “method” of the list `my_list`. This is a common programming feature in Python and we will use it often.

Example A.3. The `.append` method can be used to append an element to the end of a list.

```
my_list = [0,1,2,3]
print(my_list)
# Append the string 'a' to the end of the list
my_list.append('a')
print(my_list)
# Do it again ... just for fun
my_list.append('a')
print(my_list)
# Append the number 15 to the end of the list
my_list.append(15)
print(my_list)
```

Example A.4. The `.remove` method can be used to remove an element from a list.

```
# Let us remove the 3
my_list.remove(3)
my_list
```

Example A.5. The `.insert` method can be used to insert an element at a location in a list.

```
# insert the letter `A` at the 0-indexed spot
my_list.insert(0, 'A')
# insert the letter `B` at the spot with index 3
my_list.insert(3, 'B')
# remember that index 3 means the fourth spot in the list
my_list
```

Exercise A.3. In this exercise you will go a bit further with your list operation skills.

1. Create the list of the first several Lucas Numbers: 1, 3, 4, 7, 11, 18, 29, 47.
 2. Add the next three Lucas Numbers to the end of the list.
 3. Remove the number 3 from the list.
 4. Insert the 3 back into the list in the correct spot.
 5. Print the list in reverse order.
 6. Do a few other list operations to this list and report your findings.
-

A.2.4. Tuples

In Python, a “tuple” is like an ordered pair (or ordered triple, or ordered quadruple, ...) in mathematics. We will occasionally see tuples in our work in numerical analysis so for now let us just give a couple of code snippets showing how to store and read them.

We can define the tuple of numbers (10, 20) in Python as follows:

Example A.6.

```
point = 10, 20
print(point, type(point))
```

We can also define a tuple with parenthesis if we like. Python does not care.

```
point = (10, 20) # now we define the tuple with parenthesis
print(point, type(point))
```

We can then unpack the tuple into components if we wish:

```
x, y = point
print("x = ", x)
print("y = ", y)
```

There are other important data structures in Python that we will not use in this module. These include dictionaries and sets. We will not cover these here because we are trying to keep things simple so that we can concentrate on Numerical Analysis instead. If you are interested in learning more about these data structures, you can find a lot of information about them in the Python documentation.

A.2.5. Control Flow: Loops and If Statements

Any time you need to do some repetitive task with a programming language you can use a loop. Just like in other programming languages, we can do loops and conditional statements in very easy ways in Python. The thing to keep in mind is that the Python language is very white-space-dependent. This means that your indentations need to be correct in order for a loop to work. You could get away with sloppy indentation in other languages but not so in Python. Also, in some languages (like R and Java) you need to wrap your loops in curly braces. Again, not so in Python.

Caution: Be really careful of the white space in your code when you write loops.

A.2.5.1. for Loops

A `for` loop is designed to do a task a certain number of times and then stop. This is a great tool for automating repetitive tasks, but it is also nice numerically for building sequences, summing series, or just checking lots of examples. The following are some examples of Python `for` loops.

Example A.7. Print the first 6 perfect squares.

```
for x in [1,2,3,4,5,6]:
    print(x**2)
```

Often instead of writing the list of integers explicitly one uses the `range()` function, so that this example would be written as

```
for x in range(1,7):
    print(x**2)
```

Note that `range(1,7)` produces the integers from 1 to 6, not from 1 to 7. This is another manifestation of Python's weird 0-based indexing. Of course it is only weird to people who are new to Python. For Pythonists it is perfectly natural.

You can also use `range()` to generate a sequence of numbers with a specific step size. For example, `range(1, 10, 2)` will generate the odd integers from 1 to 9.

```
for x in range(1, 10, 2):
    print(x)
```

Take careful note of the syntax for a `for` loop as it is the same as for other loops and conditional statements:

- a control statement,
- a colon, a new line,
- indent four spaces,
- some programming statements

When you are done with the loop, just back out of the indentation. There is no need for an `end` command or a curly brace. All of the control statements in Python are white-space-dependent.

Example A.8. Print the names in a list.

```
NamesList = ['Alice', 'Billy', 'Charlie', 'Dom', 'Enrique', 'Francisco']
for name in NamesList:
    print(name)
```

In Python you can use a more compact notation for for loops sometimes. This takes a bit of getting used to, but is super slick!

Example A.9. Create a list of the perfect squares from 1 to 9.

```
# create a list of the perfect squares from 1 to 9
CoolList = [x**2 for x in range(1,10)]
print(CoolList)
# Then print the sum of this list
print("The sum of the first 9 perfect squares is", sum(CoolList))
```

for loops can also be used to build sequences, as can be seen in the next couple of examples.

Example A.10. In the following code we write a for loop that outputs a list of the first 7 iterations of the sequence $x_{n+1} = -0.5x_n + 1$ starting with $x_0 = 3$. Notice that we are using the command `x.append` instead of `x[n + 1]` to append the new term to the list. This allows us to grow the length of the list dynamically as the loop progresses.

```
x=[3.0]
for n in range(0,7):
    x.append(-0.5*x[n] + 1)
    print(x) # print the whole list x at each step of the loop
```

Example A.11. As an alternative to the code from the previous example we can pre-allocate the memory in a list of zeros. This is done with the clever code `x = [0] * 10`. Literally multiplying a list by some number, like 10, says to repeat that list 10 times.

Now we will build the sequence with pre-allocated memory.

```
x = [0] * 7
x[0] = 3.0
for n in range(0,6):
    x[n+1] = -0.5*x[n]+1
    print(x) # This print statement shows x at each iteration
```

Exercise A.4. We want to sum the first 100 perfect cubes. Let us do this in two ways.

1. Start off a variable called `total` at 0 and write a `for` loop that adds the next perfect cube to the running total.
2. Write a `for` loop that builds the sequence of the first 100 perfect cubes. After the list has been built find the sum with the `sum()` function.

The answer is: 25,502,500 so check your work.

Exercise A.5. Write a `for` loop that builds the first 20 terms of the sequence $x_{n+1} = 1 - x_n^2$ with $x_0 = 0.1$. Pre-allocate enough memory in your list and then fill it with the terms of the sequence. Only print the list after all of the computations have been completed.

A.2.5.2. while Loops

A **while** loop repeats some task (or sequence of tasks) while a logical condition is true. It stops when the logical condition turns from true to false. The structure in Python is the same as with **for** loops.

Example A.12. Print the numbers 0 through 4 and then the word “done.” we will do this by starting a counter variable, *i*, at 0 and increment it every time we pass through the loop.

```
i = 0
while i < 5:
    print(i)
    i += 1 # increment the counter
print("done")
```

Example A.13. Now let us use a while loop to build the sequence of Fibonacci numbers and stop when the newest number in the sequence is greater than 1000. Notice that we want to keep looping until the condition that the last term is greater than 1000 – this is the perfect task for a **while** loop, instead of a **for** loop, since we do not know how many steps it will take before we start the task

```
Fib = [1,1]
while Fib[-1] <= 1000:
    Fib.append(Fib[-1] + Fib[-2])
print("The last few terms in the list are:\n",Fib[-3:])
```

Exercise A.6. Write a **while** loop that sums the terms in the Fibonacci sequence until the sum is larger than 1000

A.2.5.3. if Statements

Conditional (`if`) statements allow you to run a piece of code only under certain conditions. This is handy when you have different tasks to perform under different conditions.

Example A.14. Let us look at a simple example of an `if` statement in Python.

```
Name = "Alice"
if Name == "Alice":
    print("Hello, Alice. Isn't it a lovely day to learn Python?")
else:
    print("You're not Alice. Where is Alice?")
```

```
Name = "Billy"
if Name == "Alice":
    print("Hello, Alice. Isn't it a lovely day to learn Python?")
else:
    print("You're not Alice. Where is Alice?")
```

Example A.15. For another example, if we get a random number between 0 and 1 we could have Python print a different message depending on whether it was above or below 0.5. Run the code below several times and you will see different results each time.

Note: We have to import the `numpy` package to get the random number generator in Python. Do not worry about that for now. We will talk about packages in a moment.

```
import numpy as np
x = np.random.uniform() # get a random number between 0 and 1
if x < 0.5:
    print(x, " is less than a half")
else:
    print(x, "is NOT less than a half")
```

(Take note that the output will change every time you run it.)

A. Python

Example A.16. In many programming tasks it is handy to have several different choices between tasks instead of just two choices as in the previous examples. This is a job for the `elif` command.

This is the same code as last time except we will make the decision at 0.33 and 0.67.

```
import numpy as np
x = np.random.rand(1,1) # get a random 1x1 matrix using numpy
x = x[0,0] # pull the entry from the first row and first column
if x < 0.33:
    print(x, " < 1/3")
elif x < 0.67:
    print("1/3 <= ",x,"< 2/3")
else:
    print(x, ">= 2/3")
```

(Take note that the output will change every time you run it.)

Exercise A.7. Write code to give the Collatz Sequence

$$x_{n+1} = \begin{cases} x_n/2, & x_n \text{ is even} \\ 3x_n + 1, & \text{otherwise} \end{cases} \quad (\text{A.2})$$

starting with a positive integer of your choosing. The sequence will converge¹ to 1 so your code should stop when the sequence reaches 1.

Hints: To test whether a number `x` is even you can test whether the remainder after dividing by 2 is zero with `(x % 2) == 0`. Also you will want to use the integer division `//` when calculating $x_n/2$.

¹Actually, it is still an open mathematical question whether every integer seed will converge to 1. The Collatz sequence has been checked for many millions of initial seeds and they all converge to 1, but there is no mathematical proof that it will always happen. You will check the conjecture numerically in ?@exr-1.50

A.2.6. Functions

Mathematicians and programmers talk about functions in very similar ways, but they are not exactly the same. When we say “function” in a programming sense we are talking about a chunk of code that you can pass parameters and expect an output of some sort. This is not unlike the mathematician’s version. But unlike a mathematical function, a Python function can also have side effects, like plotting a graph for example. So Python’s definition of a function is a bit more flexible than that of a mathematician.

In Python, to define a function we start with `def`, followed by the function’s name, any input variables in parenthesis, and a colon. The indented code after the colon is what defines the actions of the function.

Example A.17. The following code defines the polynomial $f(x) = x^3 + 3x^2 + 3x + 1$ and then evaluates the function at a point $x = 2.3$.

```
def f(x):
    return(x**3 + 3*x**2 + 3*x + 1)
f(2.3)
```

Take careful note of several things in the previous example:

- To define the function we cannot just type it like we would see it on paper. This is not how Python recognizes functions.
 - Once we have the function defined we can call upon it just like we would on paper.
 - We cannot pass symbols into this type of function.²
-

Exercise A.8. Define the function $g(n) = n^2 + n + 41$ as a Python function. Write a loop that gives the output for this function for integers from $n = 0$ to $n = 39$. Euler noticed that each of these outputs is a prime number (check this on your own). Will the function produce a prime for $n = 40$? For $n = 41$?

²There is the `sympy` package if you want to do symbolic computations, but we will not use that in this module.

A. Python

Example A.18. One cool thing that you can do with functions is call them recursively. That is, you can call the same function from within the function itself. This turns out to be really handy in several mathematical situations.

Let us define a function for the factorial. This function is naturally going to be recursive in the sense that it calls on itself!

```
def factorial(n):
    if n==0:
        return(1)
    else:
        return(n*factorial(n-1))
    # Note: we are calling the function recursively.
```

When you run this code there will be no output. You have just defined the function so you can use it later, as follows:

```
factorial(12)
```

Example A.19. For this next example let us define a function to calculate the next element in the sequence

$$x_{n+1} = \begin{cases} 2x_n, & x_n \in [0, 0.5] \\ 2x_n - 1, & x_n \in (0.5, 1] \end{cases} \quad (\text{A.3})$$

and then build a loop to find the first several elements of the sequence starting at any real number between 0 and 1.

```
# Define the function
def my_seq(xn):
    if xn <= 0.5:
        return(2*xn)
    else:
        return(2*xn-1)
# Now build a sequence with this function
x = [0.125] # arbitrary starting point
for n in range(0,5): # Let us only build the first 5 terms
    x.append(my_seq(x[n]))
print(x)
```


Example A.20. A fun way to approximate the square root of two is to start with any positive real number and iterate over the sequence

$$x_{n+1} = \frac{1}{2}x_n + \frac{1}{x_n} \quad (\text{A.4})$$

until we are within any tolerance we like of the square root of 2. Write code that defines the sequence as a function and then iterates in a while loop until we are within 10^{-8} of the square root of 2.

We import the `math` package so that we get the square root function. More about packages in the next section.

```
from math import sqrt
def f(x):
    return(0.5*x + 1/x)
x = 1.1 # arbitrary starting point
print(f"{'Approximation':<20} | {'Exact':<20} | {'Absolute error':<20}")
print("-" * 68)
while abs(x-sqrt(2)) > 10**(-8):
    x = f(x)
    print(f"{x:<20} | {sqrt(2):<20} | {abs(x - sqrt(2)):<20}")
```

This example also shows how to format the output of a print statement so that it is nicely aligned in columns. The `:<20` means that the string will be left-aligned and padded with spaces to a total width of 20 characters. The `f` before the string allows us to use curly braces `{}` to insert variables into the string. You will see an alternative way for displaying tabular data in Example A.48.

Exercise A.9. The previous example is a special case of the Babylonian Algorithm for calculating square roots. If you want the square root of S then iterate the sequence

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{S}{x_n} \right) \quad (\text{A.5})$$

until you are within an appropriate tolerance.

Modify the code given in the previous example to give a list of approximations of the square roots of the natural numbers 2 through 20, each to within 10^{-8} . This problem will require that you build a function, write a ‘for’ loop (for the integers 2 through 20), and write a ‘while’ loop inside your ‘for’ loop to do the iterations.

A.2.7. Lambda Functions

Using `def` to define a function as in the previous subsection is really nice when you have a function that is complicated or requires some bit of code to evaluate. However, in the case of mathematical functions we have a convenient alternative: `lambda` Functions.

The basic idea of a `lambda` Function is that we just want to state what the variable is and what the rule is for evaluating the function. This is closest to the way that we write mathematical functions. For example, we can define the mathematical function $f(x) = x^2 + 3$ in two different ways.

- Using `def`:

```
def f(x):  
    return(x**2+3)
```

- Using `lambda`:

```
f = lambda x: x**2+3
```

You can see that in the Lambda Function we are explicitly stating the name of the variable immediately after the word `lambda`, then we put a colon, and then the function definition. This is somewhat similar to but also annoyingly different from the mathematicians notation $f : x \mapsto x^2 + 3$.

No matter whether we use `def` or `lambda` to define the function `f`, if we want to evaluate the function at a point, say $x = 1.5$, then we can write code just like we would mathematically: $f(1.5)$

```
f(1.5) # evaluate the function at x=1.5
```

We can also define Lambda Functions of several variables. For example, if we want to define the mathematical function $f(x, y) = x^2 + xy + y^3$ we could write the code

```
f = lambda x, y: x**2 + x*y + y**3
```

If we wanted the value $f(2, 4)$ we would now write the code `f(2,4)`.

Exercise A.10. Go back to Exercise A.8 and repeat this exercise using a `lambda` function.

Exercise A.11. Go back to Exercise A.9 and repeat this exercise using a `lambda` function.

A.2.8. Packages

Python was not created as a scientific programming language. The reason Python can be used for scientific computing is that there are powerful extension packages that define additional functions that are needed for scientific calculations.

Let us start with the `math` package.

Example A.21. The code below imports the `math` package into your instance of Python and calculates the cosine of $\pi/4$.

```
import math
x = math.cos(math.pi / 4)
print(x)
```

The answer, unsurprisingly, is the decimal form of $\sqrt{2}/2$.

You might already see a potential disadvantage to Python's packages: there is now more typing involved! Let us fix this. When you import a package you could just import all of the functions so they can be used by their proper names.

Example A.22. Here we import the entire `math` package so we can use every one of the functions therein without having to use the `math` prefix.

```
from math import * # read this as: from math import everything
x = cos(pi / 4)
print(x)
```

A. Python

The end result is exactly the same: the decimal form of $\sqrt{2}/2$, but now we had less typing to do.

Now you can freely use the functions that were imported from the `math` package. There is a disadvantage to this, however. What if we have two packages that import functions with the same name. For example, in the `math` package and in the `numpy` package there is a `cos()` function. In the next block of code we will import both `math` and `numpy`, but instead we will import them with shortened names so we can type things a bit faster.

Example A.23. Here we import `math` and `numpy` under aliases so we can use the shortened aliases and not mix up which functions belong to which packages.

```
import math as ma
import numpy as np
# use the math version of the cosine function
x = ma.cos(ma.pi / 4)
# use the numpy version of the cosine function
y = np.cos(np.pi / 4)
print(x, y)
```

Both `x` and `y` in the code give the decimal approximation of $\sqrt{2}/2$. This is clearly pretty redundant in this really simple case, but you should be able to see where you might want to use this and where you might run into troubles.

Example A.24 (Contents of a package). Once you have a package imported you can see what is inside of it using the `dir` command. The following block of code prints a list of all of the functions inside the `math` package.

```
import math
print(dir(math))
```

By the way: you only need to import a package once in a session. The only reason we are repeating the `import` statement in each code block is to make it easier to come back to this material later in a new session, where you will need to import the packages again.

Of course, there will be times when you need help with a function. You can use the `help` function to view the help documentation for any function. For example, you can run the code `help(math.acos)` to get help on the arc cosine function from the `math` package.

Exercise A.12. Import the `math` package, figure out how the `log` function works, and write code to calculate the logarithm of the number 8.3 in base 10, base 2, base 16, and base e (the natural logarithm).

A.3. Numerical Python with NumPy

The base implementation of Python includes the basic programming language, the tools to write loops, check conditions, build and manipulate lists, and all of the other things that we saw in the previous section. In this section we will explore the package `numpy` that contains optimized numerical routines for doing numerical computations in scientific computing.

Example A.25. To start with, let us look at a really simple example. Say you have a list of real numbers and you want to take the sine of every element in the list. If you just try to take the sine of the list you will get an error. Try it yourself.

```
from math import pi, sin
my_list = [0, pi/6, pi/4, pi/3, pi/2, 2*pi/3, 3*pi/4, 5*pi/6, pi]
sin(my_list)
```

You could get around this error using some of the tools from base Python, but none of them are very elegant from a programming perspective.

```
from math import pi, sin
my_list = [0, pi/6, pi/4, pi/3, pi/2, 2*pi/3, 3*pi/4, 5*pi/6, pi]
sine_list = [sin(n) for n in my_list]
sine_list
```

A. Python

```
from math import pi, sin
my_list = [0, pi/6, pi/4, pi/3, pi/2, 2*pi/3, 3*pi/4, 5*pi/6, pi]
sine_list = [ ]
for n in range(0, len(my_list)):
    sine_list.append(sin(my_list[n]))
sine_list
```

Perhaps more simply, say we wanted to square every number in a list. Just appending the code `**2` to the end of the list will fail!

```
my_list = [1, 2, 3, 4]
my_list**2 # This will produce an error
```

If, instead, we define the list as a `numpy` array instead of a Python list then everything will work mathematically exactly the way that we intend.

```
import numpy as np
my_list = np.array([1, 2, 3, 4])
my_list**2 # This will work as expected!
```

Exercise A.13. See if you can take the sine of a full list of numbers that are stored in a `numpy` array.

Hint: you will now see why the `numpy` package provides its own version of the sine function.

The package `numpy` is used in many (most) mathematical computations in numerical analysis using Python. It provides algorithms for matrix and vector arithmetic. Furthermore, it is optimized to be able to do these computations in the most efficient possible way (both in terms of memory and in terms of speed).

Typically when we import `numpy` we use `import numpy as np`. This is the standard way to name the `numpy` package. This means that we will have lots of function with the prefix “np” in order to call on the `numpy` functions. Let us first see what is inside the package

```
import numpy as np
dir(np)
```

A brief glimpse through the list reveals a huge wealth of mathematical functions that are optimized to work in the best possible way with the Python language. (We are intentionally not showing the output here since it is quite extensive, run it so you can see.)

A.3.1. Numpy Arrays, Array Operations, and Matrix Operations

In the previous section you worked with Python lists. As we pointed out, the shortcoming of Python lists is that they do not behave well when we want to apply mathematical functions to the vector as a whole. The “numpy array”, `np.array`, is essentially the same as a Python list with the notable exceptions that

- In a **numpy** array every entry is a floating point number
- In a **numpy** array the memory usage is more efficient (mostly since Python is expecting data of all the same type)
- With a **numpy** array there are ready-made functions that can act directly on the array as a matrix or a vector

Let us just look at a few examples using **numpy**. What we are going to do is to define a matrix A and vectors v and w as

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad v = \begin{pmatrix} 5 \\ 6 \end{pmatrix} \quad \text{and} \quad w = v^T = (5 \quad 6). \quad (\text{A.6})$$

Then we will do the following

- Get the size and shape of these arrays
- Get individual elements, rows, and columns from these arrays
- Treat these arrays as with linear algebra to
 - do element-wise multiplication
 - do matrix a vector products
 - do scalar multiplication
 - take the transpose of matrices
 - take the inverse of matrices

A. Python

Example A.26 (numpy Matrices). The first thing to note is that a matrix is a list of lists (each row is a list).

```
import numpy as np
A = np.array([[1,2],[3,4]])
print("The matrix A is:\n",A)
v = np.array([[5],[6]]) # this creates a column vector
print("The vector v is:\n",v)
w = np.array([5,6]) # this creates a row vector
print("The vector w is:\n",w)
```

Example A.27 (.shape). The .shape attribute can be used to give the shape of a numpy array. Notice that the output is a tuple showing the size (rows, columns).

```
print("The shape of the matrix A is ", A.shape)
print("The shape of the column vector v is ", v.shape)
print("The shape of the row vector w is ", w.shape)
```

Example A.28 (.size). The .size attribute can be used to give the size of a numpy array. The size of a matrix or vector will be the total number of elements in the array. You can think of this as the product of the values in the tuple coming from the shape method.

```
print("The size of the matrix A is ", A.size)
print("The size of the column vector v is ", v.size)
print("The size of the row vector w is ", w.size)
```

Reading individual elements from a numpy array is the same, essentially, as reading elements from a Python list. We will use square brackets to get the row and column. Remember that the indexing all starts from 0, not 1!

Example A.29. Let us read the top left and bottom right entries of the matrix A .


```
import numpy as np
A = np.array([[1,2],[3,4]])
print(A[0,0]) # top left
print(A[1,1]) # bottom right
```

Example A.30. Let us read the first row from that matrix A .

```
import numpy as np
A = np.array([[1,2],[3,4]])
print(A[0,:])
```

Example A.31. Let us read the second column from the matrix A .

```
import numpy as np
A = np.array([[1,2],[3,4]])
print(A[:,1])
```

Notice when we read the column it was displayed as a row. Be careful. Reading a row or a column from a matrix will automatically flatten it into a 1-dimensional array.

If we try to multiply either A and v or A and A we will get some funky results. Unlike in some programming languages like MATLAB, the default notion of multiplication is NOT matrix multiplication. Instead, the default is element-wise multiplication. You may be familiar with this from R.

Example A.32. If we write the code $A*A$ we do NOT do matrix multiplication. Instead we do element-by-element multiplication. This is a common source of issues when dealing with matrices and Linear Algebra in Python.

```
import numpy as np
A = np.array([[1,2],[3,4]])
print("Element-wise multiplication:\n", A * A)
print("Matrix multiplication:\n", A @ A)
```

Example A.33. If we write $A * v$ Python will do element-wise multiplication across each column since v is a column vector. If we want the matrix A to act on v we write $A @ v$.

```
import numpy as np
A = np.array([[1,2],[3,4]])
v = np.array([[5],[6]])
print("Element-wise multiplication on each column:\n", A * v)
# A @ v will do proper matrix multiplication
print("Matrix A acting on vector v:\n", A @ v)
```

It is up to you to check that these products are indeed correct from the definitions of matrix multiplication from Linear Algebra.

It remains to show some of the other basic linear algebra operations: inverses, determinants, the trace, and the transpose.

Example A.34 (Transpose). Taking the transpose of a matrix (swapping the rows and columns) is done with the `.T` attribute.

```
A.T # The transpose is relatively simple
```

Example A.35 (Trace). The trace is done with `matrix.trace()`

```
A.trace() # The trace is pretty darn easy too
```

Oddly enough, the trace returns a matrix, not a scalar Therefore you will have to read the first entry (index `[0,0]`) from the answer to just get the trace.

Example A.36 (Determinant). The determinant function is hiding under the `linalg` subpackage inside `numpy`. Therefore we need to call it as such.

```
np.linalg.det(A)
```

You notice an interesting numerical error here. You can do the determinant easily by hand and so know that it should be exactly -2 . We'll discuss the source of these kinds of errors in Chapter 1.

Example A.37 (Inverse). In the `linalg` subpackage there is also a function for taking the inverse of a matrix.

```
A_inv = np.linalg.inv(A)
A_inv
```

We can check that we get the identity matrix back:

```
A @ A_inv
```

Exercise A.14. Now that we can do some basic linear algebra with `numpy` it is your turn. Define the matrix B and the vector u as

$$B = \begin{pmatrix} 1 & 4 & 8 \\ 2 & 3 & -1 \\ 0 & 9 & -3 \end{pmatrix} \quad \text{and} \quad u = \begin{pmatrix} 6 \\ 3 \\ -7 \end{pmatrix}. \quad (\text{A.7})$$

Then find

1. Bu
2. B^2 (in the traditional linear algebra sense)
3. The size and shape of B
4. $B^T u$
5. The element-by-element product of B with itself
6. The dot product of u with the first row of B

A.3.2. arange, linspace, zeros, ones, and meshgrid

There are a few built-in ways to build arrays in `numpy` that save a bit of time in many scientific computing settings.

Example A.38. The `np.arange` (array range) function is great for building sequences.

```
import numpy as np
x = np.arange(0,0.6,0.1)
x
```

`np.arange` builds an array of floating point numbers with the arguments `start`, `stop`, and `step`. Note that the `stop` value itself is not included in the result.

Example A.39. The `np.linspace` function builds an array of floating point numbers starting at one point, ending at the next point, and have exactly the number of points specified with equal spacing in between: `start`, `stop`, `number of points`.

```
import numpy as np
y = np.linspace(0,5,11)
y
```

In a linear space you are always guaranteed to hit the stop point exactly, but you do not have direct control over the step size.

Example A.40. The `np.zeros` function builds an array of zeros. This is handy for pre-allocating memory.

```
import numpy as np
z = np.zeros((3,5)) # create a 3x5 matrix of zeros
z
```

If you already have an array and want to create an array of zeros with the same shape, you can use `np.zeros_like(array)`.

```
import numpy as np
x = np.linspace(0,5,11)
z = np.zeros_like(x)
z
```

Similarly there are the functions `np.ones` and `np.ones_like` to build arrays of ones.

Example A.41. The `np.meshgrid` function builds two arrays that when paired make up the ordered pairs for a 2D (or higher D) mesh grid of points. This is handy for building 2D (or higher dimensional) arrays of data for multi-variable functions. Notice that the output is defined as a tuple.

```
import numpy as np
x, y = np.meshgrid(np.linspace(0,5,6), np.linspace(0,5,6))
print("x = ", x)
print("y = ", y)
```

The thing to notice with the `np.meshgrid()` function is that when you lay the two arrays on top of each other, the matching entries give every ordered pair in the domain.

If the purpose of this is not clear to you yet, don't worry. You will see it used a lot later in the module.

Exercise A.15. Now it is time to practice with some of these `numpy` functions.

- Create a `numpy` array of the numbers 1 through 10 and square every entry in the list without using a loop.
 - Create a 10×10 identity matrix and change the top right corner to a 5. Hint: `np.identity()`
 - Find the matrix-vector product of the answer to part (b) and the answer to part (a).
 - Change the bottom row of your matrix from part (b) to all 3's, then change the third column to all 7's, and then find the 5^{th} power of this matrix.
-

A.4. Plotting with Matplotlib

A key part of scientific computing is plotting your results or your data. The tool in Python best-suited to this task is the package `matplotlib`. As with all of the other packages in Python, it is best to learn just the basics first and then to dig deeper later. One advantage to using `matplotlib` in Python is that it is modelled off of MATLAB's plotting tools. People coming from a MATLAB background should feel pretty comfortable here, but there are some differences to be aware of.

A.4.1. Basics with `plt.plot()`

We are going to start right away with an example. In this example, however, we will walk through each of the code chunks one-by-one so that we understand how to set up a proper plot.

Below we will mention some tricks for getting the plots to render that only apply to Jupyter Notebooks. If you are using Google Colab then you may not need some of these little tricks.

Example A.42 (Plotting with `matplotlib`). In the first example we want to simply plot the sine function on the domain $x \in [0, 2\pi]$, colour it green, put a grid on it, and give a meaningful legend and axis labels. To do so we first need to take care of a couple of housekeeping items.

- Import `numpy` so we can take advantage of some good numerical routines.
- Import `matplotlib`'s `pyplot` module. The standard way to pull it in is with the nickname `plt` (just like with `numpy` when we import it as `np`).

In Jupyter Notebooks the plots will not show up unless you tell the notebook to put them “inline.” Usually we will use the following command to get the plots to show up. You do not need to do this in Google Colab. The percent sign is called a *magic* command in Jupyter Notebooks. This is not a Python command, but it is a command for controlling the Jupyter IDE specifically.

```
%matplotlib inline
```

Now we will build a `numpy` array of x values (using the `np.linspace` function) and a `numpy` array of y values from the sine function.

- Next, build the plot with `plt.plot()`. The syntax is: `plt.plot(x, y, 'color', ...)` where you have several options that you can pass (more on that later).

- We send the plot label directly to the plot function. This is optional and we could set the legend up separately if we like.
- Then we will add the grid with `plt.grid()`
- Then we will add the legend to the plot
- Finally we will add the axis labels
- We end the plotting code with `plt.show()` to tell Python to finally show the plot. This line of code tells Python that you are done building that plot.

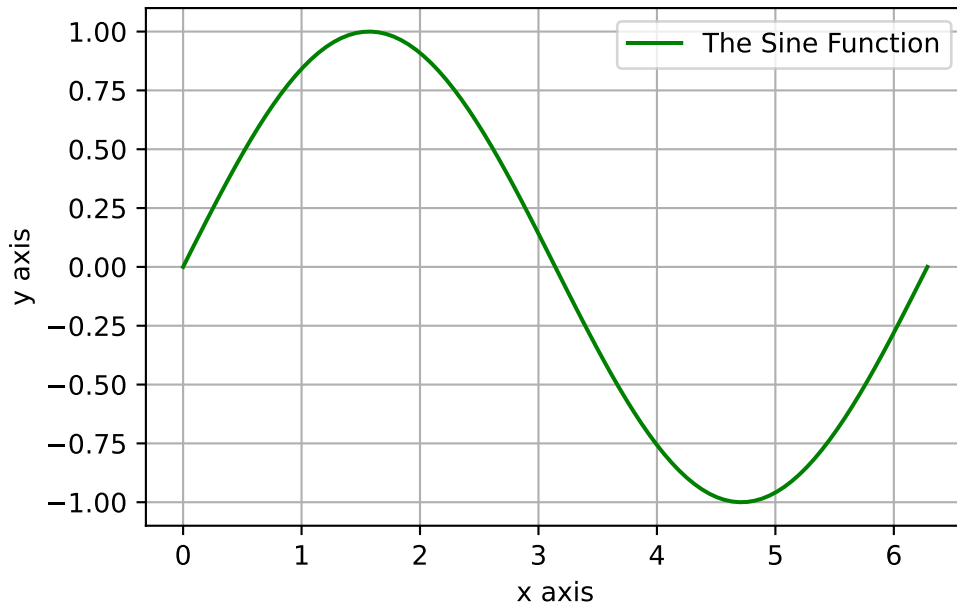


Figure A.1.: The sine function

Example A.43. Now let us do a second example, but this time we want to show four different plots on top of each other. When you start a figure, `matplotlib` is expecting all of those plots to be layered on top of each other. (Note:For MATLAB users, this means that you do not need the `hold on` command since it is automatically “on.”)

In this example we will plot

$$y_0 = \sin(2\pi x) \quad y_1 = \cos(2\pi x) \quad y_2 = y_0 + y_1 \quad \text{and} \quad y_3 = y_0 - y_1 \quad (\text{A.8})$$

on the domain $x \in [0, 1]$ with 100 equally spaced points. we will give each of the plots a different line style, built a legend, put a grid on the plot, and give axis labels.

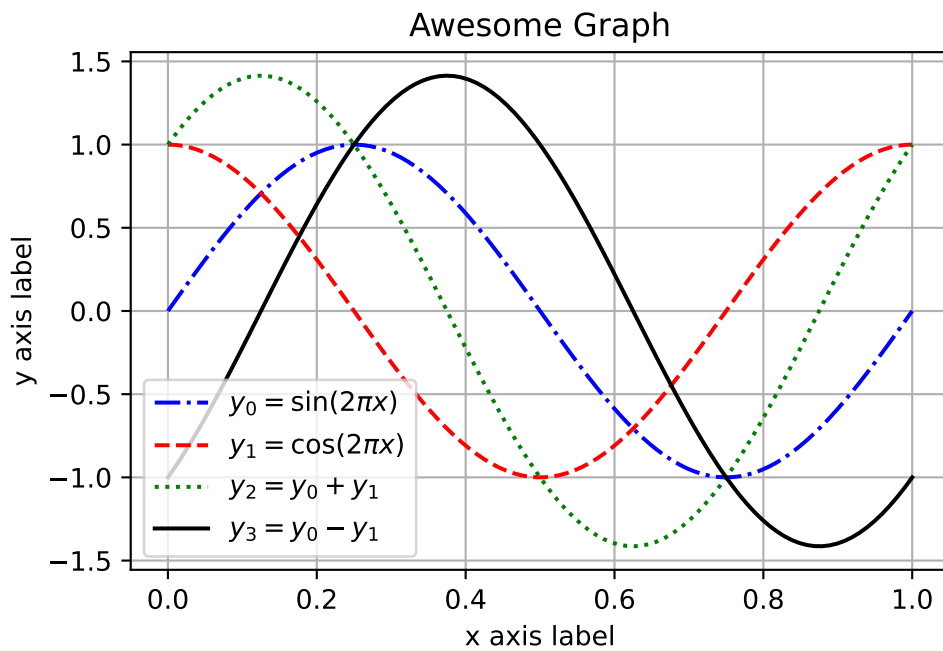


Figure A.2.: Plots of the sine, cosine, and sums and differences.

Notice the `r` in front of the strings defining the legend. This prevents the backslash that is used a lot in LaTeX to be interpreted as an escape character. These strings are referred to as raw strings.

The legend was placed automatically at the lower left of the plot. There are ways to control the placement of the legend if you wish, but for now just let Python and `matplotlib` have control over the placement.

Example A.44. Now let us create the same plot with slightly different code. The `plot` function can take several (x, y) pairs in the same line of code. This can really shrink the amount of coding that you have to do when plotting several functions on top of each other.

Exercise A.16. Plot the functions $f(x) = x^2$, $g(x) = x^3$, and $h(x) = x^4$ on the same axes. Use the domain $x \in [0, 1]$. Put a grid, a legend, a title, and appropriate labels on the axes.

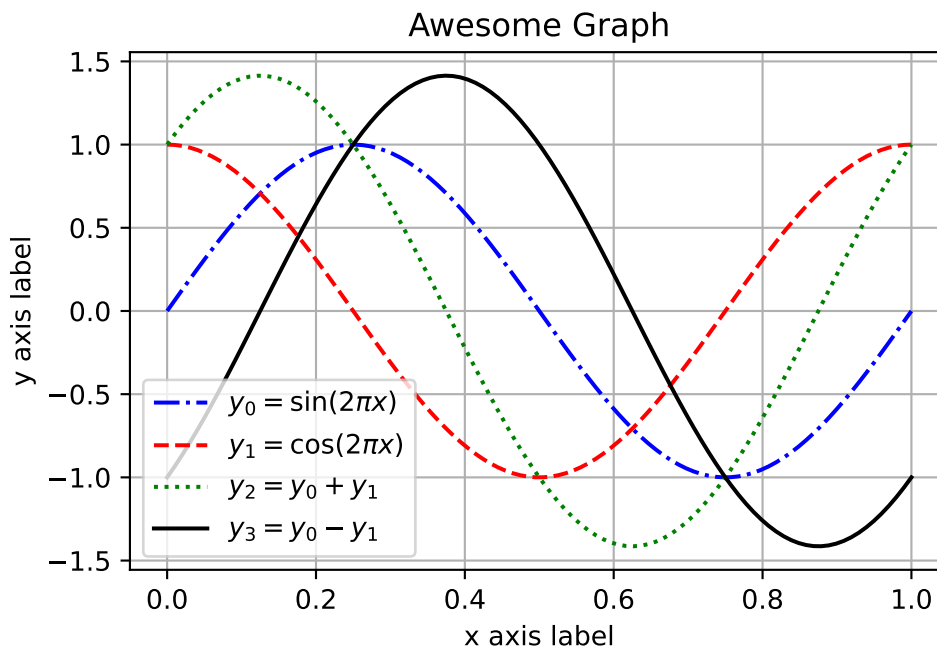


Figure A.3.: A second plot of the sine, cosine, and sums and differences.

A.4.2. Subplots

It is often very handy to place plots side-by-side or as some array of plots. The `subplots` command allows us that control. The main idea is that we are setting up a matrix of blank plots and then populating the axes with the plots that we want.

Example A.45. Let us repeat the previous exercise, but this time we will put each of the plots in its own subplot. There are a few extra coding quirks that come along with building subplots so we will highlight each block of code separately.

- First we set up the plot area with `plt.subplots()`. The first two inputs to the `subplots` command are the number of rows and the number of columns in your plot array. For the first example we will do 2 rows of plots with 2 columns – so there are four plots total.
- Then we build each plot individually telling `matplotlib` which axes to use for each of the things in the plots.
- Notice the small differences in how we set the titles and labels

A. Python

- In this example we are setting the y -axis to the interval $[-2, 2]$ for consistency across all of the plots.

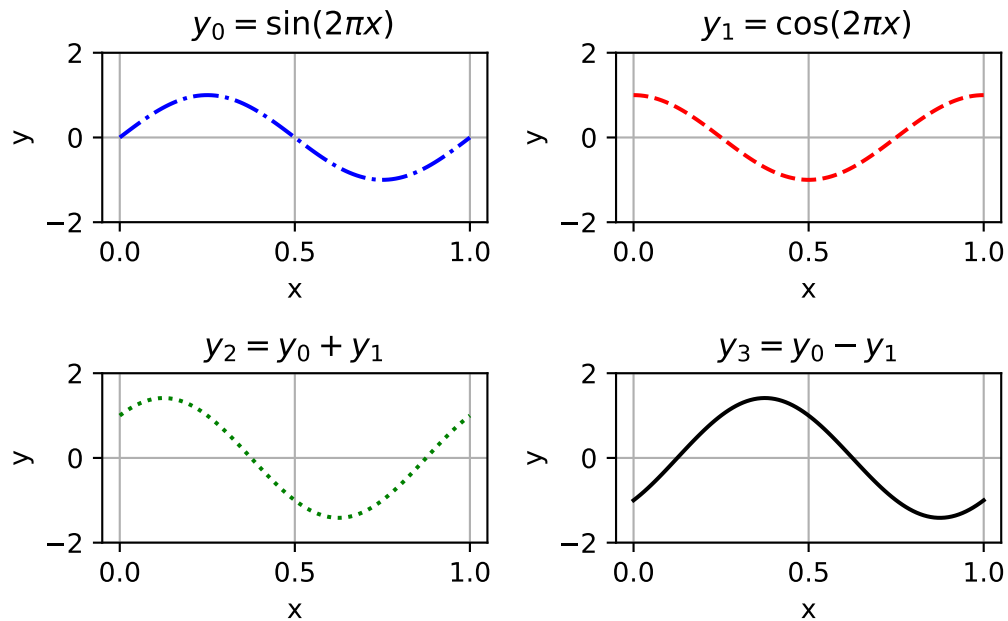


Figure A.4.: An example of subplots

The `fig.tight_layout()` command makes the plot labels a bit more readable in this instance (again, something you can play with).

Exercise A.17. Put the functions $f(x) = x^2$, $g(x) = x^3$ and $h(x) = x^4$ in a subplot environment with 1 row and 3 columns of plots. Use the unit interval as the domain and range for all three plot. Make sure that each plot has a grid, appropriate labels, an appropriate title, and the overall figure has a title.

A.4.3. Logarithmic Scaling with `semilogy`, `semilogx`, and `loglog`

It is occasionally useful to scale an axis logarithmically. This arises most often when we are examining an exponential function, or some other function, that is close to zero for much of the domain. Scaling logarithmically allows us to see how small the function is getting in orders of magnitude instead of as a raw real number. we will use this often in numerical methods.

Example A.46. In this example we will plot the function $y = 10^{-0.01x}$ on a regular (linear) scale and on a logarithmic scale on the y axis. We use the interval $[0, 500]$ on the x axis.

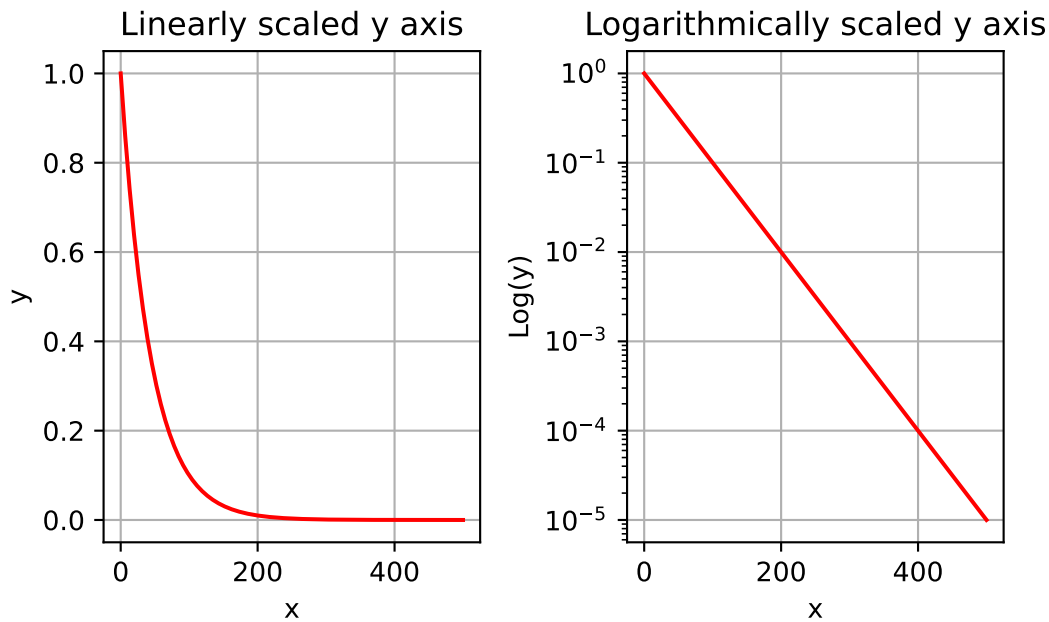


Figure A.5.: An example of using logarithmic scaling.

It should be noted that the same result can be achieved using the `yscale` command along with the `plot` command instead of using the `semilogy` command. So you could replace

```
axis[1].semilogy(x,y, 'r')
```

by

```
axis[1].plot(x,y, 'r')
axis[1].set_yscale("log")
```

to produce identical results.

Exercise A.18. Plot the function $f(x) = x^3$ for $x \in [0, 1]$ on linearly scaled axes, logarithmic axis in the y direction, logarithmically scaled axes in the x direction, and a log-log plot with logarithmic scaling on both axes. Use `subplots` to put your plots side-by-side. Give appropriate labels, titles, etc.

A.5. Dataframes with Pandas

The Pandas package provides Python with the ability to work with tables of data similar to what R provides via its dataframes. As we will not work much with data in this module, we do not need to dive deep into the Pandas package. In some of the optional exercises you will load in data from files using `pd.read_csv()`.

Example A.47.

	Time (sec)	Speed (ft/sec)
0	0	34
1	10	32
2	20	29
3	30	33
4	40	37
5	50	40
6	60	41
7	70	36
8	80	38
9	90	39

Example A.48. Pandas can also be useful to us for collecting computational results into tables for easier display. In this example we will build a table of the first 10 natural numbers and their squares and cubes. We then display the table.

	n	n ²	n ³
0	1	1	1
1	2	4	8
2	3	9	27
3	4	16	64
4	5	25	125
5	6	36	216
6	7	49	343
7	8	64	512
8	9	81	729
9	10	100	1000

This provides an alternative to how we created a tabular display with the `print()` function in Example A.20.

B. Exam solutions

In this appendix you find example solutions to the exam-style questions.

B.1. Numbers

(a) Machine Precision

Machine precision is the gap between the number 1 and the next larger floating point number. With 4 bits for the mantissa, the smallest number greater than 1 that can be represented is 1.0001_2 . So machine precision is $\epsilon = 0.0001_2 = 1/16$. The **relative** rounding error can never be larger than $\epsilon/2$.

(b) Conversion of 13.5

- Sign: Positive (+13.5), so $s = 0$.
- Convert 13.5 to binary:
 - $13_{10} = 1101_2$.
 - $0.5_{10} = 0.1_2$.
 - Result: 1101.1_2 .
- Normalize: 1.1011×2^3 .
- Exponent ($E = 3$):
 - Stored exponent $e = E + 3 = 3 + 3 = 6$.
 - $6_{10} = 110_2$.
- Mantissa (m): Drop the leading 1 $\rightarrow 1011$.
- **Final Bit Pattern: 0 110 1011.**

(c) Addition: $13.5 + 0.25$

- **Convert second number (0.25):**
 - $0.25 = 1/4 = 1.0 \times 2^{-2}$.
- **Align Exponents:**
 - $x_1 = 1.1011 \times 2^3$
 - $x_2 = 1.0000 \times 2^{-2}$

B. Exam solutions

- Shift x_2 to match exponent 3: Shift right by $3 - (-2) = 5$ positions.
- $x_2 \rightarrow 0.00001 \times 2^3$.

- **Add Significands:**

1.1011	(13.5)
+ 0.00001	(0.25 aligned)

1.10111	

- **Rounding:**

- The result 1.10111_2 has 5 fractional bits, but we can only store 4.
- It lies exactly halfway between 1.1011 (13.5) and 1.1100 (14.0).
- Tie-breaking rule: “Ties to Even”.
- The Least Significant Bit (4th bit) is 1 (odd). To make it even, we round **up** (add 1 to the LSB).
- $1.1011 + 0.0001 = 1.1100$.

- **Final Result:**

- Significand: 1.1100
- Value: $1.1100_2 \times 2^3 = 1110.0_2 = \mathbf{14.0}$.

- **Error:**

- Exact sum: 13.75.
- Stored sum: 14.0.
- Absolute Error: $|13.75 - 14.0| = \mathbf{0.25}$.

(d) Loss of Significance

- **Error Type:** Catastrophic Cancellation (or Loss of Significant Digits).
- **Explanation:** When x is very large (10^8), $x^2 + 1 \approx x^2$, so $\sqrt{x^2 + 1} \approx x$. Subtracting two extremely close numbers causes the cancellation of the leading digits, leaving only the random “noise” from the least significant bits.
- **Improved (Stable) Formula:** Multiply by the conjugate:

$$f(x) = \frac{(\sqrt{x^2 + 1} - x)(\sqrt{x^2 + 1} + x)}{\sqrt{x^2 + 1} + x} = \frac{1}{\sqrt{x^2 + 1} + x}.$$

B.2. Functions

(a) Missing x^2 term

The coefficient for the x^2 term in the Taylor Series expansion around $x_0 = 0$ is given by $\frac{f''(0)}{2!}$. For $f(x) = \sin(x)$, the second derivative is $f''(x) = -\sin(x)$. Evaluating this at $x = 0$ yields $f''(0) = -\sin(0) = 0$. Hence, the coefficient is 0. Alternatively, one could note that $\sin(x)$ is an odd function, so its Taylor series only contains odd powers of x .

(b) Python Code

```
import math

def sin_taylor(x, n):
    """
    Calculate the Taylor expansion for f(x) = sin(x) centred at x_0 = 0 up to order n.
    """
    term = x / 1
    result = term
    for i in range(3, n + 1, 2):
        term = -term * x**2 / (i * (i-1))
        result += term
    return result
```

(Other valid implementations, such as computing terms from scratch using `math.factorial` or checking if `i % 2 != 0` within a regular loop, are also acceptable).

(c) Truncation Error

- **i)** If we truncate the series after the x^3 term, the next term in the mathematical Taylor series is the x^5 term (since the x^4 term is 0). Therefore, the truncation error is $\mathcal{O}(x^5)$.
- **ii)** Evaluating the function at $x = 0.05$ instead of $x = 0.1$ reduces x by a factor of $1/2$. Because the error scales as $\mathcal{O}(x^5)$, the truncation error is expected to decrease by a factor of $(1/2)^5 = 1/32$.

B.3. Roots 1

(a) Conditions for Bisection Method Convergence

- The function f must be continuous on the closed interval $[a, b]$.
- The function values at the endpoints must have opposite signs, i.e., $f(a) \cdot f(b) < 0$.

(b) Number of Iterations

- The error after n iterations is bounded by $\frac{|b-a|}{2^n} = \frac{1}{2^n}$.
- We need $\frac{1}{2^n} \leq 2^{-20}$. Therefore, $n = 20$ iterations are required.

(c) Three Iterations of Bisection Method

• **Iteration 1:**

- Interval: $[a, b] = [1, 2]$.
- Midpoint $m_1 = 1.5$.

• **Iteration 2:**

- From the graph, $f(1.5)$ is slightly below the x-axis, so $f(1.5) < 0$.
- Since $f(1.5) < 0$ and $f(2) > 0$, the root must lie in the new interval $[1.5, 2]$.
- Midpoint $m_2 = \frac{1.5+2}{2} = 1.75$.

• **Iteration 3:**

- From the graph, at $x = 1.75$, the function curve is clearly above the x-axis, so $f(1.75) > 0$.
- Since $f(1.5) < 0$ and $f(1.75) > 0$, the narrower interval containing the root is $[1.5, 1.75]$.
- Midpoint $m_3 = \frac{1.5+1.75}{2} = 1.625$.

(d) Fixed-Point Iteration

• **i) Convergence check:**

- $g(x) = (x + 2)^{1/3}$
- $g'(x) = \frac{1}{3}(x + 2)^{-2/3}$.
- For $x \in [1, 2]$, the denominator is positive and increasing.
- The maximum of $|g'(x)|$ on this interval occurs at $x = 1$, where $g'(1) = \frac{1}{3(3)^{2/3}}$.
- Since $|g'(x)| < 1$ for all $x \in [1, 2]$ and $g(x) \in [1, 2]$ on this interval (as $g(1) \approx 1.44$ and $g(2) \approx 1.58$), the Fixed-Point Theorem guarantees convergence for any $x_0 \in [1, 2]$.

• **ii) Code:**

```
def fixed_point_iteration():  
    x = 1.0  
    for i in range(10):  
        x = (x + 2)**(1/3)  
    return x
```


B.4. Roots 2

(a) Secant Method vs Newton's Method

- **Explanation:** The Secant method replaces the exact derivative $f'(x_n)$ in Newton's iteration formula with a finite difference approximation (the slope of the secant line) using the two previous iterates: $f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$.
- **Advantage:** It does not require an explicit formula for the derivative of the function, which is useful if the derivative is difficult or computationally expensive to evaluate.
- **Disadvantage:** It has a lower order of convergence (superlinear, $\alpha \approx 1.62$) compared to Newton's method (quadratic, $\alpha = 2$), so it may require more iterations to achieve the same accuracy. It also requires two initial guesses instead of one.

(b) Order of Convergence

- **Definition:** A sequence $\{x_n\}$ converging to p (with $x_n \neq p$) has an order of convergence $\alpha \geq 1$ if there exists a constant $\lambda > 0$ such that $\lim_{n \rightarrow \infty} \frac{|x_{n+1} - p|}{|x_n - p|^\alpha} = \lambda$.
- **Orders of convergence:**
 - Bisection Method: 1 (Linear)
 - Secant Method: $\frac{1+\sqrt{5}}{2} \approx 1.618$ (Superlinear)
 - Newton's Method: 2 (Quadratic)

(c) Applying the Methods to $f(x) = x^2 - 5$

- **i) Newton's method:**
 - $f(x) = x^2 - 5 \implies f'(x) = 2x$.
 - Formula: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.
 - $x_0 = 2$.
 - $f(2) = 2^2 - 5 = -1$.
 - $f'(2) = 2(2) = 4$.
 - $x_1 = 2 - \frac{-1}{4} = 2.25$.
- **ii) Secant method:**
 - Formula: $x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$.
 - $x_0 = 3, f(x_0) = 3^2 - 5 = 4$.
 - $x_1 = 2, f(x_1) = 2^2 - 5 = -1$.
 - $x_2 = 2 - \frac{(-1)(2-3)}{(-1)-4} = 2 - \frac{(-1)(-1)}{-5} = 2 - \frac{1}{-5} = 2.2$.

(d) Error Estimation

- Given $E_{n+1} \approx E_n^2$.
- $E_3 \approx E_2^2 = (10^{-3})^2 = 10^{-6}$.

B. Exam solutions

- $E_4 \approx E_3^2 = (10^{-6})^2 = 10^{-12}$.
- The expected error E_4 is roughly 10^{-12} .

(e) Order of Convergence for Newton's Method

- Newton's method is defined by $x_{n+1} = g(x_n)$ where $g(x) = x - \frac{f(x)}{f'(x)}$.
- To find the order of convergence, we evaluate the derivatives of $g(x)$ at the root p .
- First derivative: $g'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{[f'(x)]^2} = \frac{[f'(x)]^2 - [f'(x)]^2 + f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}$.
- Evaluate $g'(x)$ at p : since p is a root, $f(p) = 0$. Since it is a simple root, $f'(p) \neq 0$. Thus, $g'(p) = \frac{f(p)f''(p)}{[f'(p)]^2} = \frac{0 \cdot f''(p)}{[f'(p)]^2} = 0$.
- According to the theorem on the order of convergence for fixed-point iterations, if $g(p) = p$ and $g'(p) = 0$, but $g''(p) \neq 0$ (which is generally true unless $f''(p) = 0$), the sequence has an order of convergence of $m = 2$ (quadratic convergence).

B.5. Calculus

(a) Difference Formulas and Accuracies

- **Forward difference:** $f'(x) \approx \frac{f(x+h) - f(x)}{h}$. Order of accuracy: $\mathcal{O}(h)$ (first order).
- **Central difference:** $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$. Order of accuracy: $\mathcal{O}(h^2)$ (second order).

(b) Derivation of Central Difference

- Taylor series:
 - $f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4)$
 - $f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4)$
- Subtracting the second equation from the first: $f(x+h) - f(x-h) = 2hf'(x) + \frac{2h^3}{6}f'''(x) + \mathcal{O}(h^5)$
- Solving for $f'(x)$: $2hf'(x) = f(x+h) - f(x-h) - \frac{h^3}{3}f'''(x) + \mathcal{O}(h^5)$ $f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6}f'''(x) + \mathcal{O}(h^4)$
- Since the leading error term is proportional to h^2 , the truncation error is $\mathcal{O}(h^2)$.

(c) Error Reduction for Central Difference

- The central difference scheme is $\mathcal{O}(h^2)$, meaning the error is proportional to h^2 .
- Reducing the step size from $h = 0.2$ to $h = 0.05$ means we are dividing the step size by a factor of 4.
- Because the error scales quadratically, the new error should be reduced by a factor of $4^2 = 16$.
- Therefore, the expected new absolute error is approximately $0.016/16 = 0.001$.

(d) Trapezoidal Rule Graph

- A sketch should show the curve $y = f(x)$ over an interval, partitioned into subintervals, with straight line segments connecting the function values $(x_i, f(x_i))$ and $(x_{i+1}, f(x_{i+1}))$, forming trapezoids whose combined areas approximate the precise area under the curve.

(e) Trapezoidal Rule Value

- The function $f(x) = 5x + 3$ is a linear function. The trapezoidal rule approximates the area under a curve using straight line segments. For a linear function, the tops of the trapezoids perfectly match the graph of the function.
- Therefore, the trapezoidal rule evaluates the integral *exactly*, independently of the number of subintervals.
- The exact integral is $\int_0^5 (5x + 3)dx = \left[\frac{5}{2}x^2 + 3x\right]_0^5 = \frac{5}{2}(25) + 3(5) = 62.5 + 15 = 77.5$.
- (Alternatively, using a geometric argument: treat the entire interval $[0, 5]$ as a single trapezoid with vertices at $(0, f(0))$ and $(5, f(5))$, giving $\frac{f(0)+f(5)}{2} \times 5 = \frac{3+28}{2} \times 5 = 15.5 \times 5 = 77.5$. This works because for a linear function the composite trapezoidal rule with any number of subintervals gives the same exact answer.)

(f) Order of Accuracy of Trapezoidal Rule

- The order of accuracy for the global trapezoidal rule is $\mathcal{O}(h^2)$ (second order).
- This means that the approximation error is proportional to the square of the step size h . Doubling the number of subintervals halves the step size ($h \rightarrow h/2$), so the approximation error will decrease by a factor of $2^2 = 4$ (it will become one-quarter of its previous value).

(g) Trapezoidal Python Code

```
x = np.linspace(a, b, N+1)

dx = x[1] - x[0]

A = (f(x[:-1]) + f(x[1:])) / 2 * dx

return np.sum(A)
```

B.6. Optimisation**(a) Golden Section Search**

B. Exam solutions

- **Logic:** The Golden Section Search is a bracketing method that iteratively narrows down an interval known to contain a local minimum. It selects a new test point within the current bracket and, based on the function values, replaces one of the bracketing endpoints to create a smaller interval that is still guaranteed to bracket the minimum.
- **Condition:** For the triplet (a, c, b) with $a < c < b$ to bracket a minimum, the function value at the interior point c must be strictly less than the function values at the endpoints a and b (i.e. $f(c) < f(a)$ and $f(c) < f(b)$).

(b) Gradient Descent α Parameter

- **Role:** The parameter α (learning rate) controls the size of the steps taken in the direction of the negative gradient.
- **Too large:** If α is too large, the algorithm acts unstably and may repeatedly step past the minimum point, overshooting and potentially diverging from the minimum.
- **Too small:** If α is too small, the algorithm will confidently move toward the minimum but will require many iterations to arrive, leading to an extremely slow convergence process.

(c) Multivariable Gradient Descent

- **Iteration formula:** $x_{n+1} = x_n - \alpha \nabla f(x_n)$.
- **Geometric rationale:** The gradient vector $\nabla f(x)$ points in the direction of the steepest *ascent* of the function at the point x . Therefore, the negative gradient $-\nabla f(x)$ points directly *downhill* in the direction of steepest *descent*. Moving a small distance in this direction represents the optimal path to decrease the function's value to find a local minimum.

(d) Gradient Descent Code

```
grad = df(x)
x = x - alpha * grad
```

B.7. ODE 1

(a) Numerical Instability

- **Definition:** Numerical instability occurs when the errors in the numerical approximation grow exponentially as the computation proceeds.
- **Result:** The numerical solution diverges rapidly from the exact solution, yielding meaningless results.
- **Avoidance:** It can typically be avoided by choosing a step size h (or Δt) that is smaller than some critical threshold value, or by using an implicit method instead.

(b) Euler's Method

- Differential equation: $x' = -2x + t$, $x(0) = 1$, $h = 0.5$.
- Formula: $x_1 = x_0 + hf(t_0, x_0)$.
- $t_0 = 0$, $x_0 = 1$.
- $f(0, 1) = -2(1) + 0 = -2$.
- $x_1 = 1 + 0.5(-2) = 1 - 1 = 0$.

(c) Midpoint Method

- $m_0 = f(t_0, x_0) = -2$.
- $t_{1/2} = t_0 + h/2 = 0 + 0.25 = 0.25$.
- $x_{1/2} = x_0 + (h/2)m_0 = 1 + 0.25(-2) = 0.5$.
- $m_{1/2} = f(t_{1/2}, x_{1/2}) = -2(0.5) + 0.25 = -0.75$.
- $x_1 = x_0 + hm_{1/2} = 1 + 0.5(-0.75) = 1 - 0.375 = 0.625$.

(d) System of First-Order ODEs

- Let $\theta = \theta$ and $\omega = d\theta/dt$. Then:
- $\frac{d\theta}{dt} = \omega$
- $\frac{d\omega}{dt} = -\frac{g}{L} \sin(\theta)$

(e) Midpoint Method Python Code

```
slope = f(t[n], x[n])
x_halfstep = x[n] + (dt / 2) * slope
x[n+1] = x[n] + dt * f(t[n] + dt / 2, x_halfstep)
```

B.8. ODE 2**(a) Explicit vs Implicit Euler**

- **Geometric Difference:** The explicit (forward) Euler method estimates the next spatial/temporal point by using the derivative evaluated at the *current* known point (t_n, x_n) . The implicit (backward) Euler method, conversely, evaluates the derivative at the *unknown future* point (t_{n+1}, x_{n+1}) .
- **Stiff Equations:** Backward Euler is unconditionally stable for stable linear equations (its region of absolute stability is the entire left half-plane). Thus it avoids numerical instability when simulating stiff equations where transients would otherwise force explicit methods to use impractically small step sizes.

B. Exam solutions

(b) Unconditional Stability of Backward Euler

Apply the backward Euler method to $x' = \lambda x$:

$$x_{n+1} = x_n + h\lambda x_{n+1}.$$

Solving for x_{n+1} :

$$x_{n+1}(1 - h\lambda) = x_n \implies x_{n+1} = \frac{1}{1 - h\lambda} x_n.$$

The amplification factor is $R = \frac{1}{1 - h\lambda}$. For $\text{Re}(\lambda) < 0$ and any $h > 0$, writing $\lambda = a + bi$ with $a < 0$:

$$|1 - h\lambda|^2 = (1 - ha)^2 + (hb)^2 > 1,$$

since $1 - ha > 1$ (as $a < 0$). Therefore $|R| < 1$ for all $h > 0$, so $|x_n| \rightarrow 0$ as $n \rightarrow \infty$, and the method is unconditionally stable.

(c) Local Truncation Error of Backward Euler

The local truncation error is defined as the residual when the exact solution is substituted into the numerical scheme:

$$\tau_n = \frac{1}{h} |x(t_{n+1}) - x_{n+1}| = \left| \frac{x(t_{n+1}) - x(t_n)}{h} - f(t_{n+1}, x(t_{n+1})) \right|.$$

Taylor-expanding $x(t_{n+1})$ about t_n :

$$x(t_{n+1}) = x(t_n) + h x'(t_n) + \frac{h^2}{2} x''(t_n) + \mathcal{O}(h^3),$$

so

$$\frac{x(t_{n+1}) - x(t_n)}{h} = x'(t_n) + \frac{h}{2} x''(t_n) + \mathcal{O}(h^2).$$

Since $f(t_{n+1}, x(t_{n+1})) = x'(t_{n+1})$, Taylor-expanding x' gives:

$$x'(t_{n+1}) = x'(t_n) + h x''(t_n) + \mathcal{O}(h^2).$$

Substituting:

$$\tau_n = \left| x'(t_n) + \frac{h}{2} x''(t_n) - x'(t_n) - h x''(t_n) \right| + \mathcal{O}(h^2) = \frac{h}{2} |x''(t_n)| + \mathcal{O}(h^2) = \mathcal{O}(h).$$

(d) Runge-Kutta 4

- **Order of Accuracy:** $\mathcal{O}(h^4)$ (Fourth order).
- **Function Evaluations:** 4 evaluations of $f(t, x)$ are required at each time step.

B.9. PDE 1

(a) Finite Difference Scheme for 1D Heat Equation

- Forward difference in time: $u_t \approx \frac{U_i^{n+1} - U_i^n}{\Delta t}$.
- Centred difference in space: $u_{xx} \approx \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}$.
- Substitute into equation: $\frac{U_i^{n+1} - U_i^n}{\Delta t} = D \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}$.
- Solve for U_i^{n+1} : $U_i^{n+1} = U_i^n + D \frac{\Delta t}{\Delta x^2} (U_{i+1}^n - 2U_i^n + U_{i-1}^n)$.

(b) Truncation Errors and Overall Order

- Truncation error for u_t approximation is $\mathcal{O}(\Delta t)$.
- Truncation error for u_{xx} approximation is $\mathcal{O}(\Delta x^2)$.
- Overall order of the numerical scheme is $\mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)$.

(c) Stability Condition

- For the explicit forward difference scheme to be stable, we require $a = D \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}$.

(d) Neumann Boundary Condition

- $u_x(t, 0) \approx \frac{U_1^n - U_0^n}{\Delta x} = 0$.
- Therefore, $U_0^n = U_1^n$.

(e) Python implementation

```
a = D * dt / (dx**2)

U[n+1, 1:-1] = U[n, 1:-1] + a * (U[n, 2:] - 2*U[n, 1:-1] + U[n, :-2])
```

(f) Matrix notation and stability

- i) For $N = 4$ there are $N - 1 = 3$ interior grid points. The matrix A is 3×3 :

$$A = \begin{pmatrix} 1-2a & a & 0 \\ a & 1-2a & a \\ 0 & a & 1-2a \end{pmatrix}.$$

It is tridiagonal, with $1 - 2a$ on the main diagonal and a on the two off-diagonals.

- ii) Applying the recurrence $\mathbf{U}_{n+1} = A\mathbf{U}_n$ repeatedly gives

$$\mathbf{U}_n = A^n \mathbf{U}_0.$$

B. Exam solutions

- **iii)** After n steps the computed vector satisfies

$$\tilde{\mathbf{U}}_n = A^n \tilde{\mathbf{U}}_0 = A^n(\mathbf{U}_0 + \epsilon) = A^n \mathbf{U}_0 + A^n \epsilon = \mathbf{U}_n + A^n \epsilon.$$

Hence the error at step n is $\epsilon_n = \tilde{\mathbf{U}}_n - \mathbf{U}_n = A^n \epsilon$.

- **iv)** Expanding ϵ in the eigenbasis $\{\mathbf{v}_k\}$ of A (with eigenvalues λ_k) gives

$$\epsilon_n = A^n \epsilon = \sum_k \epsilon_k \lambda_k^n \mathbf{v}_k.$$

The k -th component of the error is multiplied by λ_k^n at each step. For stability we therefore require $|\lambda_k| \leq 1$ for all k ; if any $|\lambda_k| > 1$ that component grows without bound and the method is unstable.

B.10. PDE 2

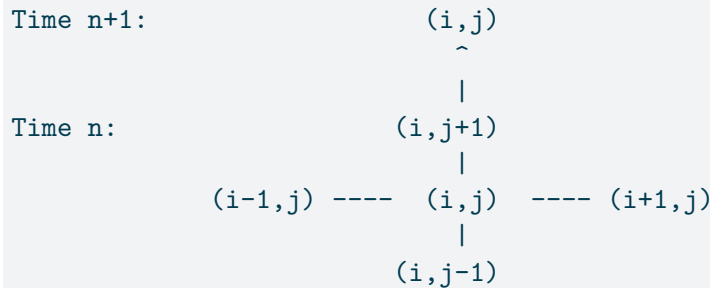
(a) Finite Difference Formula for 2D Heat Equation

- Forward difference in time: $\frac{U_{i,j}^{n+1} - U_{i,j}^n}{\Delta t}$.
- Centred differences in space: $\frac{U_{i+1,j}^n - 2U_{i,j}^n + U_{i-1,j}^n}{h^2}$ and $\frac{U_{i,j+1}^n - 2U_{i,j}^n + U_{i,j-1}^n}{h^2}$.
- Substitute and isolate $U_{i,j}^{n+1}$:

$$U_{i,j}^{n+1} = U_{i,j}^n + D \frac{\Delta t}{h^2} (U_{i+1,j}^n + U_{i-1,j}^n - 4U_{i,j}^n + U_{i,j+1}^n + U_{i,j-1}^n)$$

(b) Finite Difference Stencil

- The stencil comprises five points at the current time step n representing a spatial cross: the central node (i, j) and its four immediate neighbours in cardinal directions. These five points are used to compute the value for the single node (i, j) at the next time step $(n + 1)$.



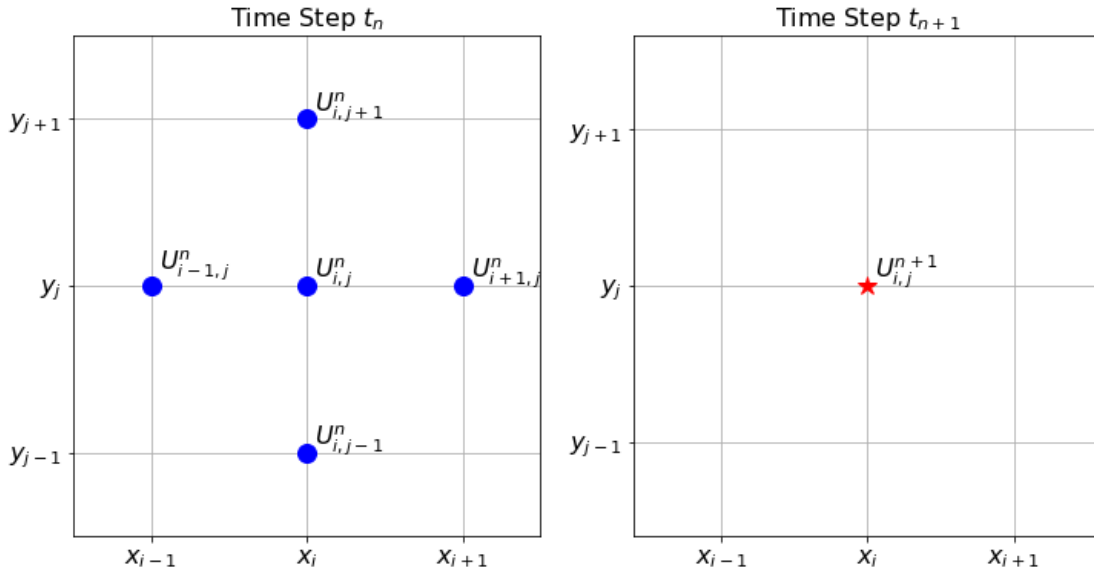


Figure B.1.: The finite difference stencil for the 2D heat equation.

(c) Stability Condition

- $a = D \frac{\Delta t}{h^2} \leq \frac{1}{4}$.

(d) Finite Difference Scheme for 1D Wave Equation

- Using centred differences in time and space:

$$\frac{U_i^{n+1} - 2U_i^n + U_i^{n-1}}{\Delta t^2} = c^2 \frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}$$

- Rearranging:

$$U_i^{n+1} = 2U_i^n - U_i^{n-1} + c^2 \frac{\Delta t^2}{\Delta x^2} (U_{i+1}^n - 2U_i^n + U_{i-1}^n)$$

- Initial data required: Because the scheme requires data from two previous time steps, both the initial position $u(0, x)$ (for U_i^0) and the initial velocity $u_t(0, x)$ (used to estimate U_i^1) are required.

(e) Python Code

```
a = D * dt / dx**2

U[0, :, :] = u_0(X, Y)
```

B. Exam solutions

```
U[n, 2:, 1:-1] +  
U[n, :-2, 1:-1] -  
4 * U[n, 1:-1, 1:-1] +  
U[n, 1:-1, 2:] +  
U[n, 1:-1, :-2]
```

C. Linear Algebra

You cannot learn too much linear algebra.
– Every mathematician

C.1. Intro to Numerical Linear Algebra

The preceding comment says it all – linear algebra is the most important of all of the mathematical tools that you can learn as a practitioner of the mathematical sciences. The theorems, proofs, conjectures, and big ideas in almost every other mathematical field find their roots in linear algebra. Numerical Linear Algebra is the study of algorithms for solving problems in linear algebra. This subject has a somewhat different flavour than the numerical analysis we are studying in the main text and hence we present it as an optional appendix.

Our goal in this appendix is to explore numerical algorithms for the primary questions of linear algebra:

- solving systems of equations,
- finding eigenvalue-eigenvector pairs for a matrix.

Take careful note that in our current digital age, numerical linear algebra and its fast algorithms are behind the scenes for wide varieties of computing applications. Applications of numerical linear algebra include:

- building neural networks and AI algorithms,
- determining the most important web page in a Google search,
- modelling realistic 3D environments in video games,
- digital image processing,
- and many many more.

What's more, researchers have found provably optimal ways to perform most of the typical tasks of linear algebra so most scientific software works very well and very quickly with linear algebra.

C.2. Notation

Throughout this chapter we will use the following notation conventions.

- A bold mathematical symbol such as x or u will represent a vector.
- If u is a vector then u_j will be the j^{th} entry of the vector.
- Vectors will typically be written vertically with parenthesis as delimiters such as

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}. \quad (\text{C.1})$$

- Two bold symbols separated by a centred dot such as $u \cdot v$ will represent the dot product of two vectors.
- A capital mathematical symbol such as A or X will represent a matrix
- If A is a matrix then A_{ij} will be the element in the i^{th} row and j^{th} column of the matrix.
- A matrix will typically be written with parenthesis as delimiters such as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & \pi \end{pmatrix}. \quad (\text{C.2})$$

- The juxtaposition of a capital symbol and a bold symbol such as Ax will represent matrix-vector multiplication.
- A lower case or Greek mathematical symbol such as x , c , or λ will represent a scalar.
- The scalar field of real numbers is given as $\mathbb{R}$ and the scalar field of complex numbers is given as $\mathbb{C}$.
- The symbol $\mathbb{R}^n$ represents the collection of all n -dimensional vectors where the elements are drawn from the real numbers.
- The symbol $\mathbb{C}^n$ represents the collection of all n -dimensional vectors where the elements are drawn from the complex numbers.

It is an important part of learning to read and write linear algebra to give special attention to the symbolic language so you can communicate your work easily and efficiently.

C.3. Vectors and Matrices in Python

We first need to understand how Python’s `numpy` library builds and stores vectors and matrices. The following exercises will give you some experience building and working with these data structures and will point out some common pitfalls that mathematicians fall into when using Python for linear algebra.

Example C.1 (numpy Arrays). In Python you can build a list using square brackets such as `[1,2,3]`. This is called a “Python list” and is NOT a vector in the way that we think about it mathematically. It is simply an ordered collection of objects. To build mathematical vectors in Python we need to use `numpy` arrays with `np.array()`. For example, the vector

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad (\text{C.3})$$

would be built with the following code.

```
import numpy as np
u = np.array([1,2,3])
print(u)
```

Notice that Python defines the vector `u` as a matrix with only one dimension. You can see that in the following code.

```
import numpy as np
u = np.array([1,2,3])
print("The length of the u vector is \n",len(u))
print("The shape of the u vector is \n",u.shape)
```

Example C.2 (numpy Matrices). In `numpy`, a matrix is validly built as a list of lists. For example, the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad (\text{C.4})$$

is defined using `np.array()` where each row is an individual list, and the matrix is a collection of these lists.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(A)
```

Moreover, we can extract the shape, the number of rows, and the number of columns of A using the `A.shape` command. To be a bit more clear on this one we will use the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad (\text{C.5})$$

```
import numpy as np
A = np.array([[1,2,3],[4,5,6]])
print("The shape of the A matrix is \n",A.shape)
print("Number of rows in A is \n",A.shape[0])
print("Number of columns in A is \n",A.shape[1])
```

Example C.3 (Row and Column Vectors in Python). You can more specifically build row or column vectors in Python using the `np.array()` command and then only specifying one row or column. But take careful note that `numpy` treats a 1D array (like `np.array([1,2,3])`) differently from a 2D array (like `np.array([[1,2,3]])`). For example, if you want the vectors

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 4 & 5 & 6 \end{pmatrix} \quad (\text{C.6})$$

then we would use the following Python code.

```
import numpy as np
u = np.array([[1],[2],[3]])
print("The column vector u is \n",u)
v = np.array([1,2,3])
print("The row vector v is \n",v)
```

Alternatively, if you want to define a column vector you can define a row vector (since there are far fewer brackets to keep track of) and then transpose the matrix to turn it into a column. Note that the `.transpose()` method (or `.T`) only swaps dimensions; it won't turn a 1D array into a 2D column vector unless it was already 2D.

```
import numpy as np
u = np.array([[1,2,3]])
u = u.transpose()
print("The column vector u is \n",u)
```

Example C.4 (Matrix Indexing). Python indexes all arrays, vectors, lists, and matrices starting from index 0. Let us get used to this fact.

Consider the matrix A defined in the previous problem. Mathematically we know that the entry in row 1 column 1 is a 1, the entry in row 1 column 2 is a 2, and so on. However, with Python we need to shift the way that we enumerate the rows and columns of a matrix. Hence we would say that the entry in row 0 column 0 is a 1, the entry in row 0 column 1 is a 2, and so on.

Mathematically we can view all Python matrices as follows. If A is an $n \times n$ matrix then

$$A = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & \cdots & A_{0,n-1} \\ A_{1,0} & A_{1,1} & A_{1,2} & \cdots & A_{1,n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{n-1,0} & A_{n-1,1} & A_{n-1,2} & \cdots & A_{n-1,n-1} \end{pmatrix} \quad (\text{C.7})$$

Similarly, we can view all vectors as follows. If u is an $n \times 1$ vector then

$$u = \begin{pmatrix} u_0 \\ u_1 \\ \vdots \\ u_{n-1} \end{pmatrix} \quad (\text{C.8})$$

The following code should help to illustrate this indexing convention.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Entry in row 0 column 0 is",A[0,0])
print("Entry in row 0 column 1 is",A[0,1])
print("Entry in the bottom right corner",A[2,2])
```

Exercise C.1. Build your own matrix in Python and practice choosing individual entries from the matrix.

Example C.5 (Matrix Slicing). The last thing that we need to be familiar with is *slicing* a matrix. The term “slicing” generally refers to pulling out individual rows, columns, entries, or blocks from a list, array, or matrix in Python. Examine the code below to see how to slice parts out of a `numpy` matrix.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(A)
print("The first column of A is \n",A[:,0])
print("The second row of A is \n",A[1,:])
print("The top left 2x2 sub matrix of A is \n",A[:-1,:-1])
print("The bottom right 2x2 sub matrix of A is \n",A[1:,1:])
u = np.array([1,2,3,4,5,6])
print("The first 3 entries of the vector u are \n",u[:3])
print("The last entry of the vector u is \n",u[-1])
print("The last two entries of the vector u are \n",u[-2:])
```

Exercise C.2. Define the matrix A and the vector u in Python. Then perform all of the tasks below.

$$A = \begin{pmatrix} 1 & 3 & 5 & 7 \\ 2 & 4 & 6 & 8 \\ -3 & -2 & -1 & 0 \end{pmatrix} \quad \text{and} \quad u = \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix} \quad (\text{C.9})$$

1. Print the matrix A , the vector u , the shape of A , and the shape of u .
2. Print the first column of A .
3. Print the first two rows of A .
4. Print the first two entries of u .
5. Print the last two entries of u .
6. Print the bottom left 2×2 submatrix of A .
7. Print the middle two elements of the middle row of A .

C.4. Matrix and Vector Operations

Now let us start doing some numerical linear algebra. We start our discussion with the basics: the dot product and matrix multiplication. The numerical routines in Python's `numpy` packages are designed to do these tasks in very efficient ways but it is a good coding exercise to build your own dot product and matrix multiplication routines just to further cement the way that Python deals with these data structures and to remind you of the mathematical algorithms. What you will find in numerical linear algebra is that the indexing and the housekeeping in the codes is the hardest part. So why do not we start "easy."

C.4.1. The Dot Product

Exercise C.3. This problem is meant to jog your memory about dot products, how to compute them, and what you might use them for. If your linear algebra is a bit rusty then read ahead a bit and then come back to this problem.

Consider two vectors u and v defined as

$$u = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 3 \\ 4 \end{pmatrix}. \quad (\text{C.10})$$

1. Draw a picture showing both u and v .
2. What is $u \cdot v$?
3. What is $\|u\|$?
4. What is $\|v\|$?
5. What is the angle between u and v ?
6. Give two reasons why we know that u is not perpendicular to v .
7. What is the scalar projection of u onto v ? Draw this scalar projections on your picture from part (1).
8. What is the scalar projection of v onto u ? Draw this scalar projections on your picture from part (1).

Now let us get the formal definitions of the dot product on the table.

C. Linear Algebra

Definition C.1 (Dot product). The **dot product** of two vectors $u, v \in \mathbb{R}^n$ is

$$u \cdot v = \sum_{j=1}^n u_j v_j. \quad (\text{C.11})$$

Without summation notation the dot product of two vectors is ,

$$u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n. \quad (\text{C.12})$$

You may also recall that the dot product of two vectors is given geometrically as

$$u \cdot v = \|u\| \|v\| \cos \theta \quad (\text{C.13})$$

where $\|u\|$ and $\|v\|$ are the magnitudes (or lengths) of u and v respectively, and θ is the angle between the two vectors. In physical applications the dot product is often used to find the angle between two vectors (e.g. between two forces). Hence, the last form of the dot product is often rewritten as

$$\theta = \cos^{-1} \left(\frac{u \cdot v}{\|u\| \|v\|} \right). \quad (\text{C.14})$$

Definition C.2 (Magnitude of a Vector). The **magnitude** of a vector $u \in \mathbb{R}^n$ is defined as ¹

$$\|u\| = \sqrt{u \cdot u}. \quad (\text{C.15})$$

Exercise C.4. Write a Python function that accepts two vectors (defined as `numpy` arrays) and returns the dot product. Write this code without the use any loops.

```
import numpy as np
def myDotProduct(u,v):
    return # the dot product formula uses a product inside a sum.
```

¹You should also note that $\|u\| = \sqrt{u \cdot u}$ is not the only definition of distance. More generally, if you let $\langle u, v \rangle$ be an inner product for u and v in some vector space $\mathcal{V}$ then $\|u\| = \sqrt{\langle u, u \rangle}$. In most cases in this text we will be using the dot product as our preferred inner product so we will not have to worry much about this particular natural extension of the definition of the length of a vector.

Exercise C.5. Test your `myDotProduct()` function on several dot products to make sure that it works. Example code to find the dot product between

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} \quad (\text{C.16})$$

is given below. Test your code on other vectors. Then implement an error catch into your code to catch the case where the two input vectors are not the same size. You will want to use the `len()` command to find the length of the vectors.

```
u = np.array([1,2,3])
v = np.array([4,5,6])
myDotProduct(u,v)
```

Exercise C.6. Try sending Python lists instead of `numpy` arrays into your `myDotProduct` function. What happens? Why does it happen? What is the cautionary tale here? Modify your `myDotProduct()` function one more time so that it starts by converting the input vectors into `numpy` arrays.

```
u = [1,2,3]
v = [4,5,6]
myDotProduct(u,v)
```

Exercise C.7. The `numpy` library in Python has a built-in command for doing the dot product: `np.dot()`. Test the `np.dot()` command and be sure that it does the same thing as your `myDotProduct()` function.

C.4.2. Matrix Multiplication

Next we will blow the dust off of your matrix multiplication skills.

C. Linear Algebra

Exercise C.8. Verify that the product of A and B is indeed what we show below. Work out all of the details by hand.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad B = \begin{pmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \quad (\text{C.17})$$

$$AB = \begin{pmatrix} 27 & 30 & 33 \\ 61 & 68 & 75 \\ 95 & 106 & 117 \end{pmatrix} \quad (\text{C.18})$$

Now that you have practised the algorithm for matrix multiplication we can formalize the definition and then turn the algorithm into a Python function.

Definition C.3 (Matrix Multiplication). If A and B are matrices with $A \in \mathbb{R}^{n \times p}$ and $B \in \mathbb{R}^{p \times m}$ then the product AB is defined as

$$(AB)_{ij} = \sum_{k=1}^p A_{ik} B_{kj}. \quad (\text{C.19})$$

A moment's reflection reveals that each entry in the matrix product is actually a dot product,

$$(\text{Entry in row } i \text{ column } j \text{ of } AB) = (\text{Row } i \text{ of matrix } A) \cdot (\text{Column } j \text{ of matrix } B). \quad (\text{C.20})$$

Exercise C.9. The definition of matrix multiplication above contains the cryptic phrase *a moment's reflection reveals that each entry in the matrix product is actually a dot product*. Let us go back to the matrices A and B defined in Exercise C.8 above and re-evaluate the matrix multiplication algorithm to make sure that you see each entry as the end result of a dot product.

We want to find the product of matrices A and B using dot products.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad B = \begin{pmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \quad (\text{C.21})$$

1. Why will the product AB clearly be a 3×3 matrix?
2. When we do matrix multiplication we take the product of a row from the first matrix times a column from the second matrix ... at least that's how many people think of it when they perform the operation by hand.

- a. The rows of A can be written as the vectors

$$a_0 = \begin{pmatrix} 1 & 2 \end{pmatrix} \quad (\text{C.22})$$

$$a_1 = \begin{pmatrix} \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \end{pmatrix} \quad (\text{C.23})$$

$$a_2 = \begin{pmatrix} \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \end{pmatrix} \quad (\text{C.24})$$

- b. The columns of B can be written as the vectors

$$b_0 = \begin{pmatrix} 7 \\ 10 \end{pmatrix} \quad (\text{C.25})$$

$$b_1 = \begin{pmatrix} \rule{1cm}{0.4pt} \\ \rule{1cm}{0.4pt} \end{pmatrix} \quad (\text{C.26})$$

$$b_2 = \begin{pmatrix} \rule{1cm}{0.4pt} \\ \rule{1cm}{0.4pt} \end{pmatrix} \quad (\text{C.27})$$

3. Now let us write each entry in the product AB as a dot product.

$$AB = \begin{pmatrix} a_0 \cdot b_0 & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} \\ \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} \\ \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} & \rule{1cm}{0.4pt} \cdot \rule{1cm}{0.4pt} \end{pmatrix} \quad (\text{C.28})$$

4. Verify that you get

$$AB = \begin{pmatrix} 27 & 30 & 33 \\ 61 & 68 & 75 \\ 95 & 106 & 117 \end{pmatrix} \quad (\text{C.29})$$

when you perform all of the dot products from part (3).

C. Linear Algebra

Exercise C.10. The observation that matrix multiplication is just a bunch of dot products is what makes the code for doing matrix multiplication very fast and very streamlined. We want to write a Python function that accepts two `numpy` matrices and returns the product of the two matrices. Inside the code we will leverage the `np.dot()` command to do the appropriate dot products.

Partial code is given below. Fill in all of the details and give ample comments showing what each line does.

```
import numpy as np
def myMatrixMult(A,B):
    # Get the shapes of the matrices A and B.
    # Then write an if statement that catches size mismatches
    # in the matrices. Next build a zeros matrix that is the
    # correct size for the product of A and B.
    AB = ???
    # AB is a zeros matrix that will be filled with the values
    # from the product
    #
    # Next we do a double for-loop that loops through all of
    # the indices of the product
    for i in range(n): # loop over the rows of AB
        for j in range(m): # loop over the columns of AB
            # use the np.dot() command to take the dot product
            AB[i,j] = ???
    return AB
```

Use the following test code to determine if you actually get the correct matrix product out of your code.

```
``` python
A = np.array([[1,2],[3,4],[5,6]])
B = np.array([[7,8,9],[10,11,12]])
AB = myMatrixMult(A,B)
print(AB)
```

---

**Exercise C.11.** Try your `myMatrixMult()` function on several other matrix multiplication problems.

---

**Exercise C.12.** Build in an error catch so that your `myMatrixMult()` function catches when the input matrices do not have compatible sizes for multiplication. Write your code so that it returns an appropriate error message in this special case.

---

Now that you have been through the exercise of building a matrix multiplication function we will admit that using it inside larger coding problems would be a bit cumbersome (and perhaps annoying). It would be nice to just type `@` and have Python just *know* that you mean to do matrix multiplication. This is where `numpy` arrays come in quite handy.

---

**Exercise C.13** (Matrix Multiplication with Python). Python will handle matrix multiplication easily so long as the matrices are defined as `numpy` arrays. For example, with the matrices  $A$  and  $B$  from above if you can just type `A @ B` (or `np.dot(A, B)`) in Python and you will get the correct result. Pretty nice!! Let us take another moment to notice, though, that regular Python lists do not behave in the same way. Can you guess what happens if you run the following Python code? (Note: `*` does strictly element-wise multiplication for `numpy` arrays, which we will see in a moment).

```
A = [[1,2],[3,4],[5,6]] # a Python list of lists
B = [[7,8,9],[10,11,12]] # a Python list of lists
A @ B
```

---

**Example C.6** (Element-by-Element Multiplication). Sometimes it is convenient to do naive multiplication of matrices when you code. That is, if you have two matrices that are the same size, “naive multiplication” would just line up the matrices on top of each other and multiply the corresponding entries.<sup>2</sup> In Python you can do this with the `*` operator (or `np.multiply()`). The code below demonstrates this tool with the matrices

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}. \quad (\text{C.30})$$

---

<sup>2</sup>You might have thought that *naive multiplication* was a much more natural way to do matrix multiplication when you first saw it. Hopefully now you see the power in the definition of matrix multiplication that we actually use. If not, then I give you this moment to ponder that (a) matrix multiplication is just a bunch of dot products, and (b) dot products can be seen as projections. Hence, matrix multiplication is really just a projection of the rows of  $A$  onto the columns of  $B$ . This has much more rich geometric flavour than *naive multiplication*.

### C. Linear Algebra

(Note that the product  $AB$  does not make sense under the mathematical definition of matrix multiplication, but it does make sense in terms of element-by-element (“naive”) multiplication.)

```
import numpy as np
A = np.array([[1,2],[3,4],[5,6]])
B = np.array([[7,8],[9,10],[11,12]])
print(A * B)
```

---

The key takeaways for doing matrix multiplication in Python are as follows:

- If you are doing linear algebra in Python then you should define vectors and matrices with `np.array()`.
  - If your matrices are defined with `np.array()` then `@` (or `np.dot()`) does regular matrix multiplication and `*` does element-by-element multiplication.
- 

## C.5. The LU Factorization

One of the many classic problems of linear algebra is to solve the linear system  $Ax = b$  where  $A$  is a matrix of coefficients and  $b$  is a vector of right-hand sides. You likely recall your go-to technique for solving systems was row reduction (or Gaussian Elimination). Furthermore, you likely rarely actually did row reduction by hand, and instead you relied on a computer to do most of the computations for you. Just what was the computer doing, exactly? Do you think that it was actually following the same algorithm that you did by hand?

### C.5.1. A Recap of Row Reduction

Let us blow the dust off your row reduction skills before we look at something better.

---



**Exercise C.14.** Solve the following system of equations by hand.

$$\begin{aligned} x_0 + 2x_1 + 3x_2 &= 1 \\ 4x_0 + 5x_1 + 6x_2 &= 0 \\ 7x_0 + 8x_1 &= 2 \end{aligned} \tag{C.31}$$

Note that the system of equations can also be written in the matrix form

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{C.32}$$

If you need a nudge to get started then jump ahead to the next problem.

**Exercise C.15.** We want to solve the system of equations

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{C.33}$$

### Row Reduction Process:

**Note:** Throughout this discussion we use Python-type indexing so the rows and columns are enumerated starting at 0. That is to say, we will talk about row 0, row 1, and row 2 of a matrix instead of rows 1, 2, and 3.

1. Augment the coefficient matrix and the vector on the right-hand side to get

$$\left( \begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 4 & 5 & 6 & 0 \\ 7 & 8 & 0 & 2 \end{array} \right) \tag{C.34}$$

2. The goal of row reduction is to perform elementary row operations until our augmented matrix gets to (or at least gets as close as possible to)

$$\left( \begin{array}{ccc|c} 1 & 0 & 0 & \star \\ 0 & 1 & 0 & \star \\ 0 & 0 & 1 & \star \end{array} \right) \tag{C.35}$$

The allowed elementary row operations are:

- a. We are allowed to scale any row.
- b. We can add two rows.

### C. Linear Algebra

- c. We can interchange two rows.
- 3. We are going to start with column 0. We already have the “1” in the top left corner so we can use it to eliminate all of the other values in the first column of the matrix.

- a. For example, if we multiply the 0<sup>th</sup> row by  $-4$  and add it to the first row we get

$$\left( \begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & -3 & -6 & -4 \\ 7 & 8 & 0 & 2 \end{array} \right). \quad (\text{C.36})$$

- b. Multiply row 0 by a scalar and add it to row 2. Your end result should be

$$\left( \begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & -3 & -6 & -4 \\ 0 & -6 & -21 & -5 \end{array} \right). \quad (\text{C.37})$$

What did you multiply by? Why?

- 4. Now we should deal with column 1.

- a. We want to get a 1 in row 1 column 1. We can do this by scaling row 1. What did you scale by? Why? Your end result should be

$$\left( \begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 2 & \frac{4}{3} \\ 0 & -6 & -21 & -5 \end{array} \right). \quad (\text{C.38})$$

- b. Now scale row 1 by something and add it to row 0 so that the entry in row 0 column 1 becomes a 0.
- c. Next scale row 1 by something and add it to row 2 so that the entry in row 2 column 1 becomes a 0.
- d. At this point you should have the augmented system

$$\left( \begin{array}{ccc|c} 1 & 0 & -1 & -\frac{5}{3} \\ 0 & 1 & 2 & \frac{4}{3} \\ 0 & 0 & -9 & 3 \end{array} \right). \quad (\text{C.39})$$

- 5. Finally we need to work with column 2.

- a. Make the value in row 2 column 2 a 1 by scaling row 2. What did you scale by? Why?
- b. Scale row 2 by something and add it to row 1 so that the entry in row 1 column 2 becomes a 0. What did you scale by? Why?

- c. Scale row 2 by something and add it to row 0 so that the entry in row 0 column 2 becomes a 0. What did you scale by? Why?
- d. By the time you have made it this far you should have the system

$$\left( \begin{array}{ccc|c} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -\frac{1}{3} \end{array} \right) \quad (\text{C.40})$$

and you should be able to read off the solution to the system.

- 6. You should verify your answer in two different ways:
  - a. If you substitute your values into the original system then all of the equal signs should be true. Verify this.
  - b. If you substitute your values into the matrix equation and perform the matrix-vector multiplication on the left-hand side of the equation you should get the right-hand side of the equation. Verify this.

---

**Exercise C.16.** Summarize the process for doing Gaussian Elimination to solve a square system of linear equations.

---

### C.5.2. The LU Decomposition

You may have used the `rref()` command either on a calculator in other software to perform row reduction in the past. You will be surprised to learn that there is no `rref()` command in Python's `numpy` library! That's because there are far more efficient and stable ways to solve a linear system on a computer. There is an `rref` command in Python's `sympy` (symbolic Python) library, but given that it works with symbolic algebra it is quite slow.

In solving systems of equations we are interested in equations of the form  $Ax = b$ . Notice that the  $b$  vector is just along for the ride, so to speak, in the row reduction process since none of the values in  $b$  actually cause you to make different decisions in the row reduction algorithm. Hence, we only really need to focus on the matrix  $A$ . Furthermore, let us change our awfully restrictive view of always seeking a matrix of the form

$$\left( \begin{array}{cccc|c} 1 & 0 & \cdots & 0 & \star \\ 0 & 1 & \cdots & 0 & \star \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & \star \end{array} \right) \quad (\text{C.41})$$

### C. Linear Algebra

and instead say:

*What if we just row reduce until the system is simple enough to solve by hand?*

That's what the next several exercises are going to lead you to. Our goal here is to develop an algorithm that is fast to implement on a computer and simultaneously performs the same basic operations as row reduction for solving systems of linear equations.

**Exercise C.17.** Let  $A$  be defined as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}. \quad (\text{C.42})$$

1. The first step in row reducing  $A$  would be to multiply row 0 by  $-4$  and add it to row 1. Do this operation by hand so that you know what the result is supposed to be. Check out the following amazing observation. Define the matrix  $L_1$  as follows:

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (\text{C.43})$$

Now multiply  $L_1$  and  $A$ .

$$L_1 A = \begin{pmatrix} \_ & \_ & \_ \\ \_ & \_ & \_ \\ \_ & \_ & \_ \end{pmatrix} \quad (\text{C.44})$$

What just happened?!

2. Let us do it again. The next step in the row reduction of your result from part (1) would be to multiply row 0 by  $-7$  and add to row 2. Again, do this by hand so you know what the result should be. Then define the matrix  $L_2$  as

$$L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -7 & 0 & 1 \end{pmatrix} \quad (\text{C.45})$$

and find the product  $L_2(L_1 A)$ .

$$L_2(L_1 A) = \begin{pmatrix} \_ & \_ & \_ \\ \_ & \_ & \_ \\ \_ & \_ & \_ \end{pmatrix} \quad (\text{C.46})$$

Pure insanity!!

3. Now let us say that you want to make the entry in row 2 column 1 into a 0 by scaling row 1 by something and then adding to row 2. Determine what the scalar would be and then determine which matrix, call it  $L_3$ , would do the trick so that  $L_3(L_2L_1A)$  would be the next row reduced step.

$$L_3 = \begin{pmatrix} 1 & \text{---} & \text{---} \\ \text{---} & 1 & \text{---} \\ \text{---} & \text{---} & 1 \end{pmatrix} \quad (\text{C.47})$$

$$L_3(L_2L_1A) = \begin{pmatrix} \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \end{pmatrix} \quad (\text{C.48})$$


---

**Exercise C.18.** Apply the same idea from the previous problem to do the first three steps of row reduction to the matrix

$$A = \begin{pmatrix} 2 & 6 & 9 \\ -6 & 8 & 1 \\ 2 & 2 & 10 \end{pmatrix} \quad (\text{C.49})$$


---

**Exercise C.19.** Now let us make a few observations about the two previous problems.

1. What will multiplying  $A$  by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (\text{C.50})$$

do?

2. What will multiplying  $A$  by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ c & 0 & 1 \end{pmatrix} \quad (\text{C.51})$$

do?

3. What will multiplying  $A$  by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & c & 1 \end{pmatrix} \quad (\text{C.52})$$

do?

### C. Linear Algebra

4. More generally: If you wanted to multiply row  $j$  of an  $n \times n$  matrix by  $c$  and add it to row  $k$ , that is the same as multiplying by what matrix?

---

**Exercise C.20.** After doing all of the matrix products,  $L_3 L_2 L_1 A$ , the resulting matrix will have zeros in the entire lower triangle. That is, all of the non-zero entries of the resulting matrix will be on the main diagonal or above. We call this matrix  $U$ , for upper-triangular. Hence, we have formed a matrix

$$L_3 L_2 L_1 A = U \quad (\text{C.53})$$

and if we want to solve for  $A$  we would get

$$A = (\text{_____})^{-1} (\text{_____})^{-1} (\text{_____})^{-1} U \quad (\text{C.54})$$

(Take care that everything is in the right order in your answer.)

---

**Exercise C.21.** It would be nice, now, if the inverses of the  $L$  matrices were easy to find. Use `np.linalg.inv()` to directly compute the inverse of  $L_1$ ,  $L_2$ , and  $L_3$  for each of the example matrices. Then complete the statement: If  $L_k$  is an identity matrix with some non-zero  $c$  in row  $i$  and column  $j$  then  $L_k^{-1}$  is what matrix?

---

**Exercise C.22.** We started this discussion with  $A$  as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \quad (\text{C.55})$$

and we defined

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -7 & 0 & 1 \end{pmatrix}, \quad \text{and} \quad L_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}. \quad (\text{C.56})$$

Based on your answer to the previous exercises we know that

$$A = L_1^{-1} L_2^{-1} L_3^{-1} U. \quad (\text{C.57})$$

Explicitly write down the matrices  $L_1^{-1}$ ,  $L_2^{-1}$ , and  $L_3^{-1}$ .

Now explicitly find the product  $L_1^{-1} L_2^{-1} L_3^{-1}$  and call this product  $L$ . Verify that  $L$  itself is also a lower-triangular matrix with ones on the main diagonal. Moreover, take note of exactly the form of the matrix. The answer should be super surprising to you!!

---

Throughout all of the preceding exercises, our final result is that we have factored the matrix  $A$  into the product of a lower-triangular matrix and an upper-triangular matrix. Stop and think about that for a minute ... we just factored a matrix!

Let us return now to our discussion of solving the system of equations  $Ax = b$ . If  $A$  can be factored into  $A = LU$  then the system of equations can be rewritten as  $LUx = b$ . As we will see in the next subsection, solving systems of equations with triangular matrices is super fast and relatively simple! Hence, we have partially achieved our modified goal of reducing the row reduction into some simpler case.<sup>3</sup>

It remains to implement the  $LU$  decomposition (also called the  $LU$  factorization) in Python.

---

**Example C.7** (The  $LU$  Factorization). The following Python function takes a square matrix  $A$  and outputs the matrices  $L$  and  $U$  such that  $A = LU$ . The entire code is given to you. It will be up to you in the next exercise to pick apart every step of the function.

```
def myLU(A):
 n = A.shape[0] # get the dimension of the matrix A
 L = np.eye(n) # Build the identity part of L
 U = np.copy(A) # start the U matrix as a copy of A
 for j in range(0,n-1):
 for i in range(j+1,n):
 mult = U[i,j] / U[j,j]
 U[i, :] = U[i, :] - mult * U[j,:]
 L[i,j] = mult
 return L,U
```

---

<sup>3</sup>Take careful note here. We have actually just built a special case of the  $LU$  decomposition. Remember that in row reduction you are allowed to swap the order of the rows, but in our  $LU$  algorithm we do not have any row swaps. The version of  $LU$  with row swaps is called  $LU$  with partial pivoting. We will not build the full partial pivoting algorithm in this text but feel free to look it up. The wikipedia page is a decent place to start. What you will find is that there are indeed many different versions of the  $LU$  decomposition.

**Exercise C.23.** Go to Example C.7 and go through every iteration of every loop **by hand** starting with the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}. \quad (\text{C.58})$$

Give details of what happens at every step of the algorithm. I will get you started.

- $n=3$ ,  $L$  starts as an identity matrix of the correct size, and  $U$  starts as a copy of  $A$ .
- Start the outer loop:  $j=0$ : ( $j$  is the counter for the column)
  - Start the inner loop:  $i=1$ : ( $i$  is the counter for the row)
    - \* `mult = A[1,0] / A[0,0]` so `mult=4/1`.
    - \* `A[1, 1:3] = A[1, 1:3] - 4 * A[0,1:3]`. Translated, this states that columns 1 and 2 of matrix  $A$  took their original value minus 4 times the corresponding values in row 0.
    - \* `U[1, 1:3] = A[1, 1:3]`. Now we replace the locations in  $U$  with the updated information from our first step of row reduction.
    - \* `L[1,0]=4`. We now fill the  $L$  matrix with the proper value.
    - \* `U[1,0]=0`. Finally, we zero out the lower triangle piece of the  $U$  matrix which we have now taken care of.
  - $i=2$ :
    - \* ... keep going from here ...

---

**Exercise C.24.** Apply your new `myLU` code to other square matrices and verify that indeed  $A$  is the product of the resulting  $L$  and  $U$  matrices. You can produce a random matrix with `np.random.randn(n,n)` where  $n$  is the number of rows and columns of the matrix. For example, `np.random.randn(10,10)` will produce a random  $10 \times 10$  matrix with entries chosen from the normal distribution with centre 0 and standard deviation 1. Random matrices are just as good as any other when testing your algorithm.

---



### C.5.3. Solving Triangular Systems

We now know that row reduction is just a collection of sneaky matrix multiplications. In the previous exercises we saw that we can often turn our system of equations  $Ax = b$  into the system  $LUx = b$  where  $L$  is lower-triangular (with ones on the main diagonal) and  $U$  is upper-triangular. But why was this important?

Well, if  $LUx = b$  then we can rewrite our system of equations as two systems:

$$\text{An upper-triangular system: } Ux = y \quad (\text{C.59})$$

and

$$\text{A lower-triangular system: } Ly = b. \quad (\text{C.60})$$

In the following exercises we will devise algorithms for solving triangular systems. After we know how to work with triangular systems we will put all of the pieces together and show how to leverage the  $LU$  decomposition and the solution techniques for triangular systems to quickly and efficiently solve linear systems.

---

**Exercise C.25.** Outline a fast algorithm (without formal row reduction) for solving the lower-triangular system

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 7 & 2 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix}. \quad (\text{C.61})$$


---

**Exercise C.26.** As a convention we will always write our lower-triangular matrices with ones on the main diagonal. Generalize your steps from the previous exercise so that you have an algorithm for solving any lower-triangular system. The most natural algorithm that most people devise here is called **forward substitution**.

---

**Definition C.4** (Forward Substitution Algorithm (`lsolve`)). The general statement of the Forward Substitution Algorithm is:

*Solve  $Ly = b$  for  $y$ , where the matrix  $L$  is assumed to be lower-triangular with ones on the main diagonal.*

The code below gives a full implementation of the **Forward Substitution** algorithm (also called the `lsolve` algorithm).

```
def lsolve(L, b):
 # L is assumed to be a lower-triangular np.array
 n = b.size # what does this do?
 y = np.zeros(n) # what does this do?
 for i in range(n):
 # start the loop by assigning y to the value on the right
 y[i] = b[i]
 for j in range(i): # now adjust y
 y[i] = y[i] - L[i,j] * y[j]
 return(y)
```

---

**Exercise C.27.** Work with your partner(s) to apply the `lsolve()` code to the lower-triangular system

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 7 & 2 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \quad (\text{C.62})$$

**by hand.** It is incredibly important to implement numerical linear algebra routines by hand a few times so that you truly understand how everything is being tracked and calculated.

I will get you started.

- Start: `i=0`:
  - `y[0]=1` since `b[0]=1`.
  - The next `for` loop does not start since `range(0)` has no elements (stop and think about why this is).
- Next step in the loop: `i=1`:
  - `y[1]` is initialized as 0 since `b[1]=0`.
  - Now we enter the inner loop at `j=0`:
    - \* What does `y[1]` become when `j=0`?
  - Does `j` increment to anything larger?
- Finally we increment `i` to `i=2`:
  - What does `y[2]` get initialized to?

- Enter the inner loop at  $j=0$ :
    - \* What does  $y[2]$  become when  $j=0$ ?
  - Increment the inner loop to  $j=1$ :
    - \* What does  $y[2]$  become when  $j=1$ ?
  - Stop
- 

**Exercise C.28.** Copy the code from Definition C.4 into a Python function but in your code write a comment on every line stating what it is doing. Write a test script that creates a lower-triangular matrix of the correct form and a right-hand side  $b$  and solve for  $y$ . Test your code by giving it a large lower-triangular system.

---

Now that we have a method for solving lower-triangular systems, let us build a similar method for solving upper-triangular systems. The merging of lower and upper-triangular systems will play an important role in solving systems of equations.

---

**Exercise C.29.** Outline a fast algorithm (without formal row reduction) for solving the upper-triangular system

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & -9 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -4 \\ 3 \end{pmatrix} \quad (\text{C.63})$$

The most natural algorithm that most people devise here is called **backward substitution**. Notice that in our upper-triangular matrix we do not have a diagonal containing all ones.

---

**Exercise C.30.** Generalize your backward substitution algorithm from the previous problem so that it could be applied to any upper-triangular system.

---

**Definition C.5** (Backward Substitution Algorithm). The following code solves the problem  $Ux = y$  using backward substitution. The matrix  $U$  is assumed to be upper-triangular. You will notice that most of this code is incomplete. It is your job to complete this code, and the next exercise should help.

```
def usolve(U, y):
 # U is assumed to be an upper-triangular np.array
 n = y.size
 x = np.zeros(n)
 for i in range(???): # what should we be looping over?
 x[i] = y[i] / ??? # what should we be dividing by?
 for j in range(???): # what should we be looping over:
 x[i] = x[i] - U[i,j] * x[j] / ??? # complete this line
 # ... what does the previous line do?
 return(x)
```

---

**Exercise C.31.** Now we will work through the backward substitution algorithm to help fill in the blanks in the code. Consider the upper-triangular system

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & -9 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -4 \\ 3 \end{pmatrix} \quad (\text{C.64})$$

Work the code from Definition C.5 to solve the system. Keep track of all of the indices as you work through the code. You may want to work this problem in conjunction with the previous two problems to unpack all of the parts of the *backward substitution* algorithm.

I will get you started.

- In your backward substitution algorithm you should have started with the last row, therefore the outer loop starts at  $n-1$  and reads backward to 0. (Why are we starting at  $n-1$  and not  $n$ ?)
- Outer loop:  $i=2$ :
  - We want to solve the equation  $-9x_2 = 3$  so the clear solution is to divide by  $-9$ . In code this means that  $x[2]=y[2]/U[2,2]$ .
  - There is nothing else to do for row 3 of the matrix, so we should not enter the inner loop. How can we keep from entering the inner loop?
- Outer loop:  $i=1$ :

- Now we are solving the algebraic equation  $-3x_1 - 6x_2 = -4$ . If we follow the high school algebra we see that  $x_1 = \frac{-4 - (-6)x_2}{-3}$  but this can be rearranged to

$$x_1 = \frac{-4}{-3} - \frac{-6x_2}{-3}. \quad (\text{C.65})$$

So we can initialize  $x_1$  with  $x_1 = \frac{-4}{-3}$ . In code, this means that we initialize with `x[1] = y[1] / U[1,1]`.

- Now we need to enter the inner loop at `j=2`: (why are we entering the loop at `j=2`?)
  - \* To complete the algebra we need to take our initialized value of `x[1]` and subtract off  $\frac{-6x_2}{-3}$ . In code this is `x[1] = x[1] - U[1,2] * x[2] / U[1,1]`
- There is nothing else to do so the inner loop should end.
- Outer loop: `i=0`:
  - Finally, we are solving the algebraic equation  $x_0 + 2x_1 + 3x_2 = 1$  for  $x_0$ . The clear and obvious solution is  $x_0 = \frac{1 - 2x_1 - 3x_2}{1}$  (why am I explicitly showing the division by 1 here?).
  - Initialize  $x_0$  at `x[0] = ???`
  - Enter the inner loop at `j=2`:
    - \* Adjust the value of `x[0]` by subtracting off  $\frac{3x_2}{1}$ . In code we have `x[0] = x[0] - ??? * ??? / ???`
  - Increment `j` to `j=1`:
    - \* Adjust the value of `x[0]` by subtracting off  $\frac{2x_1}{1}$ . In code we have `x[0] = x[0] - ??? * ??? / ???`
- Stop.
- You should now have a solution to the equation  $Ux = y$ . Substitute your solution in and verify that your solution is correct.

---

**Exercise C.32.** Copy the code from Definition C.5 into a Python function but in your code write a comment on every line stating what it is doing. Write a test script that creates an upper-triangular matrix of the correct form and a right-hand side  $y$  and solve for  $x$ . Your code needs to work on systems of arbitrarily large size.

---

### C.5.4. Solving Systems with LU Decomposition

We are finally ready for the punch line of this whole  $LU$  and triangular systems business!

---

**Exercise C.33.** If we want to solve  $Ax = b$  then

1. If we can, write the system of equations as  $LUx = b$ .
2. Solve  $Ly = b$  for  $y$  using forward substitution.
3. Solve  $Ux = y$  for  $x$  using backward substitution.

Pick a matrix  $A$  and a right-hand side  $b$  and solve the system using this process.

---

**Exercise C.34.** Try the process again on the  $3 \times 3$  system of equations

$$\begin{pmatrix} 3 & 6 & 8 \\ 2 & 7 & -1 \\ 5 & 2 & 2 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -13 \\ 4 \\ 1 \end{pmatrix} \quad (\text{C.66})$$

That is: Find matrices  $L$  and  $U$  such that  $Ax = b$  can be written as  $LUx = b$ . Then do two triangular solves to determine  $x$ .

---

Let us take stock of what we have done so far.

- Solving lower-triangular systems is super fast and easy!
- Solving upper-triangular systems is super fast and easy (so long as we never divide by zero).
- It is often possible to rewrite the matrix  $A$  as the product of a lower-triangular matrix  $L$  and an upper-triangular matrix  $U$  so  $A = LU$ .
- Now we can re-frame the equation  $Ax = b$  as  $LUx = b$ .
- Substitute  $y = Ux$  so the system becomes  $Ly = b$ . Solve for  $y$  with forward substitution.
- Now solve  $Ux = y$  using backward substitution.

We have successfully take row reduction and turned into some fast matrix multiplications and then two very quick triangular solves. Ultimately this will be a faster algorithm for solving a system of linear equations.

---

**Definition C.6** (Solving Linear Systems with the LU Decomposition). Let  $A$  be a square matrix in  $\mathbb{R}^{n \times n}$  and let  $x, b \in \mathbb{R}^n$ . To solve the problem  $Ax = b$ ,

1. Factor  $A$  into lower and upper-triangular matrices  $A = LU$ .  
 $L, U = \text{myLU}(A)$
  2. The system can now be written as  $LUx = b$ . Substitute  $Ux = y$  and solve the system  $Ly = b$  with forward substitution.  $y = \text{lsolve}(L, b)$
  3. Finally, solve the system  $Ux = y$  with backward substitution.  
 $x = \text{usolve}(U, y)$
- 

**Exercise C.35.** The  $LU$  decomposition is not perfect. Discuss where the algorithm will fail.

---

**Exercise C.36.** What happens when you try to solve the system of equations

$$\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 7 \\ 9 \\ -3 \end{pmatrix} \quad (\text{C.67})$$

with the  $LU$  decomposition algorithm? Discuss.

## C.6. The QR Factorization

In this section we will try to find an improvement on the  $LU$  factorization scheme from the previous section. What we will do here is leverage the geometry of the column space of the  $A$  matrix instead of leveraging the row reduction process.

---

**Exercise C.37.** We want to solve the system of equations

$$\begin{pmatrix} 1/3 & 2/3 & 2/3 \\ 2/3 & 1/3 & -2/3 \\ -2/3 & 2/3 & -1/3 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 6 \\ 12 \\ -9 \end{pmatrix}. \quad (\text{C.68})$$

1. We could do row reduction by hand ... yuck ... do not do this.
2. We could apply our new-found skills with the  $LU$  decomposition to solve the system, so go ahead and do that with your Python code.
3. What do you get if you compute the product  $A^T A$ ?
  - a. Why do you get what you get? In other words, what was special about  $A$  that gave such a nice result?
  - b. What does this mean about the matrices  $A$  and  $A^T$ ?
4. Now let us leverage what we found in part (3) to solve the system of equations  $Ax = b$  much faster. Multiply both sides of the matrix equation by  $A^T$ , and now you should be able to just read off the solution. This seems amazing!!
5. What was it about this particular problem that made part (4) so elegant and easy?

---

The previous exercise tells us something amazing:

**Theorem C.1** (Orthonormal Matrices). *If  $A$  is an orthonormal matrix where the columns are mutually orthogonal and every column is a unit vector, then  $A^T = A^{-1}$  and to solve the system of equation  $Ax = b$  we simply need to multiply both sides of the equation by  $A^T$ . Hence, the solution to  $Ax = b$  is just  $x = A^T b$  in this special case.*

---

Theorem C.1 begs an obvious question: *Is there a way to turn any matrix  $A$  into an orthogonal matrix so that we can solve  $Ax = b$  in this same very efficient and fast way?*

The answer: Yes. Kind of.

In essence, if we can factor our coefficient matrix into an orthonormal matrix and some other nicely formatted matrix (like a triangular matrix, perhaps) then the job of solving the linear system of equations comes down to matrix multiplication and a quick triangular solve – both of which are extremely fast!

What we will study in this section is a new matrix factorization called the  $QR$  factorization whose goal is to convert the matrix  $A$  into a product of two matrices,  $Q$  and  $R$ , where  $Q$  is orthonormal and  $R$  is upper-triangular.



---

**Exercise C.38.** Let us say that we have a matrix  $A$  and we know that it can be factored into  $A = QR$  where  $Q$  is an orthonormal matrix and  $R$  is an upper-triangular matrix. How would we then leverage this factorization to solve the system of equation  $Ax = b$  for  $x$ ?

---

Before proceeding to the algorithm for the  $QR$  factorization let us pause for a moment and review scalar and vector projections from Linear Algebra. In Figure C.1 we see a graphical depiction of the vector  $u$  projected onto vector  $v$ . Notice that the projection is indeed the perpendicular projection as this is what seems natural geometrically.

The **vector projection** of  $u$  onto  $v$  is the vector  $cv$ . That is, the vector projection of  $u$  onto  $v$  is a scalar multiple of the vector  $v$ . The value of the scalar  $c$  is called the **scalar projection** of  $u$  onto  $v$ .

Figure 4.1: Projection of one vector onto another.

We can arrive at a formula for the scalar projection rather easily if we consider that the vector  $w$  in Figure C.1 must be perpendicular to  $cv$ . Hence

$$w \cdot (cv) = 0. \quad (\text{C.69})$$

From vector geometry we also know that  $w = u - cv$ . Therefore

$$(u - cv) \cdot (cv) = 0. \quad (\text{C.70})$$

If we distribute we can see that

$$cu \cdot v - c^2 v \cdot v = 0 \quad (\text{C.71})$$

and therefore either  $c = 0$ , which is only true if  $u \perp v$ , or

$$c = \frac{u \cdot v}{v \cdot v} = \frac{u \cdot v}{\|v\|^2}. \quad (\text{C.72})$$

Therefore,

- the **scalar projection** of  $u$  onto  $v$  is

$$c = \frac{u \cdot v}{\|v\|^2} \quad (\text{C.73})$$

- the **vector projection** of  $u$  onto  $v$  is

$$cv = \left( \frac{u \cdot v}{\|v\|^2} \right) v \quad (\text{C.74})$$

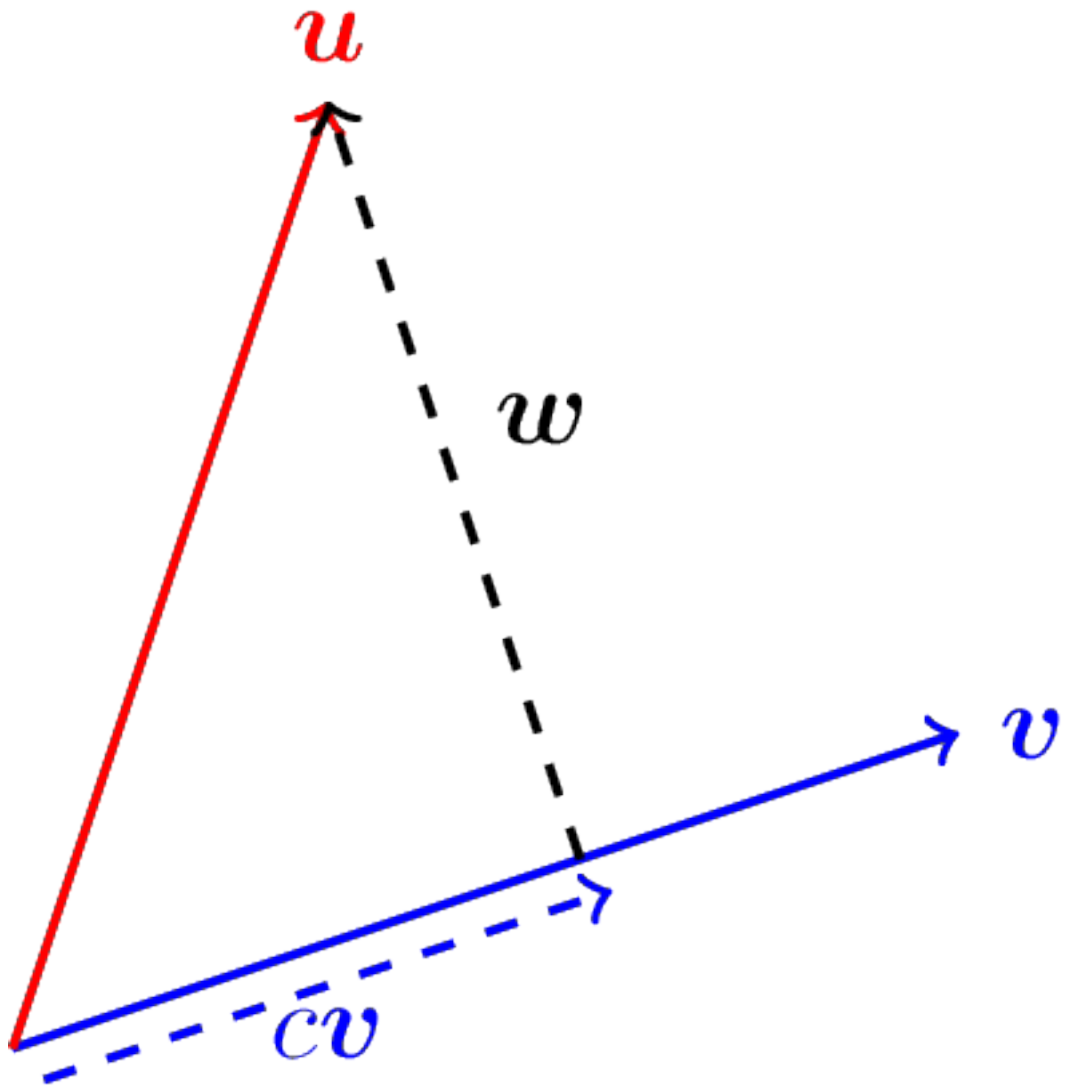


Figure C.1.: Projection of one vector onto another.

Another problem related to scalar and vector projections is to take a basis for the column space of a matrix and transform that basis into an orthogonal (or orthonormal) basis. Indeed, in Figure C.1 if we have the matrix

$$A = \begin{pmatrix} | & | \\ u & v \\ | & | \end{pmatrix} \quad (\text{C.75})$$

it should be clear from the picture that the columns of this matrix are not perpendicular. However, if we take the vector  $v$  and the vector  $w$  we do arrive at two orthogonal vector that form a basis for the same space. Moreover, if we normalize these vectors (by dividing by their respective lengths) then we can easily transform the original basis for the column space of  $A$  into an orthonormal basis. This process is called the Gram-Schmidt process, and you have encountered it in your Linear Algebra module.

Now we return to our goal of finding a way to factor a matrix  $A$  into an orthonormal matrix  $Q$  and an upper-triangular matrix  $R$ . The algorithm that we are about to build depends greatly on the ideas of scalar and vector projections.

---

**Exercise C.39.** We want to build a  $QR$  factorization of the matrix  $A$  in the matrix equation  $Ax = b$  so that we can leverage the fact that solving the equation  $QRx = b$  is easy. Consider the matrix  $A$  defined as

$$A = \begin{pmatrix} 3 & 1 \\ 4 & 1 \end{pmatrix}. \quad (\text{C.76})$$

Notice that the columns of  $A$  are NOT orthonormal (they are not unit vectors and they are not perpendicular to each other).

1. Draw a picture of the two column vectors of  $A$  in  $\mathbb{R}^2$ . we will use this picture to build geometric intuition for the rest of the  $QR$  factorization process.
2. Define  $a_0$  as the first column of  $A$  and  $a_1$  as the second column of  $A$ . That is

$$a_0 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad \text{and} \quad a_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \quad (\text{C.77})$$

Turn  $a_0$  into a unit vector and call this unit vector  $q_0$

$$q_0 = \frac{a_0}{\|a_0\|} = \begin{pmatrix} \rule{1cm}{0.4pt} \\ \rule{1cm}{0.4pt} \end{pmatrix}. \quad (\text{C.78})$$

This vector  $q_0$  will be the first column of the  $2 \times 2$  matrix  $Q$ . Why is this a nice place to start building the  $Q$  matrix (think about the desired structure of  $Q$ )?

### C. Linear Algebra

3. In your picture of  $a_0$  and  $a_1$  mark where  $q_0$  is. Then draw the orthogonal projection from  $a_1$  onto  $q_0$ . In your picture you should now see a right triangle with  $a_1$  on the hypotenuse, the projection of  $a_1$  onto  $q_0$  on one leg, and the second leg is the vector difference of the hypotenuse and the first leg. Simplify the projection formula for leg 1 and write the formula for leg 2.

$$\text{hypotenuse} = a_1 \quad (\text{C.79})$$

$$\text{leg 1} = \left( \frac{a_1 \cdot q_0}{q_0 \cdot q_0} \right) q_0 = \underline{\hspace{2cm}} \quad (\text{C.80})$$

$$\text{leg 2} = \underline{\hspace{2cm}} - \underline{\hspace{2cm}}. \quad (\text{C.81})$$

4. Compute the vector for leg 2 and then normalize it to turn it into a unit vector. Call this vector  $q_1$  and put it in the second column of  $Q$ .
5. Verify that the columns of  $Q$  are now orthogonal and are both unit vectors.
6. The matrix  $R$  is supposed to complete the matrix factorization  $A = QR$ . We have built  $Q$  as an orthonormal matrix. How can we use this fact to solve for the matrix  $R$ ?
7. You should now have an orthonormal matrix  $Q$  and an upper-triangular matrix  $R$ . Verify that  $A = QR$ .
8. An alternate way to build the  $R$  matrix is to observe that

$$R = \begin{pmatrix} a_0 \cdot q_0 & a_1 \cdot q_0 \\ 0 & a_1 \cdot q_1 \end{pmatrix}. \quad (\text{C.82})$$

Show that this is indeed true for the matrix  $A$  from this problem.

---

**Exercise C.40.** Keeping track of all of the arithmetic in the  $QR$  factorization process is quite challenging, so let us leverage Python to do some of the work for us. The following block of code walks through the previous exercise without any looping (that way we can see every step transparently). Some of the code is missing so you will need to fill it in.

```

import numpy as np
Define the matrix A
A = np.array([[3,1],[4,1]])
n = A.shape[0]
Build the vectors a0 and a1
a0 = A[??? , ???] # ... write code to get column 0 from A
a1 = A[??? , ???] # ... write code to get column 1 from A
Set up storage for Q
Q = np.zeros((n,n))
build the vector q0 by normalizing a0
q0 = a0 / np.linalg.norm(a0)
Put q0 as the first column of Q
Q[:,0] = q0
Calculate the lengths of the two legs of the triangle
leg1 = # write code to get the vector for leg 1 of the triangle
leg2 = # write code to get the vector for leg 2 of the triangle
normalize leg2 and call it q1
q1 = # write code to normalize leg2
q1 = q1 / np.linalg.norm(q1) # Just to be safe with normalization if not implied
Q[:,1] = q1 # What does this line do?
R = # ... build the R matrix out of A and Q

print("The Q matrix is \n",Q,"\n")
print("The R matrix is \n",R,"\n")
print("The A matrix is \n",A,"\n")
print("The product QR is\n",Q @ R)

```

---

**Exercise C.41.** You should notice that the code in the previous exercise does not depend on the specific matrix  $A$  that we used? Put in a different  $2 \times 2$  matrix and verify that the process still works. That is, verify that  $Q$  is orthonormal,  $R$  is upper-triangular, and  $A = QR$ . Be sure, however, that your matrix  $A$  is full rank.

---

**Exercise C.42.** Draw two generic vectors in  $\mathbb{R}^2$  and demonstrate the process outlined in the previous problem to build the vectors for the  $Q$  matrix starting from your generic vectors.

---

**Exercise C.43.** Now we will extend the process from the previous exercises to three dimensions. This time we will seek a matrix  $Q$  that has three orthonormal vectors starting from the three original columns of a  $3 \times 3$  matrix  $A$ . Perform each of the following steps **by hand** on the matrix

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}. \quad (\text{C.83})$$

In the end you should end up with an orthonormal matrix  $Q$  and an upper-triangular matrix  $R$ .

- **Step 1:** Pick column  $a_0$  from the matrix  $A$  and normalize it. Call this new vector  $q_0$  and make that the first column of the matrix  $Q$ .
- **Step 2:** Project column  $a_1$  of  $A$  onto  $q_0$ . This forms a right triangle with  $a_1$  as the hypotenuse, the projection of  $a_1$  onto  $q_0$  as one of the legs, and the vector difference between these two as the second leg. Notice that the second leg of the newly formed right triangle is perpendicular to  $q_0$  by design. If we normalize this vector then we have the second column of  $Q$ ,  $q_1$ .
- **Step 3:** Now we need a vector that is perpendicular to both  $q_0$  AND  $q_1$ . To achieve this we are going to project column  $a_2$  from  $A$  onto the plane formed by  $q_0$  and  $q_1$ . we will do this in two steps:
  - **Step 3a:** We first project  $a_2$  down onto both  $q_0$  and  $q_1$ .
  - **Step 3b:** The vector that is perpendicular to both  $q_0$  and  $q_1$  will be the difference between  $a_2$  the projection of  $a_2$  onto  $q_0$  and the projection of  $a_2$  onto  $q_1$ . That is, we form the vector  $w = a_2 - (a_2 \cdot q_0)q_0 - (a_2 \cdot q_1)q_1$ . Normalizing this vector will give us  $q_2$ . (Stop now and prove that  $q_2$  is indeed perpendicular to both  $q_1$  and  $q_0$ .)

The result should be the matrix  $Q$  which contains orthonormal columns. To build the matrix  $R$  we simply recall that  $A = QR$  and  $Q^{-1} = Q^T$  so  $R = Q^T A$ .

---

**Exercise C.44.** Repeat the previous exercise but write code for each step so that Python can handle all of the computations. Again use the matrix

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}. \quad (\text{C.84})$$


---

**Example C.8. (QR for  $n = 3$ )** For the sake of clarity let us now write down the full  $QR$  factorization for a  $3 \times 3$  matrix.

If the columns of  $A$  are  $a_0$ ,  $a_1$ , and  $a_2$  then

$$q_0 = \frac{a_0}{\|a_0\|} \quad (\text{C.85})$$

$$q_1 = \frac{a_1 - (a_1 \cdot q_0) q_0}{\|a_1 - (a_1 \cdot q_0) q_0\|} \quad (\text{C.86})$$

$$q_2 = \frac{a_2 - (a_2 \cdot q_0) q_0 - (a_2 \cdot q_1) q_1}{\|a_2 - (a_2 \cdot q_0) q_0 - (a_2 \cdot q_1) q_1\|} \quad (\text{C.87})$$

and

$$R = \begin{pmatrix} a_0 \cdot q_0 & a_1 \cdot q_0 & a_2 \cdot q_0 \\ 0 & a_1 \cdot q_1 & a_2 \cdot q_1 \\ 0 & 0 & a_2 \cdot q_2 \end{pmatrix} \quad (\text{C.88})$$

**Exercise C.45** (The QR Factorization). Now we are ready to build general code for the  $QR$  factorization. The following Python function definition is partially complete. Fill in the missing pieces of code and then test your code on square matrices of many different sizes. The easiest way to check if you have an error is to find the normed difference between  $A$  and  $QR$  with `np.linalg.norm(A - Q*R)`.

```
import numpy as np
def myQR(A):
 n = A.shape[0]
 Q = np.zeros((n,n))
 for j in range(???): # The outer loop goes over the columns
 q = A[:,j]
 # The next loop is meant to do all of the projections.
 # When do you start the inner loop and how far do you go?
 # Hint: You do not need to enter this loop the first time
 for i in range(???):
 length_of_leg = np.dot(A[:,j], Q[:,i])
 q = q - ??? * ??? # This is where we do projections
 Q[:,j] = q / np.linalg.norm(q)
 R = # finally build the R matrix
 return Q, R
```

```
Test Code
A = np.array(...)
or you can build A with use np.random.randn()
Often time random matrices are good test cases
Q, R = myQR(A)
error = np.linalg.norm(A - Q @ R)
print(error)
```

---

We now have a robust algorithm for doing  $QR$  factorization of square matrices we can finally return to solving systems of equations.

**Theorem C.2. (*Solving Systems with QR*)** Remember that we want to solve  $Ax = b$  and since  $A = QR$  we can rewrite it with  $QRx = b$ . Since we know that  $Q$  is orthonormal by design we can multiply both sides of the equation by  $Q^T$  to get  $Rx = Q^T b$ . Finally, since  $R$  is upper-triangular we can use our `usolve` code from the previous section to solve the resulting triangular system.

---

**Exercise C.46.** Solve the system of equations

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \quad (\text{C.89})$$

by first computing the  $QR$  factorization of  $A$  and then solving the resulting upper-triangular system.

---

**Exercise C.47.** Write code that builds a random  $n \times n$  matrix and a random  $n \times 1$  vector. Solve the equation  $Ax = b$  using the  $QR$  factorization and compare the answer to what we find from `np.linalg.solve()`. Do this many times for various values of  $n$  and create a plot with  $n$  on the horizontal axis and the normed error between Python's answer and your answer from the  $QR$  algorithm on the vertical axis. It would be wise to use a `plt.semilogy()` plot. To find the normed difference you should use `np.linalg.norm()`. What do you notice?



## C.7. Over-determined Systems and Curve Fitting

**Exercise C.48.** Consider the problem of finding the quadratic function  $f(x) = ax^2 + bx + c$  that *best fits* the points

$$(0, 1.07), (1, 3.9), (2, 14.8), (3, 26.8). \quad (\text{C.90})$$

We do not know the values of  $a$ ,  $b$ , or  $c$  but we do have four different  $(x, y)$  ordered pairs. Hence, we have four equations:

$$1.07 = a(0)^2 + b(0) + c \quad (\text{C.91})$$

$$3.9 = a(1)^2 + b(1) + c \quad (\text{C.92})$$

$$14.8 = a(2)^2 + b(2) + c \quad (\text{C.93})$$

$$26.8 = a(3)^2 + b(3) + c. \quad (\text{C.94})$$

There are four equations and only three unknowns. This is what is called an **over determined systems** – when there are more equations than unknowns. Let us play with this problem.

1. First turn the system of equations into a matrix equation.

$$\begin{pmatrix} 0 & 0 & 1 \\ \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 1.07 \\ 3.9 \\ 14.8 \\ 26.8 \end{pmatrix}. \quad (\text{C.95})$$

2. None of our techniques for solving systems will likely work here since it is highly unlikely that the vector on the right-hand side of the equation is in the column space of the coefficient matrix. Discuss this.
3. One solution to the unfortunate fact from part (b) is that we can project the vector on the right-hand side into the subspace spanned by the columns of the coefficient matrix. Think of this as casting the shadow of the right-hand vector down onto the space spanned by the columns. If we do this projection we will be able to solve the equation for the values of  $a$ ,  $b$ , and  $c$  that will create the projection exactly – and hence be as close as we can get to the actual right-hand side. Draw a picture of what we have said here.
4. Now we need to project the right-hand side, call it  $b$ , onto the column space of the the coefficient matrix  $A$ . Recall the following facts:

### C. Linear Algebra

- Projections are dot products
  - Matrix multiplication is nothing but a bunch of dot products.
  - The projections of  $b$  onto the columns of  $A$  are the dot products of  $b$  with each of the columns of  $A$ .
  - What matrix can we multiply both sides of the equation  $Ax = b$  by in order for the right-hand side to become the projection that we want? (Now do the projection in Python)
5. If you have done part (d) correctly then you should now have a square system (i.e. the matrix on the left-hand side should now be square). Solve this system for  $a$ ,  $b$ , and  $c$ .

---

**Theorem C.3** (Solving Overdetermined Systems). *If  $Ax = b$  is an overdetermined system (i.e.  $A$  has more rows than columns) then we first multiply both sides of the equation by  $A^T$  (why do we do this?) and then solve the square system of equations  $(A^T A)x = A^T b$  using a system solving like LU or QR. The answer to this new system is interpreted as the vector  $x$  which solves exactly for the projection of  $b$  onto the column space of  $A$ .*

*The equation  $(A^T A)x = A^T b$  is called **the normal equations** and arises often in Statistics and Machine Learning.*

---

**Exercise C.49.** Fit a linear function to the following data. Solve for the slope and intercept using the technique outlined in Theorem C.3. Make a plot of the points along with your best fit curve.

$x$	$y$
0	4.6
1	11
2	12
3	19.1
4	18.8
5	39.5
6	31.1
7	43.4
8	40.3
9	41.5

### C.7. Over-determined Systems and Curve Fitting

---

$x$	$y$
10	41.6

---

---

**Exercise C.50.** Fit a quadratic function to the following data using the technique outlined in Theorem C.3. Make a plot of the points along with your best fit curve.

---

$x$	$y$
0	-6.8
1	11.8
2	50.6
3	94
4	224.3
5	301.7
6	499.2
7	454.7
8	578.5
9	1102
10	1203.2

---

Code to download the data directly is given below.

```
import numpy as np
import pandas as pd
URL1 = 'https://raw.githubusercontent.com/NumericalMethodsSullivan'
URL2 = '/NumericalMethodsSullivan.github.io/master/data/'
URL = URL1+URL2
data = np.array(pd.read_csv(URL+'Exercise4_52.csv'))
Exercise4_52.csv
```

---

**Exercise C.51.** The Statistical technique of curve fitting is often called “linear regression.” This even holds when we are fitting quadratic functions, cubic functions, etc to the data ... we still call that linear regression! Why?

---

This section of the text on solving over determined systems is just a bit of a teaser for a bit of higher-level statistics, data science, and machine learning. The normal equations and solving systems via projections is the starting point of many modern machine learning algorithms.

---

## C.8. The Eigenvalue-Eigenvector Problem

We finally turn our attention to another major topic in numerical linear algebra.<sup>4</sup>

**Definition C.7** (The Eigenvalue Problem). Recall that the eigenvectors,  $x$ , and the eigenvalues,  $\lambda$  of a square matrix satisfy the equation  $Ax = \lambda x$ . Geometrically, the eigen-problem is the task of finding the special vectors  $x$  such that multiplication by the matrix  $A$  only produces a scalar multiple of  $x$ .

---

Thinking about matrix multiplication, the geometric notion of the eigenvalue problem is rather peculiar since matrix-vector multiplication usually results in a scaling and a rotation of the vector  $x$ . Therefore, in some sense the eigenvectors are the only special vectors which avoid geometric rotation under matrix multiplication. For a graphical exploration of this idea see:

<https://www.geogebra.org/m/JP2XZpzV>.

---

Recall that to solve the eigen-problem for a square matrix  $A$  we complete the following steps:

1. First rearrange the definition of the eigenvalue-eigenvector pair to

$$(Ax - \lambda x) = 0. \tag{C.96}$$

2. Next, factor the  $x$  on the right to get

$$(A - \lambda I)x = 0. \tag{C.97}$$

---

<sup>4</sup>Numerical Linear Algebra is a huge field and there is way more to say ... but alas, this is an introductory course in Numerical Analysis so we cannot do everything. Sigh.

### C.8. The Eigenvalue-Eigenvector Problem

3. Now observe that since  $x \neq 0$  the matrix  $A - \lambda I$  must NOT have an inverse. Therefore,

$$\det(A - \lambda I) = 0. \quad (\text{C.98})$$

4. Solve the equation  $\det(A - \lambda I) = 0$  for all of the values of  $\lambda$ .
5. For each  $\lambda$ , find a solution to the equation  $(A - \lambda I)x = 0$ . Note that there will be infinitely many solutions so you will need to make wise choices for the free variables.
- 

**Exercise C.52.** Find the eigenvalues and eigenvectors of

$$A = \begin{pmatrix} 1 & 2 \\ 4 & 3 \end{pmatrix}. \quad (\text{C.99})$$

---

**Exercise C.53.** In the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad (\text{C.100})$$

one of the eigenvalues is  $\lambda_1 = 0$ .

1. What does that tell us about the matrix  $A$ ?
  2. What is the eigenvector  $v_1$  associated with  $\lambda_1 = 0$ ?
  3. What is the null space of the matrix  $A$ ?
- 

OK. Now that you have recalled some of the basics, let us play with a little limit problem. The following exercises are going to work us toward the **power method** for finding certain eigen-structures of a matrix.

---

**Exercise C.54.** Consider the matrix

$$A = \begin{pmatrix} 8 & 5 & -6 \\ -12 & -9 & 12 \\ -3 & -3 & 5 \end{pmatrix}. \quad (\text{C.101})$$

This matrix has the following eigen-structure:

$$v_1 = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} \quad \text{with} \quad \lambda_1 = 3 \quad (\text{C.102})$$

$$v_2 = \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} \quad \text{with} \quad \lambda_2 = 2 \quad (\text{C.103})$$

$$v_3 = \begin{pmatrix} -1 \\ 3 \\ 1 \end{pmatrix} \quad \text{with} \quad \lambda_3 = -1 \quad (\text{C.104})$$

If we have

$$x = -2v_1 + 1v_2 - 3v_3 = \begin{pmatrix} 3 \\ -7 \\ -1 \end{pmatrix} \quad (\text{C.105})$$

then we want to do a bit of an experiment. What happens when we iteratively multiply  $x$  by  $A$  but at the same time divide by the largest eigenvalue. Let us see:

- What is  $A^1x/3^1$ ?
- What is  $A^2x/3^2$ ?
- What is  $A^3x/3^3$ ?
- What is  $A^4x/3^4$ ?
- ...

It might be nice now to go to some Python code to do the computations (if you have not already). Use your code to conjecture about the following limit.

$$\lim_{k \rightarrow \infty} \frac{A^k x}{\lambda_{max}^k} = ??? \quad (\text{C.106})$$

In this limit we are really interested in the direction of the resulting vector, not the magnitude. Therefore, in the code below you will see that we normalize the resulting vector so that it is a unit vector.

Note: be careful, computers do not do infinity, so for powers that are too large you will not get any results.

```

import numpy as np
A = np.array([[8,5,-6],[-12,-9,12],[-3,-3,5]])
x = np.array([[3],[-7],[-1]])
eigval_max = 3

k = 4
Note: For numpy arrays, A**k is element-wise power.
We must use np.linalg.matrix_power for matrix power.
result = np.linalg.matrix_power(A, k) @ x / eigval_max**k
print(result / np.linalg.norm(result))

```

---

**Exercise C.55.** If a matrix  $A$  has eigenvectors  $v_1, v_2, v_3, \dots, v_n$  with eigenvalues  $\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_n$  and  $x$  is in the column space of  $A$  then what will we get, approximately, if we evaluate  $A^k x / \max_j (\lambda_j)^k$  for very large values of  $k$ ?

Discuss your conjecture with your peers. Then try to verify it with several numerical examples.

---

**Exercise C.56.** Explain your result from the previous exercise geometrically.

---

**Exercise C.57.** The algorithm that we have been toying with will find the dominant eigenvector of a matrix fairly quickly. Why might you be only interested in the dominant eigenvector of a matrix? Discuss.

---

**Exercise C.58.** In this problem we will formally prove the conjecture that you just made. This conjecture will lead us to the **power method** for finding the dominant eigenvector and eigenvalue of a matrix.

1. Assume that  $A$  has  $n$  linearly independent eigenvectors  $v_1, v_2, \dots, v_n$  and choose  $x = \sum_{j=1}^n c_j v_j$ . You have proved in the past that

$$A^k x = c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \dots + c_n \lambda_n^k v_n. \quad (\text{C.107})$$

Stop and sketch out the details of this proof now.

### C. Linear Algebra

2. If we factor  $\lambda_1^k$  out of the right-hand side we get

$$A^k x = \lambda_1^k \left( c_1 \frac{v_1}{\lambda_1} + c_2 \left( \frac{v_2}{\lambda_2} \right)^k v_2 + c_3 \left( \frac{v_3}{\lambda_3} \right)^k v_3 + \dots + c_n \left( \frac{v_n}{\lambda_n} \right)^k v_n \right) \quad (\text{C.108})$$

(fill in the question marks)

3. If  $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$  then what happens to each of the  $(\lambda_j/\lambda_1)^k$  terms as  $k \rightarrow \infty$ ?
4. Using your answer to part (c), what is  $\lim_{k \rightarrow \infty} A^k x / \lambda_1^k$ ?

**Theorem C.4** (The Power Method). *The following algorithm, called **the power method** will quickly find the eigenvalue of largest absolute value for a square matrix  $A \in \mathbb{R}^{n \times n}$  as well as the associated (normalized) eigenvector. We are assuming that there are  $n$  linearly independent eigenvectors of  $A$ .*

**Step 1:** *Given a non-zero vector  $x$ , set  $v^{(1)} = x/\|x\|$ . (Here the superscript indicates the iteration number) Note that the initial vector  $x$  is pretty irrelevant to the process so it can just be a random vector of the correct size..*

**Step 2:** *For  $k = 2, 3, \dots$*

**Step 2a:** *Compute  $\tilde{v}^{(k)} = Av^{(k-1)}$  (this gives a non-normalized version of the next estimate of the dominant eigenvector.)*

**Step 2b:** *Set  $\lambda^{(k)} = \tilde{v}^{(k)} \cdot v^{(k-1)}$ . (this gives an approximation of the eigenvalue since if  $v^{(k-1)}$  was the actual eigenvector we would have  $\lambda = Av^{(k-1)} \cdot v^{(k-1)}$ . Stop now and explain this.)*

**Step 2c:** *Normalize  $\tilde{v}^{(k)}$  by computing  $v^{(k)} = \tilde{v}^{(k)} / \|\tilde{v}^{(k)}\|$ . (This guarantees that you will be sending a unit vector into the next iteration of the loop)*

**Exercise C.59.** Go through Theorem C.4 carefully and describe what we need to do in each step and why we are doing it. Then complete all of the missing pieces of the following Python function.



```

import numpy as np
def myPower(A, tol = 1e-8):
 n = A.shape[0]
 x = np.random.randn(n)
 x = # turn x into a unit vector
 # we do not actually need to keep track of the old iterates
 L = 1 # initialize the dominant eigenvalue
 counter = 0 # keep track of how many steps we have taken
 # You can build a stopping rule from the definition
 # Ax = lambda x ...
 while (???) > tol and counter < 10000:
 x = A @ x # update the dominant eigenvector
 x = ??? # normalize
 L = ??? # approximate the eigenvalue
 counter += 1 # increment the counter
 return x, L

```

---

**Exercise C.60.** Test your `myPower()` function on several matrices where you know the eigenstructure. Then try the `myPower()` function on larger random matrices. You can check that it is working using `np.linalg.eig()` (be sure to normalize the vectors in the same way so you can compare them.)

---

**Exercise C.61.** In the Power Method iteration you may end up getting a different sign on your eigenvector as compared to `np.linalg.eig()`. Why might this happen? Generate a few examples so you can see this. You can avoid this issue if you use a `while` loop in your Power Method code and the logical check takes advantage of the fact that we are trying to solve the equation  $Ax = \lambda x$ . Hint:  $Ax = \lambda x$  is equivalent to  $Ax - \lambda x = 0$ .

---

**Exercise C.62.** What happens in the power method iterations when  $\lambda_1$  is complex. The maximum eigenvalue can certainly be complex if  $|\lambda_1|$  (the modulus of the complex number) is larger than all of the other eigenvalues. It may be helpful to build a matrix specifically with complex eigenvalues.<sup>5</sup>

---

<sup>5</sup>To build a matrix with specific eigenvalues it may be helpful to recall the matrix factorization  $A = PDP^{-1}$  where the columns of  $P$  are the eigenvectors of  $A$  and the diagonal entries of  $D$  are the eigenvalues. If you choose  $P$  and  $D$  then you can build  $A$  with your specific eigen-structure. If you are looking for complex eigenvalues then remember that the eigenvectors may well be complex too.

**Exercise C.63** (Convergence Rate of the Power Method). The proof that the power method will work hinges on the fact that  $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$ . In Exercise C.58 we proved that the limit

$$\lim_{k \rightarrow \infty} \frac{A^k x}{\lambda_1^k} \quad (\text{C.109})$$

converges to the dominant eigenvector, but how fast is the convergence? What does the speed of the convergence depend on?

Take note that since we are assuming that the eigenvalues are ordered, the ratio  $\lambda_2/\lambda_1$  will be larger than  $\lambda_j/\lambda_1$  for all  $j > 2$ . Hence, the speed at which the power method converges depends mostly on the ratio  $\lambda_2/\lambda_1$ . Let us build a numerical experiment to see how sensitive the power method is to this ratio.

Build a  $4 \times 4$  matrix  $A$  with dominant eigenvalue  $\lambda_1 = 1$  and all other eigenvalues less than 1 in absolute value. Then choose several values of  $\lambda_2$  and build an experiment to determine the number of iterations that it takes for the power method to converge to within a pre-determined tolerance to the dominant eigenvector. In the end you should produce a plot with the ratio  $\lambda_2/\lambda_1$  on the horizontal axis and the number of iterations to converge to a fixed tolerance on the vertical axis. Discuss what you see in your plot.

Hint: To build a matrix with specific eigen-structure use the matrix factorization  $A = PDP^{-1}$  where the columns of  $P$  contain the eigenvectors of  $A$  and the diagonal of  $D$  contains the eigenvalues. In this case the  $P$  matrix can be random but you need to control the  $D$  matrix. Moreover, remember that  $\lambda_3$  and  $\lambda_4$  should be smaller than  $\lambda_2$ .

## C.9. Algorithm Summaries

**Exercise C.64.** Explain in clear language how to efficiently solve an upper-triangular system of linear equations.

---

**Exercise C.65.** Explain in clear language how to efficiently solve a lower-triangular system of linear equations.

---

**Exercise C.66.** Explain in clear language how to solve the equation  $Ax = b$  using an  $LU$  decomposition.

---

**Exercise C.67.** Explain in clear language how to solve an overdetermined system of linear equations (more equations than unknowns) numerically.

---

**Exercise C.68.** Explain in clear language the algorithm for finding the columns of the  $Q$  matrix in the  $QR$  factorization. Give all of the mathematical details.

---

**Exercise C.69.** Explain in clear language how to find the upper-triangular matrix  $R$  in the  $QR$  factorization. Give all of the mathematical details.

---

**Exercise C.70.** Explain in clear language how to solve the equation  $Ax = b$  using a  $QR$  decomposition.

---

**Exercise C.71.** Explain in clear language how the power method works to find the dominant eigenvalue and eigenvector of a square matrix. Give all of the mathematical details.

---



## D. Automatic Differentiation

In Chapter 5, we explored numerical differentiation through finite difference methods. These methods approximate derivatives by evaluating the function at different points and calculating differences. While straightforward to implement, they suffer from two main limitations:

1. **Truncation error:** As we've seen, the error scales with some power of the step size  $h$
2. **Round-off error:** As  $h$  gets extremely small, floating-point arithmetic leads to precision loss

Automatic differentiation (AD) is a different approach that computes derivatives exactly (to machine precision) without relying on finite differences. AD leverages the chain rule and the fact that all computer programs, no matter how complex, ultimately break down into elementary operations (addition, multiplication, sin, exp, etc.) whose derivatives are known.

Let's introduce the concept of a **computation graph**, which is fundamental to understanding automatic differentiation.

A computation graph represents a mathematical function as a directed graph where:

- **Nodes** represent variables (inputs, outputs, or intermediate values)
- **Edges** represent dependencies between variables
- Each node performs a simple operation with known derivatives

---

**Example D.1.** Consider the function  $f(x, y) = x^2y + \sin(xy)$ . We can break this down into elementary operations and visualize it as a computation graph:

This computation graph for  $f(x, y) = x^2y + \sin(xy)$  at  $(x, y) = (2, 3)$  shows:

1. Input nodes:  $x = 2, y = 3$
2. Intermediate computations:
  - $v_1 = x^2 = 4$
  - $v_3 = xy = 6$
  - $v_2 = v_1y = 12$

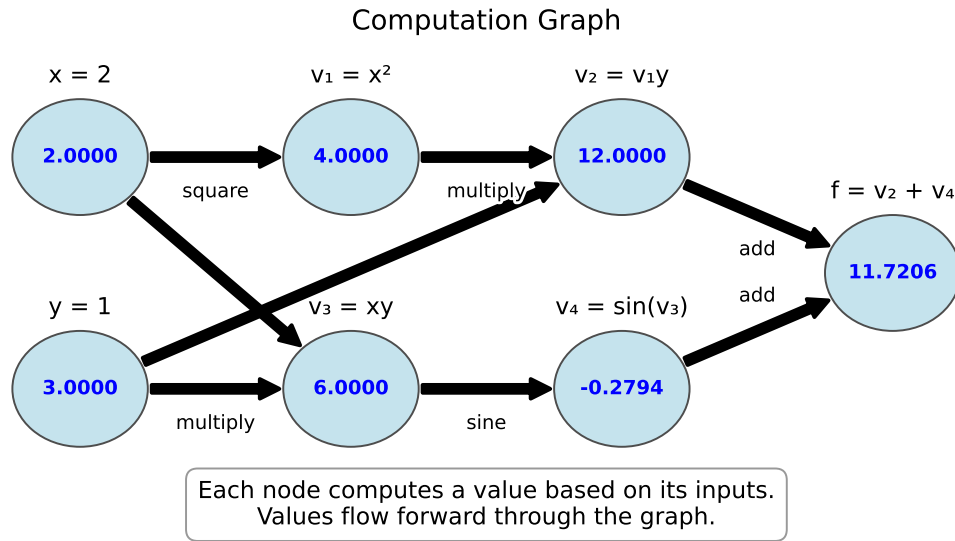


Figure D.1.: Computation graph for  $f(x,y) = x^2y + \sin(xy)$  at  $(x,y) = (2,1)$

- $v_4 = \sin(v_3) = \sin(6) \approx -0.2794$
3. Output node:  $f = v_2 + v_4 = 12 + (-0.2794) \approx 11.7206$

For each operation in the computation graph, we know:

1. How to compute the function value (forward evaluation)
2. How to compute the derivative of the operation with respect to its inputs

The computation graph is the foundation for automatic differentiation:

- **Forward mode AD:** Derivatives flow through the graph in the same direction as function evaluation
- **Reverse mode AD:** Function values flow forward through the graph, derivatives flow backward

This will become clearer in sections Section D.1 and Section D.2.

---

**Exercise D.1.** 1. Draw the computation graph for the function  $f(x) = x^2 \sin(x)$ .

2. For the function  $h(x,y,z) = xy + yz + zx$ , identify:

- The input nodes
- The intermediate nodes and their operations
- The output node

3. Explain why breaking a complex function into a computation graph of elementary operations is useful for derivative computation.

---

## D.1. Forward Mode AD

In forward mode automatic differentiation, we track both the values of variables and their derivatives with respect to the input variables. This allows us to build the derivatives as we compute the function value.

**Example D.2.** Consider a simple function  $f(x) = x^2 \cdot \sin(x)$ . We can compute both the value and the derivative at  $x = 2$  as follows:

1. Initialize:  $x = 2$ ,  $\frac{dx}{dx} = 1$  (The derivative of  $x$  with respect to itself is 1)
2. Compute  $u = x^2$ :
  - Value:  $u = 2^2 = 4$
  - Derivative:  $\frac{du}{dx} = \frac{d(x^2)}{dx} \cdot \frac{dx}{dx} = 2x \cdot 1 = 2 \cdot 2 = 4$
3. Compute  $v = \sin(x)$ :
  - Value:  $v = \sin(2) \approx 0.9093$
  - Derivative:  $\frac{dv}{dx} = \frac{d\sin(x)}{dx} \cdot \frac{dx}{dx} = \cos(x) \cdot 1 = \cos(2) \approx -0.4161$
4. Compute  $f = u \cdot v$ :
  - Value:  $f = 4 \cdot 0.9093 \approx 3.6372$
  - Derivative:  $\frac{df}{dx} = \frac{d(u \cdot v)}{dx} = \frac{du}{dx} \cdot v + u \cdot \frac{dv}{dx} = 4 \cdot 0.9093 + 4 \cdot (-0.4161) \approx 1.9728$

This is **forward mode** automatic differentiation.

The figure below illustrates the computational graph for  $f(x) = x^2 \cdot \sin(x)$  with forward mode AD. The blue values show the function evaluation, while the red values show the derivative calculation:

---

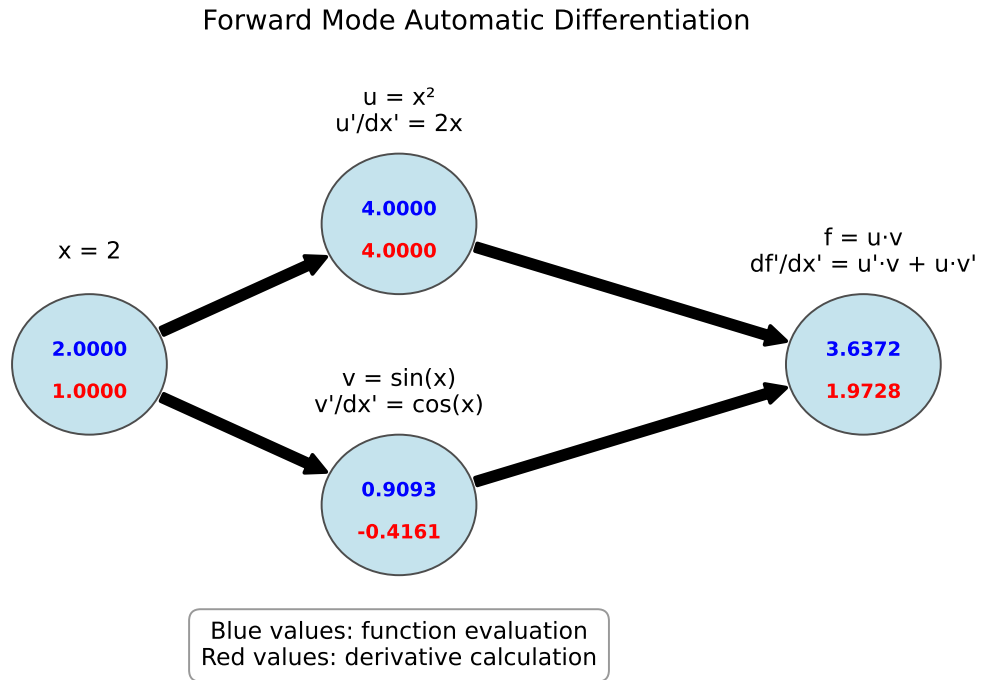


Figure D.2.: Forward mode automatic differentiation for  $f(x) = x^2 \cdot \sin(x)$  at  $x = 2$

Let's formalize the forward mode automatic differentiation process by creating a systematic approach. We'll use the concept of a "dual number" that carries both a value and its derivative.

We will represent each variable as a pair  $\begin{pmatrix} v \\ d \end{pmatrix}$  where  $v$  is the variable's value and  $d$  is its derivative with respect to the input we're differentiating against.

Consider these basic operations and their dual number representations:

1. Addition:  $\begin{pmatrix} a \\ a' \end{pmatrix} + \begin{pmatrix} b \\ b' \end{pmatrix} = \begin{pmatrix} a + b \\ a' + b' \end{pmatrix}$
2. Multiplication:  $\begin{pmatrix} a \\ a' \end{pmatrix} \cdot \begin{pmatrix} b \\ b' \end{pmatrix} = \begin{pmatrix} a \cdot b \\ a' \cdot b + a \cdot b' \end{pmatrix}$
3. Division:  $\begin{pmatrix} a \\ a' \end{pmatrix} / \begin{pmatrix} b \\ b' \end{pmatrix} = \begin{pmatrix} a/b \\ (a' \cdot b - a \cdot b')/b^2 \end{pmatrix}$
4. Power:  $\begin{pmatrix} a \\ a' \end{pmatrix}^n = \begin{pmatrix} a^n \\ n \cdot a^{n-1} \cdot a' \end{pmatrix}$
5. Sine:  $\sin \begin{pmatrix} a \\ a' \end{pmatrix} = \begin{pmatrix} \sin(a) \\ \cos(a) \cdot a' \end{pmatrix}$



6. Exponential:  $e^{\begin{pmatrix} a \\ a' \end{pmatrix}} = \begin{pmatrix} e^a \\ e^a \cdot a' \end{pmatrix}$

With this notation the calculations from Example D.2 can be written as:

$$\begin{aligned} \begin{pmatrix} x \\ x' \end{pmatrix} &= \begin{pmatrix} 2 \\ 1 \end{pmatrix} \\ \begin{pmatrix} u \\ u' \end{pmatrix} &= \begin{pmatrix} x \\ x' \end{pmatrix}^2 = \begin{pmatrix} x^2 \\ 2xx' \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix} \\ \begin{pmatrix} v \\ v' \end{pmatrix} &= \sin \begin{pmatrix} x \\ x' \end{pmatrix} = \begin{pmatrix} \sin(x) \\ \cos(x)x' \end{pmatrix} = \begin{pmatrix} 0.9093 \\ -0.4161 \end{pmatrix} \\ \begin{pmatrix} f \\ f' \end{pmatrix} &= \begin{pmatrix} u \\ u' \end{pmatrix} \cdot \begin{pmatrix} v \\ v' \end{pmatrix} = \begin{pmatrix} u \cdot v \\ u' \cdot v + u \cdot v' \end{pmatrix} = \begin{pmatrix} 3.6372 \\ 1.9728 \end{pmatrix} \end{aligned}$$


---

**Exercise D.2.** Compute the derivative of using the dual number approach:

b.  $f(x) = \frac{x^2+1}{2x-3}$  at  $x = 2$

Verify your result by computing the derivative analytically and comparing.

---

Next let us teach Python how to do this.

```
(np.float64(3.637189707302727), np.float64(1.9726023611141572))
```

```
(np.float64(3.637189707302727), np.float64(1.9726023611141572))
```

For multivariate functions, we can compute one directional derivative at a time.

For a function  $f(x, y)$ , to compute  $\frac{\partial f}{\partial x}$ , we initialize:

- $x = \begin{pmatrix} x \\ 1 \end{pmatrix}$  (value and derivative of  $x$  with respect to  $x$ )
- $y = \begin{pmatrix} y \\ 0 \end{pmatrix}$  (value and derivative of  $y$  with respect to  $x$ )

And to compute  $\frac{\partial f}{\partial y}$ , we initialize:

## D. Automatic Differentiation

- $x = \begin{pmatrix} x \\ 0 \end{pmatrix}$  (value and derivative of  $x$  with respect to  $y$ )
  - $y = \begin{pmatrix} y \\ 1 \end{pmatrix}$  (value and derivative of  $y$  with respect to  $y$ )
- 

**Exercise D.3.** 1. For  $f(x, y) = x^2y + \sin(xy)$ , compute both  $\frac{\partial f}{\partial x}$  and  $\frac{\partial f}{\partial y}$  at  $(x, y) = (2, 1)$  using forward mode AD with the dual number approach.

2. For  $f(x, y, z) = x^2y + y \sin(z) + z \cos(x)$ , compute  $\frac{\partial f}{\partial x}$ ,  $\frac{\partial f}{\partial y}$ , and  $\frac{\partial f}{\partial z}$  at  $(x, y, z) = (1, 1, 1)$ .
3. If a function has  $n$  inputs and we want all partial derivatives, how many forward mode passes do we need? What implications does this have for functions with many inputs?
- 

## D.2. Reverse Mode AD

Now let's explore **reverse mode** automatic differentiation, which is more efficient for functions with many inputs and few outputs. Reverse mode AD first computes the function value and then propagates derivatives backward from the output to the inputs.

For this exercise, we'll introduce the concept of **adjoints** or **accumulated gradients**. The adjoint of a variable  $v$  is denoted  $\bar{v}$  and represents  $\frac{\partial f}{\partial v}$  where  $f$  is the final output.

**Example D.3.** Let's trace through the reverse mode process for  $f(x, y) = x^2y + \sin(xy)$  at  $(x, y) = (2, 3)$ :

1. Define intermediate variables in the computation graph as in Figure D.1:

- $v_1 = x^2 = 4$
- $v_2 = v_1 \cdot y = 4 \cdot 3 = 12$
- $v_3 = x \cdot y = 2 \cdot 3 = 6$
- $v_4 = \sin(v_3) = \sin(6) \approx -0.2794$
- $f = v_2 + v_4 = 12 + (-0.2794) \approx 11.7206$  (final output)

2. Initialize the adjoint of the output:  $\bar{f} = 1$

3. Propagate adjoints backward using the chain rule:

- $\bar{v}_4 = \bar{f} \cdot \frac{\partial f}{\partial v_4} = 1 \cdot 1 = 1$

- $\overline{v_3} = \overline{v_4} \cdot \frac{\partial v_4}{\partial v_3} = 1 \cdot \cos(v_3) = \cos(6) \approx 0.9602$
- $\overline{v_2} = \overline{v_5} \cdot \frac{\partial f}{\partial v_2} = 1 \cdot 1 = 1$
- $\overline{v_1} = \overline{v_2} \cdot \frac{\partial v_2}{\partial v_1} = 1 \cdot y = 3$
- $\overline{x} = \overline{v_1} \cdot \frac{\partial v_1}{\partial x} + \overline{v_3} \cdot \frac{\partial v_3}{\partial x} = 3 \cdot 2x + 0.9602 \cdot y = 3 \cdot 2 \cdot 2 + 0.9602 \cdot 3 \approx 14.8806$
- $\overline{y} = \overline{v_2} \cdot \frac{\partial v_2}{\partial y} + \overline{v_3} \cdot \frac{\partial v_3}{\partial y} = 1 \cdot v_1 + 0.9602 \cdot x = 1 \cdot 4 + 0.9602 \cdot 2 \approx 5.9204$

4. The final results are  $\frac{\partial f}{\partial x} = \overline{x} \approx 14.8806$  and  $\frac{\partial f}{\partial y} = \overline{y} \approx 5.9204$

Figure D.3 illustrates the reverse mode AD computation graph for this example.

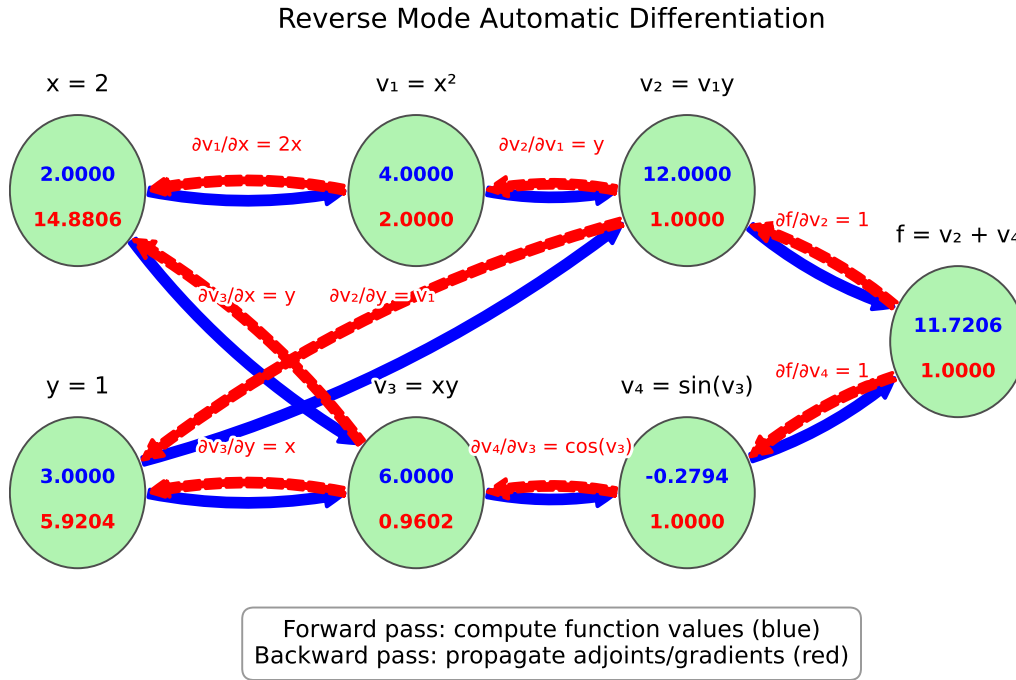


Figure D.3.: Reverse mode automatic differentiation for  $f(x,y) = x^2y + \sin(xy)$  at  $(x,y) = (2,1)$

**Exercise D.4.** Trace through the reverse mode process for:

- $f(x,y) = e^{xy}$  at  $(x,y) = (0,1)$ .
- $f(x,y) = xy^2 \cos(xy^2)$  at  $(x,y) = (\pi,3)$ .
- $f(x,y,z) = x^2 + y^2 + z^2 + xy + yz + xz$  at  $(x,y,z) = (1,1,1)$ .

#### D. Automatic Differentiation

Let's now look at how to use JAX, a library that provides automatic differentiation in Python. JAX is designed to be simple to use while providing powerful capabilities. Unlike our finite-difference methods that had truncation errors, JAX computes the derivative exactly (to machine precision).

**Example D.4.** Here's a simple example using JAX to compute derivatives:

```
import jax
import jax.numpy as jnp

Define a function
def f(x):
 return jnp.sin(x) * (1 - x)

Compute the derivative function
df = jax.grad(f)

Evaluate at x = 1
print(f"f(1) = {f(1)}")
print(f"f'(1) = {df(1.0)}")
```

Two things to note:

- We had to use the jax versions of NumPy functions, like `jnp.sin` instead of `np.sin`.
- We had to pass a float to the `df` function, rather than an integer. So we had to write `df(1.0)` instead of `df(1)`.

**Exercise D.5.** Use JAX to compute the derivatives of:

- $f(x) = x^3 - 2x^2 + 4x - 7$
- $f(x) = \sin(x) \exp(x)$
- $f(x) = \frac{x^2+1}{2x-3}$

Check that you get the same results as in Exercise D.2.

**Example D.5.** For multivariate functions, JAX allows us to compute partial derivatives:

```
def g(x, y):
 return x**2 * y + jnp.sin(x * y)

Compute partial with respect to first argument (x)
dg_dx = jax.grad(g, argnums=0)
```

```
Compute partial with respect to second argument (y)
dg_dy = jax.grad(g, argnums=1)

Evaluate at (2, 1)
print(f" g/ x at (2,1) = {dg_dx(2.0, 1.0)}")
print(f" g/ y at (2,1) = {dg_dy(2.0, 1.0)}")
```

**Exercise D.6.** Use this approach to compute the partial derivatives of:

- a.  $f(x, y) = x \exp(y + x) + y \sin(xy)$
- b.  $f(x, y, z) = x^2 y + y \sin(z) + z \cos(x)$

JAX can also compute higher-order derivatives:

```
Second derivative
d2f = jax.grad(jax.grad(f))
print(f"f''(1) = {d2f(1.0)}")

Or more concisely
d2f = jax.hessian(f)
print(f"f''(1) = {d2f(1.0)}")
```

---

**Exercise D.7.** Compare the accuracy and efficiency of the differentiation methods we've studied:

1. For the function  $f(x) = \sin(x)(1 - x)$ , compute the derivative at  $x = 1$  using:
  - a. Forward difference with step sizes  $h = 0.1, 0.01, 0.001$
  - b. Central difference with step sizes  $h = 0.1, 0.01, 0.001$
  - c. JAX automatic differentiation

Create a table giving the value of the derivative and the absolute error for each of these 7 cases.

### **D.3. Algorithm Summaries**

**Exercise D.8.** Explain how to define arithmetic operations on dual numbers and how to use them to compute the derivative of a function with forward mode automatic differentiation.

---

**Exercise D.9.** Given a computation graph of a function, explain how to accumulate the gradients in a backwards pass through the computation graph. What is the advantage of this method over forward mode automatic differentiation?

---

## E. PDE 3

### E.1. The Wave Equation

The problems that we have dealt with thus far all model natural diffusion processes: heat transport, molecular diffusion, etc. Another interesting physical phenomenon is that of wave propagation. The 1D *wave equation* is

$$u_{tt} = c^2 u_{xx}$$

where  $c$  is a parameter modelling the stiffness of the medium the wave is travelling through. With homogeneous Dirichlet boundary conditions we can think of this as the behaviour of a guitar string after it has been plucked. If the boundaries are in motion then the model might be of someone wiggling a taut string from one end.

---

**Exercise E.1.** Let us write code to numerically solve the 1D wave equation. As before, we use the notation  $U_i^n$  to represent the approximate solution  $u(t, x)$  at the point  $t = t_n$  and  $x = x_i$ .

1. Give a reasonable discretization of the second derivative in time:

$$u_{tt}(t_n, x_i) \approx \underline{\hspace{2cm}}.$$

2. Give a reasonable discretization of the second derivative in space:

$$u_{xx}(t_n, x_i) \approx \underline{\hspace{2cm}}.$$

3. Put your answers to parts 1 and 2 together with the wave equation to get

$$\frac{??? - ??? + ???}{\Delta t^2} = c^2 \frac{??? - ??? + ???}{\Delta x^2}.$$

4. Solve the equation from part 3 for  $U_i^{n+1}$ . The resulting difference equation is the finite difference scheme for the 1D wave equation.

### E. PDE 3

5. You should notice that the finite difference scheme for the wave equation references two different times:  $U_i^n$  and  $U_i^{n-1}$ . Based on this observation, what information do we need in order to actually start our numerical solution?
6. Consider the wave equation  $u_{tt} = 2u_{xx}$  in  $x \in (0, 1)$  with  $u(0, x) = 4x(1 - x)$ ,  $u_t(0, x) = 0$ , and  $u(t, 0) = u(t, 1) = 0$ . Use the finite difference scheme that you built in this problem to approximate the solution to this PDE.

---

Figure E.1 shows the finite difference stencil for the 1D wave equation. Notice that we need two prior time steps in order to advance to the new time step. This means that in order to start the finite difference scheme for the wave equation we need to have information about time  $t_0$  and also time  $t_1$ . We get this information by using the two initial conditions  $u(0, x)$  and  $u_t(0, x)$ .

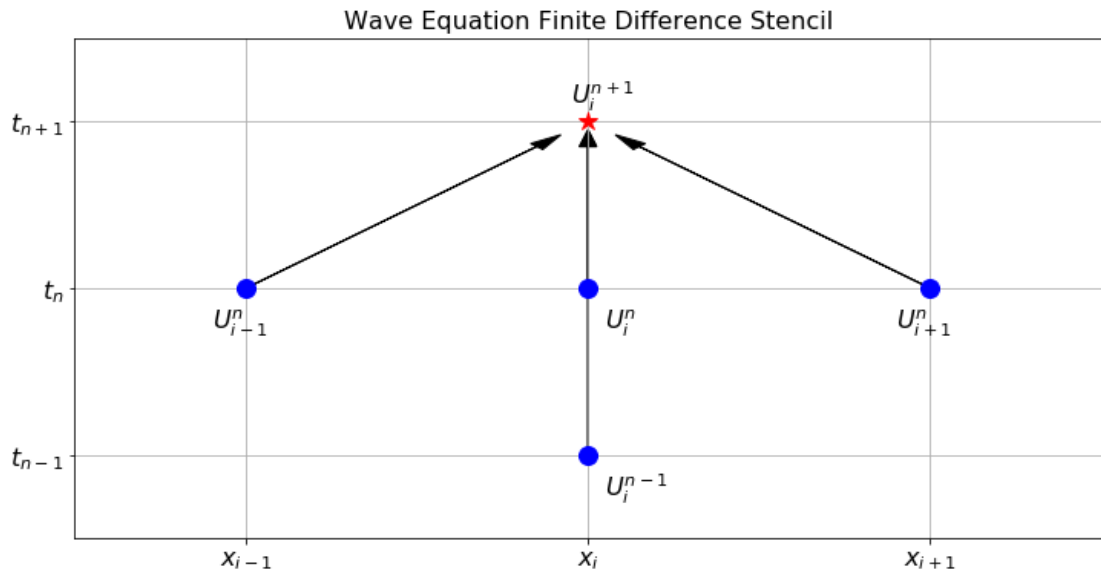


Figure E.1.: The finite difference stencil for the 1D wave equation.

---

**Exercise E.2.** The ratio  $c^2 \Delta t^2 / \Delta x^2$  shows up explicitly in the finite difference scheme for the 1D wave equation. Just like in the heat equation, this parameter controls when the finite difference solution will be stable. Experiment with your finite difference solution and conjecture a value of  $a = c^2 \Delta t^2 / \Delta x^2$  which divides the regions of stability versus instability. Your answer should be in the form:



If  $a = c^2 \Delta t^2 / \Delta x^2 < \underline{\hspace{1cm}}$  then the finite difference scheme for the 1D wave equation will be stable. Otherwise it will be unstable.

---

**Exercise E.3.** Show several plots demonstrating what occurs to the finite difference solution of the wave equation when the parameters are in the unstable region and right on the edge of the unstable region.

---

**Exercise E.4.** What is the expected error in the finite difference scheme for the 1D wave equation? What does this mean in plain English?

---

**Exercise E.5.** Use your finite difference code to solve the 1D wave equation

$$u_{tt} = c^2 u_{xx}$$

with boundary conditions  $u(t, 0) = u(t, 1) = 0$ , initial condition  $u(0, x) = 4x(1 - x)$ , and zero initial velocity. Experiment with different values of  $c^2$ . What does the parameter  $c$  do to the wave? Give a physical interpretation of  $c$ .

---

**Exercise E.6.** Solve the 1D wave equation

$$u_{tt} = u_{xx}$$

with Dirichlet boundary conditions  $u(t, 0) = 0.4 \sin(\pi t)$  and  $u(t, 1) = 0$  along with initial condition  $u(0, x) = 0$  and zero initial velocity. This time the left-hand boundary is being controlled externally and the string starts off at equilibrium. Give a physical situation where this sort of setup might arise. Then modify your solution so that both sides of the string are being wiggled at different frequencies.

## E.2. The 2D Wave Equation

**Exercise E.7.** Now consider the 2D wave equation

$$u_{tt} = c^2 (u_{xx} + u_{yy}).$$

We want to build a numerical solution to this new PDE. Just like with the 2D heat equation we propose the notation  $U_{i,j}^n$  for the approximation of the function  $u(t, x, y)$  at the point  $t = t_n$ ,  $x = x_i$ , and  $y = y_j$ .

1. Give discretizations of the derivatives  $u_{tt}$ ,  $u_{xx}$ , and  $u_{yy}$ .
2. Substitute your discretizations into the 2D wave equation, make the simplifying assumption that  $\Delta x = \Delta y$ , and solve for  $U_{i,j}^{n+1}$ . This is the finite difference scheme for the 2D wave equation.
3. Write code to implement the finite difference scheme from part 2 on the domain  $(x, y) \in (0, 1) \times (0, 1)$  with homogeneous Dirichlet boundary conditions, initial condition  $u(0, x, y) = \sin(2\pi(x - 0.5)) \sin(2\pi(y - 0.5))$ , and zero initial velocity.
4. Draw the finite difference stencil for the 2D wave equation.

**Exercise E.8.** What is the region of stability for the finite difference scheme on the 2D wave equation? Produce several plots showing what happens when we are in the unstable region as well as when we are right on the edge of the stable region.

**Exercise E.9.** Solve the 2D wave equation on the unit square with  $u$  starting at rest and being driven by a wave coming in from one boundary.

## E.3. The Transport Equation

Now we turn our attention to a new PDE: the transport equation

$$u_t + vu_x = 0.$$

In this equation  $u(t, x)$  is the density of some conserved quantity at time  $t$  and spatial location  $x$ . The parameter  $v$  is the velocity at which the quantity is transported.

**Exercise E.10.** Consider the PDE  $u_t + vu_x = 0$ . There is a very easy way to get an analytic solution to this equation that describes a travelling wave. If we have the initial condition  $u(0, x) = f(x) = e^{-(x-4)^2}$  then we claim that  $u(t, x) = f(x - vt)$  is an analytic solution to the PDE. More explicitly, we are claiming that

$$u(t, x) = e^{-(x-vt-4)^2}$$

is the analytic solution to the PDE. Let us prove this.

1. Take the  $t$  derivative of  $u(t, x)$ .
2. Take the  $x$  derivative of  $u(t, x)$ .
3. The PDE claims that  $u_t + vu_x = 0$ . Verify that this equal sign is indeed true.

**Exercise E.11.** Now we would like to visualize the solution to the PDE from the previous exercise. The Python code below gives an interactive visual of the solution. Experiment with different values of  $v$  and different initial conditions.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation, rc
from IPython.display import HTML

v = 1
f = lambda x: np.exp(-(x-4)**2)
u = lambda t, x: f(x - v*t)
x = np.linspace(0, 10, 101)
t = np.linspace(0, 10, 101)

fig, ax = plt.subplots()
plt.close()
ax.grid()
ax.set_xlabel('x')
ax.set_xlim((0, 10))
ax.set_ylim((-0.1, 1))
frame, = ax.plot([], [], linewidth=2, linestyle='--')

def animator(N):
 ax.set_title(f"Time = {t[N]:.2f}")
 frame.set_data(x, u(t[N], x)) # plot the correct time step for u(t,x)
 return (frame,)
```

```

PlotFrames = range(0,len(t),1)
anim = animation.FuncAnimation(fig,
 animator,
 frames=PlotFrames,
 interval=100,
)

rc('animation', html='jshtml') # embed in the HTML for Google Colab
anim

```

---

**Exercise E.12.** Use the chain rule to prove that for any differentiable function  $f(x)$  the function  $u(t, x) = f(x - vt)$  is an analytic solution to the transport equation  $u_t + vu_x = 0$  with initial condition  $u(0, x) = f(x)$ .

---

Thus the transport equation  $u_t + vu_x = 0$  has a very nice analytic solution which we can always find. Therefore there is no need to ever find a numerical solution – we can just write down the analytic solution if we are given the initial condition. As it turns out though, the numerical solutions exhibit some very interesting behaviour.

---

**Exercise E.13.** Consider the transport equation  $u_t + vu_x = 0$  with initial condition  $u(0, x) = f(x)$  for some given function  $f$  and boundary condition  $u(t, 0) = 0$ . To build a numerical solution we will again adopt the notation  $U_i^n$  for the approximation to  $u(t, x)$  at the point  $t = t_n$  and  $x = x_i$ .

- Write an approximation of  $u_t$  using  $U_i^{n+1}$  and  $U_i^n$ .
  - Write an approximation of  $u_x$  using  $U_{i+1}^n$  and  $U_i^n$ .
  - Substitute your answers from parts (a) and (b) into the transport equation and solve for  $U_i^{n+1}$ . This is our first finite difference scheme for the transport equation.
  - Write Python code to get the finite difference approximation of the solution to the PDE. Plot your finite difference solution on top of the analytic solution for  $f(x) = e^{-(x-4)^2}$ . What do you notice? Can you stabilize this method by changing the values of  $\Delta t$  and  $\Delta x$  like with did with the heat and wave equations?
-

The finite difference scheme that you built in the previous exercise is called the downwind scheme for the transport equation. Figure E.2 shows the finite difference stencil for this scheme. We call this scheme “downwind” since we expect the wave to travel from left to right and we can think of a fictitious wind blowing the solution from left to right. Notice that we are using information from “downwind” of the point at the new time step.

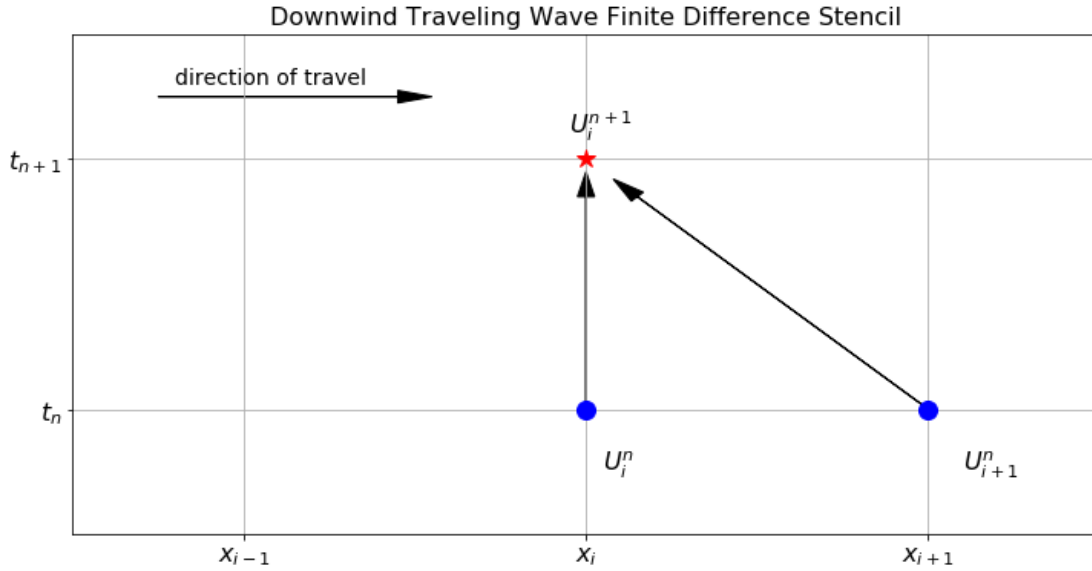


Figure E.2.: The finite difference stencil for the 1D downwind scheme on the traveling wave equation.

---

**Exercise E.14.** You should have noticed in the previous exercise that you cannot reasonably stabilize the finite difference scheme. Propose several reasons why this method appears to be unstable no matter what you use for the ratio  $v\Delta t/\Delta x$ .

---

**Exercise E.15.** One of the troubles with the finite difference scheme that we have built for the transport equation is that we are using the information at our present spatial location and the next spatial location to the right to propagate the solution forward in time. The trouble here is that the wave is moving from left to right, so the interesting information about the next time step’s solution is actually coming from the left, not the right. We call this “looking upwind” since you can think of a fictitious *wind* blowing from left to right, and we need to look “upwind” to see what is coming at us. If we write the spatial derivative as

$$u_x \approx \frac{U_i^n - U_{i-1}^n}{\Delta x}$$

### E. PDE 3

we still have a first-order approximation of the derivative but we are now looking left instead of right for our spatial derivative. Make this modification in your finite difference code for the transport equation (call it the “upwind method”). Approximate the solution to the same PDE as we worked with in the previous exercises. What do you notice now?

Figure E.3 shows the finite difference stencil for the upwind scheme. We call this scheme “upwind” since we expect the wave to travel from left to right and we can think of a fictitious wind blowing the solution from left to right. Notice that we are using information from “upwind” of the point at the new time step.

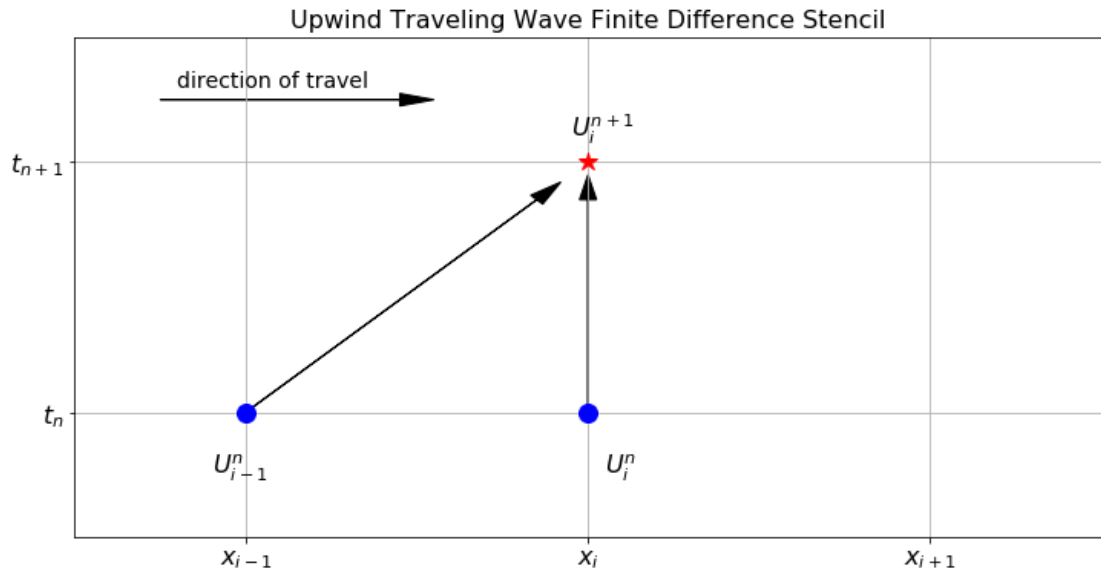


Figure E.3.: The finite difference stencil for the 1D upwind scheme on the traveling wave equation.

**Exercise E.16.** Complete the following sentences:

1. In the downwind finite difference scheme for the transport equation, the approximate solution moves at the correct speed, but ...
2. In the upwind finite difference scheme for the transport equation, the approximate solution moves at the correct speed, but ...

**Exercise E.17.** Neither the downwind nor the upwind solutions for the transport equation are satisfactory. They completely miss the interesting dynamics of the analytic solution to the PDE. Some ideas for stabilizing the finite difference solution for the transport equation are as follows. Implement each of these ideas and discuss pros and cons of each. Also draw a finite difference stencil for each of these methods.

1. Perhaps one of the issues is that we are using first-order methods to approximate  $u_t$  and  $u_x$ . What if we used a second-order approximation for these first derivatives

$$u_t \approx \frac{U_i^{n+1} - U_i^{n-1}}{2\Delta t} \quad \text{and} \quad u_x \approx \frac{U_{i+1}^n - U_{i-1}^n}{2\Delta x}?$$

Solve for  $U_i^{n+1}$  and implement this method. This is called the **leapfrog method**.

2. For this next method let us stick with the second-order approximation of  $u_x$  but we will do something clever for  $u_t$ . For the time derivative we originally used

$$u_t \approx \frac{U_i^{n+1} - U_i^n}{\Delta t}$$

what happens if we replace  $U_i^n$  with the average value from the two surrounding spatial points

$$U_i^n \approx \frac{1}{2} (U_{i+1}^n + U_{i-1}^n)?$$

This would make our approximation of the time derivative

$$u_t \approx \frac{U_i^{n+1} - \frac{1}{2}(U_{i+1}^n + U_{i-1}^n)}{\Delta t}.$$

Solve this modified finite difference equation for  $U_i^{n+1}$  and implement this method. This is called the **Lax-Friedrichs** method.

3. Finally we will do something very clever (and very counter intuitive). What if we inserted some artificial diffusion into the problem? You know from your work with the heat equation that diffusion spreads a solution out. The downwind scheme seemed to have the issue that it was *bunching up* at the beginning and end of the wave, so artificial diffusion might smooth this out. The **Lax-Wendroff method** does exactly that: take a regular Euler-type step in time

$$u_t \approx \frac{U_i^{n+1} - U_i^n}{\Delta t},$$

use a second-order centred difference scheme in space to approximate  $u_x$

$$u_x \approx \frac{U_{i+1}^n - U_{i-1}^n}{2\Delta x},$$

but add on the term

$$\left( \frac{v^2 \Delta t^2}{2\Delta x^2} \right) (U_{i-1}^n - 2U_i^n + U_{i+1}^n)$$

to the right-hand side of the equation. Notice that this new term is a scalar multiple of the second-order approximation of the second derivative  $u_{xx}$ . Solve this equation for  $U_i^{n+1}$  and implement the Lax-Wendroff method.

## E.4. The Laplace and Poisson Equations

**Exercise E.18.** Consider the 1D heat equation  $u_t = 1u_{xx}$  with boundary conditions  $u(t, 0) = 0$  and  $u(t, 1) = 1$  and initial condition  $u(0, x) = 0$ .

1. Describe the physical setup for this problem.
2. Recall that the solution to a differential equation reaches a steady state (or equilibrium) when the time rate of change is zero. Based on the physical system, what is the steady state heat profile for this PDE?
3. Use your 1D heat equation code to show the full time evolution of this PDE. Run the simulation long enough so that you see the steady state heat profile.

---

**Exercise E.19.** Now consider the forced 1D heat equation  $u_t = u_{xx} + e^{-(x-0.5)^2}$  with the same boundary and initial conditions as the previous exercise. The exponential forcing function introduced in this equation is an external source of heat (like a flame held to the middle of the metal rod).

1. Conjecture what the steady state heat profile will look like for this particular setup. Be able to defend your answer.
2. Modify your 1D heat equation code to show the full time evolution of this PDE. Run the simulation long enough so that you see the steady state heat profile.

---

**Exercise E.20.** Next we will examine 2D steady state heat profiles. Consider the PDE  $u_t = u_{xx} + u_{yy}$  with boundary conditions  $u(t, 0, y) = u(t, x, 0) = u(t, x, 1) = 0$  and  $u(t, 1, y) = 1$  with initial condition  $u(0, x, y) = 0$ .

1. Describe the physical setup for this problem.
2. Based on the physical system, describe the steady state heat profile for this PDE. Be sure that your steady state solution still satisfies the boundary conditions.
3. Use your 2D heat equation code to show the full time evolution of this PDE. Run the simulation long enough so that you see the steady state heat profile.



**Exercise E.21.** Now consider the forced 2D heat equation  $u_t = u_{xx} + u_{yy} + 10e^{-(x-0.5)^2-(y-0.5)^2}$  with the same boundary and initial conditions as the previous exercise. The exponential forcing function introduced in this equation is an external source of heat (like a flame held to the middle of the metal sheet).

1. Conjecture what the steady state heat profile will look like for this particular setup. Be able to defend your answer.
2. Modify your 2D heat equation code to show the full time evolution of this PDE. Run the simulation long enough so that you see the steady state heat profile.

Up to this point we have studied PDEs that all depend on time. In many applications, however, we are not interested in the transient (time dependent) behaviour of a system. Instead we are often interested in the steady state solution when the forces in question are in static equilibrium. Two very famous time-independent PDEs are the Laplace Equation

$$u_{xx} + u_{yy} + u_{zz} = 0$$

and the Poisson equation

$$u_{xx} + u_{yy} + u_{zz} = f(x, y, z).$$

Notice that both the Laplace and Poisson equations are the equations that we get when we consider the limit  $u_t \rightarrow 0$ . In the limit when the time rate of change goes to zero we are actually just looking at the eventual steady state heat profile resulting from the initial and boundary conditions of the heat equation. In the previous exercises you already wrote code that will show the steady state profiles in a few setups. The trouble with the approach of letting the time-dependent simulation run for a *long time* is that the finite difference solution for the heat equation is known to have stability issues. Moreover, it may take a lot of computational time for the solution to reach the eventual steady state. In the remainder of this section we look at methods of solving for the steady state directly – without examining any of the transient behaviour. We will first examine a 1D version of the Laplace and Poisson equations.

**Exercise E.22.** Consider a 1-dimensional rod that is infinitely thin and has unit length. For the sake of simplicity assume the following:

- the specific heat of the rod is exactly 1 for the entire length of the rod,
- the temperature of the left end is held fixed at  $u(0) = 0$ ,

### E. PDE 3

- the temperature of the right end is held fixed at  $u(1) = 1$ , and
- the temperature has reached a steady state.

You can assume that the temperatures are *reference temperatures* instead of absolute temperatures, so a temperature of “0” might represent room temperature.

Since there are no external sources of heat we model the steady-state heat profile we must have  $u_t = 0$  in the heat equation. Thus the heat equation collapses to  $u_{xx} = 0$ . This is exactly the one dimensional Laplace equation.

- To get an exact solution of the Laplace equation in this situation we simply need to integrate twice. Do the integration and write the analytic solution (there should be no surprises here).
- To get a numerical solution we first need to partition the domain into finitely many point. For the sake of simplicity let us say that we subdivide the interval into 5 equal sub intervals (so there are 6 points including the endpoints). Furthermore, we know that we can approximate  $u_{xx}$  as

$$u_{xx} \approx \frac{U_{i+1} - 2U_i + U_{i-1}}{\Delta x^2}.$$

Thus we have 6 linear equations:

$$\begin{aligned} U_0 &= 0 && \text{(left boundary condition)} \\ \frac{U_2 - 2U_1 + U_0}{\Delta x^2} &= 0 \\ \frac{U_3 - 2U_2 + U_1}{\Delta x^2} &= 0 \\ \frac{U_4 - 2U_3 + U_2}{\Delta x^2} &= 0 \\ \frac{U_5 - 2U_4 + U_3}{\Delta x^2} &= 0 \\ U_5 &= 1 && \text{(right boundary condition).} \end{aligned}$$

Notice that there are really only four unknowns since the boundary conditions dictate two of the temperature values. Rearrange this system of equations into a matrix equation and solve for the unknowns  $U_1$ ,  $U_2$ ,  $U_3$ , and  $U_4$ . Your coefficient matrix should be  $4 \times 4$ .

- Compare your answers from parts (a) and (b).
- Write code to build the numerical solution with an arbitrary value for  $\Delta x$  (i.e. an arbitrary number of sub intervals). You should build the linear system automatically in your code.

Solving the 1D Laplace equation with Dirichlet boundary conditions is rather uninteresting since the answer will always be a linear function connecting the two boundary conditions. The Poisson equation  $u_{xx} = f(x)$  is more interesting than the Laplace equation in 1D. The function  $f(x)$  is called a forcing function. You can think of it this way: if  $u$  is the amount of force on a linear bridge, then  $f$  might be a function that gives the distribution of the forces on the bridge due to the cars sitting on the bridge. In terms of heat we can think of this as an external source of heat energy warming up the one-dimensional rod somewhere in the middle (like a flame being held to one place on the rod).

---

**Exercise E.23.** How would we analytically solve the Poisson equation  $u_{xx} = f(x)$  in one spatial dimension? As a sample problem consider  $x \in [0, 1]$ , the forcing function  $f(x) = 5 \sin(2\pi x)$  and boundary conditions  $u(0) = 2$  and  $u(1) = 0.5$ . Of course you need to check your answer by taking two derivatives and making sure that the second derivative exactly matches  $f(x)$ . Also be sure that your solution matches the boundary conditions exactly.

---

**Exercise E.24.** Now we can solve the Poisson equation from the previous problem numerically. Let us again build this with a partition that contains only 6 points just like we did with the Laplace equation a few exercise ago. We know the approximation for  $u_{xx}$  so we have the linear system

$$\begin{aligned}
 U_0 &= 2 \quad (\text{left boundary condition}) \\
 \frac{U_2 - 2U_1 + U_0}{\Delta x^2} &= f(x_1) \\
 \frac{U_3 - 2U_2 + U_1}{\Delta x^2} &= f(x_2) \\
 \frac{U_4 - 2U_3 + U_2}{\Delta x^2} &= f(x_3) \\
 \frac{U_5 - 2U_4 + U_3}{\Delta x^2} &= f(x_4) \\
 U_5 &= 0.5 \quad (\text{right boundary condition}).
 \end{aligned}$$

- (a) Rearrange the system of equations as a matrix equation and then solve the system for  $U_1, U_2, U_3$ , and  $U_4$ . There are really only four equations so your matrix should be  $4 \times 4$ .
- (b) Compare your solution from part (a) to the function values that you found in the previous exercise.

- (c) Now generalize the process of solving the 1D Poisson equation for an arbitrary value of  $\Delta x$ . You will need to build the matrix and the right-hand side in your code. Test your code on new forcing functions and new boundary conditions.

---

**Exercise E.25.** The previous exercises only account for Dirichlet boundary conditions (fixed boundary conditions). We would now like to modify our Poisson solution to allow for a Neumann condition: where we know the derivative of  $u$  at one of the boundaries. The statement of the problem is as follows:

$$\text{Solve: } u_{xx} = f(x) \quad \text{on } x \in (0, 1) \quad \text{with } u_x(0) = \alpha \quad \text{and } u(1) = \beta.$$

The derivative condition on the boundary can be approximated by using a first-order approximation of the derivative, and as a consequence we have one new equation. Specifically, if we know that  $u_x(0) = \alpha$  then we can approximate this condition as

$$\frac{U_1 - U_0}{\Delta x} = \alpha,$$

and we simply need to add this equation to the system that we were solving in the previous exercise. If we go back to our example of a partition with 6 points the system becomes

$$\begin{aligned} \frac{U_1 - U_0}{\Delta x} &= \alpha \quad (\text{left boundary condition}) \\ \frac{U_2 - 2U_1 + U_0}{\Delta x^2} &= f(x_1) \\ \frac{U_3 - 2U_2 + U_1}{\Delta x^2} &= f(x_2) \\ \frac{U_4 - 2U_3 + U_2}{\Delta x^2} &= f(x_3) \\ \frac{U_5 - 2U_4 + U_3}{\Delta x^2} &= f(x_4) \\ U_5 &= \beta \quad (\text{right boundary condition}). \end{aligned}$$

There are 5 equations this time.

- (a) With a 6 point grid solve the Poisson equation  $u_{xx} = 5 \sin(2\pi x)$  with  $u_x(0) = 0$  and  $u(1) = 3$ .
- (b) Modify your code from part (a) to solve the same problem but with a much smaller value of  $\Delta x$ . You will need to build the matrix equation in your code.
-

**Exercise E.26** (The 2D Poisson Equation). We conclude this section, and chapter, by examining the two dimensional Poisson equations. As a sample problem, we want to solve the Poisson equation

$$u_{xx} + u_{yy} = f(x, y)$$

on the domain  $(x, y) \in (0, 1) \times (0, 1)$  with homogeneous Dirichlet boundary conditions and forcing function

$$f(x, y) = -20 \exp \left( -\frac{(x - 0.5)^2 + (y - 0.5)^2}{0.05} \right)$$

numerically.

We are going to start with a  $6 \times 6$  grid of points and explicitly write down all of the equations. In Figure E.4 the red stars represent boundary points where the value of  $u(x, y)$  is known and the blue interior points are the ones where  $u(x, y)$  is yet unknown. It should be clear that we should have two indices for each point (one for the  $x$  position and one for the  $y$  position), but it should also be clear that this will cause problems when writing down the resulting system of equations as a matrix equation (stop and think carefully about this). Therefore, in Figure E.4 we propose an index,  $k$ , starting at the top left of the unknown nodes and reading left to right (just like we do with Python arrays).

- (a) Start by discretizing the 2D Poisson equation  $u_{xx} + u_{yy} = f(x, y)$ . For simplicity we assume that  $\Delta x = \Delta y$  so that we can combine like terms from the  $x$  derivative and the  $y$  derivative. Fill in the missing coefficients and indices below.

$$U_{i+1,j} + U_{i,j-1} - (\_)U_{\_,\_} + U_{\_,\_} + U_{\_,\_} = \Delta x^2 f(x_i, y_j)$$

- (b) In Figure E.4 we see that there are 16 total equations resulting from the discretization of the Poisson equation. Your first task is to write all 16 of these equations. we will get you started:

$$\begin{aligned} k = 0: & \quad U_{k=1} + U_{i=1,j=0} - 4U_{k=0} + U_{i=0,j=1} + U_{k=4} = \Delta x^2 f(x_1, y_1) \\ k = 1: & \quad U_{k=2} + U_{k=0} - 4U_{k=1} + U_{i=0,j=2} + U_{k=5} = \Delta x^2 f(x_1, y_2) \\ & \quad \vdots \\ k = 15: & \quad U_{i=4,j=5} + U_{k=14} - 4U_{k=15} + U_{k=11} + U_{i=5,j=4} = \Delta x^2 f(x_4, y_4) \end{aligned}$$

In this particular example we have homogeneous Dirichlet boundary conditions so all of the boundary values are zero. If this was not the case then every boundary value would need to be moved to the right-hand sides of the equations.

- (c) We now have a  $16 \times 16$  matrix equation to write based on the equations from part (b). Each row and column of the matrix equation is indexed by  $k$ . The coefficient matrix  $A$  is started for you below. Write the whole thing out and fill in the blanks.

### E. PDE 3

Notice that this matrix has a much more complicated structure than the coefficient matrix in the 1D Poisson and Laplace equations.

$$A = \begin{pmatrix} -4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & \cdots & 0 \\ 1 & -4 & 1 & 0 & 0 & 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & -4 & 1 & 0 & 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & -4 & 1 & 0 & 0 & 1 & \cdots & 0 \\ 1 & 0 & 0 & 0 & -4 & 1 & 0 & 0 & \ddots & \\ 0 & & & & & & & & & \\ \vdots & & & & & & & & & \\ & & & & & & & & & -4 \end{pmatrix}$$

(d) In the coefficient matrix from part (c) notice that the small matrix

$$\begin{pmatrix} -4 & 1 & 0 & 0 \\ 1 & -4 & 1 & 0 \\ 0 & 1 & -4 & 1 \\ 0 & 0 & 1 & -4 \end{pmatrix}$$

shows up in blocks along the main diagonal. If you have a hard copy of the matrix go back and draw a box around these blocks in the coefficient matrix. Also notice that there are diagonal bands of  $1^s$ . Discuss the following:

1. Why are the blocks  $4 \times 4$ ?
2. How could you have predicted the location of the diagonal bands of  $1^s$ ?
3. What would the structure of the matrix look like if we partitioned the domain into a  $10 \times 10$  grid of points instead of a  $6 \times 6$  grid (including the boundary points)?
4. Why is it helpful to notice this structure?

(e) The right-hand side of the matrix equation resulting the your system of equations from part (b) is

$$b = \Delta x^2 \begin{pmatrix} f(x_1, y_1) \\ f(x_1, y_2) \\ f(x_1, y_3) \\ f(x_1, y_4) \\ f(x_2, y_1) \\ f(x_2, y_2) \\ \vdots \\ f(x_4, y_4) \end{pmatrix}.$$

Notice the structure of this vector. Why is it structured this way? Why is it useful to notice this?

- (f) Write Python code to solve the problem at hand. Recall that  $f(x, y) = -20 \exp\left(-\frac{(x-0.5)^2 + (y-0.5)^2}{0.05}\right)$ . Show a contour plot of your solution. This will take a little work changing the indices back from  $k$  to  $i$  and  $j$ . Think carefully about how you want to code this before you put fingers to keyboard. You might want to use the `np.block()` command to build the coefficient matrix efficiently or you can use loops with carefully chosen indices.
- (g) (Challenge) Generalize your code to solve the Poisson equation with a much smaller value of  $\Delta x = \Delta y$ .
- (h) One more significant observation should be made about the 2D Poisson equation on this square domain. Notice that the corner points of the domain (e.g.  $i = 0, j = 0$  or  $i = 5, j = 0$ ) are never included in the system of equations. What does this mean about trying to enforce boundary conditions that only apply at the corners?

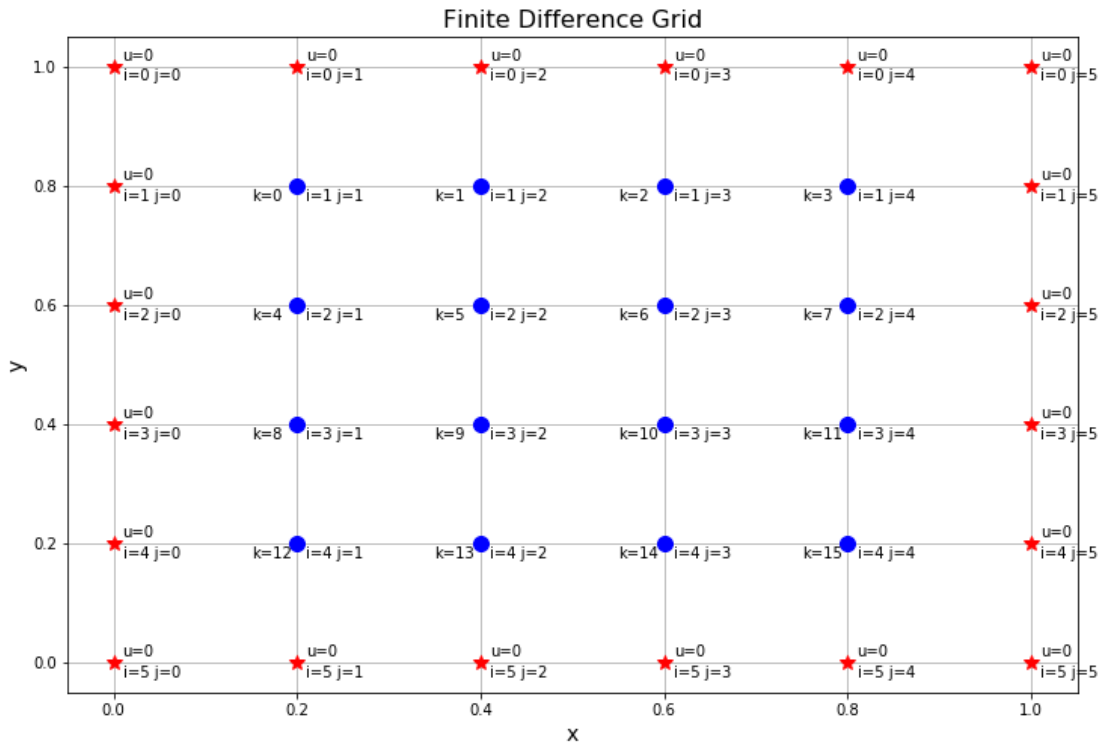


Figure E.4.: A finite difference grid for the Poisson equation with 6 grid points in each direction.

## E.5. Algorithm Summaries

**Exercise E.27.** Show the full mathematical details for building a first-order in time and second-order in space approximation method for the one-dimensional heat equation. Explain what the order of the error means in this context

---

**Exercise E.28.** Show the full mathematical details for building a second-order in time and second-order in space approximation method for the one-dimensional wave equation. Explain what the order of the error means in this context

---

**Exercise E.29.** Show the full mathematical details for building a first-order in time and second-order in space approximation method for the two-dimensional heat equation. Explain what the order of the error means in this context

---

**Exercise E.30.** Show the full mathematical details for building a second-order in time and second-order in space approximation method for the two-dimensional wave equation. Explain what the order of the error means in this context

---

**Exercise E.31.** Explain in clear language what it means for a finite difference method to be stable versus unstable.

---

**Exercise E.32.** Show the full mathematical details for solving the 1D heat equation using the implicit and Crank-Nicolson methods.

---

**Exercise E.33.** Show the full mathematical details for building a downwind finite difference scheme for the transport equation. Discuss the primary disadvantages of the downwind scheme.



---

**Exercise E.34.** Show the full mathematical details for building an upwind finite difference scheme for the transport equation. Discuss the primary disadvantages of the upwind scheme.

---

**Exercise E.35.** Show the full mathematical details for numerically solving the 1D Laplace and Poisson equations.

---

Acton, Forman S. 1990. *Numerical Methods That Work*. 1St Edition edition. The Mathematical Association of America.

Burden, Richard L., and J. Douglas Faires. 2010. *Numerical Analysis*. 9th ed. Brooks Cole.

Butcher, J. C. 2016. *Numerical Methods for Ordinary Differential Equations*. Third edition. Wiley. [https://yorsearch.york.ac.uk/permalink/f/1kq3a7l/44YORK\\_ALMA\\_DS51336126850001381](https://yorsearch.york.ac.uk/permalink/f/1kq3a7l/44YORK_ALMA_DS51336126850001381).

Greenbaum, Anne, and Tim P. Chartier. 2012. *Numerical Methods: Design, Analysis, and Computer Implementation of Algorithms*. Princeton University Press.

Kincaid, D. R., and E. W. Cheney. 2009. *Numerical Analysis: Mathematics of Scientific Computing*. Pure and Applied Undergraduate Texts. American Mathematical Society.

Press, William H., Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. 2007. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press. <https://numerical.recipes/>.

